STUDY GUIDE

PYTHON & SOFTWARE ENGINEERING: FROM ZERO TO PROFESSIONAL

A practical, hands-on textbook for a working professional moving into data roles and becoming a data, ML and AI engineer.

Taught the IIT Professor Way — Real Numbers. Real Problems. Real Understanding.

CHAPTER 1 Setup and Thinking in Python	**CHAPTER 2** Control Flow, Functions, Scope and Recursion	**CHAPTER 3** Strings, Text Processing and Regular Expressions	**CHAPTER 4** Collections, Comprehensions and the Standard-Library Toolbox
CHAPTER 5 Files, Paths, JSON, CSV, Dates and Time Zones	**CHAPTER 6** Errors, Exceptions, Logging and Debugging	**CHAPTER 7** Iterators, Generators, Decorators and Context Managers	**CHAPTER 8** Object-Oriented Programming
CHAPTER 9 Modules, Packages, Environments, Type Hints and Validation	**CHAPTER 10** Testing and Code Quality	**CHAPTER 11** Concurrency and Performance	**CHAPTER 12** APIs, the Web, Scraping and Command-Line Tools
CHAPTER 13 Professional Practice: Clean Code, CI/CD, Secrets, Security and Packaging	**CHAPTER 14** Capstone Projects and the Python Interview Bank	**CHAPTER 15** Project Lab: Four Portfolio Projects, Built and Tested Step by Step	**APPENDIX** Cheat Sheet

A Note From Your Professor

Dear learner,

You may already be good at spotting patterns, asking careful questions and checking whether a number makes sense. That is a real strength, and Python is the tool that will let you turn that strength into data, machine learning and AI work.

This document teaches Python the way a good engineer uses it: not just syntax, but how to think, test, debug, secure and ship code. Every chapter starts with something that works, shows you the real output (all code was run on Python 3.14, using only the standard library), and then explains why it works, line by line. Where a tool needs the internet or a paid service (pytest, FastAPI, pydantic and similar), the text says "not run here" and shows the standard usage, and you should run it yourself.

I will be honest: this is a large document, and some chapters are long. That is on purpose. You do not need to finish it in a week. You need to understand each part completely.

The one rule: **never move on until the current topic feels obvious.** If a section feels foggy, type the code yourself, change it, break it on purpose, and read the error message. Confusion that you fix today saves you hours in an interview and weeks on the job. Study about one hour a day. Slow and clear beats fast and fuzzy.

What Every Topic Contains

1. An **INTUITION** callout: the problem the idea solves, in everyday language.
2. A **formal definition** or "how it works", in plain words, with every new term defined.
3. **Worked examples** with real, runnable code and the exact output (programming sections) or numbered problems with steps (maths-style sections).
4. **The picture**: an ASCII diagram, table or image, explained completely.
5. A **real-world mapping** table, and the **connection to the real job**.
6. **Applications**, grouped by role.
7. **COMMON MISTAKE** callouts: what beginners get wrong and how to catch it.
8. **Check yourself**: three short questions with answers.

Every chapter ends with a **Chapter recap** (with a small cheat-sheet table), a graded **Practice set** with answers, and a **Prompt pack** of five prompts you can paste into any AI assistant.

Contents

1. **Setup and Thinking in Python.** The REPL, scripts, how Python runs code, basic types, mutability, references and copies.
2. **Control Flow, Functions, Scope and Recursion.** Decisions, loops, `match`, arguments, closures, LEGB, recursion.

3. **Strings, Text Processing and Regular Expressions.** Slicing, methods, f-strings, UTF-8, regex.
4. **Collections, Comprehensions and the Standard-Library Toolbox.** List, tuple, set, dict, `collections`, `itertools`, `functools`, `statistics`.
5. **Files, Paths, JSON, CSV, Dates and Time Zones.** Reading and writing data safely, configuration.
6. **Errors, Exceptions, Logging and Debugging.** Tracebacks, `try/except`, custom exceptions, `logging`, `pdb`.
7. **Iterators, Generators, Decorators and Context Managers.** Lazy data pipelines and reusable wrappers.
8. **Object-Oriented Programming.** Classes, dunder methods, dataclasses, inheritance, SOLID, design patterns.
9. **Modules, Packages, Environments, Type Hints and Validation.** Imports, venv, `pyproject.toml`, mypy, pydantic, notebooks.
10. **Testing and Code Quality.** Test pyramid, pytest ideas, mocks, TDD, ruff, black, pre-commit.
11. **Concurrency and Performance.** GIL, threads, processes, asyncio, profiling, vectorization, Little's law.
12. **APIs, the Web, Scraping and Command-Line Tools.** HTTP, REST, FastAPI basics, auth, responsible scraping, argparse.
13. **Professional Practice.** Clean code, architecture, CI/CD, secrets, SQL injection, hashing, packaging.
14. **Capstone Projects and the Python Interview Bank.** Four projects and 45 graded questions.

Appendix: **Cheat Sheet** (`99_cheatsheet.md`) with an honest closing note.

How To Study This Document

The learning loop for every topic:

1. **Build first.** Type the code from the section yourself. Do not copy and paste. Predict the output before you run it.
2. **Ask.** When something is unclear, use a prompt from the chapter's Prompt pack and ask an AI to explain it another way.
3. **Just-in-time theory.** Read the explanation after you have seen the code work. It will make more sense.
4. **Rebuild.** Close the document and write the same program from memory, with a small change (a new field, a new rule).
5. **Teach.** Explain the idea aloud, as if to a colleague, in simple words. If you cannot, you do not own it yet.

Suggested pace. About one chapter per week at one hour a day. Chapters 1 to 4 are the foundation; spend extra time there. Do the Practice set at the end of the chapter, then use the Prompt pack for a mock interview. Chapters 5 to 7 build real-world scripting skills. Chapters 8 to 10 make you a software engineer. Chapters 11 to 13 make you employable. Chapter 14 is where you prove it.

Using the prompt packs. Each pack has five prompts: teach me, give me graded exercises, quiz me, review my work, mock interview. Paste one into an AI assistant. When a prompt says `[PASTE HERE]`, paste your own code. Always run the code the AI gives you. Check it against the real output in this document.

Prerequisites

- Basic computer skills: files, folders, a terminal (Command Prompt, Terminal or PowerShell).
- Comfort with school mathematics. You do not need advanced maths for this document.
- No prior programming or data experience is required. Many examples use everyday data such as shop orders, delivery times and logs.

Tools you need

All free.

- **Python 3.10 or newer** (examples were run on 3.14), from python.org.
- A text editor: **VS Code** (free) with its Python extension.
- A terminal and **Git** (see the Linux and Git material in the skill map).
- Later chapters mention `pytest`, `ruff`, `black`, `mypy`, `pydantic`, `httpx` / `requests` and `FastAPI`. Install them with `pip` inside a virtual environment (Chapter 9) when you reach them. Everything runnable in the text uses only the standard library.

CHAPTER 1

SETUP AND THINKING IN PYTHON

Run code, meet the basic types, and understand what a variable really is.

Imagine your manager gives you a 2 GB file of every order an online shop took this year and says, "Tell me the average order value for each city by tomorrow." You cannot do that by hand, and Excel will not even open the file. With Python you can do it in 20 lines. But before those 20 lines make sense, you must understand the ground floor: how you run Python, what a variable really is, and why a change in one place can silently change data in another place. This chapter builds that ground floor. By the end you will run code in two ways, know the seven basic types, predict what a copy does and does not copy, and avoid the bugs that trip up most beginners.

Running example for this chapter. We use one line from an online-shop orders file: `A1042,3,480,true`. It means: order `A1042`, `3` items, amount `480` rupees, paid `true`. Python reads it as plain text, and turning that text into useful numbers is our first real job.

All code in this document was run with Python 3.14 on a Mac. If your Python is older, small details (like the bytecode listing in Section 1.1) may look different. Everything else works on any Python 3.10 or newer.

How to use the code blocks. For each example, first cover the output with your hand and **predict** what will print. Then compare. Then read the explanation. Being wrong is useful: it shows exactly where your mental model needs repair.

1.1 Running Python: REPL, Scripts and How Python Runs Your Code

INTUITION

In a kitchen you can taste one spoonful of sauce at once (that is a quick experiment), or you can write the full recipe down and cook it again later (that is a repeatable run). Python gives you the same two modes. The **REPL** is the quick taste. A **script** is the written recipe.

Formal definition

- **Python** is a programming language. The program that reads your code and runs it is called the **interpreter**. On your machine the command is usually `python3`.
- **REPL** means Read-Eval-Print Loop. You type one line, Python reads it, evaluates it, prints the

result, and waits for the next line. Start it by typing `python3` in a terminal. Leave it with `exit()` or Ctrl+D.

- A **script** is a text file ending in `.py`. You run it with `python3 file.py`.
- A **module** is any `.py` file that other code imports. A script and a module are the same kind of file. The only difference is how you use it.
- One-line mode: `python3 -c "print('hi')"` runs a string of code.

How Python runs code

Python does not turn your code directly into machine instructions the way C does. It works in three steps:

1. **Compile** your source text into **bytecode**. Bytecode is a list of small, simple instructions for a "virtual machine".
2. **Run** the bytecode on the **Python Virtual Machine (PVM)**, a loop that executes one instruction at a time.
3. **Cache** the bytecode in a `.pyc` file inside the `__pycache__` folder, so imports are faster next time.

```
your code      compile         bytecode        PVM executes
add.py  ----->  (automatic) --> LOAD, ADD, ... --> result
                     |
                     +--> __pycache__/add.cpython-314.pyc  (cache)
```

The standard Python from python.org is called **CPython** because it is written in C. There are others (PyPy, for example), but when people say "Python" they nearly always mean CPython.

Worked example 1.1.1: use the REPL

Predict: Before you look, what do you think each line prints? Which lines print nothing?

I piped three lines into the REPL to show you what a session looks like. The `>>>` is the prompt.

```
printf '2+3\nx=10\nx*x\n"ab"*3\nimport math\nmath.sqrt(16)\n' | python3 -q -i
```

```
>>> 5
>>> >>> 100
>>> 'abababab'
>>> >>> 4.0
>>>
```

The lines `x=10` and `import math` print nothing, because they are statements, not expressions. An

expression has a value (`2+3` is `5`). A **statement** does something (`x=10` stores a value). The REPL prints the value of every expression automatically. A script does not. In a script you must call `print()` .

Worked example 1.1.2: a script and its bytecode

Save this as `a.py` :

```
import sys, dis
def add(a, b):
    return a + b
dis.dis(add)
print(sys.version_info[:2] >= (3, 10))
print(__name__)
```

Run `python3 a.py` . Real output:

```
  2           RESUME                   0

  3           LOAD_FAST_BORROW_LOAD_FAST_BORROW 1 (a, b)
              BINARY_OP                0 (+)
              RETURN_VALUE
True
__main__
```

Read it like this:

- `dis.dis` shows the bytecode of the function `add` . Load `a` and `b` , add them, return the result. Exact names of instructions change between Python versions. That is normal.
- `__name__` is a built-in variable. When you run a file directly, its value is `"__main__"` . When another file imports it, the value is the module's name. We use this in Chapter 9 to write `if __name__ == "__main__":` .

Real-world mapping

Idea	Example	Where it appears in the job
REPL	`python3` , then `len(open("r.csv").readlines())`	Checking a file before you write a pipeline
Script	`python3 summarise.py results.csv`	A nightly report job
Module	`import utils`	Shared helper code in a team

		repo
Bytecode cache	`__pycache__/`	A folder you add to `.gitignore`

The picture

```
REPL                              Script
____                              ______
type -> result now                write file -> run whole file
good for: trying ideas,           good for: anything you want to
checking a function               repeat, share, test, schedule
state lost when you exit          state lost when script ends
```

Connection to the real job

- Data analysts and scientists live in the REPL and notebooks to explore data. Engineers save the final logic as scripts and modules that run on a schedule (for example, an Airflow task or a cron job).
- `python3 -c` is handy in shell pipelines and CI jobs for tiny checks.
- Knowing about `.pyc` and `__pycache__` stops you from committing them by mistake. They belong in `.gitignore`.

Applications

- Quick data checks in a terminal.
- Automation scripts (rename files, call APIs, parse logs).
- Entry points of services and batch jobs.
- Learning a new library by trying it in the REPL.

COMMON MISTAKE

Naming your own file after a standard library module, for example `json.py`, `random.py` or `email.py`. When you write `import json`, Python finds *your* file first and your program breaks in strange ways. Never use a standard library name for your own files.

PROFESSOR'S NOTE

Use the REPL for 5 minutes before writing any new function. Type small inputs and look at the output. This habit alone makes you faster than people who write 40 lines and then debug.

Check yourself

1. What is the difference between an expression and a statement?

Answer: An expression has a value (`2+3`). A statement performs an action (`x = 10`). The REPL auto-prints expression values.

1. What does `__name__` equal when you run `python3 a.py` ?

Answer: The string `"__main__"` .

1. Why does Python create `__pycache__` folders?

Answer: To store compiled bytecode (`.pyc` files) so later imports start faster.

1.2 Variables and the Basic Types

INTUITION

A cardboard box with a label is a good picture for other languages. For Python, a better picture is a **sticky note**. The value lives somewhere in memory. The variable name is a sticky note that you stick on it. You can move the note to another value any time, and you can put two notes on the same value.

Formal definition

A **variable** is a name bound to an object. Assignment `x = 10` means: create the object `10` (or reuse it), then bind the name `x` to it. Every object has a **type**. Python is **dynamically typed**: a name has no fixed type, but each object does. Python is also **strongly typed**: it will not silently mix `"5" + 5` .

The basic types:

Type	Meaning	Example
`int`	whole number, any size	`42` , `10**30`
`float`	decimal number (64-bit, IEEE 754)	`3.14`
`complex`	real + imaginary part	`2+3j`
`bool`	`True` or `False` (a subtype of `int`)	`True`
`str`	text (Unicode characters)	`"hello"`

`bytes`	raw bytes (numbers 0 to 255)	`b"hi"`
`NoneType`	the single value `None`, meaning "nothing here"	`None`

Worked example 1.2.1: numbers

Predict: Will `0.1 + 0.2 == 0.3` print `True`? What is `-7 // 2`?

```
print(type(42), type(3.14), type(2+3j), type(True), type("hi"), type(b"hi"),
type(None))
print(7/2, 7//2, 7%2, -7//2, -7%2, 2**10, divmod(17,5))
print(0.1+0.2, 0.1+0.2==0.3)
import math
print(math.isclose(0.1+0.2,0.3))
from decimal import Decimal
print(Decimal("0.1")+Decimal("0.2"))
print(10**30, (2+3j)*(1-1j), (2+3j).real, (2+3j).imag, abs(3+4j))
```

Real output:

```
<class 'int'> <class 'float'> <class 'complex'> <class 'bool'> <class 'str'>
<class 'bytes'> <class 'NoneType'>
3.5 3 1 -4 1 1024 (3, 2)
0.30000000000000004 False
True
0.3
1000000000000000000000000000000 (5+1j) 2.0 3.0 5.0
```

What to learn from this:

- `/` always gives a float. `//` is floor division (rounds *down*, so `-7//2` is `-4`, not `-3`). `%` is the remainder and always has the sign of the divisor, so `-7 % 2` is `1`.
- Integers have no size limit. `10**30` is exact.
- Floats are stored in binary. `0.1` has no exact binary form, just like `1/3` has no exact decimal form. So `0.1 + 0.2` is `0.30000000000000004`. Never compare floats with `==`. Use `math.isclose`. For money, use `decimal.Decimal` (made from strings) or integer paise.
- `(2+3j)*(1-1j) = 2 − 2j + 3j − 3j² = 2 + j + 3 = 5+1j`. Check by hand: it matches.

Worked example 1.2.2: truthiness, bool and conversion

```
print(True+True, isinstance(True,int), bool(0), bool(""), bool([]), bool("0"),
bool(None))
print(int("42")+1, float("2.5"), str(99)+"!", int(3.9), round(2.5), round(3.5),
round(2.675,2))
```

Real output:

```
2 True False False False True False
43 2.5 99! 3 2 4 2.67
```

- `True + True` is `2` because `bool` is a kind of `int`.
- **Truthiness**: in an `if`, these count as false: `0`, `0.0`, `""`, `[]`, `{}`, `()`, `set()`, `None`, `False`. Everything else is true. The string `"0"` is true because it is not empty.
- `int(3.9)` cuts off the decimals (gives `3`). It does not round.
- `round(2.5)` gives `2` and `round(3.5)` gives `4`. Python rounds exact halves to the nearest *even* number ("banker's rounding"). And `round(2.675, 2)` gives `2.67`, not `2.68`, because the float `2.675` is really a tiny bit less than 2.675.

Worked example 1.2.3: text, bytes and UTF-8

```
s = "héllo ₹"
print(len(s), len(s.encode("utf-8")), s.encode("utf-8"))
print(b"abc"[0], bytes([72,105]), b"Hi".decode())
```

Real output:

```
7 10 b'h\xc3\xa9llo \xe2\x82\xb9'
97 b'Hi' Hi
```

- The string has 7 **characters** but 10 **bytes** in UTF-8. `é` takes 2 bytes. `₹` takes 3 bytes. The rest take 1 each: 5 + 2 + 3 = 10.
- A `str` is text. `bytes` is raw data. `.encode()` turns text into bytes. `.decode()` turns bytes into text. You must name the encoding; always use `"utf-8"`.
- Indexing `bytes` gives an integer (`97` is the code of `a`).

Real-world mapping

Type	Example in our running row `A1042,3,480,true`	Where it appears in the job

`str`	`"A1042"`, and also `"480"` before conversion	Raw CSV and log fields
`int`	`3`, `480` after `int("480")`	Item counts, amounts, minutes
`float`	`411.25` (an average)	Means, percentages, model scores
`bool`	`ok == "true"` gives `True`	Flags such as `is_error`
`bytes`	`b"\xff"`	Files, network data, hashes
`None`	a missing delivery date	Missing values and "no result"

Worked example 1.2.4: our running row, with the typical mistakes

Predict: `amount` came from `split`. What is its type? What does `amount + amount` give?

```
row = "A1042,3,480,true"
order_id, items, amount, paid = row.split(",")
print(type(amount), amount + amount)
amount_rs = int(amount)
print(amount_rs + 20, paid == "true")
try:
    print("amount: " + amount_rs)
except TypeError as e:
    print("TypeError:", e)
print("amount: " + str(amount_rs))
try:
    print(undefined_name)
except NameError as e:
    print("NameError:", e)
try:
    b"\xff".decode("utf-8")
except UnicodeDecodeError as e:
    print("UnicodeDecodeError:", e)
```

Real output:

```
<class 'str'> 480480
500 True
TypeError: can only concatenate str (not "int") to str
amount: 480
NameError: name 'undefined_name' is not defined
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xff in position 0: invalid
start byte
```

Line by line:

- `split(",")` cuts the text at each comma and returns a list of four strings. We unpack them into four names. Note that `amount` is the *text* `"480"`, so `+` joins text and gives `480480`. This is the single most common beginner surprise with CSV data.
- `int(amount)` makes the number `480`. Now `+ 20` is arithmetic: `500`. Python has no `true` keyword, so we compare the text with `"true"` to get a real `bool`.
- Mixing `str` and `int` with `+` is an error: Python is strongly typed and refuses to guess. Fix: convert with `str(...)` (or use an f-string, Chapter 3).
- A `NameError` means you used a name that was never assigned. Read the name in the message: usually a typo.
- A `UnicodeDecodeError` means the bytes are not valid in the encoding you asked for. Byte `0xff` can never start a UTF-8 character.

The picture

```
name         object (in memory)
----         ------------------
x   ------>  [ int: 10 ]
y   ------>  [ str: "hi" ]
z   ------>  [ None ]

x = "now text"   moves the sticky note x to a new object.
The old int 10 is untouched (and freed if nothing else points to it).
```

Connection to the real job

- Float rounding explains why two data pipelines can give sums that differ in the 12th decimal. In tests, use `math.isclose` or `pytest.approx`.
- `None` is how Python represents missing values. Pandas uses `NaN` for the same idea in numeric columns (see the Data Analyst document).
- UTF-8 bugs (`UnicodeDecodeError`) are one of the most common failures when reading CSV files from different systems.
- Amounts and dates read from a CSV file of orders arrive as strings. You convert them with `int()` and `float()` before computing totals and averages.

Applications

- Money and billing (`Decimal`).
- Text cleaning and NLP (`str`, encoding).

- Binary files, network packets, hashes (`bytes`).
- Optional values in APIs (`None`).
- Signal processing and physics (`complex`).

COMMON MISTAKE

Using `==` to compare floats, and using `float` for money. Another very common one: reading a file without saying `encoding="utf-8"` , which works on your laptop and fails on the server.

Check yourself

1. What is `-9 // 4` and `-9 % 4` ?

Answer: `-3` and `3` . Floor of -2.25 is -3, and `-9 - (4 * -3) = 3` .

1. Why is `bool("False")` equal to `True` ?

Answer: It is a non-empty string, and every non-empty string is truthy.

1. How many bytes is `"₹"` in UTF-8?

Answer: 3 bytes.

1.3 Mutability, Identity, References and Memory

INTUITION

Two friends share one Google Doc. If one of them edits it, the other sees the change, because there is only one document. But if you each have your own printed copy, edits do not travel. A **mutable** object is the shared Google Doc. An **immutable** object is a printed page: you cannot edit it, you can only print a new one.

Formal definition

- **Mutable** objects can change after creation: `list` , `dict` , `set` , `bytearray` , and most classes you write.
- **Immutable** objects can never change: `int` , `float` , `complex` , `bool` , `str` , `bytes` , `tuple` , `frozenset` , `None` . Doing `n += 1` on an int creates a new int and moves the name.
- **Identity** is "same object in memory". `a is b` tests identity. `id(a)` gives a number for that identity.
- **Equality** is "same value". `a == b` tests equality (it calls the object's `__eq__`).

- Use `is` only for singletons: `x is None`, `x is True`. Use `==` for everything else.
- **Reference counting**: CPython counts how many names point to each object. When the count hits zero, memory is freed at once. A **garbage collector** (module `gc`) also finds and frees groups of objects that point to each other (cycles) when no outside name points to them.
- **Shallow copy** (`copy.copy`, `list[:]`, `list.copy()`) copies the outer container but shares the inner objects. **Deep copy** (`copy.deepcopy`) copies everything inside, recursively.

Worked example 1.3.1: aliasing

Predict: After `b.append(4)`, does `a` change?

```
import copy
a=[1,2,3]; b=a; c=a[:]
b.append(4)
print(a,b,c, a is b, a is c, a==c, id(a)==id(b))
```

Real output:

```
[1, 2, 3, 4] [1, 2, 3, 4] [1, 2, 3] True False False True
```

- `b = a` does not copy. It puts a second note on the same list. So `b.append(4)` changes what `a` sees.
- `c = a[:]` is a new list made earlier, so it still holds `[1, 2, 3]`. After the append, `a == c` is `False` because the values differ now, and `a is c` is `False` because they are different objects.

Worked example 1.3.2: shallow vs deep copy, and tuples with lists inside

Predict: A tuple is immutable. Can its inner list still grow?

```
import copy
nested=[[1,2],[3,4]]
sh=copy.copy(nested); dp=copy.deepcopy(nested)
nested[0].append(99)
print(sh, dp)
t=(1,[2,3]); t[1].append(9); print(t)
try:
    t[0]=5
except TypeError as e: print("TypeError:",e)
```

Real output:

```
[[1, 2, 99], [3, 4]] [[1, 2], [3, 4]]
(1, [2, 3, 9])
TypeError: 'tuple' object does not support item assignment
```

- The shallow copy `sh` has its own outer list but shares the inner lists, so it saw the `99`. The deep copy `dp` did not.
- A tuple is immutable, but it only promises that *which objects it holds* never changes. If one of those objects is a list, the list can still change. The tuple became `(1, [2, 3, 9])`.

Worked example 1.3.3: passing to functions

```
def f(lst): lst.append("x")
def g(n): n+=1; return n
L=[]; f(L); print(L); n=1; g(n); print(n)
```

Real output:

```
['x']
1
```

Python passes **the reference** (call by object reference, sometimes called "call by sharing"). Function `f` changes the same list the caller owns, so the caller sees `['x']`. Function `g` cannot change the caller's `n`, because `n += 1` makes a new int and re-points only the local name.

Worked example 1.3.4: the grid bug, memory size, and garbage collection

Predict: In `[[0]*3]*2`, if you set the first cell of row 0 to 1, how many rows change?

```
import sys, gc
grid=[[0]*3]*2; grid[0][0]=1; print(grid)
grid=[[0]*3 for _ in range(2)]; grid[0][0]=1; print(grid)
print(sys.getsizeof(0), sys.getsizeof([]), sys.getsizeof([1,2,3]),
sys.getsizeof("a"), sys.getsizeof({}))
class Node:
    def __init__(s): s.other=None
p=Node(); q=Node(); p.other=q; q.other=p
del p,q
print("collected:", gc.collect()>0)
import weakref
class K: pass
k=K(); r=weakref.ref(k); print(r() is k); del k; print(r())
```

Real output:

```
[[1, 0, 0], [1, 0, 0]]
[[1, 0, 0], [0, 0, 0]]
28 56 88 42 64
collected: True
True
None
```

- `[[0]*3]*2` repeats the *same* inner list twice. Changing one row changes both. The list comprehension builds two separate rows. This is a classic bug.
- `sys.getsizeof` shows the size of the object itself in bytes, **not** the objects it points to. An empty list is 56 bytes here and an `int` is 28 bytes. These numbers depend on Python version and platform. For a full size of a nested structure you have to add up inner objects yourself, or use `tracemalloc` (Chapter 11).
- Objects `p` and `q` point to each other. After `del p, q` reference counts do not reach zero, so only the cycle collector can free them. `gc.collect()` returned a positive count, meaning it found and freed objects.
- A **weak reference** (`weakref.ref`) does not keep an object alive. After `del k` the object is gone and `r()` returns `None` .
- `__slots__` is a way to reduce memory per instance. We cover it in Chapter 8.

Worked example 1.3.5: when `is` surprises you

```
a = 1000; b = 1000
print(a == b, a is b)
x = [1,2]; y = [1,2]
print(x == y, x is y)
v = None
print(v is None, v == None)
```

Real output:

```
True True
True False
True True
```

`a is b` printed `True` here only because Python reused one constant object when compiling this small file. Inside a function call or from user input, two equal big ints are often different objects. **The result of `is` on numbers and strings is an implementation detail. Never rely on it.** Lists made separately are never the same object.

The picture

Two names on one list, a third name on a copy

Read the diagram left to right. The blue boxes are names. The arrows go from a name to the object it points at. Names `a` and `b` have arrows to the same box, so a change through either name shows up through the other. Name `c` points to a different box that holds a copy. The text version below shows the same idea and the grid bug.

```
a = [1,2,3]
b = a            a ----+
                       +--> [1,2,3]
                 b ----+

c = a[:]         c ------> [1,2,3]   (new object)

grid = [[0]*3]*2
grid --> [ row , row ]       both slots point to ONE row
              \   /
               [0,0,0]
```

Connection to the real job

- The most common "ghost bug" in data code is a function that changes the list or DataFrame it was given. The caller's data changes and nobody knows why. Copy inputs at the start (`df = df.copy()`) or do not change them at all.
- Understanding references explains `SettingWithCopyWarning` in pandas.
- Memory awareness matters when loading millions of rows. A list of Python ints costs far more than the same numbers in a NumPy array.
- Long-running programs: if a script that processes millions of orders slowly eats memory, reference cycles and global lists are the first suspects.

Applications

- Safe function design (do not change arguments).
- Caches and shared state (dict as a cache).
- Deep copies of configuration before modifying for an experiment.
- Memory tuning for large data jobs.

COMMON MISTAKE

Thinking `b = a` makes a copy. Also: using `is` instead of `==` to compare values. A good rule: if you write `is` and the right side is not `None`, `True` or `False`, stop and ask why.

KEY INSIGHT

In Python, assignment never copies data. It only attaches a name to an object. Copies happen only when you ask (`.copy()`, slicing, `deepcopy`) or when an operation builds a new object (like `+` on strings).

Check yourself

1. After `a = [1,2]; b = a; b += [3]`, what is `a`?

Answer: `[1, 2, 3]`. `+=` on a list changes the list in place, and `a` and `b` are the same object.

1. Is `t = (1, 2)` followed by `t[0] = 9` allowed?

Answer: No. Tuples are immutable, so you get a `TypeError`.

1. What is the difference between `copy.copy` and `copy.deepcopy`?

Answer: `copy` copies only the outer container and shares inner objects. `deepcopy` copies inner objects recursively.

1.4 Unpacking, Naming Rules and First Good Habits

INTUITION

Good names are the cheapest documentation. A teammate (or you, six months later) should read `average_order_value` and know what it means without a comment. Unpacking is like handing out four parcels to four people in one move, instead of one by one.

Formal definition

- Names use letters, digits and `_`, and cannot start with a digit. Names are case sensitive. You cannot use keywords (`if`, `class`, `for`, ...).
- Style (**PEP 8**, the official Python style guide; PEP means Python Enhancement Proposal): `snake_case` for variables and functions, `PascalCase` for classes, `UPPER_CASE` for constants.
- **Unpacking** assigns several names at once: `a, b = 1, 2`. A starred name collects the rest:

`first, *rest = items`.

- **Swap** with `a, b = b, a`, no temporary variable needed.
- You can write big numbers with underscores: `1_250_000`. Python ignores them.

Worked example 1.4.1: unpacking

Predict: What do `a` and `b` hold after the swap? What is `rest`?

```
a, b = 1, 2
a, b = b, a
print(a, b)
amounts = [480, 360, 510, 295]
first, *rest = amounts
print(first, rest, sum(amounts)/len(amounts))
n = 0
n += 1; print(n)
a = [1,2]; b = a; b += [3]; print(a)
total_sales = 1_250_000
print(total_sales, f"{total_sales:,}")
```

Real output:

```
2 1
480 [360, 510, 295] 411.25
1
[1, 2, 3]
1250000 1,250,000
```

- The right side `b, a` is built first as the pair `(2, 1)`, and only then assigned. That is why no temporary variable is needed.
- `first` gets the first amount. `*rest` collects everything else into a new list.
- `n += 1` on an int makes a new int. `b += [3]` on a list changes the list in place, so `a` changes too. This is the one place where `+=` behaves differently for mutable and immutable types.
- `f"{total_sales:,}"` is an f-string with a thousands separator. We study f-strings in Chapter 3.

Problem 1.4.1 Using the four order amounts above (480, 360, 510, 295 rupees), find the mean and how much cheaper the cheapest order is than the dearest, in percent.

Formula	mean = sum / count. saving % = (dearest − cheapest) / dearest × 100.
Plug in	sum = 480 + 360 + 510 + 295. count = 4. dearest = 510. cheapest = 295.
Compute	sum = 1645, so mean = 1645 / 4 = 411.25. saving = (510 − 295) / 510 × 100 = 215 / 510 × 100 = 42.157...
Answer	The mean is 411.25 rupees. The cheapest order is about 42.2% cheaper than the dearest.
Check	In code, `sum(amounts)/len(amounts)` printed `411.25` above. And `510 * (1 - 0.42157)` is about `294.999`, which rounds to 295, so the percentage is right.
Why it matters	In real work you compute many statistics like this from a list of numbers. The mean alone hides the spread, which is why later chapters and the Statistics document use percentiles.

Real-world mapping

Habit	Bad	Good
Name a number	`x = 480`	`amount_rs = 480`
Name a constant	`limit = 2000`	`FREE_DELIVERY_LIMIT = 2000`
Unpack a record	`r[0]`, `r[2]`	`order_id, items, amount, paid = row.split(",")`

The picture

```
order_id, items, amount, paid = "A1042,3,480,true".split(",")

 "A1042,3,480,true" --split(",")--> ["A1042", "3", "480", "true"]
                                       |        |      |        |
                                   order_id  items amount  paid
```

The split produces one list. Unpacking hands one list item to each name, in order. If the counts do not match you get `ValueError: too many values to unpack` or `not enough values to unpack`. That error is useful: it tells you a data row has the wrong number of columns.

Connection to the real job

Every log parser, CSV reader and API client starts by splitting a line and naming the parts. Clear names let another engineer review your pipeline in minutes. Code reviews at most companies reject

names like `x`, `tmp2` and `data1`.

Applications

- **Data Analytics:** unpack columns from a row; name measures clearly in a report script.
- **Data Engineering:** parse log lines into named fields before loading them.
- **Machine Learning:** `X_train, X_test, y_train, y_test = split(...)` is unpacking in nearly every ML tutorial.
- **Retail operations:** `order_id, items, amount, paid = ...` for each line of an orders file.

COMMON MISTAKE

Using a built-in name as your own variable, such as `list = [1, 2]`, `str = "x"` or `max = 5`. Later `list(...)` stops working with `TypeError: 'list' object is not callable`. Use `items`, `text`, `max_value` instead.

Check yourself

1. What does `first, *rest = [7]` give for `rest`?

Answer: `[]`, an empty list.

1. Is `2nd_try = 5` a legal name?

Answer: No. A name cannot start with a digit.

1. What style does PEP 8 recommend for a function name?

Answer: `snake_case`, such as `parse_row`.

Chapter recap

- Python reads your source, compiles it to bytecode, and the virtual machine runs it. `.pyc` files in `__pycache__` are only a cache.
- Use the REPL to try ideas fast and scripts to make them repeatable. Never name your file like a standard library module.
- A variable is a name stuck to an object. Assignment never copies data.
- Types: `int` (unlimited size), `float` (binary, so never compare with `==`), `complex`, `bool`, `str` (text), `bytes` (raw data), `None` (nothing).
- Always say `encoding="utf-8"`. Text and bytes are different things.
- Immutable types (`int`, `str`, `tuple`, ...) never change. Mutable types (`list`, `dict`, `set`)

change in place, and every name pointing at them sees the change.

- Use `==` for values and `is` only for `None`, `True`, `False`.
- Copy with `.copy()` or slicing (shallow) or `copy.deepcopy` (deep). Functions receive references, so avoid changing arguments.
- Use clear `snake_case` names and unpacking.

Pattern	Example
Run a script	`python3 file.py`
Run one line	`python3 -c "print(1)"`
Float compare	`math.isclose(a, b)`
Text to number	`int("480")`, `float("2.5")`
Text to bytes and back	`s.encode("utf-8")`, `b.decode("utf-8")`
Shallow / deep copy	`x.copy()`, `copy.deepcopy(x)`
Swap	`a, b = b, a`
Collect rest	`first, *rest = items`

Practice set

Problem 1.P.1 (easy) What is the output of `print(type(5/1), 5//2, 5%2)`?

Answer: `<class 'float'> 2 1`. Division `/` always gives a float. Floor division gives 2, remainder 1.

Problem 1.P.2 (easy) What does `bool([0])` return?

Answer: `True`. The list has one element, so it is not empty. The element being `0` does not matter.

Problem 1.P.3 (easy) How many characters and how many UTF-8 bytes in `"₹5"`?

Answer: 2 characters. `₹` is 3 bytes and `5` is 1 byte, so 4 bytes.

Problem 1.P.4 (what is the output?)

```
a = [1, 2]
b = a
b = b + [3]
print(a, b)
```

Answer: `[1, 2] [1, 2, 3]`. Here `b = b + [3]` builds a *new* list and re-points `b`. Compare with `b += [3]` from Section 1.4, which changes the list in place. Same-looking code, different result.

Problem 1.P.5 (what is the output?) What does `print(round(0.5), round(1.5), round(2.5))` print?

Answer: `0 2 2`. Halves round to the nearest even number.

Problem 1.P.6 (find the bug)

```
amount = "480"
total = amount + 20
```

Answer: `TypeError: can only concatenate str (not "int") to str`. Convert first: `total = int(amount) + 20`, which is 500.

Problem 1.P.7 (find the bug) A function should return the 3 smallest items but the caller's list also ends up sorted:

```
def top(items):
    items.sort()
    return items[:3]
```

Answer: `items.sort()` changes the caller's list in place. Use `sorted(items)`, which returns a new list: `return sorted(items)[:3]`.

Problem 1.P.8 (computation) Make a 3x3 grid of zeros where changing `grid[0][0]` does not change other rows. Write the line and say why it works.

Answer: `grid = [[0]*3 for _ in range(3)]`. The comprehension runs the inner expression three times and creates three separate lists.

Problem 1.P.9 (scenario) Your function receives a config dict and must test a different timeout without changing the original. What do you do?

Answer: `import copy; test_cfg = copy.deepcopy(cfg); test_cfg["timeout"] = 5`. Use `deepcopy` if the dict contains nested dicts or lists, because a shallow `cfg.copy()` would share them.

Problem 1.P.10 (interview) "Is Python pass-by-value or pass-by-reference?" Answer in two sentences.

Answer: Neither exactly. Python passes object references by value ("call by object reference"): the function gets a name bound to the same object, so changes to a mutable object are visible to the caller, but re-assigning the parameter name inside the function does not affect the caller's name.

Problem 1.P.11 (interview) "Why does `0.1 + 0.2 != 0.3` in Python, and how do you compare floats safely?"

Answer: Floats are stored in binary and `0.1` and `0.2` cannot be represented exactly, so their sum is `0.30000000000000004`. Use `math.isclose(a, b)` (or `abs(a-b) < tolerance`), or use `decimal.Decimal` when exact decimal arithmetic matters, for example money.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional at an
Indian company moving into data and AI roles, and English is my second
language, so use simple words and short sentences. Teach me Python Chapter 1:
how Python runs code (REPL, scripts, bytecode), the basic types (int, float,
bool, str, bytes, None), and why float comparison is tricky. Explain step by
step with small runnable examples, then ask me three questions one at a time
and check my answers before moving on.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional at an
Indian company moving into data and AI roles, with English as a second
language. Topic: Python mutability, identity (`is` vs `==`), references,
shallow vs deep copy. Give me 8 graded exercises from easy to hard, each asking
me to predict the output of a short snippet. Do not give solutions yet. After I
answer, I will paste my answers and you will correct them and explain any wrong
one with a picture made of text.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data and AI roles, and English is my second language. Quiz me on Python
basics: types, truthiness, integer vs float division, string vs bytes, UTF-8,
unpacking, and naming. Ask exactly one question at a time, wait for my answer,
say whether it is right, explain the reason in simple words, then ask the next
question. Make the questions harder when I get three right in a row.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles, with English as a second language. Review my Python code for beginner problems: wrong types, confusing names, mutable-argument bugs, unsafe copies, float comparisons, and missing encoding. Explain each problem in simple words, show the fixed code, and tell me which rule I broke. Here is my code: [PASTE HERE]
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data and AI roles, with English as a second language. Run a mock Python interview for a junior data role, focused on the basics from Chapter 1: mutable vs immutable, `is` vs `==`, copy vs deepcopy, call by object reference, float precision, str vs bytes. Ask one question at a time, give me a score from 1 to 5 for each answer, and show a model answer after each. At the end give me a list of my weak points.
```

CHAPTER 2

CONTROL FLOW, FUNCTIONS, SCOPE AND RECURSION

Make your program decide, repeat, and reuse its own logic.

A delivery app does not just handle one order. It loops over many orders, checks conditions, branches when a payment fails, and reuses the same "pack the parcel" step again and again. Python programs work the same way. In this chapter you learn the building blocks that do this: **decisions** (`if` , `match`), **repetition** (`for` , `while`), and **reusable steps** (functions). You also learn the rules for where a name is visible (**scope**) and how a function can call itself (**recursion**). By the end you will write small, correct functions that process a list of results, and you will know the five classic traps: the mutable default, the late-binding closure, `UnboundLocalError` , modifying a list while looping, and runaway recursion.

Running example. A list of five online-shop orders. Each is a tuple `(item, status_code, delivery_minutes)` , where the status code is what the shop's order system reported:

```
samples = [("book",200,480),("lamp",200,1250),("phone",500,3100),
("book",200,310),("phone",503,60)]
```

A **tuple** is a fixed group of values in round brackets (Chapter 4 explains it fully). Here, think of each one as a row of an orders file.

2.1 Control Flow: Decisions and Loops

INTUITION

Think of a traffic police officer at a crossing. For each car he asks a question ("is it an ambulance?") and sends it left or right. That is `if` . A roundabout where cars go around again and again until they find their exit is a loop. **Control flow** means "the order in which lines run, and the questions that change that order".

Words you need

Word	Meaning
Condition	an expression that is true or false, such as `code < 300`

Block	the indented lines under a `:`. Python uses indentation (4 spaces) to group code
Iterable	something you can loop over: a list, a string, a file, `range(...)`
Iteration	one trip around a loop

How it works

```
if condition:        # runs if true
    ...
elif other:          # tried only if the first was false
    ...
else:                # runs if all were false
    ...
for item in iterable:    # one trip per item
    ...
while condition:         # repeat while true
    ...
```

- `break` leaves the loop at once. `continue` skips to the next iteration.
- A loop can have an `else:` block. It runs only if the loop **ended without** `break`. The name is confusing; read it as "no break happened".
- `range(start, stop, step)` gives integers from `start` up to but **not including** `stop`.
- `enumerate(items, start=1)` gives `(index, item)` pairs. `zip(a, b)` walks two iterables together and stops at the shorter one.
- `match` (Python 3.10 and newer) is **structural pattern matching**: it looks at the *shape* of a value, not just equality.
- The **walrus operator** `:=` assigns and returns a value inside an expression. Use it sparingly.
- A **ternary expression** `A if cond else B` picks one of two values.

Worked example 2.1.1: classify and loop

Predict: The status codes 500 and 503 are server errors in the order system. Which rows will print `server error`?

```
def classify(code):
    if code < 300: return "ok"
    elif code < 500: return "client error"
    else: return "server error"
for item, code, mins in samples:
    print(f"{item:<7}{code} {mins:>5} min  {classify(code)}")
for i in range(0, 10, 3): print(i, end=" ")
print()
for i, (item, _, _) in enumerate(samples, start=1): print(i, item, end="; ")
print()
print(list(zip([1,2,3],"abc")), list(zip([1,2,3],"ab")))
```

Real output:

```
book   200   480 min  ok
lamp   200  1250 min  ok
phone  500  3100 min  server error
book   200   310 min  ok
phone  503    60 min  server error
0 3 6 9
1 book; 2 lamp; 3 phone; 4 book; 5 phone;
[(1, 'a'), (2, 'b'), (3, 'c')] [(1, 'a'), (2, 'b')]
```

Line by line:

- `for item, code, mins in samples` unpacks each tuple (Chapter 1, Section 1.4) into three names.
- The f-string `{item:<7}` means "left-align in 7 characters", and `{mins:>5}` means "right-align in 5". Chapter 3 explains f-strings.
- `range(0, 10, 3)` gives 0, 3, 6, 9. It stops before 10.
- `_` is the usual name for a value you do not care about.
- The last `zip` call stopped after two pairs because `"ab"` has only two letters.

Worked example 2.1.2: `break`, loop `else`, `while`, `continue`

Predict: Our courier has a budget of 3000 delivery minutes in total. At which order does the running total pass it?

```
budget = 3000; spent = 0
for item, code, mins in samples:
    spent += mins
    if spent > budget:
        print("over budget at", item); break
else:
    print("within budget")
attempts = 0
while True:
    attempts += 1
    if attempts % 2: continue
    if attempts >= 6: break
print("attempts", attempts)
```

Real output:

```
over budget at phone
attempts 6
```

- Running total: 480, then 1730 (480+1250), then 4830 (1730+3100). 4830 is above 3000 at `phone`, so `break` fires and the `else` block is skipped.
- In the `while True` loop odd numbers hit `continue` and go back to the top. Even numbers reach the `break` test. The loop ends at 6.
- `while True` with a `break` inside is a common pattern for "retry until success".

Worked example 2.1.3: `match`, walrus, ternary

Predict: What does `route(5)` return?

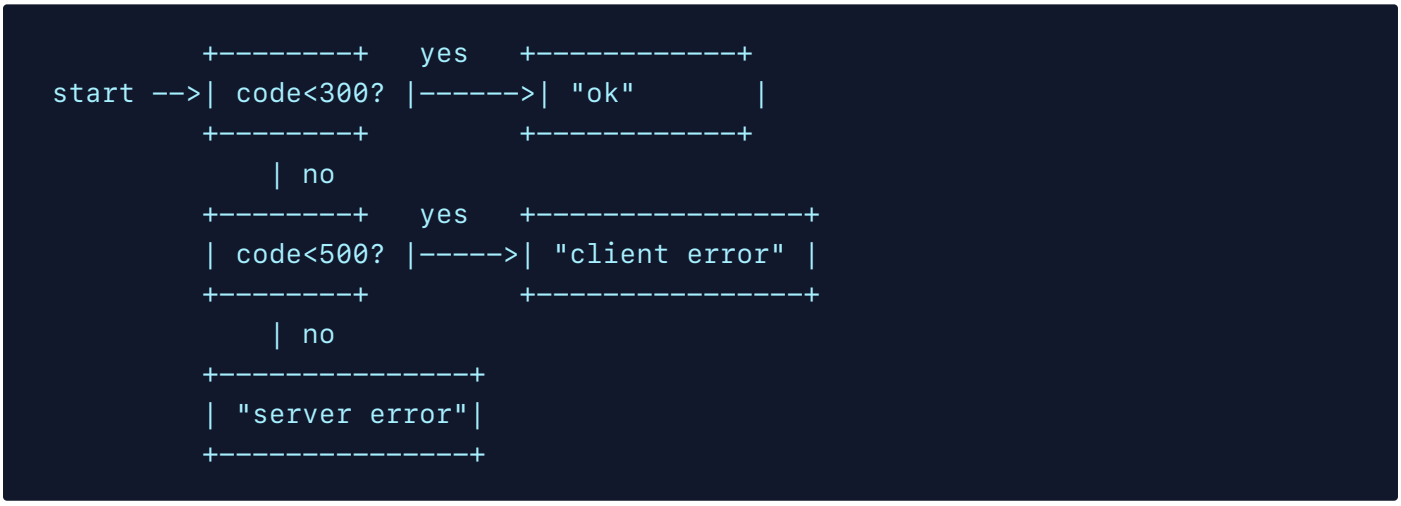
```
def route(req):
    match req:
        case {"method": "GET", "path": "/health"}: return "health"
        case {"method": "POST", "path": p} if p.startswith("/pay"): return
f"pay:{p}"
        case {"method": m}: return f"other {m}"
        case _: return "bad"
print(route({"method":"GET","path":"/health"}),
route({"method":"POST","path":"/pay/1"}), route({"method":"PUT"}), route(5))
def kind(x):
    match x:
        case 0: return "zero"
        case int() | float(): return "number"
        case [a, b]: return f"pair {a},{b}"
        case [first, *rest]: return f"list starting {first} (+{len(rest)})"
        case str(): return "text"
        case _: return "unknown"
print([kind(v) for v in (0, 2.5, [1,2], [1,2,3,4], "hi", None)])
data = [480, 1250, 3100]
if (n := len(data)) > 2: print("many samples:", n)
import re
line = "ERROR code=503"
if m := re.search(r"code=(\d+)", line): print(int(m.group(1)))
x = 500
print("slow" if x > 400 else "fast", 200 <= x < 600)
```

Real output:

```
health pay:/pay/1 other PUT bad
['zero', 'number', 'pair 1,2', 'list starting 1 (+3)', 'text', 'unknown']
many samples: 3
503
slow True
```

- `match` tries each `case` from the top. A `case` can match a dictionary shape (`{"method": m}` captures the value into `m`), a type (`int()`), a list shape (`[a, b]`, `[first, *rest]`), and can have an `if` guard. `case _` is the "anything else" default.
- Order matters: `case 0` is before `case int()`, so `0` is "zero", not "number". `route(5)` is `"bad"` because `5` is not a dict.
- `(n := len(data)) > 2` stores the length in `n` and tests it in one line. The regex example (Chapter 3) uses `:=` to test "did the search find something?" and name the result at once.
- Python allows **chained comparison**: `200 <= x < 600` means `200 <= x and x < 600`.

The picture



Each diamond is an `if/elif`. Only the first yes-branch runs. Lines of code inside a branch are indented under it.

Real-world mapping

Concept	Example	Where it appears in the job
`if/elif/else`	classify a status code	Data validation, alert rules
`for`	walk every sample	Processing rows of a file
`while` + `break`	retry an API until it answers	Robust pipelines
`for ... else`	"search; if not found, do X"	Lookup logic
`match`	route by request shape	Parsers, event handlers

Applications

- **Data Analytics:** label each row (`"fast"`, `"slow"`) before grouping.
- **Data Engineering:** loop over files in a folder; retry loop for flaky network calls.
- **Machine Learning:** training loops (`for epoch in range(10)`), stop early with `break` when loss stops improving.
- **GenAI:** route a chatbot message by shape (`match` on a parsed JSON event).

COMMON MISTAKE

Changing a list while looping over it. Removing items shifts the later ones and some are skipped. Loop over a copy (`for it in items[:]`) or build a new list with a comprehension (Chapter 4). Another one: `=` (assignment) instead of `==` (comparison) in a condition. Python reports a `SyntaxError` , which is a kindness.

PROFESSOR'S NOTE

Names like `i` leak out of a loop. After `for i in range(3): pass` , the name `i` still exists and equals 2. This is legal, but do not depend on it.

Check yourself

1. What does `list(range(2, 8, 2))` give?

Answer: `[2, 4, 6]` . It stops before 8.

1. When does a loop's `else` block run?

Answer: Only when the loop finished without hitting `break` .

1. Why must `case 0` be placed before `case int()` ?

Answer: `match` takes the first case that fits. `0` is also an `int` , so a later `case 0` would never be reached.

2.2 Functions: Arguments, Defaults and Lambdas

INTUITION

A function is a recipe card. You write the steps once, give it a name, and then you can say "make this recipe with these ingredients" as many times as you like. A restaurant kitchen does the same: one recipe for tomato sauce is reused in the pizza, the pasta and the soup.

Formal definition

A **function** is a named, reusable block of code. You define it with `def` , and you **call** it with brackets.

- **Parameter**: the name in the definition (`values`). **Argument**: the real value you pass in the call (`[1,2,3]`).
- **Positional arguments** are matched by order. **Keyword arguments** are matched by name

(`ndigits=1`).

- **Default value**: `def mean(values, ndigits=2)` . The caller may leave it out.
- `*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dict. At a call site `*list` and `**dict` do the reverse: they spread values out.
- A `/` in the parameter list means "everything before me is positional-only". A `*` means "everything after me is keyword-only".
- A function without `return` returns `None` .
- A **docstring** (a string as the first line) documents the function. **Type hints** (`old: float`) are labels for humans and tools. Python does not enforce them at run time (Chapter 9).
- **Functions are objects.** You can pass them to other functions. A **lambda** is a tiny unnamed function with one expression: `lambda p: p[1]` .

Worked example 2.2.1: arguments in every style

Predict: What happens with `mean(ndigits=1, values=[1,2])` ? And with `f(1,2,3,4)` ?

```
def mean(values, ndigits=2):
    return round(sum(values)/len(values), ndigits)
print(mean([480,1250,3100]), mean([480,1250,3100], 0), mean(ndigits=1, values=
[1,2]))
def f(a, b, /, c, *, d): return (a,b,c,d)
print(f(1,2,3,d=4), f(1,2,c=3,d=4))
for bad in ("f(1,2,3,4)", "f(a=1,b=2,c=3,d=4)", "mean()"):
    try: eval(bad)
    except TypeError as e: print("TypeError:", e)
def stats(*args, **kwargs):
    return args, kwargs
print(stats(1, 2, unit="ms", n=3))
nums = [3, 1, 2]; opts = {"ndigits": 1}
print(mean(nums, **opts), max(*nums))
```

Real output:

```
1610.0 1610.0 1.5
(1, 2, 3, 4) (1, 2, 3, 4)
TypeError: f() takes 3 positional arguments but 4 were given
TypeError: f() got some positional-only arguments passed as keyword arguments:
'a, b'
TypeError: mean() missing 1 required positional argument: 'values'
((1, 2), {'unit': 'ms', 'n': 3})
2.0 3
```

- The mean of 480, 1250, 3100 is 4830 / 3 = 1610. With `ndigits=0` , `round` still returns a float

(`1610.0`).

- In `f` , `a` and `b` can only be given by position, and `d` can only be given by name. The error messages tell you exactly which rule you broke.
- `mean(nums, **opts)` spreads the dict into `ndigits=1` . `max(*nums)` spreads the list into `max(3, 1, 2)` .

Worked example 2.2.2: the mutable default trap

Predict: What do three calls to `add_sample` print?

```
def add_sample(x, bucket=[]):
    bucket.append(x); return bucket
print(add_sample(1), add_sample(2), add_sample(3))
def add_sample2(x, bucket=None):
    if bucket is None: bucket = []
    bucket.append(x); return bucket
print(add_sample2(1), add_sample2(2))
print(add_sample.__defaults__)
```

Real output:

```
[1, 2, 3] [1, 2, 3] [1, 2, 3]
[1] [2]
([1, 2, 3],)
```

Why: the default value is built **once**, when `def` runs, not each time you call. So all calls share one list (Chapter 1: mutable objects shared by reference). All three prints show the same list, which ended as `[1, 2, 3]` . The fix is to use `None` as the default and build a fresh list inside. The last line proves the list lives in `add_sample.__defaults__` .

Worked example 2.2.3: functions as values, lambda, sorting

```
pairs = [("book", 480), ("phone", 3100), ("lamp", 1250)]
print(sorted(pairs, key=lambda p: p[1], reverse=True))
print(list(map(lambda p: p[1]/1000, pairs)), list(filter(lambda p: p[1] > 1000,
pairs)))
def minmax(v): return min(v), max(v)
lo, hi = minmax([5,2,9]); print(lo, hi)
def nothing(): pass
print(nothing())
def pct(old: float, new: float) -> float:
    """Percent improvement from old to new."""
    return (old - new) / old * 100
print(pct(480, 360), pct.__doc__)
print(pct.__annotations__)
```

Real output:

```
[('phone', 3100), ('lamp', 1250), ('book', 480)]
[0.48, 3.1, 1.25] [('phone', 3100), ('lamp', 1250)]
2 9
None
25.0 Percent improvement from old to new.
{'old': <class 'float'>, 'new': <class 'float'>, 'return': <class 'float'>}
```

- `key=` tells `sorted` what to compare. `lambda p: p[1]` means "take the second item of each pair" (the minutes).
- `map` applies a function to each item, `filter` keeps items for which the function is true. Both return lazy iterators, so we wrap them in `list(...)` (Chapter 7).
- A function can "return two values": it really returns one tuple, which the caller unpacks.
- `pct(480, 360)` is `(480-360)/480*100 = 25.0`, the same improvement formula we used in the style guide. Check: `480 * 0.75 = 360`.

Problem 2.2.1 Write a function `slow_share(samples, limit)` that returns the percentage of samples whose delivery time is above `limit`. Use our five samples and limit 1000.

Step 1	Count samples above the limit: 1250 and 3100 are above 1000. The others (480, 310, 60) are not. Count = 2.
Step 2	Divide by the total number of samples: 2 / 5 = 0.4.
Step 3	Multiply by 100: 40.0.
Answer	40% of the samples were slower than 1000 minutes (a late delivery in this example).
Check	Code below prints 40.0.
Why it matters	"Share of orders delivered later than the promised time" is a standard service KPI for any shop.

```
def slow_share(samples, limit):
    slow = [mins for _, _, mins in samples if mins > limit]
    return len(slow) / len(samples) * 100
print(slow_share(samples, 1000))
```

```
40.0
```

The picture

```
def mean(values, ndigits=2):        call: mean([1,2], ndigits=1)
        ^        ^                           ^           ^
   parameter  parameter+default       argument       argument by name

 caller ----- arguments ----> [ function body ] ----- return value ----> caller
```

Real-world mapping

Concept	Example	Where it appears in the job
Default argument	`read_csv(path, sep=",")`	Library functions you will call daily
`**kwargs`	pass options through to another function	Wrapper functions, decorators (Chapter 7)
`lambda` as `key=`	sort rows by one column	Quick data ordering
Keyword-only	`train(model, *, epochs=10)`	APIs that must stay readable

Applications

- **Data Analytics:** helper functions like `clean_amount(text)` used on every row.
- **Data Engineering:** one function per pipeline step so each can be tested.
- **Machine Learning:** `train(model, data, *, lr=0.01, epochs=10)` with keyword-only hyper-parameters.
- **Operations:** a `late_share(orders, limit)` helper reused across every weekly delivery report.

COMMON MISTAKE

Mutable default arguments (`def f(x, bucket=[])`). Never use `[]` , `{}` or `set()` as a default. Use `None` and create the object inside. Another: forgetting `return` and then wondering why you got `None` .

Check yourself

1. What are `args` and `kwargs` inside `def g(*args, **kwargs)` when called as `g(1, a=2)` ?

Answer: `args = (1,)` and `kwargs = {"a": 2}` .

1. What does a function return if it has no `return` ?

Answer: `None` .

1. What is wrong with `def f(items=[]):` ?

Answer: The default list is created once and shared by all calls, so data leaks between calls.

2.3 Scope, Closures and `global` / `nonlocal`

INTUITION

Think of rooms in a house. You look for your keys in your own room first, then in the house, then in the street. You never look in your neighbour's house. **Scope** is the set of "rooms" where Python looks for a name, and it always looks inner room first.

The LEGB rule

When Python sees a name it searches four places in order:

1. **L**ocal: inside the current function.

2. **E**nclosing: inside any function that wraps this one.
3. **G**lobal: at the top level of the file (module).
4. **B**uilt-in: names Python gives you, like `len` and `print`.

The first match wins. **Assigning** to a name inside a function makes it local for the *whole* function. To change an outer name you must declare `global name` (module level) or `nonlocal name` (enclosing function).

A **closure** is an inner function that remembers variables from the function that created it, even after the outer function has finished. Think of it as a function that carries a small backpack.

Worked example 2.3.1: local, global, UnboundLocalError

Predict: `broken()` reads `counter` and then assigns it. Does it print the global value?

```
x = "global"
def show():
    x = "local"
    return x
print(show(), x)
def broken():
    try:
        print(counter)
        counter = 1
    except UnboundLocalError as e: print("UnboundLocalError:", e)
broken()
counter = 0
def inc():
    global counter
    counter += 1
inc(); inc(); print(counter)
```

Real output:

```
local global
UnboundLocalError: cannot access local variable 'counter' where it is not
associated with a value
2
```

- `show` creates its own local `x`; the global `x` is untouched.
- In `broken`, the later line `counter = 1` makes `counter` local for the entire function. The `print(counter)` above it then reads a local name that has no value yet. That is the error. Python decides locality when it compiles the function, not line by line.
- `inc` says `global counter`, so `counter += 1` changes the module-level variable. Use

`global` rarely. A function that returns a value is easier to test.

Worked example 2.3.2: closures with `nonlocal`, and the LEGB order

```
def make_counter():
    count = 0
    def step():
        nonlocal count
        count += 1
        return count
    return step
c1 = make_counter(); c2 = make_counter()
print(c1(), c1(), c1(), c2())
print(c1.__closure__[0].cell_contents)
v = "G"
def outer():
    v = "E"
    def inner():
        v = "L"
        return v
    return inner(), v
print(outer(), v)
def make_limit(limit):
    def check(mins): return mins <= limit
    return check
within = make_limit(2000)
print(within(1250), within(3100))
```

Real output:

```
1 2 3 1
3
('L', 'E') G
True False
```

- `c1` and `c2` each have their own `count`. Calling `c1` three times gives 1, 2, 3. `c2` starts from 1. Each closure owns a separate backpack.
- `c1.__closure__[0].cell_contents` shows the backpack: the current count, 3.
- `outer()` returned `('L', 'E')` and the global stayed `'G'`: each level has its own `v`.
- `make_limit(2000)` builds a checker that remembers `limit = 2000`. This is the same pattern decorators use in Chapter 7.

Worked example 2.3.3: the late-binding trap and `partial`

Predict: What will `[f() for f in fns]` print for `lambda: i` created in a loop?

```
fns = [lambda: i for i in range(3)]
print([f() for f in fns])
fns = [lambda i=i: i for i in range(3)]
print([f() for f in fns])
from functools import partial
def power(base, exp): return base**exp
square = partial(power, exp=2)
print(square(7))
```

Real output:

```
[2, 2, 2]
[0, 1, 2]
49
```

A closure looks up `i` **when it is called**, not when it was created. By then the loop is over and `i` is 2, so all three give 2. Fix: bind the current value as a default (`i=i`), or use `functools.partial` , which stores the argument right now. `partial(power, exp=2)` is a new function with `exp` already filled in.

The picture

Nested rooms: Local inside Enclosing inside Global inside Built-in

Read the picture from the inside out. The smallest green room is the function you are in. Python searches that room first, then the next larger room, and so on out to the grey room of built-in names. The first room that contains the name wins. The text version shows the same order.

```
 built-in   [ len, print, ... ]
 global     [ x, counter, show ... ]
 enclosing      [ count ]          <- make_counter's local, kept alive
 local              [ step's own names ]

 lookup order:  L -> E -> G -> B   (inner room first)
```

Real-world mapping

Concept	Example	Where it appears in the job

Closure	`make_limit(2000)`	Configurable checks, decorators with arguments
`global`	a module-level `counter`	Quick scripts (avoid in real code)
`partial`	`partial(parse, sep=";")`	Passing a pre-configured function to `map` or a thread pool
LEGB	a local `max` hides the built-in	Debugging "object is not callable"

Applications

- **Data Engineering:** factory functions that return a parser configured for one file format.
- **Machine Learning:** `partial(train_step, lr=0.01)` passed to a runner.
- **Web/API work:** callbacks and decorators remember their settings through closures.

COMMON MISTAKE

Creating lambdas in a loop and expecting each to remember its own `i` (see 2.3.3). Also: naming a variable `list`, `max` or `sum` at module level, which hides the built-in everywhere below it.

Check yourself

1. What does LEGB stand for?

Answer: Local, Enclosing, Global, Built-in, the order Python searches names.

1. Which keyword lets an inner function change a variable of its outer function?

Answer: `nonlocal`.

1. Why does reading a variable before assigning it in the same function give `UnboundLocalError`?

Answer: The assignment makes the name local for the whole function, so the early read finds a local with no value.

2.4 Recursion

INTUITION

Russian nesting dolls. To count the dolls, you open the biggest one and say "1 plus the count of the dolls inside". You keep opening until you reach the smallest doll that holds nothing. A **recursive** function solves a problem by calling itself on a smaller version of the same problem.

Formal definition

A recursive function needs two parts:

1. **Base case**: the smallest problem, answered directly, with no further call.
2. **Recursive case**: reduce the problem, call yourself, and combine the result.

Without a base case, or if the problem does not get smaller, Python stops with `RecursionError` after about 1000 nested calls (`sys.getrecursionlimit()`). Each call uses memory on the **call stack**, the list of calls still waiting for an answer.

Worked example 2.4.1: factorial and its call trace

Predict: `fact(3)` is `3 * 2 * 1` . In what order do the "->" lines print?

```
def factorial(n):
    if n <= 1: return 1       # base case
    return n * factorial(n-1) # recursive case
print(factorial(5), factorial(20))
def trace(n, depth=0):
    print("  "*depth + f"fact({n})")
    if n <= 1:
        print("  "*depth + "-> 1"); return 1
    r = n * trace(n-1, depth+1)
    print("  "*depth + f"-> {r}"); return r
trace(3)
```

Real output:

```
120 2432902008176640000
fact(3)
  fact(2)
    fact(1)
    -> 1
  -> 2
-> 6
```

The calls go **down** (3, 2, 1), then the answers come **up** (1, 2, 6). Hand check: `5! = 5*4*3*2*1 = 120` . The result for 3 is `3 * 2 = 6` : it waited for `fact(2)` , which waited for `fact(1)` .

Worked example 2.4.2: runaway recursion, and fibonacci three ways

Predict: How many function calls does `fib_counted(20)` make? (Guess before you look.)

```
import sys
def bad(n): return n * bad(n-1)
try: bad(5)
except RecursionError as e: print("RecursionError:", str(e)[:45])
print(sys.getrecursionlimit())
calls = 0
def fib_counted(n):
    global calls; calls += 1
    return n if n < 2 else fib_counted(n-1) + fib_counted(n-2)
print(fib_counted(20), calls)
from functools import lru_cache
calls2 = 0
@lru_cache(maxsize=None)
def fib_memo(n):
    global calls2; calls2 += 1
    return n if n < 2 else fib_memo(n-1) + fib_memo(n-2)
print(fib_memo(20), calls2, fib_memo(90))
def fib_iter(n):
    a, b = 0, 1
    for _ in range(n): a, b = b, a+b
    return a
print(fib_iter(20), fib_iter(90))
```

Real output:

```
RecursionError: maximum recursion depth exceeded
1000
6765 21891
6765 21 2880067194370816120
6765 2880067194370816120
```

- `bad` never stops (no base case), so Python raises `RecursionError`.
- Plain recursive Fibonacci recomputes the same values again and again: **21,891 calls** just for `fib(20)`. The cost roughly doubles for each extra step.
- `@lru_cache` (a decorator, Chapter 7) remembers earlier answers. **Memoization** means "store a result the first time and reuse it". Only 21 calls were needed (for n = 0 to 20). So `fib_memo(90)` is instant.
- A loop (`fib_iter`) gives the same answers with no recursion and no stack. When a loop is easy to write, prefer the loop.

Worked example 2.4.3: recursion on nested data

```
def flatten(x):
    out = []
    for item in x:
        if isinstance(item, list): out.extend(flatten(item))
        else: out.append(item)
    return out
print(flatten([1,[2,[3,4]],5,[[6]]]))
def bsearch(a, t, lo=0, hi=None):
    if hi is None: hi = len(a)-1
    if lo > hi: return -1
    mid = (lo+hi)//2
    if a[mid] == t: return mid
    return bsearch(a, t, mid+1, hi) if a[mid] < t else bsearch(a, t, lo, mid-1)
arr = [2,5,8,12,16,23,38,56]
print(bsearch(arr, 23), bsearch(arr, 7))
def depth_sum(tree):
    return tree["mins"] + sum(depth_sum(c) for c in tree.get("steps", []))
recipe_tree = {"mins":50,"steps":[{"mins":120,"steps":[{"mins":30}]},
{"mins":200}]}
print(depth_sum(recipe_tree))
```

Real output:

```
[1, 2, 3, 4, 5, 6]
5 -1
400
```

- `flatten` is natural as recursion because the data is nested to an unknown depth.
- `bsearch` (binary search) halves the search range each call. It found `23` at index 5, and `7` is not there (`-1`).
- `depth_sum` adds up a recipe tree. A **recipe tree** is a dish made of steps, where some steps are themselves made of smaller steps, each with a preparation time. 50 + 120 + 30 + 200 = 400 minutes. Trees and folders are the classic use of recursion.

Problem 2.4.1 Count how many calls `factorial(5)` makes, and the deepest call depth.

Step 1	`factorial(5)` calls `factorial(4)`, which calls 3, then 2, then 1.
Step 2	That is calls for n = 5, 4, 3, 2, 1: five calls. The base case stops at n = 1.
Step 3	The call stack holds all five at the moment `factorial(1)` runs, so depth is 5.
Answer	5 calls, depth 5. The work grows in a straight line with n.
Check	Compare to `fib(20)` above: 21,891 calls for n = 20. Straight-line recursion is cheap; branching recursion with repeated work is not.
Why it matters	Count the calls before you trust a recursive solution on large input.

The picture

```
fib(4)
 +-- fib(3)
 |    +-- fib(2)
 |    |    +-- fib(1)
 |    |    +-- fib(0)
 |    +-- fib(1)
 +-- fib(2)         <- fib(2) is computed AGAIN (wasted work)
      +-- fib(1)
      +-- fib(0)
```

Each node is one call. The repeated `fib(2)` is why memoization helps: it remembers `fib(2)` after the first time.

Real-world mapping

Concept	Example	Where it appears in the job
Recursion on trees	folder walk, JSON of unknown depth	Parsing nested API data
Memoization	`lru_cache` on a slow lookup	Caching repeated calls
Divide and conquer	binary search	Interview problems, sorted data
Recursion limit	`RecursionError` on deep data	Choosing a loop or an explicit stack

Applications

- **Data Engineering:** walking nested JSON and directory trees.

- **Machine Learning:** decision trees are built and evaluated recursively.
- **Data Science:** dynamic programming (cached recursion) for optimisation problems.
- **Interview prep:** trees, graphs, backtracking.

COMMON MISTAKE

No base case, or a base case that is never reached (`factorial(-1)` with `if n == 0` loops forever). Also, recursing on huge input: Python does not optimise tail calls, so deep recursion fails around 1000 levels. Convert to a loop.

Check yourself

1. What are the two parts of every recursive function?

Answer: A base case and a recursive case that makes the problem smaller.

1. Why is plain recursive `fib(40)` slow?

Answer: It recomputes the same sub-problems an enormous number of times.

1. What does `@lru_cache` do?

Answer: It stores results of earlier calls (memoization) so repeated calls return instantly.

Chapter recap

- `if/elif/else` decide, `for` and `while` repeat, `break` / `continue` change the loop, and a loop `else` runs only when no `break` happened.
- `match` checks the shape of data. Put specific cases before general ones, and end with `case _` .
- Functions take parameters (definition) and arguments (call). Use keyword arguments for clarity, `*args` and `**kwargs` to pass many.
- Never use a mutable default value. Use `None` and create inside.
- Python looks up names in the order Local, Enclosing, Global, Built-in.
- Assigning in a function makes the name local. Use `nonlocal` or `global` only when you must.
- A closure remembers variables from where it was created. In loops, bind values early (`i=i`) or use `partial` .
- Recursion needs a base case and shrinking input. Use `lru_cache` for repeated sub-problems, or a loop when it is simple.

Pattern	Example

Loop with index	`for i, x in enumerate(items, start=1):`
Walk two lists	`for a, b in zip(xs, ys):`
Loop `else`	`for ...: ... break` / `else: not found`
Pattern match	`match v: case [a, b]: ...`
Safe default	`def f(x, bucket=None): bucket = bucket or []` (use `is None` when `[]` is valid input)
Sort by field	`sorted(rows, key=lambda r: r[1])`
Fix an argument	`partial(power, exp=2)`
Cache a function	`@lru_cache(maxsize=None)`

Practice set

Problem 2.P.1 (easy) What does this print?

```
for i in range(3):
    if i == 1: continue
    print(i)
```

Answer: `0` then `2`. When `i` is 1, `continue` skips the print.

Problem 2.P.2 (easy) What is `list(zip("ab", [10, 20, 30]))`?

Answer: `[('a', 10), ('b', 20)]`. `zip` stops at the shortest input.

Problem 2.P.3 (what is the output?)

```
def f(x, y=[]):
    y.append(x)
    return len(y)
print(f(1), f(2))
```

Answer: `1 2`. The default list is shared, so the second call sees two items.

Problem 2.P.4 (what is the output?)

```
x = 5
def g():
    x = x + 1
    return x
print(g())
```

Answer: It raises `UnboundLocalError`. The assignment makes `x` local, so `x + 1` reads a local `x` that has no value. Fix: pass `x` as a parameter, or declare `global x` (not preferred).

Problem 2.P.5 (computation) Use `match` to return `"empty"` for `[]`, `"one"` for a one-item list and `"many"` otherwise.

Answer:

```
def size(x):
    match x:
        case []: return "empty"
        case [_]: return "one"
        case _: return "many"
```

`size([])` is `"empty"`, `size([7])` is `"one"`, `size([1,2])` is `"many"`.

Problem 2.P.6 (computation) How many times is the base case reached when computing `fib(4)` recursively? (Base cases are n = 0 and n = 1.)

Answer: 5. The leaves of the tree above are `fib(1)`, `fib(0)`, `fib(1)`, `fib(1)`, `fib(0)`. That is three `fib(1)` and two `fib(0)`, so 5 leaves.

Problem 2.P.7 (find the bug)

```
fns = []
for n in [10, 20, 30]:
    fns.append(lambda: n * 2)
print([f() for f in fns])
```

Answer: It prints `[60, 60, 60]` because every lambda reads `n` when called, after the loop ended with `n = 30`. Fix: `lambda n=n: n * 2` gives `[20, 40, 60]`.

Problem 2.P.8 (find the bug)

```
def countdown(n):
    print(n)
    countdown(n - 1)
```

Answer: There is no base case, so it recurses until `RecursionError`. Add `if n < 0: return` at the top.

Problem 2.P.9 (scenario) You receive 10,000 delivery times (in minutes) and must report how many are above 2000. A teammate writes a recursive function that processes one item per call. What is wrong and what do you do?

Answer: Recursion depth would reach 10,000, far above the default limit of 1000, so it raises `RecursionError`. Use a loop or a comprehension: `sum(1 for mins in times if mins > 2000)`.

Problem 2.P.10 (interview) "What is the difference between `*args` and `**kwargs`?"

Answer: `*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dict. They let a function accept any number of inputs, and are used to forward arguments to another function.

Problem 2.P.11 (interview) "Explain Python's scoping rule and when you need `nonlocal`."

Answer: Python uses LEGB: Local, Enclosing, Global, Built-in. Assignment inside a function creates a local name. If an inner function must rebind a variable owned by an enclosing function, it must declare it `nonlocal`, otherwise Python creates a new local variable instead.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional at an
Indian company moving into data and AI roles, and English is my second
language, so use simple words. Teach me Python control flow: if/elif/else, for,
while, break, continue, loop else, enumerate, zip, and match statements. Use
one running example of online-shop delivery orders. Explain step by step with
short runnable code, ask me to predict each output first, and check my
understanding with three questions at the end.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles, with English as a second language. Topic: Python function
```

arguments (positional, keyword, defaults, *args, **kwargs, keyword-only, positional-only) and the mutable default argument bug. Give me 8 graded exercises from easy to hard with no solutions. When I paste my answers, mark each one and explain every mistake in simple words with a small diagram.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Explain Python scope (LEGB), closures, global and nonlocal with an analogy and runnable examples. Then quiz me one question at a time, including the late-binding lambda trap and UnboundLocalError. Wait for my answer before giving the next question, and explain the reason for each correct answer.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data and AI roles, with English as a second language. Teach recursion from zero: base case, recursive case, call stack, RecursionError, memoization with lru_cache, and when to use a loop instead. Walk through fib(5) by drawing the call tree in text. Then give me 5 recursion exercises on nested lists, trees, and binary search, with hints but no full solutions. Here is my attempt: [PASTE HERE]

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Run a mock Python interview on Chapter 2 topics: default arguments, *args and **kwargs, closures, scope, decorators-lite using closures, recursion limits, and loop else. Ask one question at a time, give a score from 1 to 5, show a model answer, and finish with a short list of topics I should revise.

CHAPTER 3

STRINGS, TEXT PROCESSING AND REGULAR EXPRESSIONS

Turn messy text into clean, usable fields.

Most real data starts as text: log lines, CSV rows, JSON from an API, user comments, file names. An online shop, for example, keeps an access log with one line for every visit to its website. In this chapter you learn how Python stores text, the 20 string methods that solve 90% of cleaning jobs, the f-string mini-language for printing neat reports, and **regular expressions** (regex), a compact language for describing text patterns. By the end you will be able to take one raw web-server log line and pull out the method, path, status code and processing time as proper typed values.

Running example. One line from a web-server access log. It is one visit to a shop website. It tells us who asked for what and how long the server took to answer:

```
10.0.0.5 - - [07/Nov/2026:10:15:32 +0530] "GET /api/orders/42 HTTP/1.1" 200 512 0.480
```

Reading left to right: client IP, two unused dashes, timestamp, the request in quotes (method, path, protocol), status code `200`, bytes sent `512`, and time taken `0.480` seconds.

3.1 Strings: Slicing and the Everyday Methods

INTUITION

A string is a necklace of beads, where each bead is one character and each bead has a number: 0 for the first, 1 for the second, and so on. Slicing means "give me the beads from position A to position B". You cannot change a bead in place. To change the necklace you make a new one.

Formal definition

- A `str` is an immutable sequence of Unicode characters (Chapter 1). Indexing starts at 0. Negative indexes count from the end: `-1` is the last character.
- **Slice** `s[start:stop:step]` takes characters from `start` up to but not including `stop`. Any part may be left out. `s[::-1]` reverses.
- Methods never change the original string. They return a new one.
- **Raw string** `r"..."`: backslashes are kept as they are (`r"\t"` is two characters, not a tab). You

need these for regex and Windows paths.

- **Escape sequences**: `\n` new line, `\t` tab, `\\` backslash, `\"` quote.

The methods you will use every day

Task	Method	Example result
remove spaces at both ends	`s.strip()` (also `lstrip`, `rstrip`)	`" hi "` becomes `"hi"`
cut into a list	`s.split(sep, maxsplit)`	`"a,b".split(",")` gives `['a','b']`
glue a list	`sep.join(list)`	`"-".join(["a","b"])` gives `"a-b"`
replace	`s.replace(old, new)`	all matches replaced
find position	`s.find(x)` returns `-1` if missing, `s.index(x)` raises `ValueError`	
test start / end	`s.startswith(x)`, `s.endswith(x)`	`True` / `False`
test inside	`x in s`	`True` / `False`
count	`s.count(x)`	number of non-overlapping matches
case	`upper`, `lower`, `title`, `capitalize`, `swapcase`, `casefold`	
test type	`isdigit`, `isalpha`, `isalnum`, `isspace`, `isupper`	
pad	`center`, `ljust`, `rjust`, `zfill`	`"7".zfill(3)` gives `"007"`
cut a prefix or suffix	`removeprefix`, `removesuffix`	
split in three	`partition(sep)`	`('a', '=', 'b=c')`

Worked example 3.1.1: slicing and the core methods

Predict: `t` is the request text `GET /api/orders/42?status=paid HTTP/1.1`. What does `t.split()` give? What does `t.find("zzz")` give, and what does `t.index("zzz")` do?

```
s = "  GET /api/orders/42?status=paid HTTP/1.1  "
print(repr(s.strip()), len(s))
t = s.strip()
print(t[0:3], t[4:], t[-8:], t[::-1][:5], t[::2][:6])
print(t.split(), t.split("/", 2), t.rsplit("/", 1))
print("-".join(["a","b","c"]), " ".join("hello world".split()))
print(t.replace("GET","POST"), t.find("api"), t.find("zzz"), t.index("api"))
try: t.index("zzz")
except ValueError as e: print("ValueError:", e)
print(t.startswith("GET"), t.endswith("1.1"), "api" in t, t.count("/"))
```

Real output:

```
'GET /api/orders/42?status=paid HTTP/1.1' 43
GET /api/orders/42?status=paid HTTP/1.1 HTTP/1.1 1.1/P GT/p/r
['GET', '/api/orders/42?status=paid', 'HTTP/1.1'] ['GET ', 'api', 'orders/42?
status=paid HTTP/1.1'] ['GET /api/orders/42?status=paid HTTP', '1.1']
a-b-c hello world
POST /api/orders/42?status=paid HTTP/1.1 5 -1 5
ValueError: substring not found
True True True 4
```

Line by line:

- `repr(...)` shows the quotes, so you can see that the two spaces on each side are gone. The original had length 43 (39 characters plus 4 spaces).
- `t[0:3]` is `GET`. `t[4:]` is everything from index 4. `t[-8:]` is the last 8 characters, `HTTP/1.1`. `t[::-1]` reverses the text, so the first 5 characters of it are `1.1/P`. `t[::2]` takes every second character: `GT/p/r`.
- `split()` with no argument cuts on any run of whitespace. `split("/", 2)` cuts at most 2 times, leaving the rest in the last item. `rsplit` cuts from the right.
- `" ".join("hello world".split())` is the standard trick to squeeze many spaces into one.
- `find` returns `-1` when there is no match, `index` raises an error. Prefer `in` when you only need yes/no, because `-1` is a valid-looking index and causes silent bugs.

Worked example 3.1.2: more methods, raw strings, Unicode

Predict: What is `"a,b,,c".split(",")`? And `"a b c".split(" ")` versus `"a b c".split()`?

```
print("a,b,,c".split(","), "a b  c".split(" "), "a b  c".split())
print("line1\nline2\r\nline3".splitlines(), "x".center(7,"*"), "7".zfill(3),
"ab".ljust(5,".")+"|", "ab".rjust(5)+"|")
print("www.site.com".removeprefix("www."), "file.csv".removesuffix(".csv"))
print("a\tb", r"a\tb", len("a\tb"), len(r"a\tb"))
print("abc".partition("b"), "a=b=c".partition("="), "a=b=c".rpartition("="))
print("Straße".upper(), "ß".casefold(), "STRASSE".casefold() ==
"straße".casefold())
print(ord("A"), chr(97), chr(8377), "\N{INDIAN RUPEE SIGN}")
s2 = "abc"
try: s2[0] = "z"
except TypeError as e: print("TypeError:", e)
print("abc" < "abd", "B" < "a", sorted(["b","A","a","B"]),
sorted(["b","A","a","B"], key=str.lower))
```

Real output:

```
['a', 'b', '', 'c'] ['a', 'b', '', 'c'] ['a', 'b', 'c']
['line1', 'line2', 'line3'] ***x*** 007 ab...|    ab|
site.com file
a       b a\tb 3 4
('a', 'b', 'c') ('a', '=', 'b=c') ('a=b', '=', 'c')
STRASSE ss True
65 a ₹ ₹
TypeError: 'str' object does not support item assignment
True True ['A', 'B', 'a', 'b'] ['A', 'a', 'b', 'B']
```

What to learn:

- `split(",")` keeps empty pieces between two commas. `split(" ")` also does, but `split()` with no argument skips empty pieces. For messy spaces use `split()`.
- `splitlines()` handles `\n` and `\r\n` (Windows line endings), a very common surprise in files from other systems.
- `r"a\tb"` has 4 characters (the backslash and the `t` are kept), while `"a\tb"` has 3 (a real tab).
- `casefold()` is the strong, Unicode-aware version of `lower()`. Use it to compare text without caring about case: German `ß` becomes `ss`.
- `ord` gives the number of a character, `chr` does the reverse. `₹` is code 8377.
- Strings compare by character code, so `"B" < "a"` is true (capitals come first). To sort ignoring case, pass `key=str.lower`.
- Strings are immutable, so `s2[0] = "z"` fails.

The picture

```
t =  G  E  T     /  a  p  i ...
+ve  0  1  2  3  4  5  6  7
-ve -n ...                  -2 -1

t[0:3]  -> "GET"           (index 0,1,2; stop 3 not included)
t[4:]   -> from index 4 to the end
t[-8:]  -> last 8 characters
t[::-1] -> step -1 = backwards
```

Each character has two addresses: a positive one counting from the left and a negative one counting from the right. The `stop` number is a boundary, not a character, which is why `t[0:3]` has exactly 3 characters (`stop − start`).

Real-world mapping

Method	Example	Where it appears in the job
`strip` , `lower`	clean a column of city names	Data cleaning before joins
`split` , unpack	parse `key=value` pairs	Config and log parsing
`join`	build a CSV line or SQL list	Report generation
`startswith`	filter file names	ETL file pickers
`casefold`	compare user input	Deduplicating names and emails

Applications

- **Data Analytics:** clean text columns (`str.strip().str.lower()` in pandas uses the same method names).
- **Data Engineering:** split log lines, build file paths and S3 keys.
- **NLP / GenAI:** tokenising, cleaning documents, building prompts.
- **Retail / Operations Analytics:** read order-system and website access-log lines.

COMMON MISTAKE

Building a long string with `+=` in a big loop (slow, because each step makes a new string). Collect parts in a list and use `"".join(parts)` once. Another: forgetting that methods return new strings, so `s.strip()` alone does nothing; you need `s = s.strip()` .

Check yourself

1. What is `"hello"[1:4]`?

Answer: `"ell"` (indexes 1, 2, 3).

1. Why use `casefold()` instead of `lower()` for comparison?

Answer: It handles Unicode special cases (like `ß`) so text with different case compares equal.

1. What is the difference between `find` and `index` when the text is missing?

Answer: `find` returns `-1`; `index` raises `ValueError`.

3.2 Formatting Text: f-strings and Friends

INTUITION

A mail-merge letter: one template, many names. Formatting lets you write the template once ("{name} took {mins} mins") and Python fills in the holes, aligned and rounded the way you ask.

Formal definition

An **f-string** is a string with `f` before the quote. Inside `{ }` you put any expression and, after a colon, a **format spec**: `{value:fill align width , .precision type}`. Common pieces:

Spec	Meaning	Example
`.1f`	1 decimal place	`480.4567` gives `480.5`
`>10` `<10` `^10`	right, left, center in 10 characters	
`,` or `_`	thousands separator	`1,250,000`
`.1%`	multiply by 100, 1 decimal, add `%`	`0.25` gives `25.0%`
`03d`	integer, zero padded to width 3	`5` gives `005`
`x`, `b`	hexadecimal, binary	`255` gives `ff`
`=` after name	print name and value (debugging)	`{mins=}`
`!r`	use `repr()`	shows quotes

Older styles still appear in old code: `"{} {}".format(a, b)` and `"%s %d" % (a, b)`. Learn to read them, write f-strings.

Worked example 3.2.1: format specs

Predict: What does `f"{pct:.1%}"` give for `pct = 0.25` ?

```
name, mins, pct = "apple", 480.4567, 0.25
print(f"{name} took {mins:.1f} mins")
print(f"{name:>10}|{name:<10}|{name:^10}|")
print(f"{1250000:,} {1250000:_} {pct:.1%} {255:x} {255:#b} {5:03d}
{mins:10.2f}|")
print(f"{mins=}", f"{name!r}", f"{2+3*4}", f"{{literal}}")
w = 8; print(f"{name:>{w}}|")
print("{} took {} mins".format(name, 480), "{0}-{0}-{1}".format("a","b"), "{n}:
{v:.2f}".format(n="x", v=1.5))
print("%s took %d mins (%.1f%%)" % (name, 480, 25.0))
```

Real output:

```
apple took 480.5 mins
     apple|apple     |  apple   |
1,250,000 1_250_000 25.0% ff 0b11111111 005     480.46|
mins=480.4567 'apple' 14 {literal}
   apple|
apple took 480 mins a-a-b x:1.50
apple took 480 mins (25.0%)
```

- `480.4567` rounded to one decimal is `480.5` , and to two decimals `480.46` .
- `{mins:10.2f}` means width 10, 2 decimals, so the number is padded with spaces on the left.
- `{{` and `}}` print literal braces. `{2+3*4}` runs the expression and shows 14. The width itself can be a variable: `{name:>{w}}` .

Worked example 3.2.2: a neat report table, and a wrong format code

Predict: What happens with `f"{'abc':d}"` ?

```
rows = [("book", 480, 0.0), ("shoes", 1250, 0.5), ("lamp", 3100, 12.25)]
print(f"{'item':<8}{'mins':>7}{'late%':>8}")
for l, m, e in rows: print(f"{l:<8}{m:>7,}{e:>8.2f}")
try: print(f"{'abc':d}")
except ValueError as e: print("ValueError:", e)
```

Real output:

```
item       mins   late%
book        480    0.00
shoes     1,250    0.50
lamp      3,100   12.25
ValueError: Unknown format code 'd' for object of type 'str'
```

Text columns are left-aligned (`<8`) and numbers right-aligned (`>7`), so digits line up. Asking for `d` (integer) on a string is an error, and the message names both the code and the type. Fix by using `s` for strings or no code at all.

The picture

```
f"{mins:>10,.2f}"
    |  | | | |
    |  | | | +-- type: f = fixed decimals
    |  | | +---- precision: 2 digits after the point
    |  | +------ thousands separator
    |  +-------- width: 10 characters, align right (>)
    +----------- the value (any expression)
```

Read the spec from left to right: align, width, separator, precision, type. Most of the time you only need two or three of them.

Real-world mapping

Spec	Example	Where it appears in the job
`:.2f`	`f"{rmse:.2f}"`	Model metrics in reports
`:,`	`f"{rows:,}"`	Row counts in logs
`:.1%`	`f"{err:.1%}"`	Error rates, accuracy
`{x=}`	`f"{n_rows=}"`	Quick debugging output

Applications

- **Data Analytics:** report tables and e-mail summaries.
- **Machine Learning:** training logs like `epoch 3 loss 0.2154` .
- **GenAI:** building prompts from templates with `f"""..."""` .
- **Data Engineering:** building file names (`f"{date:%Y/%m/%d}/part-{i:04d}.csv"`).

COMMON MISTAKE

Putting user-supplied text into a SQL string with an f-string. That is an injection risk (Chapter 13). Format for **display** only; use parameters for databases. Another: using the same quote type inside an f-string on old Python versions (before 3.12 you must switch quote style).

Check yourself

1. How do you print `3.14159` with 2 decimals?

Answer: `f"{3.14159:.2f}"` gives `3.14`.

1. What does `f"{0.07:.0%}"` print?

Answer: `7%`.

1. How do you print literal curly braces in an f-string?

Answer: Double them: `{{` and `}}`.

3.3 Encoding: Text, Bytes and UTF-8 in Practice

INTUITION

A computer can only store numbers. An **encoding** is a code book that says "number 97 means the letter a". Everyone must use the same code book, or text turns into garbage (the famous `Ã©` instead of `é`).

Formal definition

- **Unicode** is a list that gives every character in the world a number (a **code point**). `₹` is code point 8377.
- **UTF-8** is the most common way to turn those numbers into bytes. English letters take 1 byte, `é` takes 2, `₹` takes 3, most emoji take 4. It is the standard for the web and for new files.
- `str.encode("utf-8")` gives `bytes`. `bytes.decode("utf-8")` gives `str`. If the code book is wrong, you get `UnicodeDecodeError`, or silent garbage ("mojibake").
- `open(path, encoding="utf-8")` always names the encoding explicitly. Do not trust the default, it differs between systems.
- The decode error policy: `errors="replace"` puts `�` in place of bad bytes, `errors="ignore"` drops them. Both hide problems, so use them only when you accept data loss.

Worked example 3.3.1: mojibake and error policies

Predict: If we encode `"café ₹"` as UTF-8 and then wrongly decode it with Latin-1 (an older code book), do we get an error or garbage?

```
s = "café ₹"
b = s.encode("utf-8")
print(b, len(s), len(b))
print(b.decode("latin-1"))
print(b"caf\xe9".decode("latin-1"))
try: b"caf\xe9".decode("utf-8")
except UnicodeDecodeError as e: print("UnicodeDecodeError:", e)
print(b"caf\xe9".decode("utf-8", errors="replace"), b"caf\xe9".decode("utf-8",
errors="ignore"))
```

Real output:

```
b'caf\xc3\xa9 \xe2\x82\xb9' 6 9
cafÃ© â¹
café
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9 in position 3:
unexpected end of data
caf caf
```

- 6 characters become 9 bytes (`é` is 2 bytes and `₹` is 3).
- Decoding those bytes with the wrong code book (Latin-1) does **not** fail. It silently gives `cafÃ© â¹` . That is **mojibake**, and it is worse than an error because nobody notices. If you see `Ã©` in your data, someone read UTF-8 as Latin-1.
- The byte `\xe9` is the letter `é` in Latin-1, but in UTF-8 it is the start of a longer character, so UTF-8 raises an error.
- `errors="replace"` gave `caf` and `errors="ignore"` gave `caf` . Both lose data.

Worked example 3.3.2: files and look-alike characters

Predict: Two strings both look like `é` . Are they equal?

```
import tempfile, os, unicodedata
with tempfile.TemporaryDirectory() as d:
    p = os.path.join(d, "x.txt")
    with open(p, "w", encoding="utf-8") as f: f.write("café ₹\n")
    print(open(p, "rb").read())
    print(open(p, encoding="utf-8").read().strip())
    print(open(p, encoding="latin-1").read().strip())
a = "e\u0301"   # letter e + combining accent
b = "\u00e9"     # one single character
print(a, b, a == b, len(a), len(b), unicodedata.normalize("NFC", a) == b)
```

Real output:

```
b'caf\xc3\xa9 \xe2\x82\xb9\n'
café ₹
cafÃ© â¹
é é False 2 1 True
```

- The file on disk holds UTF-8 bytes. Reading it with `encoding="utf-8"` gives the right text. Reading it as Latin-1 gives mojibake again.
- `a` and `b` print identically but are **different strings**: one is two code points, the other one. `==` is `False`. `unicodedata.normalize("NFC", ...)` converts both to one standard form so they compare equal. Normalise text before you compare or deduplicate it.

The picture

```
 "é"  --encode("utf-8")-->  b'\xc3\xa9'   --decode("utf-8")-->  "é"   (correct)
                               |
                               +--decode("latin-1")-->  "Ã©"      (mojibake)
```

The same two bytes can mean different text, depending on which code book you use to read them. So the reader must know the encoding. UTF-8 is the safe default.

Real-world mapping

Situation	Symptom	Fix
CSV exported from Excel on Windows	`UnicodeDecodeError`	try `encoding="utf-8-sig"` or `"cp1252"`
Names in two systems	`José` fails to match `José`	`unicodedata.normalize("NFC", s)`

API returns bytes	you see `b'...'`	`.decode("utf-8")`
Text shows `Ã©`	wrong decoding upstream	find where UTF-8 was read as Latin-1

Applications

- **Data Engineering:** ingesting files from many source systems with different encodings.
- **Data Science / NLP:** cleaning text before tokenising; deduplicating names.
- **GenAI:** token counts depend on bytes and characters, and prompts contain non-English text.

COMMON MISTAKE

Using `errors="ignore"` to make an error go away. You hide data loss. Find the true encoding first (ask the source team, or check the file's byte-order mark).

Check yourself

1. Why does reading UTF-8 text as Latin-1 not raise an error?

Answer: Latin-1 assigns a character to every byte value, so any bytes decode. The text is simply wrong.

1. What is the safest way to open a text file?

Answer: `open(path, encoding="utf-8")`, naming the encoding every time.

1. Why can `"é" == "é"` be `False`?

Answer: One may be a single code point and the other a letter plus a combining accent. Normalise with `unicodedata.normalize`.

3.4 Regular Expressions

INTUITION

`split` and `find` work when the text has a fixed shape ("cut at the comma"). But what if you must find "any 3 digits that stand alone" or "an e-mail address"? A **regular expression** (regex) is a mini language to describe the *shape* of text, like a search form: "digits, a dot, digits". You write the shape once, and the engine finds every piece of text with that shape.

Words you need

Word	Meaning
Pattern	the regex string, e.g. `r"\d+"`
Match	a piece of text that fits the pattern
Group	a part of the pattern in `( )` whose matched text you want back
Greedy	takes as much text as it can (default). **Lazy** takes as little as it can (add `?`)
Anchor	a position, not a character: `^` start, `$` end, `\b` word edge

The pattern language (the 20% that does 80%)

Piece	Meaning
`.`	any one character (except new line)
`\d` `\w` `\s`	a digit, a word character (letter, digit, `_`), a whitespace
`\D` `\W` `\S`	the opposite of the above
`[abc]` `[a-z]` `[^abc]`	one of those characters / a range / none of those
`*` `+` `?`	0 or more, 1 or more, 0 or 1
`{3}` `{2,4}`	exactly 3, between 2 and 4
`a\	b`
`( )` and `(?P<name>)`	a group, and a named group
`^` `$` `\b`	start of text, end of text, word edge
`(?<=x)` `(?=x)`	look-behind and look-ahead: "preceded by x" / "followed by x", without taking x
`\1`	the same text as group 1 again (back-reference)

Always write patterns as **raw strings** (`r"..."`) so Python does not eat the backslashes.

The functions in module `re`:

Function	What it does
`re.search(p, s)`	first match anywhere, or `None`
`re.match(p, s)`	match only at the start
`re.fullmatch(p, s)`	the whole string must match
`re.findall(p, s)`	list of all matches (or of group contents)
`re.finditer(p, s)`	iterator of match objects
`re.sub(p, repl, s)`	replace matches; `repl` may be text or a function
`re.split(p, s)`	split at matches
`re.compile(p, flags)`	prepare a pattern once and reuse it

Flags: `re.I` ignore case, `re.M` `^` and `$` per line, `re.S` dot matches new line, `re.X` verbose (allows spaces and comments in the pattern).

Worked example 3.4.1: find things in the log line

Predict: `re.findall(r"\b\d{3}\b", log)` looks for stand-alone 3-digit numbers. Which values will it return?

```
import re
log = '10.0.0.5 - - [07/Nov/2026:10:15:32 +0530] "GET /api/orders/42 HTTP/1.1"
200 512 0.480'
print(re.search(r"\d+", log).group(), re.match(r"\d+", log).group(),
re.match(r"GET", log))
print(re.findall(r"\d+\.\d+\.\d+\.\d+", log))
print(re.findall(r"\b\d{3}\b", log))
```

Real output:

```
10 10 None
['10.0.0.5']
['200', '512', '480']
```

- `\d+` means "one or more digits". The first match is `10` at the very start. `re.match` also gives `10` because the log starts with digits. `re.match(r"GET", log)` is `None` because `GET` is not

at the start. This is the main difference between `match` and `search`.

- `\d+\.\d+\.\d+\.\d+` is "digits dot digits dot digits dot digits": the IP address. The `\.` is a literal dot, because a bare `.` would mean "any character".
- `\b\d{3}\b` finds three digits with a word edge on both sides. It found `200` (status), `512` (bytes) and `480` (the digits after the dot in `0.480`). It did not return `2026` (4 digits), `+0530` (4 digits), or `42` (2 digits). Notice `480` was a **false positive**: the pattern knows nothing about meaning. That is why real parsing uses a full pattern with named groups, as below.

Worked example 3.4.2: named groups parse a log line

Predict: What will `m.groupdict()` contain for `status`: the number 200 or the text `'200'`?

```
m = re.search(r'"(?P<method>[A-Z]+) (?P<path>\S+) HTTP/[\d.]+" (?
P<status>\d{3}) (?P<bytes>\d+) (?P<secs>[\d.]+)', log)
print(m.groupdict())
print(m.group("path"), m.start("path"), m.end("path"), m.span())
d = m.groupdict(); print(int(d["status"]) + 1, float(d["secs"])*1000)
```

Real output:

```
{'method': 'GET', 'path': '/api/orders/42', 'status': '200', 'bytes': '512',
'secs': '0.480'}
/api/orders/42 47 61 (42, 85)
201 480.0
```

- Regex always returns **text**. `status` is `'200'`, not the number 200. You convert it yourself (`int(...)`). This is the same lesson as Chapter 1.
- `(?P<name>...)` gives a group a name, so you read `m.group("path")` instead of counting brackets.
- `m.span()` is `(42, 85)`: the matched text starts at index 42 and ends before index 85. `m.start("path")` is 47.
- `float("0.480") * 1000 = 480.0` ms.

Problem 3.4.1 Parse the full log line into a record with typed values: IP, timestamp, method, path, status (int), bytes (int) and processing time in whole milliseconds. Then try three lines, one of them garbage, and count the parsed lines.

Step 1	Describe the line piece by piece: IP `\S+`, two ignored words `\S+ \S+`, timestamp inside `[ ]` as `[^\]]+` (any characters except `]`), request in quotes, status `\d{3}`, bytes `\d+`, seconds `[\d.]+`.
Step 2	Anchor with `^` and `$` so the whole line must match, and name every group.
Step 3	Convert: `int(status)`, `int(bytes)`, `round(float(secs) * 1000)`.
Step 4	Loop over lines and use the walrus `:=` (Chapter 2) to keep only lines that match.
Answer	The code below parses two lines, skips the garbage line, and produces `ms = 480` for the first.
Check	`float("0.480") * 1000` is 480.0, which rounds to 480. The two parsed status codes are `'200'` and `'500'`, and the sum of seconds is 0.480 + 3.100 = 3.58, matching the output.
Why it matters	This is the first step of every log-analysis pipeline. Skipping bad lines (instead of crashing) is what makes a parser usable on real files.

```
pat = re.compile(r'^(?P<ip>\S+) \S+ \S+ \[(?P<ts>[^\]]+)\] "(?P<method>[A-Z]+)
(?P<path>\S+) [^"]+" (?P<status>\d{3}) (?P<bytes>\d+) (?P<secs>[\d.]+)$')
rec = pat.match(log).groupdict()
rec["status"] = int(rec["status"]); rec["bytes"] = int(rec["bytes"]); rec["ms"]
= round(float(rec.pop("secs"))*1000)
print(rec)
lines = [log, '10.0.0.7 - - [07/Nov/2026:10:15:40 +0530] "POST /api/pay
HTTP/1.1" 500 87 3.100', 'garbage line']
parsed = [m.groupdict() for l in lines if (m := pat.match(l))]
print(len(parsed), [p["status"] for p in parsed], sum(float(p["secs"]) for p in
parsed))
```

```
{'ip': '10.0.0.5', 'ts': '07/Nov/2026:10:15:32 +0530', 'method': 'GET', 'path':
'/api/orders/42', 'status': 200, 'bytes': 512, 'ms': 480}
2 ['200', '500'] 3.58
```

Worked example 3.4.3: sub, split, greedy vs lazy, lookaround, flags

Predict: For `<b>bold</b> and <i>it</i>`, does `<.+>` find 4 tags or 1 piece?

```
print(re.sub(r"\d+\.\d+\.\d+\.\d+", "x.x.x.x", log)[:20])
print(re.sub(r"(\w+)@(\w+)\.com", r"\2:\1", "asha@acme.com, ravi@zeta.com"))
print(re.sub(r"\d+", lambda m: str(int(m.group())*2), "a1 b22 c333"))
print(re.split(r"[,;]\s*", "a, b;c,   d"), re.split(r"(,)", "a,b"))
text = "<b>bold</b> and <i>it</i>"
print(re.findall(r"<.+>", text), re.findall(r"<.+?>", text))
print(re.findall(r"^\w+", "ab cd\nef gh", re.M), re.findall(r"HELLO", "hello
Hello", re.I))
print(re.findall(r"(?<=\$)\d+", "cost $45 and $120"), re.findall(r"\d+(?=%)",
"5% of 20 and 30%"))
print(re.findall(r"(\w)\1", "aabbcd hello"), re.sub(r"\s+", " ", "a   b \n c"))
```

Real output:

```
x.x.x.x - - [07/Nov/
acme:asha, zeta:ravi
a2 b44 c666
['a', 'b', 'c', 'd'] ['a', ',', 'b']
['<b>bold</b> and <i>it</i>'] ['<b>', '</b>', '<i>', '</i>']
['ab', 'ef'] ['hello', 'Hello']
['45', '120'] ['5', '30']
['a', 'b', 'l'] a b c
```

- `sub` can use group numbers in the replacement (`\2:\1` swaps the parts) and can take a **function** that receives each match and returns the new text (`a1 b22 c333` becomes `a2 b44 c666`). A function is the tool for "replace using a calculation". Use `sub` to mask PII (personal data): replace IP addresses or phone numbers before sharing logs.
- Wrapping the separator in `( )` in `split` keeps it in the result.
- **Greedy** `<.+>` took as much as possible, from the first `<` to the very last `>` , giving one big piece. **Lazy** `<.+?>` stops at the first `>` , giving four tags.
- `re.M` makes `^` match at the start of every line (`ab` and `ef`). `re.I` ignores case.
- Look-behind `(?<=\$)\d+` finds numbers preceded by `$` without including the `$` . Look-ahead `\d+(?=%)` finds numbers followed by `%` : `5` and `30` , but not `20` .
- `(\w)\1` finds a character followed by itself: `a` , `b` , `l` (from `aa` , `bb` , `ll` in hello).

Worked example 3.4.4: validate, escape, verbose, and the slow-regex danger

Predict: Is `a.b+c@x.co.in` a valid e-mail for our simple pattern?

```
print(re.fullmatch(r"\d{10}", "9876543210") is not None,
re.fullmatch(r"\d{10}", "98765432100") is not None)
email = re.compile(r"^[\w.+-]+@[\w-]+(\.[\w-]+)+$")
print([bool(email.match(x)) for x in
("a@b.com","a.b+c@x.co.in","bad@","@x.com")])
print(re.escape("a.b*c"), re.findall(re.escape("1.5"), "1.5 and 125"))
verbose = re.compile(r"""
   (?P<h>\d{2}):   # hours
   (?P<m>\d{2})    # minutes
""", re.X)
print(verbose.search("at 14:05 sharp").groupdict())
import time
t=time.perf_counter(); re.search(r"(a+)+b", "a"*24+"c"); bad_seconds =
time.perf_counter()-t
t=time.perf_counter(); re.search(r"a+b", "a"*24+"c"); good_seconds =
time.perf_counter()-t
print(bad_seconds > 0.05, good_seconds < 0.01)
```

Real output:

```
True False
[True, True, False, False]
a\.b\*c ['1.5']
{'h': '14', 'm': '05'}
True True
```

- `fullmatch` demands the whole string match. A 10-digit phone number passes, an 11-digit one does not.
- The e-mail pattern accepts the first two and rejects `bad@` and `@x.com`. Real e-mail rules are much more complicated, so this is a **sanity check**, not a full validator. The only real proof is to send an e-mail.
- `re.escape` adds backslashes so special characters are read literally. Use it when the search text comes from a user or a file, otherwise `1.5` would also match `125`.
- `re.X` lets you lay out a long pattern over several lines with comments. Do this for any pattern longer than a line.
- The last line is a warning. `(a+)+b` on `aaaa...c` makes the engine try an exponential number of ways to split the `a`s before giving up (**catastrophic backtracking**). Both checks printed `True`: the bad pattern took more than 0.05 seconds on just 25 characters (the exact time depends on your machine, and it doubles with every extra `a`), while the safe pattern took under 0.01 seconds. Avoid nested repeats like `(x+)+`. On web servers, a bad regex plus user input can freeze a service (a "ReDoS" attack).

The picture

```
Pattern:  \b \d{3} \b          Text:  ... "HTTP/1.1" 200 512 0.480
           |   |    |                                ^^^ ^^^     ^^^
        edge 3 digits edge                            |   |       |
                                                   match match  match (false
positive!)

Groups:   "(?P<method>[A-Z]+) (?P<path>\S+)"   on   "GET /api/orders/42 ..."
                 |                  |
              'GET'           '/api/orders/42'
```

Think of the engine as a cursor that slides over the text. At each position it asks, "can the pattern start here?" It reports a match when yes, and moves on when no.

Real-world mapping

Regex idea	Example	Where it appears in the job
Named groups	parse an access log	Log analysis, data ingestion
`sub`	mask phone numbers or IPs	Privacy engineering (PII masking)
`fullmatch`	validate an ID format	Data validation rules
`findall`	pull all hashtags or numbers from text	NLP preprocessing, feature extraction
lazy `.+?`	extract text between HTML tags	Quick scrapers (but prefer a real parser for HTML)

Applications

- **Data Analytics:** extract numbers, IDs and dates from free-text columns.
- **Data Engineering:** parse semi-structured logs into tables; validate file names.
- **Data Science / NLP:** cleaning text, finding patterns before modelling.
- **MLOps / Security:** scan code or logs for leaked secrets with patterns (Chapter 13).
- **Retail / Operations Analytics:** pull `order_id` and `delivery_minutes` out of application logs to join with the orders table.

COMMON MISTAKE

Parsing HTML or JSON with regex. They have nested structure that regex cannot handle reliably. Use `html.parser` or BeautifulSoup for HTML, and `json` for JSON. Another mistake: forgetting the raw string (`"\d"` instead of `r"\d"`), which gives a warning in recent Python versions. And forgetting that `.` needs `\.` when you mean a literal dot.

PROFESSOR'S NOTE

Build a regex in small steps. Start with one piece, test it in the REPL, add the next piece, test again. Never write a 100-character regex in one go. Try `re.X` and comments to keep it readable.

Check yourself

1. What is the difference between `re.match` and `re.search` ?

Answer: `match` only tries at the start of the string. `search` looks anywhere.

1. What does `.+?` do differently from `.+` ?

Answer: `.+` is greedy (takes as much as possible). `.+?` is lazy (as little as possible).

1. Why should you wrap user-supplied search text in `re.escape` ?

Answer: So characters like `.` and `*` are read as plain characters and not as regex operators.

Chapter recap

- A string is an immutable sequence of characters. Index from 0, slice with `s[start:stop:step]` , and remember `stop` is not included.
- String methods return new strings. Learn `strip` , `split` , `join` , `replace` , `find` , `startswith` , `lower` , `casefold` first.
- Build big strings with `"".join(list)` , not with `+=` in a loop.
- Use f-strings for output. The format spec `{value:>10,.2f}` controls alignment, separator, decimals and type.
- Text is `str` , data is `bytes` . Always name the encoding, and use UTF-8.
- If you see `Ã©` , UTF-8 was read as Latin-1. If two identical-looking strings differ, normalise with `unicodedata` .
- Regex describes the shape of text. Use raw strings, named groups, `fullmatch` to validate, `sub` with a function to transform.

- Regex returns text. Convert to numbers yourself. Do not use regex for HTML or JSON.
- Avoid nested repeats such as `(a+)+` : they can freeze a program.

Pattern	Example
Slice	`s[2:5]` , `s[-3:]` , `s[::-1]`
Clean and split	`s.strip().split(",")`
Join	`",".join(parts)`
Format	`f"{x:,.2f}"` , `f"{p:.1%}"` , `f"{n:05d}"`
Encode / decode	`s.encode("utf-8")` , `b.decode("utf-8")`
Find one	`re.search(r"(?P<n>\d+)", s)`
Find all	`re.findall(r"\d+", s)`
Validate	`re.fullmatch(r"\d{10}", s)`
Replace	`re.sub(r"\s+", " ", s)`
Compile once	`pat = re.compile(r"...", re.X)`

Practice set

Problem 3.P.1 (easy) What is `"performance"[2:6]` ?

Answer: `"rfor"` . Indexes 2, 3, 4, 5 are `r` , `f` , `o` , `r` .

Problem 3.P.2 (easy) What is `"a-b-c".split("-", 1)` ?

Answer: `['a', 'b-c']` . The second argument is the maximum number of cuts.

Problem 3.P.3 (easy) Write an f-string that prints `0.4567` as a percentage with one decimal.

Answer: `f"{0.4567:.1%}"` gives `45.7%` .

Problem 3.P.4 (what is the output?)

```
s = "hello"
s.upper()
print(s)
```

Answer: `hello.upper()` returns a new string and we threw it away. Use `s = s.upper()`.

Problem 3.P.5 (what is the output?) `import re; print(re.findall(r"\d+", "a12b345c6"))`

Answer: `['12', '345', '6']`. Each run of digits is one match.

Problem 3.P.6 (computation) How many bytes does `"₹100"` take in UTF-8?

Answer: 6 bytes. `₹` is 3 bytes, and each of the digits `1`, `0`, `0` is 1 byte: 3 + 3 = 6.

Problem 3.P.7 (find the bug)

```
import re
print(re.findall("\d+\.\d+", "version 3.14 and 2.7"))
```

Answer: The pattern is not a raw string. Recent Python versions warn about the invalid escape `\d`. Use `r"\d+\.\d+"`. The result is `['3.14', '2.7']`.

Problem 3.P.8 (find the bug) A regex to get the text inside the first bracket pair from `"[a] and [b]"` returns the wrong text:

```
re.search(r"\[(.+)\]", "[a] and [b]").group(1)
```

Answer: It returns `a] and [b` because `.+` is greedy. Use the lazy form `r"\[(.+?)\]"`, which returns `a`.

Problem 3.P.9 (scenario) You read a CSV from a partner and names like `Zoë` show as `ZoÃ«`. What happened and what do you do?

Answer: The file is UTF-8 but was read as Latin-1 (or similar). Re-read with `open(path, encoding="utf-8")` (or `pandas.read_csv(..., encoding="utf-8")`). Do not "fix" it with `errors="ignore"`.

Problem 3.P.10 (scenario) Write one line that extracts all status codes (3-digit numbers after a closing quote and a space) from a log string `text`.

Answer: `re.findall(r'" (\d{3}) ', text)`. For our sample line it returns `['200']`. The group in `( )` makes `findall` return only the digits.

Problem 3.P.11 (interview) "What is the difference between `str` and `bytes` in Python 3?"

Answer: `str` is text, a sequence of Unicode characters. `bytes` is raw binary data, a sequence of numbers 0-255. You convert with `encode` (str to bytes) and `decode` (bytes to str), and must name an encoding such as UTF-8.

Problem 3.P.12 (interview) "Why can a regex be dangerous in production, and how would you reduce the risk?"

Answer: A pattern with nested repeats, like `(a+)+b`, can take exponential time on some inputs (catastrophic backtracking), which can freeze a service. Avoid nested quantifiers, keep patterns simple, limit input length, use `fullmatch` with anchors, and test with long non-matching input.

Prompt pack

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles, and English is my second language, so use simple words and
short sentences. Teach me Python string slicing and the 15 most useful string
methods for cleaning text. Use a web-server log line as the running example.
For each method show a small example, ask me to predict the output first, and
explain with a picture of index positions. End with three questions to check
me.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles, and English is my second language. Teach me Python f-string
formatting: alignment, width, decimals, thousands separator, percent, zero
padding and the {x=} debug form. Give me 8 graded exercises, from easy to hard,
where I must write the format string for a given output. Do not give solutions
until I answer. Then review my answers and explain mistakes in simple words.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles, with English as a second language. Explain text encoding in
Python: Unicode, UTF-8, bytes vs str, mojibake, UnicodeDecodeError,
errors="replace", and Unicode normalisation. Use real examples such as a CSV
with Indian names. Then quiz me one question at a time, wait for my answer, and
explain why each answer is right or wrong.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles, and English is my second language. Teach me regular
expressions in Python from zero: character classes, quantifiers, groups, named
groups, greedy vs lazy, anchors, look-ahead, re.sub with a function, and
catastrophic backtracking. Build one pattern step by step to parse an access-
log line. Then give me 6 graded regex exercises with no solutions. Here is a
```

```
pattern I am stuck on: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles, with English as a second language. Run a mock interview on
Python text processing: strings, immutability, f-strings, encoding, and regular
expressions. Ask one question at a time, include two small coding questions
that I answer by typing code, give a score from 1 to 5 for each answer, show a
model answer, and finish with a list of my weak points and what to practise
next.
```

CHAPTER 4

COLLECTIONS, COMPREHENSIONS AND THE STANDARD-LIBRARY TOOLBOX

Choose the right container, and let `collections`, `itertools` and `functools` do the heavy lifting.

An online shop records thousands of delivery times. To answer "which product is delivered slowest?" you must group numbers by item, count them, sort them, remove duplicates and find the top few. In Python every one of those jobs has a ready-made container or tool, and picking the right one can make a program 100 times faster or ten times shorter. In this chapter you learn the four core containers (list, tuple, set, dict), the one-line loops called comprehensions, and three standard-library modules that professionals use every day. By the end you will group, count, rank and deduplicate data without writing clumsy loops.

Running example. The same shop delivery records as before, as `(item, delivery_mins)` pairs, where the number is minutes from order to doorstep. Keep this table in your head:

item	delivery_mins
book	480
lamp	3100
book	310
lamp	60
shoes	1250
book	290

4.1 Lists and Tuples: Ordered Sequences

INTUITION

A list is a shopping list on paper: you can add items, cross them out, reorder them. A tuple is a printed receipt: it records exactly what was bought, and you do not edit it. Use a list for "a pile of similar things that may change", and a tuple for "one record with fixed parts".

Formal definition

- **list** `[a, b, c]` : ordered, **mutable**, can hold anything (even mixed types). Indexing and slicing work exactly like strings (Chapter 3).
- **tuple** `(a, b)` : ordered, **immutable**. A one-item tuple needs a trailing comma: `(5,)` . Without it, `(5)` is just the number 5.
- **namedtuple**: a tuple whose fields also have names (`s.item`).
- Both support `len` , `in` , `+` , `*` , `min` , `max` , `sum` (numbers), `count` , `index` .

Main list methods:

Method	What it does	Returns
`append(x)`	add one item at the end	`None`
`extend(iterable)`	add many items at the end	`None`
`insert(i, x)`	insert before position `i`	`None`
`pop()` / `pop(i)`	remove and return the last / the i-th item	the item
`remove(x)`	remove the first item equal to `x`	`None`
`sort(key=, reverse=)`	sort **in place**	`None`
`sorted(list)`	a **new** sorted list	new list
`reverse()`	reverse in place	`None`

Notice that methods that change the list in place return `None` . This is a deliberate signal: "I changed the thing, there is no new thing."

Worked example 4.1.1: a list of delivery times

Predict: After `mins.sort(reverse=True)` , which index does the value `480` have?

```
mins = [480, 1250, 3100, 310, 60]
mins.append(95)
mins.extend([700, 220])
mins.insert(0, 5)
print(mins, len(mins))
print(mins[0], mins[-1], mins[1:4])
last = mins.pop()
first = mins.pop(0)
mins.remove(95)
print(last, first, mins)
copy_sorted = sorted(mins)
print(copy_sorted, mins)
mins.sort(reverse=True)
print(mins, mins.index(480), mins.count(60), min(mins), max(mins), sum(mins))
try:
    mins.remove(9999)
except ValueError as e:
    print("ValueError:", e)
try:
    mins[100]
except IndexError as e:
    print("IndexError:", e)
```

Real output:

```
[5, 480, 1250, 3100, 310, 60, 95, 700, 220] 9
5 220 [480, 1250, 3100]
220 5 [480, 1250, 3100, 310, 60, 700]
[60, 310, 480, 700, 1250, 3100] [480, 1250, 3100, 310, 60, 700]
[3100, 1250, 700, 480, 310, 60] 3 1 60 3100 5900
ValueError: list.remove(x): x not in list
IndexError: list index out of range
```

Step by step:

- After four changes the list has 9 items. `insert(0, 5)` put the `5` in front.
- `pop()` removed `220` (last), `pop(0)` removed `5` (first), `remove(95)` deleted the value 95. Six items remain.
- `sorted(mins)` made a **new** list and left `mins` unchanged. `mins.sort(reverse=True)` changed `mins` itself. After sorting, `480` is at index 3 (after 3100, 1250, 700).
- Check the sum: 3100 + 1250 + 700 + 480 + 310 + 60 = 5900. Matches.
- Asking for a missing value or a too-big index raises `ValueError` or `IndexError`. Read the message: it tells you what was missing.

Worked example 4.1.2: tuples and named tuples

```
pt = (480, "book")
one = (5,)
print(type((5)), type(one), pt[0])
item_mins, name = pt
from collections import namedtuple
S = namedtuple("S", "item mins")
s = S("book", 480)
print(s, s.item, s[1], s._asdict())
```

Real output:

```
<class 'int'> <class 'tuple'> 480
S(item='book', mins=480) book 480 {'item': 'book', 'mins': 480}
```

- `(5)` is an `int`; `(5,)` is a tuple. The comma makes the tuple, not the brackets.
- The `namedtuple` lets you read `s.item` instead of `s[0]`. Code with names is much easier to read. In Chapter 8 you will meet `dataclass`, which is the more modern choice for records.
- Tuples can be **dictionary keys** and **set members** (lists cannot), because they never change.

The picture

```
 list   [ 480 | 1250 | 3100 | 310 | 60 ]     can grow, shrink, reorder
 index     0      1      2      3     4
 neg      -5     -4     -3     -2    -1

 tuple  ( "book", 480 )                      fixed shape, one record
```

A list is a row of numbered boxes. Python stores in each box not the value itself but a reference to it (Chapter 1), so a list of 1,000 items holds 1,000 arrows.

Real-world mapping

Container	Example	Where it appears in the job
list	all delivery times of one day	Column of data before it becomes a DataFrame
tuple	`(item, mins)` one row	Rows returned by a database cursor
namedtuple	`S("book", 480)`	Light records with readable fields

list as stack	`append` / `pop`	Undo, parsing nested brackets

Applications

- **Data Analytics:** rows read from a file are a list of tuples.
- **Data Engineering:** batches of records are lists; database rows are tuples.
- **Machine Learning:** a list of feature names; a tuple for an array shape such as `(1000, 20)`.
- **Operations Analytics:** the list of delivery times is the input of every percentile.

COMMON MISTAKE

Writing `sorted_list = mins.sort()`. `sort()` returns `None`, so `sorted_list` becomes `None`. Use `sorted(mins)` when you want a new list. Another: `list.pop(0)` and `list.insert(0, x)` on a huge list are slow (every item shifts). Use `deque` (Section 4.4).

Check yourself

1. What is `type((7))` and `type((7,))`?

Answer: `int` and `tuple`.

1. What does `[1, 2, 3].append(4)` return?

Answer: `None`. It changes the list in place.

1. Why can a tuple be a dictionary key but a list cannot?

Answer: A tuple is immutable (hashable); a list can change, so Python refuses to hash it.

4.2 Sets: Uniqueness and Fast Membership

INTUITION

A set is a guest list at the door. Each name is on it at most once, and the guard can check "is this person on the list?" instantly, without reading the whole list. The order of names does not matter.

Formal definition

- **set** `{a, b}`: unordered, no duplicates, mutable. An empty set is `set()`, because `{}` makes an empty **dict**.
- **frozenset**: an immutable set. It can be a dict key or a member of another set.
- Members must be **hashable**: Python can compute a fixed number (a **hash**) from them. Numbers,

strings and tuples of those are hashable. Lists, sets and dicts are not.

- Set maths: `a | b` union, `a & b` intersection, `a - b` difference, `a ^ b` symmetric difference (in exactly one), `a <= b` subset.
- Why fast? A set stores items in a **hash table**: the hash tells Python which "drawer" to open, so a lookup takes about the same time for 10 items or 10 million (called O(1), "constant time").

Worked example 4.2.1: set operations

Predict: `a` has three product names and `b` has two. How many items are in `a ^ b`?

```
a = {"book", "shoes", "lamp"}
b = {"lamp", "pen"}
print(sorted(a | b), sorted(a & b), sorted(a - b), sorted(a ^ b))
print("lamp" in a, len(a), {1, 2} <= {1, 2, 3}, a.isdisjoint({"x"}))
print(set([1, 1, 2, 2, 3]), sorted(set("hello")))
print({frozenset([1, 2]): "ok"})
print(type({}), type(set()))
try:
    {[1, 2]}
except TypeError as e:
    print("TypeError:", e)
```

Real output:

```
['book', 'lamp', 'pen', 'shoes'] ['lamp'] ['book', 'shoes'] ['book', 'pen',
'shoes']
True 3 True True
{1, 2, 3} ['e', 'h', 'l', 'o']
{frozenset({1, 2}): 'ok'}
<class 'dict'> <class 'set'>
TypeError: cannot use 'list' as a set element (unhashable type: 'list')
```

- The union has 4 names. Only `lamp` is in both. `a ^ b` has 3 names: `book`, `shoes`, `pen` (the ones in exactly one set).
- I used `sorted(...)` when printing because the printed order of a set is not guaranteed and can change between runs for text values. Never depend on set order.
- `set("hello")` has only 4 letters, because the second `l` is dropped. `{}` is a `dict`, `set()` is the empty set.

Worked example 4.2.2: why sets are fast

Problem 4.2.1 You have 200,000 IDs and must ask "is 199,999 in the collection?" 200 times. Compare

a list and a set.

Step 1	Build both: `big = list(range(200_000))` and `bigset = set(big)`.
Step 2	Time 200 lookups of the last item in each. In a list, Python compares the item with each element from the left: up to 200,000 comparisons per lookup. In a set it hashes the item once.
Step 3	Compare the two times as a ratio.
Answer	In my run the list was more than 100 times slower (the script printed `True` for "ratio above 100"). The exact ratio depends on your machine; I measured only that it exceeds 100.
Check	The code is below. Run it yourself and print `tl/ts` to see your own number.
Why it matters	A join or a filter such as "keep rows whose id is in this other list" can take minutes with a list and under a second with a set. This is the single most useful speed-up for a beginner.

```
import time
big = list(range(200_000)); bigset = set(big)
t=time.perf_counter(); [199_999 in big for _ in range(200)];
tl=time.perf_counter()-t
t=time.perf_counter(); [199_999 in bigset for _ in range(200)];
ts=time.perf_counter()-t
print("list slower than set by more than 100x:", tl/ts > 100)
```

```
list slower than set by more than 100x: True
```

The picture

```
 list lookup:   [ . . . . . . . . . X ]     walk from the left: many steps
 set  lookup:   hash("X") -> drawer 7 -> open drawer 7 -> found
```

Real-world mapping

Set idea	Example	Where it appears in the job
deduplicate	`set(user_ids)`	Unique visitors, distinct customers
membership	`if id in blocked_ids`	Filtering rows quickly

test		
`a - b`	products on the price list but never sold	Data reconciliation between two systems
`a & b`	customers in both files	Join checks

Applications

- **Data Analytics:** count distinct users; find users who bought in both months.
- **Data Engineering:** compare yesterday's and today's key sets to find new and deleted rows.
- **Machine Learning:** build a vocabulary of unique words; train/test leakage checks (`train_ids & test_ids` must be empty).
- **Security:** blocklists and allowlists.

COMMON MISTAKE

Using a list for repeated membership tests (`if x in big_list` inside a loop). Convert once to a set. Another: expecting a set to keep order, or writing `{}` to create an empty set.

Check yourself

1. How do you create an empty set?

Answer: `set()` . The literal `{}` is an empty dict.

1. What is `{1, 2, 3} & {2, 3, 4}` ?

Answer: `{2, 3}` .

1. Why is `x in my_set` fast?

Answer: Python hashes `x` and jumps straight to the right slot of a hash table instead of scanning.

4.3 Dictionaries: Key-Value Lookup

INTUITION

A dictionary is a phone book. You do not read every page: you look up the name (the **key**) and get the number (the **value**). Because the lookup uses a hash table (the same trick as a set), it is instant even for millions of entries.

Formal definition

- **dict** `{key: value}` : maps keys to values. Keys must be hashable and unique. Values can be anything.
- Since Python 3.7 a dict **remembers insertion order**.
- `d[k]` raises `KeyError` if missing. `d.get(k)` returns `None` ; `d.get(k, default)` returns your default.
- `d.setdefault(k, default)` returns `d[k]` , first inserting the default if the key is missing.
- `keys()` , `values()` , `items()` give **views** (live windows onto the dict). `items()` gives `(key, value)` pairs.
- Merge: `a | b` (right side wins on clashes), or `{**a, **b}` . `d.update(...)` changes in place.
- `pop(k)` , `del d[k]` , `d.fromkeys(keys, value)` , `dict(zip(keys, values))` .

Worked example 4.3.1: the basics

Predict: What does `stat.get("zzz")` return, and what does `stat["zzz"]` do?

```
stat = {"book": 480, "lamp": 3100}
stat["shoes"] = 1250
print(stat, stat["lamp"], stat.get("zzz"), stat.get("zzz", 0), len(stat))
try:
    stat["zzz"]
except KeyError as e:
    print("KeyError:", e)
print(list(stat), list(stat.values()), list(stat.items()))
print(stat.setdefault("pen", 0), stat)
print(stat.pop("pen"), stat.pop("nope", "none"))
print({"a": 1, "b": 2} | {"b": 3, "c": 4}, {**{"a": 1}, **{"a": 9}})
stat.update(lamp=2900)
del stat["book"]
print(stat, "lamp" in stat)
print(dict(zip("abc", [1, 2, 3])), dict.fromkeys(["a", "b"], 0))
print(sorted(stat.items(), key=lambda kv: kv[1]))
```

Real output:

```
{'book': 480, 'lamp': 3100, 'shoes': 1250} 3100 None 0 3
KeyError: 'zzz'
['book', 'lamp', 'shoes'] [480, 3100, 1250] [('book', 480), ('lamp', 3100),
('shoes', 1250)]
0 {'book': 480, 'lamp': 3100, 'shoes': 1250, 'pen': 0}
0 none
{'a': 1, 'b': 3, 'c': 4} {'a': 9}
{'lamp': 2900, 'shoes': 1250} True
{'a': 1, 'b': 2, 'c': 3} {'a': 0, 'b': 0}
[('shoes', 1250), ('lamp', 2900)]
```

Explanation:

- `d[k]` with a missing key gives `KeyError: 'zzz'`. `get` is the safe choice when "missing" is normal.
- `list(stat)` lists the keys (looping over a dict gives keys). `items()` gives pairs; use `for k, v in stat.items()`.
- In the merge, `b` appears in both dicts and the **right** one wins: `3`.
- `sorted(stat.items(), key=lambda kv: kv[1])` sorts pairs by value: 1250 before 2900. A dict itself has no sort; you sort its items.
- `"lamp" in stat` tests the **keys**, and is fast.

Worked example 4.3.2: grouping, the most common dictionary task

Problem 4.3.1 From the running table, group the elapsed times by item and compute the average per item.

Step 1	Start with an empty dict. For each `(item, mins)`, make sure the item has a list, then append `mins`. `setdefault(item, [])` does both in one line.
Step 2	After the loop, each item maps to its list: book → [480, 310, 290], lamp → [3100, 60], shoes → [1250].
Step 3	Average = sum / count. book: (480 + 310 + 290) / 3 = 1080 / 3 = 360. lamp: (3100 + 60) / 2 = 3160 / 2 = 1580. shoes: 1250 / 1 = 1250.
Answer	book averages 360 mins, lamp 1580 mins, shoes 1250 mins.
Check	The code prints exactly these values (`360.0`, `1580.0`, `1250.0`).
Why it matters	Interpretation: `lamp` has the worst average, but with only two samples (3100 and 60) the average hides a huge spread. Always look at the spread and the count, not just the average.

The code in 4.4 does the same with `defaultdict`.

The picture

```
key       value
"book"  ->  480
"lamp"    ->  3100          hash("lamp") picks the slot, so lookup is O(1)
"shoes" ->  1250
```

A dict is a two-column table where you can jump from a key to its row without scanning.

Real-world mapping

Dict idea	Example	Where it appears in the job
lookup table	`{"US": "United States"}`	Mapping codes to names
record	`{"id": 7, "name": "Asha"}`	One JSON object from an API
group-by	item → list of times	Aggregations before pandas
config	`{"timeout": 5}`	Settings passed into functions

Applications

- **Data Engineering:** every JSON document you read becomes dicts and lists.
- **Data Analytics:** `df.to_dict()`, and mapping values with `Series.map(dict)`.
- **Machine Learning:** hyper-parameter dictionaries; word-to-index vocabularies.
- **GenAI:** chat messages are lists of dicts (`{"role": ..., "content": ...}`).

COMMON MISTAKE

Changing the size of a dict while looping over it. Python stops you with `RuntimeError: dictionary changed size during iteration`. Loop over `list(d)` (a copy of the keys) or build a new dict. Another: using `d[k]` where `d.get(k)` is the right tool, and getting `KeyError` in production.

Check yourself

1. How do you read a key that may be missing, with a default of 0?

Answer: `d.get(key, 0)`.

1. In `{"a": 1} | {"a": 2}`, what is the result?

Answer: `{"a": 2}`. The right dict wins.

1. Does a dict keep insertion order?

Answer: Yes, since Python 3.7.

4.4 The `collections` Module and Friends

INTUITION

The built-in containers are hand tools. The `collections` module is the power-tool shelf: a dict that makes missing lists by itself (`defaultdict`), a tally counter (`Counter`), and a queue that is fast at both ends (`deque`).

Formal definition

Tool	What it is	Use it for
`defaultdict(factory)`	dict that creates a default value for a missing key by calling `factory`	grouping, counting
`Counter(iterable)`	dict subclass that counts items; `most_common(n)`	frequency tables
`deque(maxlen=n)`	double-ended queue; O(1) at both ends; optional fixed size	queues, sliding windows
`OrderedDict`	dict with extra order tools like `move_to_end`	LRU-style tricks (a normal dict already keeps order)
`heapq`	keeps the smallest item first (a **heap**)	top-N, schedulers
`bisect`	binary search in a sorted list	insert while keeping sorted order

Worked example 4.4.1: defaultdict, Counter, deque, heapq, bisect

Predict: After appending 0 to 4 into a `deque(maxlen=3)`, which numbers remain?

```
from collections import defaultdict, Counter, deque, OrderedDict
rows = [("book",480),("lamp",3100),("book",310),("lamp",60),("shoes",1250),
("book",290)]
dd = defaultdict(list)
for l, v in rows:
    dd[l].append(v)
print(dict(dd))
print({k: sum(v)/len(v) for k, v in dd.items()})
c = Counter(l for l, _ in rows)
print(c, c.most_common(2), c["book"], c["zzz"])
print(Counter("abracadabra").most_common(3), Counter(a=3) + Counter(a=1, b=2))
q = deque(maxlen=3)
for x in range(5):
    q.append(x)
print(q)
print(q.popleft())
print(q)
q.appendleft(9)
print(q)
q.rotate(1)
print(q)
import heapq, bisect
h = [5, 1, 8, 3]
heapq.heapify(h)
print(heapq.heappop(h), heapq.nsmallest(2, [5, 1, 8, 3]), heapq.nlargest(2, [5,
1, 8, 3]))
srt = [10, 20, 30, 40]
print(bisect.bisect_left(srt, 25), bisect.bisect_right(srt, 30))
bisect.insort(srt, 25)
print(srt)
```

Real output:

```
{'book': [480, 310, 290], 'lamp': [3100, 60], 'shoes': [1250]}
{'book': 360.0, 'lamp': 1580.0, 'shoes': 1250.0}
Counter({'book': 3, 'lamp': 2, 'shoes': 1}) [('book', 3), ('lamp', 2)] 3 0
[('a', 5), ('b', 2), ('r', 2)] Counter({'a': 4, 'b': 2})
deque([2, 3, 4], maxlen=3)
2
deque([3, 4], maxlen=3)
deque([9, 3, 4], maxlen=3)
deque([4, 9, 3], maxlen=3)
1 [1, 3] [8, 5]
2 3
[10, 20, 25, 30, 40]
```

- `defaultdict(list)` creates an empty list the first time a key appears, so `dd[l].append(v)`

needs no `setdefault`. The result is the same grouping as 4.3.1.

- `Counter` counts: `book` 3 times, `lamp` twice, `shoes` once. A missing key gives `0` (not an error). `most_common(2)` gives the top two. Counters can be added. In `"abracadabra"` the letter `a` appears 5 times (a-b-r-**a**-c-**a**-d-**a**-b-r-**a**: 5), `b` 2 times, `r` 2 times.
- A `deque` with `maxlen=3` keeps only the last 3 items: 2, 3, 4. This is a **sliding window**, handy for "average of the last 3 samples". `rotate(1)` moves items one place to the right.
- `heapq.heappop` returns the smallest item (1). `nsmallest(2, ...)` is `[1, 3]`, `nlargest(2, ...)` is `[8, 5]`.
- `bisect_left(srt, 25)` gives index 2: the place where 25 would go in the sorted list. `insort` inserts it there and keeps the list sorted.

The picture

```
deque(maxlen=3)        append(3) ->  [1,2,3]
oldest falls out  <--  append(4) ->  [2,3,4]   1 dropped from the left
```

Real-world mapping

Tool	Example	Where it appears in the job
`Counter`	count status codes per test run	Quick value counts, error summaries
`defaultdict`	group lines by session id	ETL grouping
`deque(maxlen)`	rolling window of last 100 delivery times	Moving averages, rate limits
`heapq.nlargest`	10 slowest deliveries	Top-N reports

Applications

- **Data Analytics:** `Counter` for quick frequency checks before using pandas.
- **Data Engineering:** `deque` as an in-memory buffer; `heapq` for merging sorted streams.
- **Machine Learning:** `Counter` for word counts and class balance.
- **Operations Analytics:** rolling windows of delivery times to detect a slow-down.

COMMON MISTAKE

Reading `defaultdict` entries just to test them (`if d[k]:`) creates the key as a side effect. Use `k in d` or `d.get(k)` for checks. Another: using `sort()` after every insert; use `heapq` or `bisect.insort` .

Check yourself

1. What does `Counter("aab")["z"]` return?

Answer: `0` . A missing key counts as zero.

1. What is the oldest item of `deque([1,2,3], maxlen=3)` after `append(4)` ?

Answer: `1` is dropped, so the deque is `[2, 3, 4]` .

1. When is `defaultdict(list)` better than `dict.setdefault` ?

Answer: When you group many items: it avoids repeating `setdefault` and reads more clearly.

4.5 Comprehensions and Generator Expressions

INTUITION

A comprehension is a factory line in one sentence: "from this input, take each item, keep only those that pass the test, transform them, and put them in a new container". It replaces a four-line loop with one line that reads like English.

Formal definition

- **List comprehension:** `[expression for item in iterable if condition]` .
- **Set comprehension:** `{expression for ...}` . **Dict comprehension:** `{key: value for ...}` .
- **Generator expression:** `(expression for ...)` . It does not build a container. It produces items **one at a time on demand** (Chapter 7), using almost no memory.
- A **nested comprehension** has two `for` parts. Read it left to right, like nested loops, with the outer loop first.
- The loop variable of a comprehension stays inside it (it does not leak, unlike a normal `for` loop).

Worked example 4.5.1: the common shapes

Predict: In `{l: m for l, m in zip(items, mins)}` , the item `book` appears twice. Which value

does the dict keep?

```
mins = [480, 1250, 3100, 310, 60]
print([x / 1000 for x in mins])
print([x for x in mins if x > 400])
print(["slow" if x > 1000 else "ok" for x in mins])
items = ["book", "shoes", "lamp", "book", "lamp"]
print({l: m for l, m in zip(items, mins)})
print(sorted({l.upper() for l in items}))
print([(i, j) for i in range(3) for j in range(i)])
matrix = [[1, 2, 3], [4, 5, 6]]
print([x for row in matrix for x in row], [[r[i] for r in matrix] for i in
range(3)])
x = 10
res = [x for x in range(3)]
print(x)
print([x for x in mins if x > 100 and x < 2000], [y for x in mins if (y := x //
100) > 10])
```

Real output:

```
[0.48, 1.25, 3.1, 0.31, 0.06]
[480, 1250, 3100]
['ok', 'slow', 'slow', 'ok', 'ok']
{'book': 310, 'shoes': 1250, 'lamp': 60}
['BOOK', 'LAMP', 'SHOES']
[(1, 0), (2, 0), (2, 1)]
[1, 2, 3, 4, 5, 6] [[1, 4], [2, 5], [3, 6]]
10
[480, 1250, 310] [12, 31]
```

- The condition goes at the **end** for filtering (`if x > 400`). A `if ... else ...` that **chooses a value** goes at the **front** (`"slow" if x > 1000 else "ok"`).
- Later keys overwrite earlier ones: `book` was first 480 and then 310; the dict keeps `310` . Duplicated keys silently lose data, so check for duplicates before building a dict from pairs.
- The nested comprehension `[(i, j) for i in range(3) for j in range(i)]` is the same as two loops: for each `i` , `j` goes from 0 up to `i − 1` . That gives `(1,0)` , `(2,0)` , `(2,1)` .
- Flatten a matrix with `[x for row in matrix for x in row]` (outer loop first), and transpose with `[[r[i] for r in matrix] for i in range(3)]` .
- `x` is still `10` after the comprehension that uses `x` : the comprehension has its own scope.
- The last line uses the walrus `:=` (Chapter 2) to compute `x // 100` once and also filter on it.

Worked example 4.5.2: generator expressions save memory

Predict: If I run `sum(gen)` twice on the same generator, will both give 30?

```
gen = (x * x for x in range(5))
print(type(gen).__name__, sum(gen), sum(gen))
import sys
print(sys.getsizeof([x for x in range(100000)]), sys.getsizeof(x for x in
range(100000)))
print(max(mins, default=None), max([], default=None), any(x > 3000 for x in
mins), all(x > 50 for x in mins))
```

Real output:

```
generator 30 0
800984 200
3100 None True True
```

- A generator can be used **once**. After `sum(gen)` consumed it, the second `sum(gen)` finds nothing and gives `0` . This is a classic surprise. 0 + 1 + 4 + 9 + 16 = 30 for the first call, which checks out.
- The list of 100,000 numbers takes about 800 KB for the list object (`800984` bytes; the exact figure depends on your Python version). The generator object is about 200 bytes, whatever the size of the range. The generator makes numbers only when asked.
- `any(...)` and `all(...)` stop as soon as the answer is known. With a generator expression they do not build a list first. `max(..., default=None)` avoids a `ValueError` on empty input.

The picture

```
[ f(x) for x in data if test(x) ]
  |          |          |
what goes   where it   which items
in (output) comes from  pass (filter)

equals:   out = []
          for x in data:
              if test(x): out.append(f(x))
```

Real-world mapping

Form	Example	Where it appears in the job
list comp	`[float(x) for x in row]`	Converting a column of text to numbers

dict comp	`{name: i for i, name in enumerate(vocab)}`	Building lookup tables
set comp	`{u.lower() for u in users}`	Deduplicating with cleaning
generator	`sum(len(l) for l in lines)`	Streaming huge files

Applications

- **Data Analytics:** clean and convert lists before creating a DataFrame.
- **Data Engineering:** stream lines of a huge file lazily with a generator expression.
- **Machine Learning:** build index maps, one-hot dictionaries, feature lists.
- **Interviews:** most Python coding tasks are shorter with comprehensions.

COMMON MISTAKE

Writing a comprehension so long or nested that nobody can read it (three `for` s and two `if` s). If it does not fit in one readable line, use a normal loop. Also: using a comprehension just for its side effects (`[print(x) for x in data]`), which builds a useless list.

Check yourself

1. Where does the filter `if` go in a list comprehension?

Answer: At the end: `[x for x in data if x > 0]` .

1. Why does the second `sum(gen)` print 0?

Answer: A generator is used up after one pass.

1. How do you flatten `[[1,2],[3]]` with a comprehension?

Answer: `[x for row in data for x in row]` .

4.6 `itertools` and `functools` : Ready-Made Loop Tools

INTUITION

Before you write a loop, check whether the toolbox already has a tool. `itertools` has tools for walking and combining sequences. `functools` has tools for treating functions as values (caching, fixing arguments, folding a list into one value).

Formal definition

`itertools` tool	What it does
`chain(a, b)`	walk `a` then `b` as one sequence
`islice(it, n)`	take the first `n` items of any iterator
`count(start)`, `cycle(x)`	endless counters / repeaters (use with `islice`)
`product(a, b)`	every combination, one from each (like nested loops)
`combinations(x, r)`, `permutations(x)`	choose `r` items, order ignored / order matters
`accumulate(x, func)`	running totals (or running max)
`pairwise(x)`	neighbouring pairs (Python 3.10 and newer)
`batched(x, n)`	chunks of size `n` (Python 3.12 and newer)
`zip_longest(a, b, fillvalue=)`	zip that does not stop at the shorter input
`groupby(x, key)`	group **adjacent** equal keys

`functools` tool	What it does
`lru_cache` / `cache`	remember results of a function by its arguments
`partial(f, ...)`	a new function with some arguments pre-filled
`reduce(f, x, start)`	fold a sequence into one value by applying `f` repeatedly

Worked example 4.6.1: itertools

Predict: `groupby` on an **unsorted** list of items `book, lamp, book`. How many groups come out: 2 or 3?

```
from itertools import chain, groupby, product, combinations, permutations,
islice, accumulate, count, cycle, zip_longest, pairwise, batched
print(list(chain([1,2],[3],"ab")), list(islice(count(10), 3)),
list(islice(cycle("ab"), 5)))
print(list(product("ab", [1,2])), list(combinations("abc", 2)),
len(list(permutations("abc"))))
print(list(accumulate([1,2,3,4])), list(accumulate([3,1,4], max)),
list(pairwise([1,4,9,16])))
print(list(zip_longest([1,2,3], "ab", fillvalue="-")), list(batched(range(7),
3)))
rows = [("book",480),("book",310),("lamp",3100),("lamp",60),("shoes",1250)]
for key, grp in groupby(rows, key=lambda r: r[0]):
    print(key, [v for _, v in grp])
unsorted = [("book",480),("lamp",3100),("book",310)]
print([(k, len(list(g))) for k, g in groupby(unsorted, key=lambda r: r[0])])
```

Real output:

```
[1, 2, 3, 'a', 'b'] [10, 11, 12] ['a', 'b', 'a', 'b', 'a']
[('a', 1), ('a', 2), ('b', 1), ('b', 2)] [('a', 'b'), ('a', 'c'), ('b', 'c')] 6
[1, 3, 6, 10] [3, 3, 4] [(1, 4), (4, 9), (9, 16)]
[(1, 'a'), (2, 'b'), (3, '-')] [(0, 1, 2), (3, 4, 5), (6,)]
book [480, 310]
lamp [3100, 60]
shoes [1250]
[('book', 1), ('lamp', 1), ('book', 1)]
```

- `accumulate([1,2,3,4])` gives running totals `1, 3, 6, 10`. With `max` it gives a running maximum: `3, 3, 4`.
- `pairwise` gives neighbours. Differences between neighbours (`b - a`) are easy to get from it, for example the change between consecutive measurements.
- `product("ab", [1,2])` has 2 x 2 = 4 pairs. `combinations("abc", 2)` has 3 (`ab`, `ac`, `bc`). `permutations("abc")` has 3! = 6.
- `groupby` only groups **neighbours**. With sorted data it gives 3 groups. With the unsorted list it gives **three** groups, and `book` appears twice. **Always sort by the key first**, or use `defaultdict`. This is the number-one `groupby` bug.

Worked example 4.6.2: functools

```
from functools import reduce, cache, partial
print(reduce(lambda a, b: a + b, [1,2,3,4]), reduce(max, [3,9,2]),
reduce(lambda a, b: a*b, range(1,6), 1))
@cache
def sq(n):
    print("computing", n); return n*n
print(sq(4), sq(4))
print(sq.cache_info())
hexint = partial(int, base=16)
print(hexint("ff"), hexint.func.__name__, hexint.keywords)
```

Real output:

```
10 9 120
computing 4
16 16
CacheInfo(hits=1, misses=1, maxsize=None, currsize=1)
255 int {'base': 16}
```

- `reduce` folds: `1+2+3+4 = 10`, the biggest of `3, 9, 2` is 9, and `1*2*3*4*5 = 120` (that is 5!). The start value `1` in the last call is the "nothing yet" value.
- `@cache` ran the function body once ("computing 4" printed once) and served the second call from memory. `cache_info()` shows 1 hit and 1 miss.
- `partial(int, base=16)` is a new function that reads hexadecimal: `ff` is 255 (15 x 16 + 15). `lru_cache(maxsize=N)` is the same as `cache` but forgets old entries, so memory stays bounded.

The picture

```
 groupby on sorted rows:   book book | lamp lamp | shoes      -> 3 groups
 groupby on UNsorted rows: book | lamp | book                 -> 3 groups (book
twice!)
```

Real-world mapping

Tool	Example	Where it appears in the job
`product`	all combinations of 3 learning rates and 4 batch sizes	Grid search for models
`batched` /	send 1000 rows at a time to an API	Batch loading

`islice`		
`accumulate`	running total of revenue	Cumulative metrics
`cache`	cache an expensive lookup	Speeding up repeated calls
`chain`	read several files as one stream	ETL

Applications

- **Machine Learning:** `product` to generate hyper-parameter grids; `combinations` for feature pairs.
- **Data Engineering:** `batched` for chunked inserts; `chain.from_iterable` to join file streams.
- **Data Analytics:** `accumulate` for running totals; `pairwise` for period-over-period changes.
- **Backend / APIs:** `lru_cache` for config and lookup caching.

COMMON MISTAKE

Using `groupby` on unsorted data (see above). Also: using `@cache` on a function with a changing result (like "current time" or a database read). It will return the old answer forever. Cache only pure functions: same inputs, same output.

Check yourself

1. How many items does `list(combinations("abcd", 2))` have?

Answer: 6 (`4 x 3 / 2`).

1. What must you do before `groupby` ?

Answer: Sort the data by the same key.

1. What does `partial(int, base=2)("101")` return?

Answer: `5` .

4.7 A Tour of Other Standard-Library Modules

INTUITION

Python ships with "batteries included": hundreds of ready modules that need no installation. A professional first asks "is there a standard module for this?" before writing code or installing a package. This section is a map, not a manual. You will use the modules marked in later chapters.

The map

Module	What it gives you	Chapter where you go deeper
`math`	`sqrt`, `ceil`, `floor`, `log10`, `pi`, `gcd`, `isclose`	this one
`statistics`	`mean`, `median`, `stdev`, `quantiles`	this one
`random`	random numbers; `seed` makes them repeatable	this one
`time`	`perf_counter` (timing), `sleep`, `time` (seconds since 1970)	11
`datetime`, `zoneinfo`	dates, times, time zones	5
`os`, `sys`, `pathlib`, `shutil`, `glob`, `tempfile`	files, paths, the system	5
`json`, `csv`, `sqlite3`	data formats and a small database	5, 12
`re`, `string`	text patterns	3
`collections`, `itertools`, `functools`, `heapq`, `bisect`, `copy`	data tools	1, 4
`logging`, `unittest`, `argparse`, `subprocess`	professional tooling	6, 10, 12
`threading`, `multiprocessing`, `concurrent.futures`, `asyncio`	doing many things at once	11
`urllib`, `http`	web access and a tiny web server	12
`hashlib`, `uuid`	fingerprints (hashes) and unique ids	13

Worked example 4.7.1: a mini delivery report with `statistics`

Problem 4.7.1 Ten delivery times (mins): `[480, 1250, 3100, 310, 60, 290, 700, 220, 95, 540]`. Find the mean, the median and the 95th percentile.

Formula	mean = sum / count. The median is the middle value of the sorted list (average of the two middle ones when the count is even). A **percentile** is the value below which that percent of the data falls. The `inclusive` method puts the p-th percentile at position `(n − 1) × p` in the sorted list and blends the two neighbours.
Plug in	The sum is 480 + 1250 + 3100 + 310 + 60 + 290 + 700 + 220 + 95 + 540 = 7045 and n = 10. Sorted list: `[60, 95, 220, 290, 310, 480, 540, 700, 1250, 3100]`.
Compute	mean = 7045 / 10 = 704.5. Median = (310 + 480) / 2 = 395. For p95: position = (10 − 1) × 0.95 = 8.55 (counting from 0). The values at positions 8 and 9 are 1250 and 3100. So p95 = 1250 + 0.55 × (3100 − 1250) = 1250 + 1017.5 = 2267.5.
Answer	Mean 704.5 mins, median 395 mins, p95 2267.5 mins.
Check	The code below prints the same three numbers, and `statistics.stdev` gives 910.6.
Why it matters	The mean (704.5) is nearly twice the median (395) because one very late delivery (3100) pulls it up. The p95 shows the tail that customers feel. This is why service reports use percentiles, such as "95% of orders arrive within X minutes".

```
import math, statistics
mins = [480, 1250, 3100, 310, 60, 290, 700, 220, 95, 540]
print(math.sqrt(144), math.ceil(2.1), math.floor(-2.1), math.log10(1000),
math.gcd(12, 18))
print(statistics.mean(mins), statistics.median(mins),
round(statistics.stdev(mins), 1))
print(statistics.quantiles(mins, n=20, method="inclusive")[18])
```

Real output:

```
12.0 3 -3 3.0 6
704.5 395.0 910.6
2267.5
```

`quantiles(mins, n=20)` cuts the data into 20 equal parts and returns the 19 cut points, so index 18 is the 95th percentile. Different tools use slightly different percentile methods (NumPy, pandas and spreadsheets each have their own default), so the numbers can differ in small ways. Always state your

method.

Worked example 4.7.2: random, time, uuid, hashlib

Predict: If you call `random.seed(42)` before two `randint` calls, will they give the same number?

```
import random, time, uuid, hashlib
random.seed(42); a = random.randint(1, 100)
random.seed(42); b = random.randint(1, 100)
print(a == b, len(random.sample(range(100), 5)))
t0 = time.perf_counter(); time.sleep(0.01); el = time.perf_counter() - t0
print(el >= 0.01)
u = uuid.uuid4(); print(len(str(u)), u.version)
print(hashlib.sha256(b"hello").hexdigest()[:16],
hashlib.md5(b"hello").hexdigest())
```

Real output:

```
True 5
True
36 4
2cf24dba5fb0a30e 5d41402abc4b2a76b9719d911017c592
```

- Same seed, same "random" numbers. That is how you make an experiment or a test **repeatable**. `random` is not safe for passwords or tokens (use `secrets`, Chapter 13).
- `time.perf_counter()` is the right clock for timing code, and `sleep(0.01)` really waited at least 10 ms.
- A `uuid4` is a random 36-character id (`version` 4). The chance of two being equal is negligible.
- `sha256(b"hello")` always gives the same fingerprint, starting `2cf24dba5fb0a30e`. The MD5 value of `hello` is `5d41402abc4b2a76b9719d911017c592` (a well-known value). MD5 is broken for security, but still fine for a quick checksum.

Real-world mapping

Module	Example	Where it appears in the job
`statistics`	mean, median, p95 of delivery times	Quick reports without installing anything
`random.seed`	reproducible train/test split	Machine learning experiments
`uuid`	give each request a trace id	Logging and distributed tracing
`hashlib`	fingerprint a file or a row	Data deduplication and change detection

Applications

- **Data Analytics:** check a number by hand with `statistics` before trusting a library.
- **Data Engineering:** row hashes to detect changed records; uuids as keys.
- **Machine Learning:** seeds for reproducibility.
- **Service Operations:** `perf_counter` for timing code, percentiles for service-level promises.

COMMON MISTAKE

Reporting only the average delivery time. Averages hide tails. Always report the median and a high percentile (p90, p95, p99) together with the sample count. Another: using `random` for secrets, or `time.time()` to measure short durations (use `perf_counter`).

Check yourself

1. Why is the mean larger than the median in our ten values?

Answer: One very large value (3100) pulls the mean up, but the median ignores how large the extremes are.

1. What does `random.seed(1)` do?

Answer: It sets the starting point of the random sequence, so the same numbers repeat on every run.

1. Which clock should you use to time a piece of code?

Answer: `time.perf_counter()`.

Chapter recap

- A **list** is an ordered, changeable sequence. A **tuple** is fixed. `sort()` changes in place and returns `None`; `sorted()` returns a new list.
- A **set** keeps unique items and tests membership fast (hash table). Use `set()` for an empty one and never rely on its order.
- A **dict** maps keys to values, keeps insertion order, and looks up by key in constant time. Use `get` for safe reads and `|` to merge.
- `defaultdict` for grouping, `Counter` for counting, `deque` for queues and windows, `heapq` for top-N, `bisect` for sorted inserts.
- Comprehensions build lists, sets and dicts in one readable line. The filter `if` goes at the end; a choosing `if/else` goes at the front.

- Generator expressions produce items lazily, use almost no memory, and can be used only once.
- `itertools` and `functools` replace many hand-written loops. `groupby` needs sorted input.
- Pick the container by the question you ask most often: order, uniqueness, or lookup by key.
- The standard library already has `math`, `statistics`, `random`, `uuid` and `hashlib`. Report median and percentiles, not only the mean, and seed random numbers when you need repeatability.

Need	Tool
Ordered, changeable	`list`
One fixed record	`tuple` / `namedtuple`
Unique + fast `in`	`set`
Lookup by key	`dict`, `d.get(k, default)`
Group by key	`defaultdict(list)`
Count items	`Counter(items).most_common(n)`
Sliding window / queue	`deque(maxlen=n)`
Smallest/largest n	`heapq.nsmallest(n, x)` / `nlargest`
Filter and transform	`[f(x) for x in data if test(x)]`
Running totals	`itertools.accumulate`
Mean, median, p95	`statistics.mean`, `median`, `quantiles(x, n=20)[18]`
Cache a function	`@functools.cache`

Practice set

Problem 4.P.1 (easy) What is `[1, 2, 3][::-1]`?

Answer: `[3, 2, 1]`. The step `-1` walks backwards.

Problem 4.P.2 (easy) What does `len({1, 2, 2, 3, 3, 3})` return?

Answer: `3`. A set keeps each value once.

Problem 4.P.3 (easy) Write a dict comprehension that maps each word in `["a", "bb", "ccc"]` to

its length.

Answer: `{w: len(w) for w in ["a", "bb", "ccc"]}` gives `{'a': 1, 'bb': 2, 'ccc': 3}`.

Problem 4.P.4 (what is the output?)

```
a = [3, 1, 2]
b = a.sort()
print(a, b)
```

Answer: `[1, 2, 3] None`. `sort()` changes `a` and returns `None`.

Problem 4.P.5 (what is the output?)

```
d = {"x": 1}
d2 = d
d2["y"] = 2
print(d, len(d))
```

Answer: `{'x': 1, 'y': 2} 2`. `d2` is the same dict (Chapter 1), not a copy. Use `d.copy()` for a copy.

Problem 4.P.6 (computation) Use `Counter` to find the most common word in `"to be or not to be"`.

Answer: `Counter("to be or not to be".split()).most_common(1)` gives `[('to', 2)]`. In this text `to` and `be` both appear twice, and `most_common` lists equal counts in the order first seen, so `to` is first.

Problem 4.P.7 (computation) Given `mins = [480, 1250, 3100, 310, 60]`, write one line that gives the share of values above 400 as a percentage.

Answer: `sum(1 for x in mins if x > 400) / len(mins) * 100`. Three values (480, 1250, 3100) are above 400, so 3 / 5 * 100 = 60.0.

Problem 4.P.8 (find the bug)

```
users = ["a", "b", "a"]
for u in users:
    if users.count(u) > 1:
        users.remove(u)
```

Answer: The loop changes the list while looping (Chapter 2 mistake), so items can be skipped. Deduplicate with `list(dict.fromkeys(users))`, which keeps the first occurrence and the order,

giving `['a', 'b']`.

Problem 4.P.9 (find the bug) Why is this slow on a million rows?

```
seen = []
for row_id in ids:
    if row_id not in seen:
        seen.append(row_id)
```

Answer: `row_id not in seen` scans the whole list every time, so the work grows with the square of the data size. Use `seen = set()` and `seen.add(row_id)`, where each test is constant time.

Problem 4.P.10 (scenario) You have 5 million log lines and need the 10 most frequent IP addresses. Which tools do you use and why?

Answer: Stream the lines with a generator and count with `Counter(ip for ip in ips_generator)`, then `.most_common(10)`. A generator keeps memory small; `Counter` counts in one pass.

Problem 4.P.11 (interview) "What is the difference between a list and a tuple, and when would you choose each?"

Answer: A list is mutable, a tuple is immutable (and hashable if its items are). Choose a list for a collection that grows or changes, and a tuple for a fixed record, a dict key, or a value that must not change by mistake. Tuples are also slightly smaller and faster to create.

Problem 4.P.12 (interview) "How does a dict find a key so quickly, and what is required of the keys?"

Answer: It uses a hash table: it computes the hash of the key to find the slot directly, so lookup takes constant time on average. Keys must be hashable (immutable, with a stable hash), and equal keys must have equal hashes. That is why lists cannot be keys.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles, and English is my second language, so use simple words and
short sentences. Teach me the four Python containers: list, tuple, set, dict.
For each, explain with an everyday analogy, show when to choose it, and give a
small runnable example using delivery-time data. Ask me to predict outputs
before showing them, then give me three questions to check my understanding.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles, with English as a second language. Topic: Python dictionaries and sets and why lookups are fast (hash tables). Explain hashing in simple words, then give me 8 graded exercises about grouping, counting, deduplicating and merging data using dict and set. Do not give solutions at first. When I paste my answers, correct them and explain each mistake step by step.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Teach me list, set and dict comprehensions and generator expressions in Python. Show how to turn a normal loop into a comprehension step by step, explain when NOT to use one, and explain why a generator can be used only once. Then quiz me with 6 short "what is the output?" questions, one at a time, waiting for my answer each time.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles, with English as a second language. Teach me the Python collections module (defaultdict, Counter, deque, OrderedDict), heapq and bisect, plus itertools (groupby, product, combinations, accumulate, batched) and functools (cache, partial, reduce). For each tool give one realistic data-engineering use case and a runnable example. Then review this code of mine and suggest better tools: [PASTE HERE]
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Run a mock Python interview on collections: list vs tuple, mutability, set operations, dict internals, comprehensions, the groupby-needs-sorted-input trap, and time complexity of list vs set lookups. Ask one question at a time, score each answer from 1 to 5, give a model answer, and end with my top three weak areas.
```

CHAPTER 5

FILES, PATHS, JSON, CSV, DATES AND TIME ZONES

Real programs read data from disk, talk in JSON and CSV, and live in a world of clocks and time zones. This chapter makes you safe in all four.

Imagine your manager hands you a folder of 400 daily log files, a partner's CSV with quotes inside the cells, an API reply in JSON, and a question: "How many orders did we get yesterday in New York time?" Before you can use any analysis tool, you must get the data in, get the text apart correctly, and get the clock right. This chapter gives you those four skills: files and paths, JSON, CSV, and dates with time zones. We finish with how to keep settings and secrets out of your code. By the end you can read and write files safely, convert between JSON and Python objects, parse a CSV with tricky cells, and do date maths without the classic daylight saving traps.

Running example for this chapter: a small online shop. It has an `orders.csv` file (order id, city, amount), JSON logs with delivery times in minutes, and servers in India and New York.

Every example in this chapter was run with `python3` (Python 3.14). The output shown is the real output. Folders are created with `tempfile` (a module that makes folders that delete themselves), so nothing is left on your computer. For each example, try the **Predict** step first: cover the output, guess it, then compare.

5.1 Files and Paths with `pathlib`

INTUITION

A file path is an address. If you write the address by hand as a string, you must remember that Windows uses `\` and Mac/Linux use `/`. `pathlib` is a small address book that does this for you. Think of a courier given a delivery address: if the address is wrong, the whole delivery fails. In Python it is the same, so we handle paths with care.

How it works

- `Path("data") / "raw" / "x.csv"` builds a path. The `/` operator joins parts with the right separator for your computer.
- `p.name` is the file name, `p.stem` is the name without extension, `p.suffix` is the extension, `p.parent` is the folder.
- `p.exists()`, `p.is_file()`, `p.is_dir()` ask questions about the disk.
- `p.read_text(encoding="utf-8")` and `p.write_text(...)` read or write a whole small file

in one call.

- `p.read_bytes()` and `p.write_bytes(...)` do the same for binary data (images, zip files).
- `open(p, mode, encoding=...)` gives a file object. Modes: `"r"` read (default), `"w"` write (erases old content!), `"a"` append, `"b"` binary (add to the others, like `"rb"`).
- Always use `with open(...) as f:` so the file is closed even if an error happens (more in Chapter 7).
- Always pass `encoding="utf-8"` for text. Without it, Python uses the computer's default, and your code may behave differently on another machine.
- `p.rglob("*.log")` finds files in all sub-folders. `p.glob("*.log")` looks in one folder only. The older `glob.glob(pattern)` returns strings.
- `os` is the older module (`os.path.join`, `os.listdir`, `os.environ`). `shutil` copies and deletes whole trees (`shutil.copy`, `shutil.rmtree`, `shutil.move`). `tempfile` makes temporary files and folders that clean up after themselves.
- `sys.argv` is the list of command-line words. `sys.argv[0]` is the script name.

Words you need

Word	Plain meaning
path	The address of a file or folder on disk
encoding	The rule that turns letters into bytes (UTF-8 is the standard that handles every language)
byte	One number from 0 to 255; files are made of bytes
mode	The word that tells `open` what you want: read, write, append

Item	Example value	Where it appears in the job
Path	`data/raw/orders_2026-11-06.csv`	Daily batch jobs name files by date
Suffix	`.parquet`, `.csv`, `.log`	Pipelines pick a reader by the suffix
Mode `"a"`	append to `app.log`	Log files that grow all day
Temp folder	`/tmp/tmpab12`	Tests that must not touch real data

Worked examples

Problem 5.1.1 Create a temporary folder with a `logs` sub-folder, write two log files, then inspect and

read them.

Step 1	Use `tempfile.TemporaryDirectory()` so the folder vanishes at the end.
Step 2	Build paths with `/`, write text, and ask for name, stem, suffix, size.
Step 3	Find files with `rglob` and `glob.glob`, and read line by line with `enumerate`.

Predict: Before you run it, how many bytes do you think `app.log` has? (Hint: count letters and newline characters.)

```
from pathlib import Path
import tempfile, shutil, glob, os

with tempfile.TemporaryDirectory() as d:
    root = Path(d)
    (root / "logs").mkdir()
    (root / "logs" / "app.log").write_text("line1\nline2\nline3\n",
encoding="utf-8")
    (root / "logs" / "old.log").write_text("x\n", encoding="utf-8")
    (root / "notes.txt").write_text("hello", encoding="utf-8")
    p = root / "logs" / "app.log"
    print(p.name, p.stem, p.suffix, p.parent.name)
    print(p.exists(), p.is_file(), p.stat().st_size)
    print(sorted(x.name for x in root.rglob("*.log")))
    print(sorted(os.path.basename(x) for x in glob.glob(str(root / "logs" /
"*.log"))))
    with open(p, encoding="utf-8") as f:
        for n, line in enumerate(f, start=1):
            print(n, line.rstrip())
```

```
app.log app .log logs
True True 18
['app.log', 'old.log']
['app.log', 'old.log']
1 line1
2 line2
3 line3
```

- **Explain** Line by line: `mkdir()` makes the folder; `write_text` makes and fills a file; `stat().st_size` is the size in bytes; `rglob("*.log")` searches the folder and everything under it for names ending in `.log`; `enumerate(f, start=1)` numbers the lines from 1; `rstrip()` removes the newline at the end so `print` does not add a blank line.
- **Answer** The file is 18 bytes: three lines of 5 letters plus a newline each is 3 × 6 = 18. Reading with `for line in f` gives one line at a time, so even a 10 GB file would not fill memory.

- **Check** `"line1\nline2\nline3\n"` has 5+1+5+1+5+1 = 18 characters, and all are ASCII, so bytes equal characters. The output 18 matches.
- **Why it matters** The same loop works on a 3-line file and a 30 GB log, so your code does not need to change when data grows.

The typical wrong version. You open a file that is not there:

```
open("/no/such/dir/data.csv", encoding="utf-8")
```

```
FileNotFoundError: [Errno 2] No such file or directory: '/no/such/dir/data.csv'
```

The fix is to check the path first (`Path(x).exists()`), or to catch `FileNotFoundError` and give a clear message (Chapter 6).

Problem 5.1.2 Copy a file, delete a folder, append twice to a file, and write raw bytes.

Step 1	`shutil.copy(src, dst)` copies one file. `shutil.rmtree(folder)` deletes a folder and everything inside it. There is no recycle bin, so be careful.
Step 2	Open with mode `"a"` twice. The second open does not erase the first write.
Step 3	Write `bytes([72, 105, 255])`. 72 is `H`, 105 is `i`, and 255 is not a printable ASCII letter.

Predict: What does `a.txt` contain after two appends? Will the first `"one"` survive?

```
from pathlib import Path
import tempfile, shutil, os

with tempfile.TemporaryDirectory() as d:
    root = Path(d)
    (root / "logs").mkdir()
    (root / "logs" / "app.log").write_text("line1\n", encoding="utf-8")
    (root / "notes.txt").write_text("hello", encoding="utf-8")
    shutil.copy(root / "logs" / "app.log", root / "backup.log")
    print(sorted(x.name for x in root.iterdir()))
    shutil.rmtree(root / "logs")
    print(sorted(x.name for x in root.iterdir()))
    with open(root / "a.txt", "a", encoding="utf-8") as f:
        f.write("one\n")
    with open(root / "a.txt", "a", encoding="utf-8") as f:
        f.write("two\n")
    print(repr((root / "a.txt").read_text()))
    (root / "b.bin").write_bytes(bytes([72, 105, 255]))
    print((root / "b.bin").read_bytes())
print(Path("data") / "raw" / "x.csv")
print(os.sep, os.path.join("data", "raw", "x.csv"))
```

```
['backup.log', 'logs', 'notes.txt']
['backup.log', 'notes.txt']
'one\ntwo\n'
b'Hi\xff'
data/raw/x.csv
/ data/raw/x.csv
```

- **Explain** `shutil.copy` made `backup.log`. After `rmtree` only `backup.log` and `notes.txt` remain. `repr(...)` shows the hidden `\n` characters so you can see them.
- **Answer** After `rmtree`, the `logs` folder is gone. Both appends are kept (`'one\ntwo\n'`). Binary data prints as `b'Hi\xff'`, where `\xff` means the single byte 255. On Windows, the last two lines would show `data\raw\x.csv` and `\`.
- **Check** Two writes of 4 characters each should give 8 characters: `one\n` + `two\n`. The repr shows exactly that.
- **Why it matters** Mode `"w"` here would have left only `'two\n'`. One wrong letter in a log writer can erase a day of evidence.

The picture

```
Path("data") / "raw" / "x.csv"
        |           |        |
      folder      folder     file
 p.parent -> data/raw     p.name -> x.csv
 p.stem   -> x            p.suffix -> .csv
```

Task	pathlib	older style
join	`Path("a") / "b"`	`os.path.join("a","b")`
exists	`p.exists()`	`os.path.exists(p)`
list folder	`p.iterdir()`	`os.listdir(p)`
find by pattern	`p.rglob("*.csv")`	`glob.glob("**/*.csv", recursive=True)`
read all	`p.read_text()`	`open(p).read()`

Connection to the real job

Data engineers read thousands of files from folders: raw logs, daily exports, Parquet files. Pipelines list files with `rglob`, process them one at a time, and move finished ones with `shutil.move`. Think of a bank clerk working through a long stack of paper slips one at a time; the same loop-over-lines pattern applies, and `for line in f` is the safe way for huge files.

Applications

- **Data Analytics:** open the monthly export a business user emails you.
- **Data Engineering:** a nightly job lists `raw/2026-11-06/*.csv`, processes each file, and moves it to `done/`.
- **Machine Learning:** load thousands of image files from class folders (`cats/`, `dogs/`).
- **MLOps / Testing:** a unit test builds a temporary folder, runs the code, and deletes everything.

COMMON MISTAKE

Opening a file with mode `"w"` when you meant `"a"`. `"w"` erases the file at once, before you write anything. If you are unsure, write to a new file name first.

COMMON MISTAKE

Forgetting `encoding="utf-8"`. The script works on your laptop and breaks on the server when a name like "café" appears.

COMMON MISTAKE

Reading a huge file with `f.read()` or `readlines()`. That loads everything into memory. Loop over the file object instead.

Check yourself

1. What is `Path("reports/q1.final.csv").stem`? Answer: `q1.final` (only the last suffix `.csv` is removed).
2. Which mode adds to the end of a file without erasing it? Answer: `"a"`.
3. Why use `with open(...)`? Answer: it closes the file automatically, even when an error happens inside the block.

5.2 JSON

INTUITION

Two programs, maybe written in different languages, need to exchange data. They need a common written language. JSON is that language: plain text, easy for humans, and every language can read it. When a shop's website asks a courier service for a delivery status, the reply is almost always JSON.

Formal definition

JSON stands for JavaScript Object Notation, but it is not tied to JavaScript. It has only a few types. Here is how they map to Python:

JSON	Python
object `{}`	`dict`
array `[]`	`list`
string	`str`
number	`int` or `float`

`true` / `false`	`True` / `False`
`null`	`None`

Four functions matter:

- `json.dumps(obj)` turns a Python object into a JSON **string**. (s = string)
- `json.loads(text)` turns a JSON string into Python objects.
- `json.dump(obj, file)` and `json.load(file)` do the same with a file object.

Why "dumps" and "loads"? The `s` stands for *string*. "Dump" means "pour out into text", "load" means "read back in". Think of packing a suitcase (`dumps`) and unpacking it at the other end (`loads`).

Item	Example value	Where it appears in the job
JSON object	`{"id": 7, "delivery_minutes": 20}`	One API reply or one log event
JSON array	`[{"id":1},{"id":2}]`	A page of results from an API
`null`	`"manager": null`	A missing value in an event
JSON Lines	one object per line	Event logs in a data lake

Useful options: `indent=2` (pretty print), `sort_keys=True` , `ensure_ascii=False` (keep non-English letters readable), `default=function` (tell it how to handle types it does not know).

Worked examples

Problem 5.2.1 Convert a dictionary to JSON and back. Check what is preserved.

Step 1	Build a dict with a list, a bool, `None` , and a float.
Step 2	Call `dumps` , then `loads` , and compare with `==` .
Step 3	Notice that JSON uses lowercase `true` / `null` , and that dict keys become strings.

Predict: Will `back == d` be `True` ? What happens to the key `1` in `{1: "a"}` ?

```
import json
d = {"name": "Asha", "age": 29, "skills": ["python", "sql"],
     "active": True, "manager": None, "score": 91.5}
s = json.dumps(d)
print(s)
print(json.dumps(d, indent=2, sort_keys=True))
back = json.loads(s)
print(back == d, type(back["active"]), back["manager"])
print(json.loads('{"1": "a"}'), json.dumps({1: "a"}))
print(json.dumps((1, 2)), json.loads(json.dumps((1, 2))))
```

```
{"name": "Asha", "age": 29, "skills": ["python", "sql"], "active": true,
"manager": null, "score": 91.5}
{
  "active": true,
  "age": 29,
  "manager": null,
  "name": "Asha",
  "score": 91.5,
  "skills": [
    "python",
    "sql"
  ]
}
True <class 'bool'> None
{'1': 'a'} {"1": "a"}
[1, 2] [1, 2]
```

- **Explain** `dumps` writes `True` as `true` and `None` as `null`, because those are the JSON spellings. `sort_keys=True` orders the keys A to Z, which makes files easy to compare. `indent=2` adds line breaks and two-space indents.
- **Answer** Round trip kept equality. But two things changed on the way: the integer key `1` became the string `"1"`, and the tuple `(1, 2)` became a list `[1, 2]`. JSON has no tuple and no integer keys.
- **Check** `back == d` printed `True`, so for plain types nothing is lost. The tuple and key examples show the exceptions.
- **Why it matters** If you save `{1: "a"}` and read it back, `data[1]` raises `KeyError` because the key is now `"1"`. Surprises like this cause real bugs in pipelines.

Problem 5.2.2 Handle things JSON does not know: a `datetime`, a bad JSON string, non-English text.

Step 1	Dump a `datetime` and watch the `TypeError`.
Step 2	Fix it with `default=`, which is called for unknown objects.
Step 3	Load a string with single quotes, which is not valid JSON, and read the error.
Step 4	Compare `ensure_ascii` settings.

Predict: What error will the first `dumps` raise? Which Python type is it complaining about?

```
import json
from datetime import datetime
try:
    json.dumps({"when": datetime(2026, 1, 5)})
except TypeError as e:
    print("TypeError:", e)
print(json.dumps({"when": datetime(2026, 1, 5)}, default=lambda o:
o.isoformat()))
try:
    json.loads("{'a': 1}")
except json.JSONDecodeError as e:
    print("JSONDecodeError:", e)
print(json.dumps({"city": "Chennai", "word": "café"}))
print(json.dumps({"word": "café"}, ensure_ascii=False))
```

```
TypeError: Object of type datetime is not JSON serializable
{"when": "2026-01-05T00:00:00"}
JSONDecodeError: Expecting property name enclosed in double quotes: line 1
column 2 (char 1)
{"city": "Chennai", "word": "caf\u00e9"}
{"word": "café"}
```

- **Explain** `datetime` is not one of the six JSON types, so `dumps` stops. The `default` function is called only for objects it does not understand; here it returns the ISO text. The `JSONDecodeError` message tells you the line and column of the problem, so read it.
- **Answer** Dates must be turned into strings (ISO format is the standard choice). JSON requires double quotes. By default non-ASCII letters are escaped (for example `é` becomes `\u00e9`); both forms are valid JSON.
- **Check** The date `2026-01-05` became `2026-01-05T00:00:00`, with time at midnight, which matches what `datetime(2026,1,5)` means.
- **Why it matters** API responses often contain dates and prices. Knowing this error saves you an hour the first time it happens.

Problem 5.2.3 A JSON Lines log has 3 records. Find the average delivery time, and safely read a deep key.

Step 1	JSON Lines means one JSON object per line. Parse each line with `json.loads`.
Step 2	Average = (20 + 55 + 15) / 3 = 90 / 3 = 30.0.
Step 3	Use chained `.get(key, {})` so a missing key gives a default, not a crash.

```
import json
lines =
'{"id":1,"delivery_minutes":20}\n{"id":2,"delivery_minutes":55}\n{"id":3,"deliv
ery_minutes":15}\n'
rows = [json.loads(l) for l in lines.splitlines()]
print(sum(r["delivery_minutes"] for r in rows) / len(rows))
nested = {"user": {"address": {"city": "Pune"}}}
print(nested.get("user", {}).get("address", {}).get("zip", "n/a"))
```

```
30.0
n/a
```

- **Answer** The average is 30.0 minutes, and the missing `zip` safely returns `"n/a"`.
- **Check** 20 + 55 + 15 = 90, and 90 / 3 = 30. The code printed 30.0.
- **Why it matters** In plain words: the average delivery took 30 minutes. But look at the 55-minute one: an average hides slow outliers, which is why you also report percentiles.

The typical wrong version: `nested["user"]["address"]["zip"]` would stop the program with `KeyError: 'zip'`. Use `.get` with defaults when data may be incomplete.

The picture

```
Python object --dumps/dump--> text (JSON) --send/save--> other system
Python object <--loads/load-- text (JSON) <--receive/read--
        dict <-> {}     list <-> []     None <-> null
```

Connection to the real job

APIs return JSON. Config files, MLflow metadata, notebook files, and many logs are JSON. Event data in data lakes is often JSON Lines (one record per line), because you can append and stream it. In pandas you later use `pd.json_normalize` for nested data.

Applications

- **Data Analytics:** download JSON from a web service and flatten it into a table.
- **Data Engineering:** land event data as JSON Lines, one record per line, then load it into a warehouse.
- **Machine Learning:** save experiment settings and metrics as a small JSON file.
- **GenAI:** requests and replies of LLM APIs are JSON.

COMMON MISTAKE

Mixing up `dumps` / `loads` (strings) with `dump` / `load` (files). The `s` means string.

COMMON MISTAKE

Expecting tuples, sets, dates or integer keys to survive. Sets and dates raise errors; tuples and integer keys change silently.

COMMON MISTAKE

Using `data["a"]["b"]["c"]` on API data. One missing key crashes the job. Use `.get` with defaults, or catch `KeyError`.

Check yourself

1. Which function turns a JSON string into a dict? Answer: `json.loads`.
2. What does `None` become in JSON? Answer: `null`.
3. How do you save a `datetime` in JSON? Answer: convert to an ISO string (for example with `default=` and `isoformat()`).

5.3 CSV

INTUITION

CSV is a spreadsheet saved as text: one row per line, cells split by commas. It looks too simple to go wrong, but a city name like "Delhi, North" has a comma inside it. Splitting on commas by hand would cut it in two. The `csv` module handles quotes for you.

How it works

CSV means *comma-separated values*. Why a module and not `split(",")`? Because the format has rules: a cell that contains a comma, a quote or a line break must be wrapped in double quotes, and a

quote inside a cell is doubled. The module knows all these rules.

Item	Example value	Where it appears in the job
Header row	`order_id,city,amount`	Names the columns for `DictReader`
Quoted cell	`"Delhi, North"`	City names, addresses, free text
Delimiter	`;` or tab	European exports and `.tsv` files
Customer list	`customers.csv`	Sample data for testing a program

- `csv.reader(file)` gives each row as a list of strings.
- `csv.DictReader(file)` uses the header row and gives each row as a dict. Best for readable code.
- `csv.writer` and `csv.DictWriter` write rows and add quotes when needed.
- When opening a real file for CSV, use `open(path, newline="", encoding="utf-8")`. The `newline=""` stops blank lines on Windows.
- **Everything is read as a string.** You must convert: `float(row["amount"])`.
- Another delimiter (like `;` or tab) is set with `delimiter=";"`.

Here we use `io.StringIO`, which is a fake file in memory, so the examples run without any file on disk. A real file works the same way.

Worked examples

Problem 5.3.1 Read an orders CSV with a quoted comma. Compute the total amount and the total per city.

Step 1	Use `DictReader`. Row 3 has `"Delhi, North"` inside quotes, so it stays as one cell.
Step 2	Convert amounts with `float`. Total = 250.50 + 1200 + 75.25 = 1525.75.
Step 3	Use `defaultdict(float)` to sum per city. (A `defaultdict(float)` is a dict that creates `0.0` for a missing key. Chapter 4 explains it fully.)

Predict: What will `rows[2]["city"]` be? One cell or two?

```
import csv, io
from collections import defaultdict
text = """order_id,city,amount
1,Pune,250.50
2,Chennai,1200
3,"Delhi, North",75.25
"""
rows = list(csv.DictReader(io.StringIO(text)))
print(rows[2])
print(sum(float(x["amount"]) for x in rows))
for row in csv.reader(io.StringIO(text)):
    print(row)
agg = defaultdict(float)
for x in rows:
    agg[x["city"]] += float(x["amount"])
print(dict(agg))
```

```
{'order_id': '3', 'city': 'Delhi, North', 'amount': '75.25'}
1525.75
['order_id', 'city', 'amount']
['1', 'Pune', '250.50']
['2', 'Chennai', '1200']
['3', 'Delhi, North', '75.25']
{'Pune': 250.5, 'Chennai': 1200.0, 'Delhi, North': 75.25}
```

- **Explain** `StringIO` pretends to be a file. `DictReader` reads the header, then makes one dict per row. The quoted comma stays inside one cell. `csv.reader` shows the same data as plain lists. The loop adds each amount to its city's running total.
- **Answer** The total is 1525.75. Notice every value in the dict is still a string until we convert it. Pune brought 250.5, Chennai 1200.0, and Delhi, North 75.25.
- **Check** By hand: 250.50 + 1200 = 1450.50, and 1450.50 + 75.25 = 1525.75. The code agrees, and 250.5 + 1200.0 + 75.25 from the dict also equals 1525.75.
- **Why it matters** In plain words: Chennai gave about 79% of the revenue (1200 / 1525.75). That kind of "group and sum" is the heart of analytics.

The typical wrong version. You forget that CSV values are strings and add them directly:

```
total = 0
for r in rows:
    total += r["amount"]
```

```
TypeError: unsupported operand type(s) for +=: 'int' and 'str'
```

The fix is `total += float(r["amount"])`. And splitting by hand also fails: `"Delhi, North".split(",")` gives `['Delhi', ' North']`, two pieces, which is wrong.

Problem 5.3.2 Write rows that contain quotes and commas, change the delimiter, and meet the "numbers as text" trap.

Step 1	`DictWriter` quotes cells automatically. A quote inside a cell is doubled (`""`).
Step 2	Read `"a;b;c"` with `delimiter=";"`.
Step 3	Sort the text values `"10", "9", "100"` and see the wrong order, then sort with `key=int`.

```
import csv, io
out = io.StringIO()
w = csv.DictWriter(out, fieldnames=["order_id", "city", "amount"],
lineterminator="\n")
w.writeheader()
w.writerow({"order_id": 4, "city": 'Goa "Beach"', "amount": 10})
w.writerow({"order_id": 5, "city": "Kochi, Kerala", "amount": 20})
print(out.getvalue())
print(next(csv.reader(io.StringIO("a;b;c"), delimiter=";")))
print(sorted(["10", "9", "100"]), sorted(["10", "9", "100"], key=int))
```

```
order_id,city,amount
4,"Goa ""Beach""",10
5,"Kochi, Kerala",20

['a', 'b', 'c']
['10', '100', '9'] ['9', '10', '100']
```

- **Explain** `DictWriter` wrote the quote inside `Goa "Beach"` as `""` and wrapped both cells in quotes. `delimiter=";"` changed the separator. Sorting text compares the first character first: `"1"` is less than `"9"`, so `"10"` and `"100"` come before `"9"`.
- **Answer** Text sorts character by character ("10" < "100" < "9"), so convert to numbers first.
- **Check** Sorted as numbers, 9 < 10 < 100, which matches the second list.
- **Why it matters** A "top 5 orders" report that sorts text would show wrong orders and nobody would see an error message.

The picture

```
order_id,city,amount          ->  DictReader row:
3,"Delhi, North",75.25            {"order_id":"3",
     ^ comma inside quotes         "city":"Delhi, North",
       is NOT a separator          "amount":"75.25"}
```

Connection to the real job

CSV is the most common exchange format in analytics. You will load it with `pandas.read_csv`, but knowing the plain module helps when the file is too big, too messy, or you do not want to install pandas in a small script. Exports from a shop's order system are read the same way.

Applications

- **Data Analytics:** read Excel exports and BI tool downloads.
- **Data Engineering:** small ETL (extract, transform, load) jobs that clean a file and re-save it.
- **Software Testing:** build sample-data CSV files to feed automated tests.
- **Machine Learning:** read small training sets and label files.

COMMON MISTAKE

Doing `line.split(",")`. It breaks on quoted commas. Use the `csv` module.

COMMON MISTAKE

Forgetting that all CSV values are strings. `"250.50" + "1200"` joins text; it does not add.

COMMON MISTAKE

Wrong encoding on Excel files (they may be `utf-8-sig` or `cp1252`). If you see strange characters at the start, try `encoding="utf-8-sig"`.

Check yourself

1. What type is every value from `csv.reader`? Answer: `str`.
2. Which class gives dict rows? Answer: `csv.DictReader`.
3. What does the writer do with a cell containing a comma? Answer: it wraps the cell in double quotes.

5.4 Dates, Times and Time Zones

INTUITION

"9:00" means nothing until you say where. A test that starts at 9:00 in Chennai is 3:30 in the morning in London. Servers usually store time in UTC (Coordinated Universal Time, the world's reference clock), and show local time only when a person looks at it. Think of UTC as the one common ruler that every system measures against.

Words you need

Word	Plain meaning
UTC	The world's reference clock. No daylight saving. India is UTC+5:30
offset	How far a place is from UTC (for example +05:30)
time zone	A named region with its own offset rules, like `Asia/Kolkata`
daylight saving (DST)	Some places move clocks forward one hour in summer
timestamp	Seconds since 1 Jan 1970 UTC, as one number
ISO 8601	The standard text form: `2026-11-07T14:05:00+05:30`

Item	Example value	Where it appears in the job
ISO datetime	`2026-11-07T14:05:00+05:30`	API fields and log lines
Timestamp	`1767225600`	Timestamp column in a log export (some systems use milliseconds)
timedelta	`0:00:02.350000`	Delivery time or job duration
Time zone	`America/New_York`	Joining data from several regions

Formal definition

The `datetime` module has these classes:

- `date` : year, month, day.
- `datetime` : date plus time.
- `timedelta` : a length of time (days, seconds). Subtracting two dates gives a `timedelta` .

- `timezone.utc` : the UTC zone.
- `zoneinfo.ZoneInfo("Asia/Kolkata")` : a named time zone from the standard IANA database. It knows daylight saving rules.

A datetime is **naive** if it has no zone (`tzinfo` is empty) and **aware** if it has one. You cannot mix them in arithmetic.

Converting text and dates: `strptime(text, format)` parses text; `strftime(format)` makes text. Common codes: `%Y` year, `%m` month, `%d` day, `%H` hour (24h), `%M` minute, `%S` second, `%A` weekday name, `%B` month name. `datetime.fromisoformat(...)` reads ISO 8601 text such as `2026-11-07T14:05:00+05:30` .

Worked examples

Problem 5.4.1 Do date arithmetic and parsing.

Step 1	Make a date and ask for the weekday (`weekday()` gives Monday=0, so Sunday=6).
Step 2	Add `timedelta(days=20, hours=5)` to 15 March 2026 09:30.
Step 3	Count days from 7 Nov 2026 to 25 Dec 2026: 23 days left in November (30 - 7) plus 25 = 48.
Step 4	Parse `07/11/2026 14:05` with the day-first format.

Predict: What day of the week is 15 March 2026? (`weekday()` counts Monday as 0.)

```
from datetime import datetime, date, timedelta
d = date(2026, 3, 15)
print(d, d.weekday(), d.strftime("%A, %d %B %Y"))
dt = datetime(2026, 3, 15, 9, 30)
print(dt.isoformat(), dt + timedelta(days=20, hours=5))
print(date(2026, 12, 25) - date(2026, 11, 7))
p = datetime.strptime("07/11/2026 14:05", "%d/%m/%Y %H:%M")
print(p, p.year)
print(datetime.fromisoformat("2026-11-07T14:05:00+05:30"))
print(date(2026, 2, 28) + timedelta(days=1))
start = datetime(2026, 1, 1, 10, 0, 0)
end = datetime(2026, 1, 1, 10, 0, 2, 350000)
print((end - start).total_seconds(), end - start)
```

```
2026-03-15 6 Sunday, 15 March 2026
2026-03-15T09:30:00 2026-04-04 14:30:00
48 days, 0:00:00
2026-11-07 14:05:00 2026
2026-11-07 14:05:00+05:30
2026-03-01
2.35 0:00:02.350000
```

- **Explain** `strftime` turns a date into text using codes. `strptime` does the reverse, and its format must match the text exactly (`%d/%m/%Y` means day first). Adding a `timedelta` moves a date by a length of time. Subtracting two dates gives a `timedelta`.
- **Answer** The gap is 48 days, and 28 Feb 2026 plus one day is 1 March (2026 is not a leap year). `total_seconds()` gives a number you can use in delivery-time maths. 15 March 2026 is a Sunday.
- **Check** Days left: 30 - 7 = 23 in November, plus 25 in December = 48. And `2026 % 4 = 2`, so 2026 is not a leap year, so February has 28 days.
- **Why it matters** In plain words: a delivery-form step that took `0:00:02.350000` took 2.35 seconds, or 2350 ms. Converting to one unit makes it comparable with other results.

The typical wrong version. Parse a day-first date with the wrong format:

```
from datetime import datetime
datetime.strptime("07/11/2026", "%m/%d/%Y")   # reads 11 July
datetime.strptime("25/11/2026", "%m/%d/%Y")
```

```
ValueError: time data '25/11/2026' does not match format '%m/%d/%Y'
```

The first line does not fail, it silently reads 11 July instead of 7 November, which is worse than an error. Always know whether your source is day-first or month-first.

Problem 5.4.2 Convert 9:00 in India on 1 July 2026 to New York and UTC.

Step 1	India is UTC+5:30 all year. 09:00 IST is 03:30 UTC.
Step 2	New York in July is on daylight saving time, UTC-4. So 03:30 UTC is 23:30 on the previous day, 30 June.
Step 3	Let Python confirm with `astimezone`.

Predict: Which calendar day is it in New York when it is 09:00 on 1 July in India?

```
from datetime import datetime, timezone
from zoneinfo import ZoneInfo
ist = ZoneInfo("Asia/Kolkata")
ny = ZoneInfo("America/New_York")
t = datetime(2026, 7, 1, 9, 0, tzinfo=ist)
print(t, t.astimezone(ny), t.astimezone(timezone.utc))
```

```
2026-07-01 09:00:00+05:30 2026-06-30 23:30:00-04:00 2026-07-01 03:30:00+00:00
```

- **Explain** `tzinfo=ist` tells Python what zone the clock reading is in. `astimezone(zone)` keeps the instant and changes only how it is shown.
- **Answer** It is 23:30 on 30 June in New York. The same moment has three faces but is one instant.
- **Check** 09:00 - 5:30 = 03:30 UTC. 03:30 - 4:00 = -00:30, which is 23:30 of the previous day. Matches.
- **Why it matters** "Orders per day" for a New York customer differs from "orders per day" in India. You must decide the zone before you group by day.

Problem 5.4.3 Show the daylight saving trap and the naive/aware error.

Step 1	In New York, clocks jumped forward on 8 March 2026 at 02:00 (to 03:00). So 02:30 does not exist that day.
Step 2	Take 01:30 and add one hour with plain wall-clock arithmetic. Python gives 02:30, a time that never happened.
Step 3	Convert both to UTC and subtract. This is the safe way to measure real time passed.
Step 4	Try subtracting a naive and an aware datetime.

```
from datetime import datetime, timedelta, timezone
from zoneinfo import ZoneInfo
ny = ZoneInfo("America/New_York")
a = datetime(2026, 3, 8, 1, 30, tzinfo=ny)
b = a + timedelta(hours=1)
print(a, b, b.utcoffset())
print(b.astimezone(timezone.utc) - a.astimezone(timezone.utc))
try:
    datetime(2026, 1, 1) - datetime(2026, 1, 1, tzinfo=timezone.utc)
except TypeError as e:
    print("TypeError:", e)
print(datetime.fromtimestamp(1_800_000_000, tz=timezone.utc))
print(datetime(2026, 1, 1, tzinfo=timezone.utc).timestamp())
```

```
2026-03-08 01:30:00-05:00 2026-03-08 02:30:00-05:00 -1 day, 19:00:00
1:00:00
TypeError: can't subtract offset-naive and offset-aware datetimes
2027-01-15 08:00:00+00:00
1767225600.0
```

- **Explain** Python's `+ timedelta(hours=1)` moves the *clock reading* by one hour and does not check if that reading exists. Only converting to UTC reveals the true time passed.
- **Answer** `-1 day, 19:00:00` is how Python prints a negative offset of -5 hours. The "02:30" result is a wall-clock time that does not exist in real New York time, so the lesson is: do arithmetic in UTC and convert to local only for display. A Unix timestamp (seconds since 1 Jan 1970 UTC) is a plain number, for example 1767225600.0 for 1 Jan 2026.
- **Check** UTC difference printed `1:00:00` ; the offset `-1 day, 19:00:00` equals -24h + 19h = -5h. Correct for New York before DST starts.
- **Why it matters** Twice a year, hourly reports from daylight saving regions get 23 or 25 hours in a day. Using UTC avoids this.

For timing code (how long did this take), use `time.perf_counter()` , which is a steady clock that does not jump. Do not use dates for that.

The picture

```
Store/compute in UTC:   2026-07-01 03:30 UTC   <- one real instant
Show to people:   Kolkata 09:00 (+5:30)   New York 23:30 prev day (-4:00)

naive (no zone)  --X-->  mixing with aware = TypeError
```

Connection to the real job

Logs from servers in different regions need one clock before you can join them. Daily aggregations ("orders per day") depend on which time zone defines "day". Time-series features for ML (hour of day, day of week) are wrong if time zones are mixed. Timestamps from many systems are stored as seconds or milliseconds since 1970 (a timestamp).

Applications

- **Data Analytics:** daily, weekly and monthly reports with a clear time zone for "day".
- **Data Science:** time features such as hour of day and weekday for forecasting.
- **Data Engineering:** schedulers (cron, Airflow) run jobs in UTC, and partitions are named by date.
- **Performance Testing:** line up test start and end times with server monitoring graphs.

COMMON MISTAKE

Using naive datetimes and hoping everyone means the same zone. Make datetimes aware, and store UTC.

COMMON MISTAKE

Adding timedelta to local aware times near a daylight saving change. Convert to UTC first.

COMMON MISTAKE

Writing `%m` when you mean minutes. Month is `%m`, minute is `%M`.

Check yourself

1. What does `date(2026,3,1) - date(2026,2,1)` give? Answer: `timedelta(days=28)`.
2. Why store times in UTC? Answer: it is one common ruler without daylight saving jumps.
3. What error appears when you subtract naive from aware? Answer: `TypeError: can't subtract offset-naive and offset-aware datetimes`.

5.5 Configuration: Environment Variables, `.env` Files, YAML and TOML

INTUITION

The same program runs on your laptop, a test server, and production. The database address is different in each. If you type the address inside the code, you must edit code for each place. Instead, keep settings outside the code and let each machine provide its own. This is like a restaurant kitchen reading the day's specials from a board instead of rewriting the recipe book.

How it works

Words you need: an *environment variable* is a named text value that the operating system attaches to every program it starts (like a sticky note on the program's desk). A *secret* is a value that must stay private, such as a password or API key.

Item	Example value	Where it appears in the job
Env variable	`DB_HOST=db.internal`	Docker, Kubernetes, GitHub Actions

`.env` file	`API_KEY=fake-key-123`	Your laptop during development
TOML file	`[db] port = 5432`	`pyproject.toml` and tool settings
YAML file	`db:` then `port: 5432`	Airflow, Kubernetes, CI pipelines

Three common places for settings, from most to least specific:

1. **Environment variables**: named values the operating system gives to a program. In Python: `os.environ.get("NAME", "default")`. `os.environ["NAME"]` raises `KeyError` if it is missing (good when the setting is required).
2. `.env` **file**: a text file of `NAME=value` lines used on a developer machine. A helper library (`python-dotenv`) loads it into the environment. The file must be listed in `.gitignore` and **never committed**, because it often holds secrets.
3. **Config files**: YAML (needs the third-party `PyYAML` package, not run here), TOML (`tomllib` is in the standard library from Python 3.11, read-only), or JSON.

Environment variables are always strings. Convert: `int(os.environ.get("DB_PORT", "5432"))`.

Worked examples

Problem 5.5.1 Write a small `.env` loader, read the file, and let a real environment variable override it.

Step 1	Skip blank lines and `#` comments. Split each line at the first `=`.
Step 2	Strip quotes around values.
Step 3	Convert the port to an `int`. Then override `APP_ENV` from the real environment. The key below is fake.

Predict: After we set `os.environ["APP_ENV"] = "prod"`, which value of `APP_ENV` wins, `dev` from the file or `prod`?

```
import os, tempfile
from pathlib import Path

def load_dotenv(path):
    env = {}
    for line in Path(path).read_text().splitlines():
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        k, v = line.split("=", 1)
        env[k.strip()] = v.strip().strip('"').strip("'")
    return env

with tempfile.TemporaryDirectory() as d:
    f = Path(d) / ".env"
    f.write_text('#
demo\nAPP_ENV=dev\nDB_HOST="localhost"\nDB_PORT=5432\nAPI_KEY=fake-key-123\n')
    cfg = load_dotenv(f)
print(cfg)
port = int(os.environ.get("DB_PORT", cfg["DB_PORT"]))
print(port + 1)
os.environ["APP_ENV"] = "prod"
merged = {**cfg, **{k: v for k, v in os.environ.items() if k in cfg}}
print(merged["APP_ENV"], merged["DB_HOST"])
try:
    os.environ["REQUIRED_X"]
except KeyError as e:
    print("KeyError", e)
```

```
{'APP_ENV': 'dev', 'DB_HOST': 'localhost', 'DB_PORT': '5432', 'API_KEY': 'fake-
key-123'}
5433
prod localhost
KeyError 'REQUIRED_X'
```

- **Explain** `split("=", 1)` cuts at the first `=` only, so values may contain `=`. The `{**cfg, **other}` merge keeps `cfg` first and lets the second dict overwrite same keys. `os.environ["X"]` fails loudly, `os.environ.get("X")` returns `None`.
- **Check** 5432 + 1 = 5433, matching the output. `merged["APP_ENV"]` is `prod`, so the override worked, and `DB_HOST` stayed `localhost`.
- **Why it matters** The same code now runs in dev and prod with no edits, and the secret is never in the code.
- **Answer** The environment value `prod` beat the file value `dev`, which is the usual rule: real environment wins over `.env`. The required missing variable fails loudly with `KeyError`.

Problem 5.5.2 Read a TOML config and understand what a YAML file looks like.

Step 1	`tomllib.loads` parses a TOML string into nested dicts.
Step 2	YAML expresses the same data with indentation (shown as text only; PyYAML is not installed here, so we do not run it).

```
import tomllib
print(tomllib.loads('[db]\nhost="h"\nport=5432\n'))
```

```
{'db': {'host': 'h', 'port': 5432}}
```

```
# not run here: needs PyYAML (yaml.safe_load)
db:
  host: h
  port: 5432
```

- **Answer** Both forms give `{"db": {"host": "h", "port": 5432}}`. If you use PyYAML, always call `yaml.safe_load`, never plain `yaml.load` on untrusted text.

The picture

```
code  <-- reads --  environment variables (set by server, CI, Docker)
                    .env file (laptop only, in .gitignore)
                    config.toml / config.yaml (non-secret defaults)
Priority:  env var  >  .env  >  config file  >  default in code
```

Connection to the real job

Every deployed data pipeline, API and ML service takes its database URL, bucket name and API keys from the environment. Docker, Kubernetes, GitHub Actions and Airflow all inject settings as environment variables. Chapter 13 covers secrets in depth.

The typical wrong version. Trusting that an environment value is a bool:

```
import os
print(bool(os.environ.get("DEBUG_X", "False")))
```

```
True
```

The text `"False"` is a non-empty string, so it is truthy. Compare the text instead:

`os.environ.get("DEBUG_X", "false").lower() == "true"`.

Applications

- **Data Engineering:** pipeline settings (bucket names, schedule) differ per environment.
- **MLOps:** model-serving containers receive their model path and API keys as env variables.
- **Machine Learning:** notebooks read API keys with `os.environ` instead of pasting them in cells.
- **Performance Testing:** point the same test script at dev, QA or staging with one variable.

COMMON MISTAKE

Committing `.env` to Git. Add `.env` to `.gitignore` before the first commit, and commit a `.env.example` with fake values instead.

COMMON MISTAKE

Forgetting that environment values are strings. `os.environ.get("DEBUG")` returns `"False"`, which is truthy!

COMMON MISTAKE

Hard-coding a "temporary" password in code. It always stays.

Check yourself

1. What type is `os.environ["PORT"]`? Answer: `str`.
2. What does `os.environ["X"]` do if `X` is missing? Answer: raises `KeyError`.
3. Which should win: `.env` or a real environment variable? Answer: the real environment variable.

Chapter recap

- Use `pathlib.Path` and the `/` operator for paths; it works on every operating system.
- Always open text files with `encoding="utf-8"` and use `with` so they close.
- Loop over a file object line by line for big files; do not read it all at once.
- `"w"` erases, `"a"` appends. `shutil` copies and deletes trees; `tempfile` cleans up after itself.
- JSON maps to dict, list, str, number, bool, None. `dumps/loads` work on strings, `dump/load` on files.
- Tuples become lists in JSON, integer keys become strings, and dates need `default=` or conversion.

- The `csv` module handles quotes and commas; every value arrives as a string.
- Store and compute time in UTC with aware datetimes; convert to local time only for display.
- Do arithmetic across daylight saving changes in UTC, and time code with `time.perf_counter()`.
- Keep settings in environment variables or config files, never in code, and never commit `.env`.

Cheat-sheet

Task	Pattern
Build a path	`Path("a") / "b.csv"`
Read / write small text	`p.read_text(encoding="utf-8")`, `p.write_text(...)`
Loop over big file	`with open(p, encoding="utf-8") as f: for line in f:`
Find files	`root.rglob("*.log")`
JSON to / from string	`json.dumps(obj)`, `json.loads(text)`
CSV rows as dicts	`csv.DictReader(f)` then `float(row["x"])`
Aware time	`datetime(..., tzinfo=ZoneInfo("Asia/Kolkata"))`
Change zone	`dt.astimezone(ZoneInfo("America/New_York"))`
Setting with default	`os.environ.get("NAME", "default")`

Practice set

Problem 5.P.1 What is `Path("/data/raw/sales_2026.csv").suffix`?

Answer: `.csv`.

Problem 5.P.2 What is the output? `json.dumps({"a": (1, 2), "b": None})`

Answer: `{"a": [1, 2], "b": null}`.

Problem 5.P.3 What does `date(2026, 3, 1) - date(2026, 2, 1)` print?

Answer: `28 days, 0:00:00`.

Problem 5.P.4 Find the bug: `total = 0` then `for row in csv.DictReader(f): total += row["amount"]`.

Answer: `row["amount"]` is a string, so `0 + "250"` raises `TypeError`. Use `float(row["amount"])`.

Problem 5.P.5 What is the output? `sorted(["b10", "b9", "b100"])`

Answer: `['b10', 'b100', 'b9']`, because strings compare character by character.

Problem 5.P.6 Convert 18:00 UTC on 1 January 2026 to India time. How would you write it?

Answer: India is UTC+5:30, so it is 23:30 the same day. In code: `datetime(2026,1,1,18,0,tzinfo=timezone.utc).astimezone(ZoneInfo("Asia/Kolkata"))`.

Problem 5.P.7 Find the bug: `with open("log.txt", "w") as f:` was used to add one line to an existing log.

Answer: Mode `"w"` erases the file when opened. Use `"a"`.

Problem 5.P.8 Scenario: a daily job fails on Mondays only, with `UnicodeDecodeError`. The file comes from a partner. What do you check first?

Answer: The file encoding. Open with `encoding="utf-8"` and check the raw bytes; the partner's Monday export may use another encoding (such as `cp1252`) or add a BOM (try `utf-8-sig`). Log the file name when it fails.

Problem 5.P.9 Write a function that returns the number of lines in a text file without loading all of it.

Answer:

```
def count_lines(path):
    with open(path, encoding="utf-8") as f:
        return sum(1 for _ in f)
```

Problem 5.P.10 Interview: why should servers store timestamps in UTC?

Answer: UTC has no daylight saving jumps, so ordering and subtraction are always correct. Different regions can be compared. Convert to the user's zone only when displaying.

Problem 5.P.11 Interview: your API response contains `datetime` and `Decimal` values and `json.dumps` fails. What are two ways to fix it?

Answer: Pass `default=` to `json.dumps` with a function that converts them (`isoformat()` for dates, `str` or `float` for `Decimal`), or convert the data to simple types before dumping.

Prompt pack

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles in India, and English is my second language, so use simple
words. Teach me Python file handling and paths: pathlib, open modes, encodings,
glob/rglob, shutil and tempfile. Explain step by step with small runnable
examples, then check my understanding with 5 short questions one at a time.
Wait for my answer before the next question and correct me kindly.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles in India, and English is my second language. Topic: Python JSON
handling with json.dumps, loads, dump, load, indent, default and ensure_ascii.
Give me 8 graded exercises from easy to hard, using realistic API responses and
JSON Lines logs. Do not give solutions yet. After I answer each one, I will
paste my code and you will review it for bugs and style.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles in India. Topic: reading and writing CSV in Python with the csv
module (reader, DictReader, writer, DictWriter), quoting, delimiters, encodings
and type conversion. Review my work: [PASTE HERE]. Find bugs, explain why they
are bugs in simple English, and show a cleaner version. Then give me one harder
follow-up task.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles in India, and English is my second language. Topic: Python
datetime, timedelta, strptime/strftime, zoneinfo, naive versus aware datetimes,
UTC, and daylight saving traps. Quiz me one question at a time with real
numbers (for example converting times between India, UTC and New York).
Increase difficulty slowly, and explain each wrong answer step by step.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles in India. Topic: configuration in Python using environment
variables, .env files, TOML and YAML, and keeping secrets out of Git. Run a
mock interview with me: ask 6 questions about configuration and secrets, one at
a time, grade each answer out of 10, and tell me what a strong answer would
include. End with a summary of my weak points.
```

CHAPTER 6

ERRORS, EXCEPTIONS, LOGGING AND DEBUGGING

Programs fail. Professionals are the people who know how to read the failure, handle it calmly, leave a trail in the logs, and find the cause fast.

Think of the worst moment of a big task: the program is half done and it dies with a wall of red text. If you can read that text, you fix the problem in two minutes. If you cannot, you lose the afternoon. In data work it is the same: a nightly job reads 2 million rows, row 1,400,001 has a strange value, and the job stops. This chapter teaches you four skills: reading a traceback, handling errors with `try/except`, writing your own errors, and leaving evidence with `logging`. Then you learn `pdb`, Python's built-in debugger, so you can stop a program in the middle and look inside it. By the end you can take any crash and walk to its cause step by step.

Running example: a small function that adds up shop prices like `"Rs100"`. We make it fail, read the failure, handle it, log it and debug it. All code was run with `python3` (Python 3.14). File paths in tracebacks (like `/private/tmp/w6/tb.py`) will be different on your computer.

6.1 Reading Tracebacks and the Common Exceptions

INTUITION

A traceback is like the receipt of a failed journey. It lists every road the program took, in order, and says where the car finally broke down. A beginner reads the top and panics. A professional reads the **last line first** (what broke), then the line above it (where it broke), then walks up the list (how we got there).

Words you need

Word	Plain meaning
exception	An object Python creates when something goes wrong; it stops the normal flow
raise	To create and throw an exception (Python does it for you, or you do it)
traceback	The printed list of function calls that led to the exception
call stack	The chain of "function A called B, B called C" at one moment
bug	A mistake in your code. An exception is how Python tells you about it

How to read one

Here is a real failure. `total` calls `parse_price` for each text. The third text is `"N/A"`.

Predict: Before you run it, which line of the three functions do you think will be named last in the traceback?

```
def parse_price(text):
    return float(text.replace("Rs", ""))

def total(prices):
    s = 0
    for p in prices:
        s += parse_price(p)
    return s

print(total(["Rs100", "Rs250", "N/A"]))
```

```
Traceback (most recent call last):
  File "/private/tmp/w6/tb.py", line 10, in <module>
    print(total(["Rs100", "Rs250", "N/A"]))
          ~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/private/tmp/w6/tb.py", line 7, in total
    s += parse_price(p)
         ~~~~~~~~~~~^^^
  File "/private/tmp/w6/tb.py", line 2, in parse_price
    return float(text.replace("Rs", ""))
ValueError: could not convert string to float: 'N/A'
```

- **Explain** Read from the bottom. The last line says the *type* (`ValueError`) and the *message* (`could not convert string to float: 'N/A'`). The block above it points to line 2 in `parse_price` : that is where Python stopped. The two blocks above that show the callers: `total` at line 7, and the main script at line 10. The `~~~~^^^^` marks show which part of the line failed (newer Python versions draw them).
- **Answer** The program failed in `parse_price` because `"N/A"` is not a number. The bad value is in the *input data*, but the failing line is in *our function*. The fix is to handle the case, which is the topic of 6.2.

The common exception types

Learn these ten by sight. Each line below was produced by running code and catching the error.

```
def attempt(f):
    try:
        print("ok:", f())
    except Exception as e:
        print(type(e).__name__, "->", e)

attempt(lambda: 1/0)
attempt(lambda: int("abc"))
attempt(lambda: [1,2,3][5])
attempt(lambda: {"a":1}["b"])
attempt(lambda: None.upper())
attempt(lambda: "5"+5)
attempt(lambda: undefined_name)
attempt(lambda: open("/no/such/file.txt"))
attempt(lambda: __import__("not_a_module"))
attempt(lambda: (1,2).append(3))
```

```
ZeroDivisionError -> division by zero
ValueError -> invalid literal for int() with base 10: 'abc'
IndexError -> list index out of range
KeyError -> 'b'
AttributeError -> 'NoneType' object has no attribute 'upper'
TypeError -> can only concatenate str (not "int") to str
NameError -> name 'undefined_name' is not defined
FileNotFoundError -> [Errno 2] No such file or directory: '/no/such/file.txt'
ModuleNotFoundError -> No module named 'not_a_module'
AttributeError -> 'tuple' object has no attribute 'append'
```

Exception	Usually means	Typical cause in data work
`ValueError`	Right type, wrong value	`int("abc")`, a bad cell in a CSV
`TypeError`	Wrong type	adding a string from a CSV to a number
`KeyError`	Missing dict key	an API reply without an expected field
`IndexError`	List position too big	an empty result list
`AttributeError`	Object has no such attribute	a function returned `None` and you used its result
`NameError`	Name not defined	typo in a variable name
`FileNotFoundError`	Path is wrong	wrong working folder

`ModuleNotFoundError`	Package not installed	wrong virtual environment (Chapter 9)
`ZeroDivisionError`	Divide by zero	average of an empty list

All of these are subclasses of `Exception`. Think of a family tree: `Exception` is the parent, `LookupError` is the parent of `KeyError` and `IndexError`, and so on. `except LookupError` catches both children.

The picture

```
<module> line 10  ──calls──▶  total line 7  ──calls──▶  parse_price line 2  ×
ValueError
   (oldest call)                                          (where it broke: look
here first,
                                                           then walk back up to
find who passed 'N/A')
```

Read the traceback *bottom to top*: the last line is the symptom, the frames above are the history.

Connection to the real job

In production you rarely sit next to the program. The traceback in a log file or an Airflow task page is often all you get. A teammate who reads it correctly fixes the job in minutes. It is like a hospital reading a patient's chart to find where a problem began; reading a traceback is the same skill.

Applications

- **Data Analytics:** a notebook cell dies with `KeyError: 'revenue'`; you check the column names.
- **Data Engineering:** an Airflow task fails with `FileNotFoundError`; you check whether the upstream job produced the file.
- **Machine Learning:** `ValueError: shapes not aligned` points to a matrix mismatch.
- **MLOps:** a deployed API returns 500; the error type in the logs tells you which input broke it.

COMMON MISTAKE

Reading from the top and getting lost in library code. Start at the last line, then find the *first* frame that is your own file.

COMMON MISTAKE

Fixing the line named in the traceback when the real problem is in the data or in a caller. Ask who passed the bad value.

COMMON MISTAKE

Copying only the last line when asking for help. Always paste the whole traceback.

Check yourself

1. In a traceback, which line tells you the exception type? Answer: the last line.
2. Which exception does `{"a": 1}["b"]` raise? Answer: `KeyError`.
3. `None.upper()` raises which error, and what does it usually mean? Answer: `AttributeError`; a variable you thought held text is `None`.

6.2 Handling Errors: `try`, `except`, `else`, `finally`

INTUITION

Picture a shop counter. Most customers pay normally. Sometimes a card is declined. A good cashier does not close the shop; she says "please try another card" and carries on. `try/except` lets your program react to a *known* kind of problem and keep working, while still stopping on problems you did not expect.

How it works

```
try:        code that might fail
except SomeError:   what to do if THAT error happens
else:       runs only if the try block had NO error
finally:    runs ALWAYS (cleanup: close, release, unlock)
```

- Catch **specific** exceptions. `except ValueError` is good. A bare `except:` or `except Exception:` hides bugs.
- You can catch several types: `except (ValueError, TypeError) as e:`. The `as e` gives you the exception object.
- `else` is for code that should run only on success. It keeps the `try` block short.
- `finally` runs even when there is a `return` or an error. Use it for cleanup.

- Python style prefers **EAFP**: "Easier to Ask Forgiveness than Permission" (just try, and catch the error). The other style is **LBYL**: "Look Before You Leap" (check first with `if`). EAFP is often shorter and avoids race conditions (the file could vanish between the check and the use).

Worked examples

Problem 6.2.1 Make `parse_price` safe: return `None` for bad text, and show what `else` and `finally` do.

Step 1	Wrap the risky call (`int(s)`) in `try`. Catch only `ValueError`.
Step 2	Put the "success" message in `else`.
Step 3	Put the "always" message in `finally`.

Predict: For the input `"x"`, does the `finally` message print? In what order do the lines appear?

```
def read_num(s):
    try:
        n = int(s)
    except ValueError:
        print("  bad number")
        return None
    else:
        print("  parsed", n)
        return n
    finally:
        print("  finally runs for", repr(s))

read_num("42")
read_num("x")
```

```
  parsed 42
  finally runs for '42'
  bad number
  finally runs for 'x'
```

- **Explain** For `"42"`, `int` works, so Python skips `except`, runs `else`, and then `finally`. For `"x"`, `int` raises `ValueError`, `except` runs and returns `None`, and `finally` still runs *before* the function actually returns.
- **Answer** `finally` ran in both cases, and after the `except` or `else` block.
- **Check** Count the lines: two calls produce two `finally` lines. They did.
- **Why it matters** Anything that must happen (closing a connection, releasing a lock) belongs in

`finally`, because it runs even on failure.

The typical wrong version. A bare `except:` hides real mistakes:

```
try:
    total = prise + 1      # typo: should be price
except:
    total = 0
```

This never shows an error, and the total is silently 0. Python would have told you `NameError: name 'prise' is not defined`. Catching everything turns a loud bug into a quiet one. The fix: `except ValueError:` (or the exact type you expect).

Problem 6.2.2 Run `100 // int(v)` for the inputs `"10"`, `"0"`, `"zz"` and `None`, and handle each kind of failure separately.

Step 1	`int("zz")` raises `ValueError`. `int(None)` raises `TypeError`. `100 // 0` raises `ZeroDivisionError`.
Step 2	Group the first two into one `except` with a tuple, because we treat them the same.
Step 3	Handle `ZeroDivisionError` on its own.

```
for v in ["10", "0", "zz", None]:
    try:
        print(100 // int(v))
    except (ValueError, TypeError) as e:
        print("input problem:", type(e).__name__)
    except ZeroDivisionError:
        print("zero")
```

```
10
zero
input problem: ValueError
input problem: TypeError
```

- **Answer** `100 // 10 = 10`. `"0"` gives `zero`. `"zz"` and `None` are input problems.
- **Check** `100 // 10` is 10 by hand. Each of the other three inputs hit exactly the branch we predicted.
- **Why it matters** In plain words: bad rows are separated by cause, so you can count them and report "3 bad numbers, 1 zero" instead of crashing.

Problem 6.2.3 Compare LBYL and EAFP for a dictionary lookup, and use the loop `else`.

Step 1	LBYL: `if "b" in d`. EAFP: `try ... except KeyError`.
Step 2	Both give the same answer here. EAFP is better when the check itself is as costly as the action, or when things change between check and use.
Step 3	A `for ... else` runs its `else` only if the loop did not `break`.

```
d = {"a": 1}
if "b" in d:
    x = d["b"]
else:
    x = 0
try:
    y = d["b"]
except KeyError:
    y = 0
print(x, y)

for n in [3, 5, 7]:
    if n % 2 == 0:
        print("found even"); break
else:
    print("no even")
```

```
0 0
no even
```

- **Answer** Both styles gave `0`. The loop printed `no even` because no number was even, so `break` never ran.
- **Check** 3, 5, 7 are all odd: `n % 2` is 1 each time, so the condition is never true.
- **Why it matters** For simple dict defaults you will use `d.get("b", 0)` instead; both patterns are good to know for interviews.

The picture

```
try block ──no error──▶ else block ──▶ finally ──▶ continue
    │
  error ──▶ matching except block ──▶ finally ──▶ continue
    │
  no match ──▶ finally ──▶ error keeps travelling up to the caller
```

Connection to the real job

ETL jobs process millions of rows. A good job catches the expected row-level problem (`ValueError` on a bad cell), counts it, and moves on, but lets unexpected problems (a database outage) stop the job loudly. Retry loops (Chapter 7) are built from `try/except` .

Applications

- **Data Engineering:** skip and quarantine bad rows; always close database connections in `finally` .
- **Data Analytics:** cleaning scripts that convert messy text to numbers and report the failures.
- **Machine Learning:** a training loop that catches a corrupt image file and continues.
- **API / Backend:** return a clear 400 message for bad input instead of crashing with a 500.

COMMON MISTAKE

`except Exception: pass` . You are hiding the problem. At least log it (6.4).

COMMON MISTAKE

Putting a lot of code inside `try` . Then you do not know which line failed. Keep `try` small, and use `else` for the rest.

COMMON MISTAKE

Catching an exception and then using a variable that was never set (because the failing line was the one that set it).

Check yourself

1. When does `else` in a try block run? Answer: only if no exception happened in `try` .
2. When does `finally` run? Answer: always, even after `return` or an unhandled error.
3. What does EAFP stand for? Answer: Easier to Ask Forgiveness than Permission.

6.3 Raising, Custom Exceptions and Chaining

INTUITION

Sometimes your code finds the problem, not Python. A function that receives an age of -3 must refuse it with a clear message. Raising your own exception is like a quality inspector stopping the production line and writing a note about *why*.

How it works

- `raise ValueError("age must be >= 0")` throws an exception you choose.
- A bare `raise` inside an `except` block re-throws the *same* exception. Use it after logging.
- A **custom exception** is a small class that inherits from `Exception`. Give your project one base error (`AppError`) and children for each case. Callers can then catch the whole family or one member.
- **Chaining:** `raise NewError("...") from e` links the new error to the old cause. The traceback then says "The above exception was the direct cause of the following exception", so nothing is lost. Without `from`, Python still links them as `__context__`, but with the message "During handling of the above exception, another exception occurred".
- `assert condition, "message"` raises `AssertionError` if the condition is false. Use it for checks of your own assumptions in tests and quick scripts, not for validating user input (Python can run with asserts switched off).
- `e.add_note("text")` adds extra hints to an exception (Python 3.11 and later). `ExceptionGroup` and `except*` handle several errors that happened together (3.11 and later; you will meet them with `asyncio`).

Worked examples

Problem 6.3.1 Create `AppError` and `ValidationError`, raise one, and catch it through the base class.

Step 1	Define `class AppError(Exception)` as the base.
Step 2	Define `ValidationError(AppError)` that stores the `field`.
Step 3	Catch `AppError` (the parent) and read the child's `field`.

Predict: The `except` names `AppError`, but we raise `ValidationError`. Will it be caught?

```
class AppError(Exception):
    """Base error for our app."""

class ValidationError(AppError):
    def __init__(self, field, message):
        super().__init__(f"{field}: {message}")
        self.field = field

def validate(rec):
    if rec.get("age", 0) < 0:
        raise ValidationError("age", "must be >= 0")
    return rec

try:
    validate({"age": -3})
except AppError as e:
    print(type(e).__name__, e, e.field)
```

```
ValidationError age: must be >= 0 age
```

- **Explain** `ValidationError` is a kind of `AppError`, so `except AppError` catches it. `super().__init__(...)` sets the message that `print(e)` shows (the chapter on classes explains `super()` fully). We added our own attribute `field` so callers can know *which* field failed.
- **Answer** It was caught, and we could read both the message and `e.field`.
- **Check** The printed class name is `ValidationError` (the child), proving that a parent `except` catches children.
- **Why it matters** In plain words: your API can turn any `AppError` into a clean "400 Bad Request" and show `field` to the user.

Problem 6.3.2 Translate a low-level error into a clear one and keep the original cause.

Step 1	Parse a bad JSON string. Python raises `JSONDecodeError`.
Step 2	Catch it, and raise `AppError("config is not valid JSON") from e`.
Step 3	Print the cause, then print the full traceback to see the chain.

```
import json, traceback

def load(text):
    try:
        return json.loads(text)
    except json.JSONDecodeError as e:
        raise AppError("config is not valid JSON") from e

try:
    load("{bad")
except AppError as e:
    print(e, "| cause:", type(e.__cause__).__name__)
    traceback.print_exc()
```

```
config is not valid JSON | cause: JSONDecodeError
Traceback (most recent call last):
  File "/private/tmp/w6/c.py", line 19, in load
    return json.loads(text)
           ~~~~~~~~~~^^^^^^
  File ".../json/__init__.py", line 352, in loads
    return _default_decoder.decode(s)
    ...
json.decoder.JSONDecodeError: Expecting property name enclosed in double
quotes: line 1 column 2 (char 1)

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "/private/tmp/w6/c.py", line 23, in <module>
    load("{bad")
  File "/private/tmp/w6/c.py", line 21, in load
    raise AppError("config is not valid JSON") from e
AppError: config is not valid JSON
```

The `...` lines are shortened by hand to save space; everything else is the real output.

- **Answer** The new error has a business meaning ("config is bad") and the old technical error is kept as its cause.
- **Check** `e.__cause__` is a `JSONDecodeError`, matching the first line of output.
- **Why it matters** You get friendly messages for users *and* full detail for the engineer reading the log.

Problem 6.3.3 Re-raise after logging, add a note, use `assert`, and handle an `ExceptionGroup`.

Step 1	In `except`, do your action (print "log"), then a bare `raise`.
Step 2	`assert 1 + 1 == 3, "math broke"` raises `AssertionError` with your text.
Step 3	`raise ExceptionGroup("batch", [ValueError("a"), TypeError("b")])` and catch with `except*`.

```
def risky():
    try:
        1/0
    except ZeroDivisionError:
        print("log and re-raise")
        raise

try:
    risky()
except ZeroDivisionError as e:
    print("caught again:", e)

try:
    assert 1 + 1 == 3, "math broke"
except AssertionError as e:
    print("AssertionError:", e)

try:
    raise ExceptionGroup("batch", [ValueError("a"), TypeError("b")])
except* ValueError as eg:
    print("VE", eg.exceptions)
except* TypeError as eg:
    print("TE", eg.exceptions)

try:
    try:
        int("x")
    except ValueError as e:
        e.add_note("while reading row 7")
        raise
except ValueError as e:
    print(e.__notes__)
```

```
log and re-raise
caught again: division by zero
AssertionError: math broke
VE (ValueError('a'),)
TE (TypeError('b'),)
['while reading row 7']
```

- **Answer** Each tool did its job. The caller still saw the `ZeroDivisionError` after our logging.
- **Check** `1 + 1 == 3` is false, so the assert fired, as it should.
- **Why it matters** The note "while reading row 7" tells you *where in the data* the failure happened, which `int("x")` alone never says.

The picture

```
ValueError (low-level)  ──from e──▶  AppError (meaningful)
      ▲ __cause__ keeps the link          │
                                          ▼
                          caught by the caller that understands AppError

Exception
  └─ AppError
       ├─ ValidationError
       └─ DatabaseError
```

Connection to the real job

Libraries like `requests`, `pydantic` and `sqlalchemy` each have their own exception families. Your own services should too, so that a failure in a data-quality check is different from a database outage and gets a different reaction (skip, retry, page someone).

Applications

- **Data Engineering:** `DataQualityError` fails the pipeline with a precise message; `TransientError` triggers a retry.
- **Machine Learning:** raise early when the input features have the wrong columns, before a silent wrong prediction.
- **API / Backend:** map custom exceptions to HTTP status codes.
- **Testing:** `assert` states what you expect after the code runs.

COMMON MISTAKE

Raising a bare `Exception("something failed")`. Callers cannot catch it precisely. Use a specific type.

COMMON MISTAKE

`raise NewError(...)` inside `except` without `from e`. It still works, but the message says "during handling... another exception occurred", which suggests a bug in your handler, not a translation.

COMMON MISTAKE

Using `assert` to check user input. Python started with `-O` removes all asserts.

Check yourself

1. What does a bare `raise` inside `except` do? Answer: re-raises the same exception.
2. What does `raise X from e` add? Answer: it records `e` as the direct cause of `X`.
3. Why have one base exception class for your project? Answer: callers can catch all your errors with one `except`.

6.4 Logging

INTUITION

`print` is shouting in a room. `logging` is a flight recorder: every message has a time, a severity, a source, and can be sent to the screen, a file or a monitoring system, and switched on or off without editing code. When a nightly job fails at 3 a.m., the log is the only witness.

Words you need

Word	Plain meaning
logger	The object you call (`log.info(...)`); it has a name like `etl.load`
level	Importance: DEBUG 10, INFO 20, WARNING 30, ERROR 40, CRITICAL 50
handler	Decides *where* a message goes (screen, file)
formatter	Decides *how* a message looks
propagate	A child logger passes its messages up to its parent's handlers

Level	Use for	Example

DEBUG	Details for developers	`row 5 parsed as ...`
INFO	Normal milestones	`loaded 120 rows`
WARNING	Strange but continuing	`missing column city`
ERROR	An operation failed	`cannot connect`
CRITICAL	The program may stop	`disk full`

How it works

- `logging.basicConfig(level=..., format=...)` sets up logging quickly. Call it once, at the start of the program.
- `logging.getLogger("etl.load")` returns a named logger. Convention in a module: `log = logging.getLogger(__name__)`. Dots in names make a tree: `etl.load` is a child of `etl`.
- Messages below the chosen level are ignored. At level INFO, `debug` calls do nothing.
- `log.exception("message")` logs at ERROR level and adds the traceback. Use it inside `except`.
- Use **lazy formatting**: `log.info("loaded %d rows", n)`, not `log.info(f"loaded {n} rows")`. With the first form, the text is built only if the message is actually emitted.
- Never log secrets, passwords, or personal data.

Worked examples

Problem 6.4.1 Set up basic logging at level INFO and write one message of each level, plus an exception.

Step 1	`basicConfig` with a format that shows level, logger name and message.
Step 2	Call `debug`, `info`, `warning`, `error`.
Step 3	In an `except` block call `log.exception`.

Predict: Which of these will NOT appear: debug, info, warning, error?

```
import logging, sys
logging.basicConfig(stream=sys.stdout, level=logging.INFO,
    format="%(levelname)s | %(name)s | %(message)s")
log = logging.getLogger("etl.load")
log.debug("hidden at INFO level")
log.info("loaded %d rows from %s", 120, "orders.csv")
log.warning("missing column %r", "city")
log.error("cannot connect")
try:
    1/0
except ZeroDivisionError:
    log.exception("calc failed")
print(logging.getLevelName(30), logging.WARNING, logging.ERROR,
logging.CRITICAL)
```

```
INFO | etl.load | loaded 120 rows from orders.csv
WARNING | etl.load | missing column 'city'
ERROR | etl.load | cannot connect
ERROR | etl.load | calc failed
Traceback (most recent call last):
  File "/private/tmp/w6/lg.py", line 10, in <module>
    1/0
    ~^~
ZeroDivisionError: division by zero
WARNING 30 40 50
```

- **Explain** The debug line is missing because DEBUG (10) is below INFO (20). `%d` and `%r` are placeholders filled from the extra arguments. `log.exception` printed an ERROR line and the full traceback. By default `basicConfig` writes to stderr; we used `stream=sys.stdout` only so the output appears in one place.
- **Answer** Only the debug message was hidden.
- **Check** The numbers: 30, 40, 50 are the real values of WARNING, ERROR, CRITICAL.
- **Why it matters** Change `level=logging.DEBUG` and all details appear, with no other code change. That is how you investigate a problem in production.

The typical wrong version. Using `print` everywhere. It has no level, no time, no way to send to a file, and you must delete the lines later. Also avoid `logging.info(f"...")` with f-strings for expensive objects: the text is built even when the message is dropped.

Problem 6.4.2 Send everything (DEBUG) to a file and only warnings to the screen, using two handlers, and see how a child logger propagates.

Step 1	Make a logger `svc` at level DEBUG.
Step 2	Add a `FileHandler` (level DEBUG) and a `StreamHandler` (level WARNING).
Step 3	Log at debug, info, warning. Print the file. Then log an error from the child `svc.db`.

```
import logging, sys, tempfile, os
d = tempfile.mkdtemp(); fn = os.path.join(d, "app.log")
log = logging.getLogger("svc")
log.setLevel(logging.DEBUG)
fh = logging.FileHandler(fn, encoding="utf-8"); fh.setLevel(logging.DEBUG)
ch = logging.StreamHandler(sys.stdout); ch.setLevel(logging.WARNING)
fmt = logging.Formatter("%(levelname)s:%(name)s:%(message)s")
fh.setFormatter(fmt); ch.setFormatter(fmt)
log.addHandler(fh); log.addHandler(ch)
log.debug("detail for file only")
log.info("info for file only")
log.warning("warning to both")
print("--- file content ---")
fh.flush(); print(open(fn).read(), end="")
child = logging.getLogger("svc.db")
child.error("child logger propagates to parent handlers")
```

```
WARNING:svc:warning to both
--- file content ---
DEBUG:svc:detail for file only
INFO:svc:info for file only
WARNING:svc:warning to both
ERROR:svc.db:child logger propagates to parent handlers
```

- **Explain** The first line is the screen copy of the warning (the screen only shows WARNING and above). The three lines after the divider are the file copy (all levels). The final ERROR line is printed to the *screen* by the child logger `svc.db` : it has no handlers of its own, so its message travels up to `svc` and uses its handlers (it was also written to the file after we printed it).
- **Answer** One logger tree, two destinations, each with its own threshold.
- **Check** The file shows 3 lines at the time we read it (debug, info, warning), matching the 3 calls.
- **Why it matters** In plain words: operators see only warnings and errors on the console, while engineers can search the file for the full story.

Problem 6.4.3 Produce structured (JSON) log lines so a tool can parse them.

Step 1	Subclass `logging.Formatter` and override `format`.
Step 2	Return `json.dumps` of level, message and logger name.
Step 3	Attach it to a handler. Set `propagate = False` to avoid double printing.

```
import logging, sys, json
logging.basicConfig(stream=sys.stdout, level=logging.INFO, format="%
(message)s")
class JsonFormatter(logging.Formatter):
    def format(self, r):
        return json.dumps({"level": r.levelname, "msg": r.getMessage(),
"logger": r.name})
h = logging.StreamHandler(sys.stdout); h.setFormatter(JsonFormatter())
l2 = logging.getLogger("json"); l2.addHandler(h); l2.propagate = False;
l2.setLevel(logging.INFO)
l2.info("order %s paid", 17)
```

```
{"level": "INFO", "msg": "order 17 paid", "logger": "json"}
```

- **Answer** The message became one JSON object per line, which log tools (ELK, CloudWatch, Datadog) can index.
- **Check** `"order %s paid" % 17` is `order 17 paid`. Matches.
- **Why it matters** Searching for `level = ERROR` in JSON logs is easy; searching free text is slow and fragile.

The picture

```
 your code: log.info("..")
        |
     [Logger "svc.db"] ──level check──► pass? ──► [Handler: file, level DEBUG]
──► app.log
        | propagate                                [Handler: screen, level
WARNING] ──► terminal
        ▼
     [Logger "svc"]  (its handlers do the work)
```

Connection to the real job

Every production system writes logs: Airflow tasks, FastAPI services, Spark jobs, training runs. During an incident, you search logs by time and level. Logging itself has a cost, like a restaurant kitchen that stops to write a note for every single egg it cracks: a DEBUG line inside a loop of 10 million rows slows

the job. Set INFO in production and DEBUG when investigating.

Applications

- **Data Engineering:** row counts and durations of each pipeline step, with warnings for rejected rows.
- **MLOps:** log model version, input size and prediction time of each request.
- **Machine Learning:** log loss and metrics per epoch to a file.
- **Security / Compliance:** audit trail of who ran what (without logging secrets).

COMMON MISTAKE

Calling `basicConfig` after other code already configured logging. It then does nothing. Call it once, first.

COMMON MISTAKE

Logging passwords, tokens or customer personal data. Logs are copied around and kept for years.

COMMON MISTAKE

Using `log.error(str(e))` inside `except` and losing the traceback. Use `log.exception(...)`.

Check yourself

1. At level WARNING, is `log.info(...)` shown? Answer: no.
2. Which call logs a message together with the traceback? Answer: `log.exception(...)`.
3. Why write `log.info("n=%d", n)` rather than an f-string? Answer: the text is built only if the message is emitted (lazy formatting).

6.5 Debugging with `pdb`

INTUITION

`print` debugging is like looking at a crime scene through a keyhole. A debugger lets you walk in: freeze the program at a line, look at every variable, move forward one line at a time, and ask "what is the value now?". It is the difference between guessing and seeing.

How it works

`pdb` is Python's built-in debugger. Three ways to start it:

1. Put `breakpoint()` in your code. Execution stops there.
2. Run `python3 -m pdb script.py` to start the script under the debugger.
3. **Post-mortem:** `python3 -m pdb -c continue script.py`. If the script crashes, pdb opens *at the crash line* and you can inspect the variables.

Set `PYTHONBREAKPOINT=0` to switch every `breakpoint()` off without editing code.

Command	Short	Meaning
`help`	`h`	list commands
`list` / `ll`	`l`	show source around the current line / the whole function
`next`	`n`	run the current line, stay at this level
`step`	`s`	run the current line, go *into* a function call
`continue`	`c`	run until the next breakpoint or the end
`print x`	`p x`	show the value of an expression
`break 12`	`b 12`	set a breakpoint at line 12
`where`	`w`	show the call stack
`up` / `down`	`u` / `d`	move between stack frames
`quit`	`q`	leave the debugger

The VS Code debugger does the same with clicks: click in the gutter to set a breakpoint, press F5, look at the Variables panel, and use the step buttons (not run here, because it needs the VS Code window; the ideas are identical to pdb).

Worked examples

For these examples we feed the commands to pdb through a pipe, so you can see a real session. When you work by hand, you type the same commands at the `(Pdb)` prompt.

Problem 6.5.1 Stop inside `average` just before the division, inspect the variables, and move one line.

Step 1	The function has a `breakpoint()` before `return total / len(nums)`.
Step 2	At the prompt we type `p nums`, `p total`, `n` (next), `ll` (list function), `c` (continue).
Step 3	The second call, `average([])`, stops at the breakpoint again. We look at `nums`.

```
def average(nums):
    total = 0
    for n in nums:
        total += n
    breakpoint()
    return total / len(nums)

print(average([10, 20, 30]))
print(average([]))
```

Predict: What will `p total` show for `[10, 20, 30]`?

```
> /private/tmp/w6/bug.py(5)average()
-> breakpoint()
(Pdb) [10, 20, 30]
(Pdb) 60
(Pdb) > /private/tmp/w6/bug.py(6)average()
-> return total / len(nums)
(Pdb)   1       def average(nums):
  2         total = 0
  3         for n in nums:
  4             total += n
  5         breakpoint()
  6  ->     return total / len(nums)
(Pdb) 20.0
> /private/tmp/w6/bug.py(5)average()
-> breakpoint()
(Pdb) []
(Pdb) Quitting pdb will kill the process. Quit anyway? [y/n]
```

- **Explain** The line `> ...bug.py(5)average()` says: file, line number, function. The `->` line is the *next line to run*. `p nums` printed the list, `p total` printed 60. `n` moved to line 6. `ll` printed the function with an arrow at the current line. `c` ran on: the program printed `20.0` (60 / 3), then reached the breakpoint for the empty list. `p nums` shows `[]`: the cause of the next crash is visible before it happens. (The last line appears because our pipe ended; typing `y` quits.)
- **Answer** The average of `[10, 20, 30]` is 20.0, and for `[]` we can see that `len(nums)` would be 0.
- **Check** 10 + 20 + 30 = 60 and 60 / 3 = 20.0. The debugger's values match.

- **Why it matters** In plain words: you *saw* the empty list before the program crashed, instead of guessing from the error message.

The typical wrong version. Without looking, the second call fails:

```
ZeroDivisionError: division by zero
```

and with `PYTHONBREAKPOINT=0` the `breakpoint()` line does nothing, so the script runs straight to that same `ZeroDivisionError` (we ran it: the last lines were `return total / len(nums)` and `ZeroDivisionError: division by zero`). The fix is a guard such as `if not nums: return 0.0` (or raise a clear error). Remove `breakpoint()` before you commit anyway.

Problem 6.5.2 Find a logic bug with no crash. `buggy_max([-5, -2, -9])` should return -2, but returns 0.

Step 1	Start `python3 -m pdb -c "b 4" -c continue bug2.py`. This sets a breakpoint on line 4 (the `if`) and runs.
Step 2	At each stop, print `n` and `best`.
Step 3	Notice `best` starts at 0, bigger than every number in the list.

```
def buggy_max(nums):
    best = 0
    for n in nums:
        if n > best:
            best = n
    return best
print(buggy_max([-5, -2, -9]))
```

Predict: What is `best` at the first stop?

```
Breakpoint 1 at /private/tmp/w6/bug2.py:4
> /private/tmp/w6/bug2.py(4)buggy_max()
-> if n > best:
(Pdb) (-5, 0)
(Pdb) > /private/tmp/w6/bug2.py(3)buggy_max()
-> for n in nums:
(Pdb) 0
(Pdb) > /private/tmp/w6/bug2.py(4)buggy_max()
-> if n > best:
(Pdb) (-2, 0)
```

- **Explain** The command `p n, best` printed `(-5, 0)`: n is -5, best is 0. `-5 > 0` is false, so

`best` never changes. We continued to the next stop: `(-2, 0)`, again false. The initial value `0` is the bug.

- **Answer** The function returns 0 because `best` starts at 0. Fix: `best = nums[0]` (and handle the empty list), or use the built-in `max(nums)`.
- **Check** With the fix, `max([-5, -2, -9])` is -2. Run `print(max([-5, -2, -9]))` to confirm.
- **Why it matters** There was no error message at all. Only looking at the variables revealed the problem. Many data bugs are silent like this.

Problem 6.5.3 Post-mortem: let a script crash and inspect it afterwards.

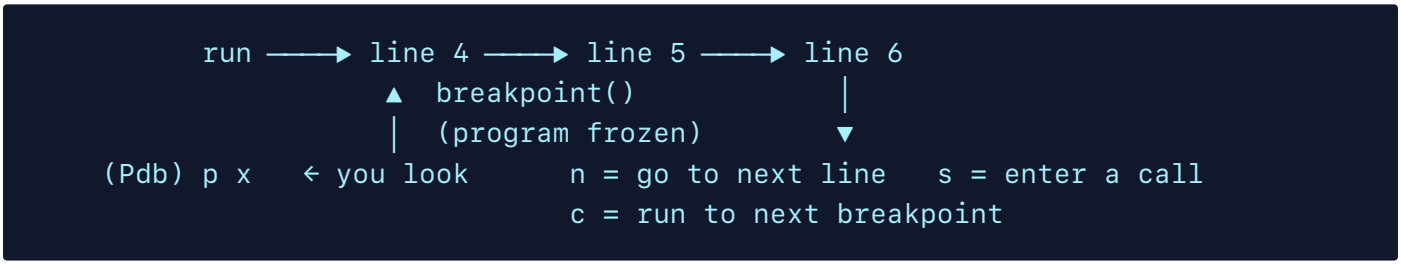
Step 1	Script: `data = {"a": [1, 2]}` then `print(data["a"][3])`.
Step 2	Run with `python3 -m pdb -c continue crash.py`. Pdb catches the crash and stops at the failing line.
Step 3	Type `p data` and `w` to see the stack.

```
IndexError: list index out of range
Uncaught exception. Entering post mortem debugging
Running 'cont' or 'step' will restart the program
> /private/tmp/w6/crash.py(2)<module>()
-> print(data["a"][3])
(Pdb) {'a': [1, 2]}
(Pdb)   /Library/.../pdb.py(3652)main()
-> pdb._run(target)
  ...
> /private/tmp/w6/crash.py(2)<module>()
-> print(data["a"][3])
```

(The lines inside the pdb library and the `...` were shortened by hand.)

- **Answer** `p data` shows the list has only 2 items, so index 3 is out of range. The bottom frame, marked with `>`, is your own line.
- **Check** A list with 2 items has valid indexes 0 and 1. Index 3 fails.
- **Why it matters** In plain words: you see the exact state at the moment of failure, not a reconstruction.

The picture



Connection to the real job

Notebook debugging uses the same ideas (`%debug` in Jupyter opens pdb after an error). For bugs that only appear on a server, you add logging (6.4) since you cannot attach a debugger. In a data pipeline, write the failing row to a file and reproduce it locally with `pdb` .

Applications

- **Data Analytics:** step through a cleaning function on one failing row.
- **Machine Learning:** stop inside a loss function to see tensor shapes.
- **Data Engineering:** reproduce a bad-record failure from yesterday's job with the saved record.
- **Backend:** debug a failing request by running the handler with a breakpoint.

COMMON MISTAKE

Committing code with `breakpoint()` left in. A server would freeze waiting for input. Search for it before you commit (a pre-commit hook can check this, Chapter 10).

COMMON MISTAKE

Typing `n` when you wanted `s` . `n` runs a whole function call as one step; `s` goes inside it.

COMMON MISTAKE

Using variable names that match pdb commands (like `c` or `n`) and then printing them. Use `p n` explicitly with `p` .

Check yourself

1. How do you stop the program at a chosen line in code? Answer: write `breakpoint()` there.
2. What is the difference between `n` and `s` ? Answer: `n` runs the line as one step; `s` steps into a function call.
3. Which command shows the call stack? Answer: `w` (where).

Chapter recap

- Read a traceback from the last line upward: type and message first, then the first frame that is your own code.
- Learn the ten common exceptions: `ValueError`, `TypeError`, `KeyError`, `IndexError`, `AttributeError`, `NameError`, `FileNotFoundError`, `ModuleNotFoundError`, `ZeroDivisionError`, `AssertionError`.
- Catch specific exceptions, keep the `try` block small, use `else` for success code and `finally` for cleanup.
- Never use a bare `except:`; it hides bugs.
- EAFP (try and catch) is the Python style; LBYL (check first) is fine for simple cases.
- Raise your own exceptions with a clear message; make a base `AppError` class and children.
- Use `raise X from e` to keep the original cause, and bare `raise` to re-raise after logging.
- Use `logging`, not `print`: levels, handlers, `log.exception`, lazy `%s` formatting, and never log secrets.
- Use `breakpoint()`, `python3 -m pdb` and post-mortem to look inside a running program.

Cheat-sheet

Task	Pattern
Handle a known error	`try: ... except ValueError as e: ...`
Cleanup always	`try: ... finally: close()`
Raise	`raise ValidationError("age", "must be >= 0")`
Keep cause	`raise AppError("...") from e`
Log with traceback	`log.exception("what failed")`
Basic log setup	`logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")`
Stop in debugger	`breakpoint()` or `python3 -m pdb script.py`
pdb moves	`n` next, `s` step, `c` continue, `p x` print, `w` stack, `q` quit

Practice set

Problem 6.P.1 What exception does `int("3.5")` raise?

Answer: `ValueError` (`int` does not accept text with a decimal point; use `int(float("3.5"))`).

Problem 6.P.2 What is the output?

```
try:
    print("A")
except ValueError:
    print("B")
else:
    print("C")
finally:
    print("D")
```

Answer: `A`, `C`, `D` on separate lines. No error, so `else` runs, then `finally`.

Problem 6.P.3 What is the output?

```
def f():
    try:
        return "try"
    finally:
        print("cleanup")
print(f())
```

Answer: `cleanup` then `try`. The `finally` block runs before the function returns.

Problem 6.P.4 Find the bug:

```
try:
    value = int(text)
except:
    pass
print(value * 2)
```

Answer: The bare `except` hides all errors, and if `int(text)` failed, `value` is not defined, so `print` raises `NameError`. Catch `ValueError` and set a default or stop.

Problem 6.P.5 Which exception does `d = {}; d["x"] += 1` raise?

Answer: `KeyError: 'x'`, because `d["x"]` is read first. Use `d.get("x", 0) + 1` or `defaultdict(int)`.

Problem 6.P.6 Write a function `to_float(s)` that returns `None` for text that is not a number, but lets other errors through.

Answer:

```
def to_float(s):
    try:
        return float(s)
    except ValueError:
        return None
```

Note that `to_float(None)` raises `TypeError`, which is correct: `None` is a programming error, not bad data. Decide this on purpose.

Problem 6.P.7 Logging level is WARNING. Which of these appear: `debug`, `info`, `warning`, `error`?

Answer: `warning` and `error`.

Problem 6.P.8 Find the bug: a function does `except ValueError as e: raise AppError("bad input")` and an engineer says the original error is lost.

Answer: Without `from e` the link is only implicit. Write `raise AppError("bad input") from e` so the traceback shows the direct cause.

Problem 6.P.9 Scenario: your nightly job crashes at 2 a.m. with `KeyError: 'amount'` after 1.2 million rows. You have only the log. List your steps.

Answer: (1) Read the traceback bottom-up and find the line in your code. (2) Check the log for which file or row it was processing (this is why you log file names and row counts). (3) Reproduce with that one row on your laptop. (4) Use `pdb` or a print of the row to see which keys exist. (5) Decide: skip and count bad rows with a warning, or fail the job with a clear `DataQualityError`. (6) Add a test with that bad row.

Problem 6.P.10 Interview: what is the difference between `except Exception` and a bare `except:`? Why avoid both when possible?

Answer: A bare `except:` also catches `KeyboardInterrupt` and `SystemExit` (they inherit from `BaseException`), so you cannot stop the program with Ctrl+C. `except Exception` skips those but still hides bugs. Catch the specific exception you can handle.

Problem 6.P.11 Interview: explain EAFP versus LBYL with an example, and say when LBYL is better.

Answer: EAFP: `try: value = d[key] except KeyError: ...`. LBYL: `if key in d: value = d[key]`. EAFP is idiomatic and avoids a race (a file may disappear between `exists()` and

`open()`). LBYL is clearer when failure is common and cheap to check, and when an exception would be costly in a hot loop.

Problem 6.P.12 Interview: you must find why a function returns the wrong number, but there is no error. How do you debug it?

Answer: Reproduce with the smallest input. Put `breakpoint()` before the suspicious line, print the inputs and intermediate values (`p`), step with `n`, and compare with what you expect by hand. Then write a failing test that captures the bug, fix, and keep the test.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles in India, and English is my second language, so use simple
words. Teach me how to read a Python traceback and the ten most common
exceptions (ValueError, TypeError, KeyError, IndexError, AttributeError,
NameError, FileNotFoundError and others). Explain step by step with small
examples, then show me 6 tracebacks one at a time and ask me what the cause and
the fix are. Wait for my answer, and correct me kindly.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles in India. Topic: Python try/except/else/finally, EAFP versus
LBYL, catching specific exceptions, and why bare except is dangerous. Give me 8
graded exercises from easy to hard about cleaning messy data rows safely. Do
not give solutions yet. When I paste my answers, review them for bugs and style
and explain every problem in simple English.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles in India, and English is my second language. Topic: custom
exceptions in Python, raising errors, exception hierarchies, raise from, re-
raising, assert, and ExceptionGroup. Explain with a data pipeline example, then
review my design for the custom errors of a small ETL job: [PASTE HERE]. Tell
me what is good, what is confusing, and how a senior engineer would improve it.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles in India. Topic: the Python logging module: levels, loggers,
```

handlers, formatters, propagation, log.exception, lazy formatting and JSON logs. Quiz me one question at a time, starting easy and getting harder. After each answer, explain the right answer step by step. After 8 questions, give me a small project to set up logging for a script with a file handler and a console handler.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data roles in India, and English is my second language. Topic: debugging Python with pdb and breakpoint(): commands n, s, c, p, ll, w, b, post-mortem debugging. Run a mock debugging interview: describe a buggy function in words, let me ask for variable values, and guide me to find the bug. Do three rounds of increasing difficulty and give me honest feedback on my method at the end.

CHAPTER 7

ITERATORS, GENERATORS, DECORATORS AND CONTEXT MANAGERS

Four features that turn beginner Python into professional Python: process data lazily, wrap behaviour around functions, and guarantee cleanup.

Suppose you must read a 20 GB log file, keep only the delivered orders, and compute their average delivery time. If you load the file into a list, your laptop runs out of memory. With iterators and generators you read one line at a time and the job runs in a few megabytes. Now suppose five of your functions need the same extras: timing, retry on network failure, and a log line. Copy-pasting those into every function is a maintenance disaster. A decorator adds them in one line each. Finally, every file, database connection and lock must be released even when errors happen, and context managers guarantee it. By the end of this chapter you can build lazy data pipelines, write your own timing and retry decorators, and write your own `with` blocks.

Running example: a tiny order log with lines like `OK north A1 120` (status, region, order id, delivery time in minutes). We read it lazily, time the functions, retry the flaky ones, and manage resources safely. Every program here was run with `python3` (Python 3.14) and the output is real.

7.1 Iterators

INTUITION

Think of a vending machine that gives one item each time you press the button, and says "empty" when it is done. You never see the whole stock at once. An iterator is that machine: you ask for the next item, it gives one, and at the end it says "no more". A `for` loop is just a person pressing the button until the machine says empty.

Words you need

Word	Plain meaning
iterable	Anything you can loop over: list, string, dict, file, `range`
iterator	An object that hands out items one at a time with `next()`
`iter(x)`	Asks an iterable for a fresh iterator

`next(it)`	Asks the iterator for the next item
`StopIteration`	The exception that means "no more items"
exhausted	An iterator that has given all its items; it cannot restart

How it works

A list is an iterable, but it is not an iterator. The rules (called the *iterator protocol*) are simple:

- An **iterable** has an `__iter__` method that returns an iterator.
- An **iterator** has `__next__`, which returns the next item or raises `StopIteration`, and `__iter__` that returns itself.
- A `for x in thing:` loop calls `iter(thing)`, then calls `next` again and again, and stops quietly when `StopIteration` is raised.

Why does this exist? So that one `for` loop works on lists, files, database cursors, and network streams alike, and so that huge or endless sources do not need to be stored in memory.

Worked examples

Problem 7.1.1 Use `iter` and `next` by hand on `[10, 20, 30]`, and see what happens at the end.

Step 1	`it = iter(nums)` creates the iterator.
Step 2	Call `next(it)` three times.
Step 3	Call it a fourth time and catch `StopIteration`.
Step 4	Rebuild the `for` loop from `while True`.

Predict: What does the fourth `next(it)` do? What does `list(it)` give after the iterator is already used up?

```
nums = [10, 20, 30]
it = iter(nums)
print(type(it).__name__, next(it), next(it), next(it))
try:
    next(it)
except StopIteration:
    print("StopIteration: exhausted")
print(iter(it) is it, iter(nums) is nums)

it = iter(nums)
while True:
    try:
        x = next(it)
    except StopIteration:
        break
    print("got", x)

it = iter([1, 2, 3])
print(list(it), list(it))
print(next(iter([]), "default"))
```

```
list_iterator 10 20 30
StopIteration: exhausted
True False
got 10
got 20
got 30
[1, 2, 3] []
default
```

- **Explain** `iter(nums)` gave a `list_iterator`. `iter(it) is it` is `True`: an iterator is its own iterator. `iter(nums) is nums` is `False`: the list makes a new iterator each time, so you can loop over a list many times. The `while True` loop is exactly what `for` does inside. The second `list(it)` is empty because the first call used up all items. `next(iter([]), "default")` gives a fallback instead of raising.
- **Answer** The fourth `next` raises `StopIteration`. A used iterator cannot be reused.
- **Check** `[1, 2, 3]` then `[]` proves the iterator was spent after one pass.
- **Why it matters** In plain words: if you pass an iterator to two functions, the second one sees nothing. This is a very common silent bug.

The typical wrong version. Loop twice over a generator or iterator:

```
it = iter([1, 2, 3])
print(sum(it))   # 6
print(sum(it))   # 0, silently
```

No error appears, only a wrong result (0). The fix: keep the data in a list, or recreate the iterator.

Problem 7.1.2 Build your own iterator class `Countdown` and use it with `list`, `sum` and `max`.

Step 1	Store the start value in `__init__`.
Step 2	`__iter__` returns `self`.
Step 3	`__next__` raises `StopIteration` at 0, otherwise returns the next number down.

```
class Countdown:
    def __init__(self, start):
        self.n = start
    def __iter__(self):
        return self
    def __next__(self):
        if self.n <= 0:
            raise StopIteration
        self.n -= 1
        return self.n + 1

print(list(Countdown(4)))
print(sum(Countdown(4)), max(Countdown(3)))
d = {"a": 1, "b": 2}
print(list(iter(d)), list(d.items()))
print(list(enumerate("ab", start=1)), list(zip("ab", [1, 2, 3])))
```

```
[4, 3, 2, 1]
10 3
['a', 'b'] [('a', 1), ('b', 2)]
[(1, 'a'), (2, 'b')] [('a', 1), ('b', 2)]
```

- **Answer** `list(Countdown(4))` is `[4, 3, 2, 1]`, and `sum` is 4+3+2+1 = 10.
- **Check** `max(Countdown(3))` is 3 (values 3, 2, 1). `zip` stops at the shorter input, so `3` has no partner and is dropped.
- **Why it matters** Anything that follows the protocol works with all built-ins (`list`, `sum`, `max`, `sorted`, `for`, `in`). You rarely write classes like this (7.2 is easier), but you now know what happens under every `for` loop.

The picture

```
iterable ──iter()──▶ iterator ──next()──▶ 10
                          │       next()──▶ 20
                          │       next()──▶ 30
                          └─ next() ──▶ raises StopIteration  (for-loop ends
quietly)
```

Item	Example value	Where it appears in the job
Iterable	a `list`, a `str`, a DataFrame's columns	passed to `for`, `sum`, `sorted`
Iterator	an open file, `zip(...)`, `map(...)`	reading big files, streaming rows
`enumerate` / `zip`	`enumerate(rows, 1)`	line numbers, pairing columns

Connection to the real job

An open file is an iterator over lines; a database cursor is an iterator over rows; `pandas.read_csv(..., chunksize=100_000)` returns an iterator of DataFrames. Understanding the protocol explains why these tools can handle data bigger than memory.

Applications

- **Data Engineering:** stream rows from a database cursor into a file in batches.
- **Data Analytics:** `for chunk in pd.read_csv(path, chunksize=...)` aggregates a file too big for memory.
- **Machine Learning:** a data loader is an iterator that yields batches.
- **GenAI:** streaming an LLM reply is an iterator of text pieces.

COMMON MISTAKE

Using an iterator twice. After the first pass it is empty. Convert to a list if you need two passes.

COMMON MISTAKE

Confusing iterable and iterator. A list is iterable, not an iterator: `next([1,2])` raises `TypeError`.

COMMON MISTAKE

Raising `StopIteration` inside a generator by accident. Since Python 3.7 this becomes a `RuntimeError`.

Check yourself

1. What does `for` call first on the object? Answer: `iter()`.
2. What exception ends iteration? Answer: `StopIteration`.
3. Can you loop over a list twice? Answer: yes, because each loop gets a new iterator; but an iterator itself can be used once.

7.2 Generators

INTUITION

A normal function is a cook who prepares the whole meal and brings every dish at once. A generator is a cook who brings one dish, waits until you ask, and then cooks the next. Nothing is cooked before you ask, and nothing is stored. This is called **lazy evaluation**: work is done only when the result is needed.

How it works

A **generator function** looks like a normal function but uses `yield` instead of (or in addition to) `return`. Calling it does *not* run the body. It returns a **generator object** (an iterator). Each `next()` runs the body until the next `yield`, hands out that value, and freezes the function with all its local variables. The next `next()` unfreezes it.

- A **generator expression** is a comprehension in round brackets: `(i*i for i in range(10))`. It makes a generator, not a list.
- `yield from other` hands out everything from another iterable (a shortcut for a loop with `yield`).
- `gen.send(value)` sends a value *into* a paused generator (advanced; used for coroutines).
- Why it exists: you can describe an endless or huge sequence in a few lines and use only constant memory.

Worked examples

Problem 7.2.1 Write `squares(n)` as a generator, and see that the body runs only on demand.

Step 1	Use `yield i * i` in a loop.
Step 2	Call it and look at the type.
Step 3	Make a `trace` generator with `print` calls to see *when* the code runs.

Predict: After `t = trace()`, does anything print?

```
def squares(n):
    for i in range(n):
        yield i * i

g = squares(4)
print(type(g).__name__, next(g), next(g), list(g))

def trace():
    print("start"); yield 1
    print("middle"); yield 2
    print("end")

t = trace()
print("created, nothing printed yet")
print(next(t)); print(next(t))
try:
    next(t)
except StopIteration:
    print("done")
```

```
generator 0 1 [4, 9]
created, nothing printed yet
start
1
middle
2
end
done
```

- **Explain** `g` is a `generator`. `next(g)` gave 0 and 1 (0^2 and 1^2). `list(g)` collected the rest: $2^2 = 4$ and $3^2 = 9$. For `trace`, creating it prints nothing. The first `next` runs until the first `yield` (printing `start`, giving 1). The third `next` runs to the end of the function (printing `end`) and Python raises `StopIteration`.
- **Answer** Nothing prints at creation. Code runs only when you ask for the next value.
- **Check** `0, 1, 4, 9` are the first four squares; the output shows `0 1 [4, 9]`: 0 and 1 from `next`, then 4 and 9 from `list`.

- **Why it matters** In plain words: you can set up a long chain of steps and no work happens until the final consumer asks for results.

The typical wrong version. Build a list when a generator is enough:

```
big_list = [i*i for i in range(1_000_000)]   # all 1 million numbers in memory
big_gen  = (i*i for i in range(1_000_000))   # nothing computed yet
```

Problem 7.2.2 Measure the memory difference between a list and a generator of one million squares.

Step 1	Use `sys.getsizeof` on both. (It measures the container only, not the integers inside the list, so the real gap is even larger.)
Step 2	Confirm both give the same sum.

Predict: Roughly how many bytes will each take? (Think: 8 bytes per list slot.)

```
import sys
big_list = [i*i for i in range(1_000_000)]
big_gen = (i*i for i in range(1_000_000))
print(sys.getsizeof(big_list), sys.getsizeof(big_gen))
print(sum(big_gen) == sum(big_list))
```

```
8448728 208
True
```

- **Explain** One million slots at 8 bytes each is about 8,000,000 bytes, plus some growth room, giving 8,448,728 here. The generator is 208 bytes, no matter how many items it will produce. The exact numbers depend on your Python version and computer; the size of the gap does not.
- **Answer** The list costs about 8.4 MB for the container, the generator 208 bytes, and the sums are equal.
- **Check** 1,000,000 × 8 = 8,000,000 bytes, close to the measured 8,448,728 (the extra is spare capacity and the list header).
- **Why it matters** Scale this to 1 billion order lines: 8 GB versus 208 bytes. This is why data code uses generators for big inputs.

Problem 7.2.3 Build a generator pipeline over a log: parse, keep status OK, take the delivery times, average them.

Step 1	`parse` yields `(status, region, order, minutes)` and skips bad lines.
Step 2	`only_ok` keeps status OK.
Step 3	`times` extracts the `minutes` values.
Step 4	Chain them. Compute the average by hand: (120 + 340 + 95) / 3.

```
lines = ["OK north A1 120", "LATE south B2 900", "OK east C3 340",
         "bad line", "OK west D4 95"]

def parse(ls):
    for l in ls:
        parts = l.split()
        if len(parts) == 4:
            yield parts[0], parts[1], parts[2], int(parts[3])

def only_ok(rs):
    return (r for r in rs if r[0] == "OK")

def times(rs):
    return (r[3] for r in rs)

ts = list(times(only_ok(parse(lines))))
print(ts, sum(ts) / len(ts))
```

```
[120, 340, 95] 185.0
```

- **Answer** Three delivered orders took 120, 340 and 95 minutes; the average is 185.0 minutes.
- **Check** 120 + 340 + 95 = 555 and 555 / 3 = 185. The late order (900 minutes) and the bad line were correctly left out.
- **Why it matters** In plain words: delivered orders average 185 minutes. Each line flows through all stages one at a time, so with a real 20 GB file memory stays tiny. Each stage is small and testable on its own.

Problem 7.2.4 More generator tools: `yield from`, an infinite generator with `islice`, `send`, and batching.

Step 1	`yield from` flattens a list of lists.
Step 2	The Fibonacci generator never ends; `itertools.islice` takes the first ten.
Step 3	`running_avg` receives numbers with `send` and yields the running average.
Step 4	`gen_read` groups items into batches of 3.

```
from itertools import islice

def inner():
    yield from [1, 2]
    yield from "ab"
print(list(inner()))

def chain_flat(nested):
    for sub in nested:
        yield from sub
print(list(chain_flat([[1, 2], [3], [4, 5]])))

def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
print(list(islice(fib(), 10)))

def running_avg():
    total = n = 0
    avg = None
    while True:
        x = yield avg
        total += x
        n += 1
        avg = total / n
r = running_avg(); next(r)
print(r.send(10), r.send(20), r.send(60))

def gen_read(lines, size):
    batch = []
    for l in lines:
        batch.append(l)
        if len(batch) == size:
            yield batch
            batch = []
    if batch:
        yield batch
print(list(gen_read(range(7), 3)))
```

```
[1, 2, 'a', 'b']
[1, 2, 3, 4, 5]
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
10.0 15.0 30.0
[[0, 1, 2], [3, 4, 5], [6]]
```

- **Answer** All five outputs match our expectations. The running averages are 10/1 = 10.0, (10+20)/2 = 15.0, (10+20+60)/3 = 30.0.
- **Check** Fibonacci: each number is the sum of the two before it: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34. And 7 items in batches of 3 give 3 + 3 + 1.
- **Why it matters** Batching is how you insert millions of rows into a database in chunks of 1,000 instead of one at a time. An infinite generator plus `islice` is a safe way to take "the first N" of a stream.

The picture

```
 file lines → parse → only_ok → times → sum / list
   (lazy)      (lazy)    (lazy)     (lazy)    (consumer pulls: nothing runs
until here)

 one line travels through ALL stages before the next line is read
```

Item	Example value	Where it appears in the job
Generator function	`def read_rows(path): ... yield row`	Row-by-row file readers
Generator expression	`(r[3] for r in rows)`	One-off lazy filters
`yield from`	`yield from sub`	Flattening nested results
Batching	`gen_read(rows, 1000)`	Bulk database inserts

Connection to the real job

Pipelines are the shape of data engineering: read, parse, validate, transform, write. Writing each step as a generator keeps memory flat and makes steps easy to test. Frameworks such as Airflow, Spark and PyTorch data loaders follow the same stream-of-items idea.

Applications

- **Data Engineering:** read 50 GB of JSON Lines logs, filter errors, write a summary, using constant memory.
- **Data Science:** generate sliding windows of a time series without storing all of them.
- **Machine Learning:** `yield` training batches forever, one epoch after another.
- **Backend / APIs:** stream a big response row by row instead of building it in memory.

COMMON MISTAKE

Calling `len()` on a generator (`TypeError: object of type 'generator' has no len()`) or indexing it. A generator has no size and no positions.

COMMON MISTAKE

Forgetting that a generator is used up after one pass. Printing it with `list(g)` for debugging empties it for the real consumer.

COMMON MISTAKE

Using a generator for tiny data or when you need to loop many times; a list is simpler and faster there.

Check yourself

1. What does calling a generator function return before any code runs? Answer: a generator object (no body code has run yet).
2. What does `yield` do to the function? Answer: it hands out a value and pauses the function, keeping its local variables.
3. What is the difference between `[x for x in a]` and `(x for x in a)` ? Answer: the first builds a whole list now; the second makes a lazy generator.

7.3 Decorators

INTUITION

Imagine gift-wrapping. The gift (your function) stays the same, but you wrap it in paper that adds something: a bow for timing, a ribbon for retry, a card for logging. Anyone who receives the wrapped gift uses it exactly as before. A decorator is a function that takes your function and returns a wrapped version of it. In everyday terms, it is like a price tag or gift note attached to a product without changing the product.

Words you need

Word	Plain meaning
first-class function	A function is a value; you can pass it around and return it, like a number
closure	An inner function that remembers variables from the function that made it
wrapper	The inner function that runs extra code around the original
`*args, **kwargs`	"Accept any positional and keyword arguments and pass them on"
`@name`	Short syntax: `@shout` above `def add` means `add = shout(add)`

How it works

The `@` line is only a shortcut. These two are identical:

```
@shout
def add(a, b): ...
# same as:
def add(a, b): ...
add = shout(add)
```

Every decorator has the same skeleton:

```
def decorator(func):
    @functools.wraps(func)            # copy name and docstring of func onto
wrapper
    def wrapper(*args, **kwargs):
        ... do something before ...
        result = func(*args, **kwargs)
        ... do something after ...
        return result
    return wrapper
```

A decorator that takes its *own* arguments (like `@retry(3)`) needs one more level: a function that returns the decorator.

Worked examples

Problem 7.3.1 Write `shout`, which prints the function name before each call. Then see why `functools.wraps` is needed.

Step 1	Define `shout(func)` with an inner `wrapper`.
Step 2	Decorate `add` and print `add.__name__` and `add.__doc__`.
Step 3	Repeat with `@functools.wraps(func)` added.

Predict: What will `add.__name__` print for the first version?

```
import functools

def shout(func):
    def wrapper(*args, **kwargs):
        print("calling", func.__name__)
        return func(*args, **kwargs)
    return wrapper

@shout
def add(a, b):
    "Add two numbers."
    return a + b
print(add(2, 3), add.__name__, add.__doc__)

def shout2(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@shout2
def add2(a, b):
    "Add two numbers."
    return a + b
print(add2.__name__, add2.__doc__)
```

```
calling add
5 wrapper None
add2 Add two numbers.
```

- **Explain** After `@shout`, the name `add` points to `wrapper`, so it reports itself as `wrapper` with no docstring. That breaks help text, logging, and debugging tools. `functools.wraps(func)` copies the original name and docstring to the wrapper.
- **Answer** Without `wraps` you get `wrapper None`; with it you get the real name and docstring.
- **Check** The call returned 5, which is 2 + 3, so the wrapped function still works.
- **Why it matters** Without `wraps`, every logged function name is "wrapper" and you cannot tell which function was slow. Always use it.

Problem 7.3.2 Write a `timed` decorator and a `repeat(times)` decorator with an argument.

Step 1	`timed` measures with `time.perf_counter()` and uses `try/finally`, so the time is reported even if the function fails.
Step 2	`repeat(times)` is a decorator factory: it returns `deco`, which returns `wrapper`.
Step 3	Test with a function that sleeps 10 ms, and a function that greets.

```
import functools, time

def timed(func):
    @functools.wraps(func)
    def wrapper(*a, **k):
        t0 = time.perf_counter()
        try:
            return func(*a, **k)
        finally:
            ms = (time.perf_counter() - t0) * 1000
            print(f"{func.__name__} took >=5ms: {ms >= 5}")
    return wrapper

@timed
def slow():
    time.sleep(0.01)
    return "done"
print(slow())

def repeat(times):
    def deco(func):
        @functools.wraps(func)
        def wrapper(*a, **k):
            return [func(*a, **k) for _ in range(times)]
        return wrapper
    return deco

@repeat(3)
def hi(name):
    return f"hi {name}"
print(hi("Ravi"))
```

```
slow took >=5ms: True
done
['hi Ravi', 'hi Ravi', 'hi Ravi']
```

(We print `>=5ms: True` instead of the exact milliseconds because the exact number changes on every run. In real code you would log the number itself.)

- **Explain** `@repeat(3)` first *calls* `repeat(3)`, which returns `deco`, and then applies `deco` to `hi`. So three layers of functions: factory, decorator, wrapper.
- **Answer** `slow` took at least 5 ms (it sleeps 10 ms), and `hi("Ravi")` ran three times.
- **Check** `time.sleep(0.01)` is 10 ms, which is greater than 5 ms, so `True` is correct.
- **Why it matters** In plain words: you got timing in one line (`@timed`) on any function, without editing its body.

The typical wrong version. Forgetting the parentheses on a decorator that takes arguments:

```
@repeat          # should be @repeat(3)
def bad(): pass
bad()
```

```
TypeError: repeat.<locals>.deco() missing 1 required positional argument:
'func'
```

`@repeat` passes the function as the `times` argument, so `bad` becomes `deco`, and calling it fails. Read the message: it names `deco` and the missing `func`. The fix is `@repeat(3)`.

Problem 7.3.3 Write a `retry` decorator for a flaky network call that fails twice, then works. Show the case where all attempts fail.

Step 1	Loop `attempt` from 1 to `max_tries`, call the function in `try`.
Step 2	On the listed exception types, print the failure; on the last attempt re-raise.
Step 3	`flaky` raises `ConnectionError` on calls 1 and 2, then returns.

```
import functools

def retry(max_tries=3, exceptions=(Exception,)):
    def deco(func):
        @functools.wraps(func)
        def wrapper(*a, **k):
            for attempt in range(1, max_tries + 1):
                try:
                    return func(*a, **k)
                except exceptions as e:
                    print(f"  attempt {attempt} failed: {e}")
                    if attempt == max_tries:
                        raise
        return wrapper
    return deco

calls = {"n": 0}

@retry(max_tries=3, exceptions=(ConnectionError,))
def flaky():
    calls["n"] += 1
    if calls["n"] < 3:
        raise ConnectionError("network down")
    return "ok on call %d" % calls["n"]
print(flaky())

@retry(max_tries=2)
def always_bad():
    raise ValueError("nope")
try:
    always_bad()
except ValueError as e:
    print("gave up:", e)
```

```
  attempt 1 failed: network down
  attempt 2 failed: network down
ok on call 3
  attempt 1 failed: nope
  attempt 2 failed: nope
gave up: nope
```

- **Answer** The third attempt succeeded. When every attempt fails, the last exception is re-raised to the caller.
- **Check** `calls["n"]` was 1, 2 (errors), then 3 (success), so `ok on call 3` is right. With `max_tries=2` the loop stops after two failures.
- **Why it matters** In plain words: temporary network problems no longer kill your program, but real

bugs still stop it. Only retry errors that can be temporary (connection, timeout), never `ValueError`; our demo does this only to show the re-raise. Real retries add a waiting time that grows between attempts (called backoff); this demo leaves it out.

Problem 7.3.4 Cache results with `functools.lru_cache`, stack two decorators, use a class decorator and a registry.

Step 1	`fib(80)` is computed with and without a cache. Without a cache the work doubles at each step; with `lru_cache` each `n` is computed once.
Step 2	Stacking: `@tag("b")` above `@tag("i")` means `text = tag("b")(tag("i")(text))`, so `i` is applied first (inner).
Step 3	A class decorator receives a class and returns a class; here it adds `__repr__`.
Step 4	A decorator can *register* functions in a dictionary and return them unchanged.

```
import functools, time

@functools.lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n-1) + fib(n-2)
print(fib(80), fib.cache_info())

def fib_slow(n):
    return n if n < 2 else fib_slow(n-1) + fib_slow(n-2)
t0 = time.perf_counter(); fib_slow(27); slow_t = time.perf_counter() - t0
print("uncached fib(27) slower than 5ms:", slow_t > 0.005)

@functools.cache
def sq(x):
    print("computing", x)
    return x * x
print(sq(4), sq(4))

def tag(name):
    def deco(f):
        @functools.wraps(f)
        def w(*a, **k):
            return f"<{name}>{f(*a, **k)}</{name}>"
        return w
    return deco

@tag("b")
@tag("i")
def text():
    return "hi"
```

```
print(text())

def add_repr(cls):
    def __repr__(self):
        items = ", ".join(f"{k}={v!r}" for k, v in vars(self).items())
        return f"{cls.__name__}({items})"
    cls.__repr__ = __repr__
    return cls

@add_repr
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
print(Point(1, 2))

registry = {}
def register(f):
    registry[f.__name__] = f
    return f

@register
def clean(): return "cleaning"
@register
def load(): return "loading"
print(sorted(registry), registry["load"]())
```

```
23416728348467685 CacheInfo(hits=78, misses=81, maxsize=None, currsize=81)
uncached fib(27) slower than 5ms: True
computing 4
16 16
<b><i>hi</i></b>
Point(x=1, y=2)
['clean', 'load'] loading
```

- **Explain** `fib(80)` is a 17-digit number, computed instantly because each of 81 values (0 to 80) was calculated once (`misses=81`) and reused 78 times (`hits=78`). `sq(4)` printed `computing 4` only once; the second call came from the cache. The stacked decorators wrapped from the bottom up: `i` inside, `b` outside. The registry pattern is how web frameworks collect routes (`@app.get("/users")`).
- **Answer** Caching turns an exponential job into a linear one; stacked decorators apply bottom-up; class decorators and registries change or collect things at definition time.
- **Check** 81 misses is exactly the number of distinct inputs (0 to 80). The uncached version of `fib(27)` is not demonstrated with a number here because timing varies; try `fib_slow(35)` yourself to feel the difference.
- **Why it matters** Caching a function is only safe if it is *pure*: the same inputs always give the same

output and no side effects. Never cache a function that reads the current time or a database that changes. Also, `lru_cache` needs hashable arguments (a list argument raises `TypeError`).

The picture

```
caller ──▶ [ wrapper ]  ──▶  [ original function ]
             │  before: start timer / check cache / log
             │  after:  stop timer / store result / retry on error
            ◀── result returned to caller unchanged

@a        a(b(f)):   caller ▶ a's wrapper ▶ b's wrapper ▶ f
@b
def f
```

Item	Example value	Where it appears in the job
`@timed`	prints durations	Finding slow pipeline steps
`@retry(3)`	repeats on `ConnectionError`	API calls, database connects
`@lru_cache`	remembers results	Repeated lookups, recursive math
`@app.get("/x")`	registers a route	FastAPI / Flask (Chapter 12)
`@pytest.fixture`	marks a test helper	Testing (Chapter 10)
`@property`, `@dataclass`	built-in decorators	Classes (Chapter 8)

Connection to the real job

You will *use* decorators daily (`@pytest.fixture`, `@app.get`, `@dataclass`, `@task` in Airflow) and *write* them for cross-cutting needs: timing, retry, logging, access checks. A `@timed` decorator that logs the duration of each pipeline step is a natural first tool.

Applications

- **Data Engineering:** retry decorators around API and database calls; `@task` in Airflow.
- **Backend / APIs:** `@app.get("/orders")` routes; an `@require_login` check.
- **Machine Learning:** `@torch.no_grad()`, `@tf.function` modify how a function runs.
- **Performance Testing:** `@timed` on functions inside a Python benchmark script (for example Locust tasks, named only as an example tool).

COMMON MISTAKE

Forgetting `functools.wraps`. Names and docstrings are lost.

COMMON MISTAKE

Forgetting to `return` the result inside `wrapper`. The decorated function then always returns `None`.

COMMON MISTAKE

Retrying everything, including non-temporary errors and non-idempotent actions (such as "charge the customer"). A retry can double the effect. Idempotent means repeating gives the same result as doing it once; retry only those.

Check yourself

1. What is `@shout` above `def add` equal to? Answer: `add = shout(add)`.
2. Why do we write `*args, **kwargs` in the wrapper? Answer: so the wrapper accepts and passes on whatever arguments the original function takes.
3. What does `functools.wraps` do? Answer: it copies the name, docstring and other details of the original function to the wrapper.

7.4 Context Managers

INTUITION

When you rent a hall for an event, there is a checkout rule: you must switch off the lights and return the key, even if the party went badly. A context manager is that rule in code. The `with` block says: "set things up, let me do my work, and *always* tidy up at the end, even if something exploded in the middle."

Words you need

Word	Plain meaning
resource	Something you must give back: a file, a connection, a lock
context manager	An object that sets up and cleans up around a block of code

`__enter__`	Runs at the start of `with`; its return value goes to `as name`
`__exit__`	Runs at the end, always; it receives the exception details, if any
suppress	Swallow an exception so it does not reach the caller

How it works

`with thing as x:` means: call `thing.__enter__()`, run the block, then call `thing.__exit__(exc_type, exc, traceback)` whatever happened. If `__exit__` returns `True`, the exception is swallowed; if it returns `False` (or nothing), the exception continues.

Two ways to make one:

1. **A class** with `__enter__` and `__exit__`.
2. **A generator** with `@contextlib.contextmanager`: code before `yield` is the setup, the `yield` gives the value, and code after `yield` (in `finally`) is the cleanup. This is shorter and used most often.

Ready-made helpers in `contextlib`: `suppress(...)`, `redirect_stdout`, `ExitStack` (a stack of many managers when you do not know how many beforehand).

Why does it exist? Before it, people wrote `try/finally` by hand and forgot one. `with` makes the right way the easy way.

Worked examples

Problem 7.4.1 Write a class-based `Timer` context manager. See what happens with and without an error.

Step 1	`__enter__` records the start time and returns `self`.
Step 2	`__exit__` records the elapsed time and prints whether an exception happened. It returns `False`, so errors still propagate.
Step 3	Run a normal block, and then a block that divides by zero.

Predict: In the failing block, does `__exit__` run? Does the `ZeroDivisionError` still reach us?

```
import time

class Timer:
    def __enter__(self):
        self.t0 = time.perf_counter()
        print("enter")
        return self
    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.t0
        print("exit, exception:", exc_type.__name__ if exc_type else None)
        return False

with Timer() as t:
    time.sleep(0.01)
print(t.elapsed > 0.005)

try:
    with Timer():
        1/0
except ZeroDivisionError:
    print("error still propagates")
```

```
enter
exit, exception: None
True
enter
exit, exception: ZeroDivisionError
error still propagates
```

- **Explain** In both runs `__exit__` ran, so cleanup is guaranteed. In the failing run it received `ZeroDivisionError` as `exc_type`. Because we returned `False`, the error continued outwards and our `except` caught it.
- **Answer** `__exit__` always runs, and errors are not hidden unless you return `True`.
- **Check** The `sleep(0.01)` is 10 ms, so `elapsed > 0.005` (5 ms) is `True`.
- **Why it matters** In plain words: the timer reports even when the code fails, which is exactly when you want a measurement.

Problem 7.4.2 Swallow only one kind of error with a class, and with `contextlib.suppress`.

Step 1	`Suppress.__exit__` returns `True` only when the exception is a `KeyError`.
Step 2	`with contextlib.suppress(FileNotFoundError):` removes a file that may not exist.

```
import contextlib, os

class Suppress:
    def __enter__(self):
        return self
    def __exit__(self, et, e, tb):
        return et is KeyError

with Suppress():
    {}["x"]
print("KeyError swallowed")

with contextlib.suppress(FileNotFoundError):
    os.remove("/tmp/definitely_not_here_123")
print("suppress ok")
```

```
KeyError swallowed
suppress ok
```

- **Answer** Both blocks swallowed their expected error and the program carried on.
- **Check** Removing a missing file would normally raise `FileNotFoundError`; since we saw `suppress ok`, it was swallowed.
- **Why it matters** `suppress` is perfect for "delete if exists" and similar harmless cases. Never use it for errors you do not understand.

Problem 7.4.3 Write context managers with `@contextlib.contextmanager`: one that changes into a temporary folder and always returns, one that prints open/close, and combine several in one `with`.

Step 1	Save the old folder, create a temp folder, `os.chdir` into it, `yield`, and in `finally` go back and remove it.
Step 2	`opened(name)` prints `open` before the `yield` and `close` after, inside `finally`.
Step 3	`with opened("a") as a, opened("b") as b:` enters left to right and exits right to left.
Step 4	`redirect_stdout` captures prints; `ExitStack` handles a list of managers.

```
import contextlib, os, tempfile, io

@contextlib.contextmanager
def chdir_temp():
    old = os.getcwd()
    d = tempfile.mkdtemp()
    os.chdir(d)
    try:
        yield d
    finally:
        os.chdir(old)
        os.rmdir(d)
        print("restored")

with chdir_temp() as d:
    print(os.path.samefile(os.getcwd(), d))

@contextlib.contextmanager
def opened(name):
    print("open", name)
    try:
        yield name
    finally:
        print("close", name)

with opened("a") as a, opened("b") as b:
    print("inside", a, b)

buf = io.StringIO()
with contextlib.redirect_stdout(buf):
    print("captured")
print(repr(buf.getvalue()))

with contextlib.ExitStack() as st:
    fs = [st.enter_context(opened(n)) for n in "xy"]
    print("both open", fs)
```

```
True
restored
open a
open b
inside a b
close b
close a
'captured\n'
open x
open y
both open ['x', 'y']
close y
close x
```

- **Explain** Inside `chdir_temp` the working folder really is the temp folder (`samefile` is `True`), and afterwards we are back (`restored`). `a` is opened first and closed last, like putting on and taking off a coat and a jacket in the right order. `redirect_stdout` sent the `print` into the buffer, which is useful in tests. `ExitStack` opened `x` and `y` and closed them in reverse order.
- **Answer** Cleanup always ran, in reverse order of setup.
- **Check** The open/close order is a stack: `open a, open b ... close b, close a` . The same pattern appears for `x` and `y` .
- **Why it matters** The `try/finally` around `yield` is what guarantees the cleanup if the block raises an error. Leave it out and a failure leaves your program in the wrong folder.

The typical wrong version. Opening a file without `with` , when an error happens before `close()` :

```
f = open("data.txt", encoding="utf-8")
total = 1 / 0        # error: the next line never runs
f.close()
```

The file stays open until the program ends. We checked this behaviour: after an error that was caught elsewhere the file object still reported `closed == False` (our run printed `without with, still open: True`). With `with open(...) as f:` the file is closed on the way out. Leaked handles become "Too many open files" errors in long-running jobs.

The picture

```
with A() as x:          __enter__()  ──► x
    body                run body ──► (ok or error)
                        __exit__(error info)  ◄── ALWAYS runs
                            return True  → error swallowed
                            return False → error continues up

@contextmanager:  [setup] ── yield value ── [cleanup in finally]
```

Item	Example value	Where it appears in the job
File	`with open(p) as f:`	Reading and writing files
DB connection	`with engine.begin() as conn:`	Commit on success, rollback on error
Lock	`with lock:`	Threads (Chapter 11)
Temp folder	`with tempfile.TemporaryDirectory()`	Tests
`mock.patch`	`with patch("x.y"):`	Testing (Chapter 10)
GPU / no-grad	`with torch.no_grad():`	Machine learning

Connection to the real job

Database sessions, HTTP sessions, file handles, locks, temporary folders, test mocks and ML inference modes are all context managers. A data engineer who forgets to close connections eventually brings down a database. A senior reviewer looks for `with` around every resource.

Applications

- **Data Engineering:** `with` around database transactions: commit if fine, roll back on error.
- **Machine Learning:** `with torch.no_grad():` switches off gradient tracking for inference.
- **Testing / MLOps:** temporary directories and patched settings that are restored after each test.
- **Performance Testing:** a `Timer` block that records the duration of a code section.

COMMON MISTAKE

Returning `True` from `__exit__` by accident. Every error then disappears silently. Return `False` or `None` unless you mean to swallow.

COMMON MISTAKE

In `@contextmanager`, putting the cleanup after `yield` without `try/finally`. If the block raises, the cleanup is skipped.

COMMON MISTAKE

Using the resource after the `with` block. The file is closed; reading it gives `ValueError: I/O operation on closed file`.

Check yourself

1. When does `__exit__` run? Answer: always at the end of the `with` block, even after an error.
2. How do you swallow an exception in `__exit__`? Answer: return `True`.
3. In `@contextmanager`, what does the `yield` mark? Answer: the point where the `with` body runs; code before is setup, code after is cleanup.

Chapter recap

- An iterable can be looped over; an iterator hands out items with `next()` and raises `StopIteration` at the end.
- An iterator can be used only once. A list gives a fresh iterator for each loop.
- A generator function uses `yield`; calling it runs no code until you ask for items (lazy evaluation).
- Generators use tiny, constant memory: 208 bytes versus about 8.4 MB for one million squares (container only).
- Chain generators into a pipeline (parse, filter, extract) so each item flows through all stages one by one.
- A decorator is a function that wraps a function; `@d` is `f = d(f)`.
- Always use `functools.wraps`, `*args, **kwargs`, and remember to return the result.
- A decorator with arguments needs a third layer; do not forget the parentheses: `@retry(3)`.
- Useful ones: timing, retry (only for temporary errors), `lru_cache` (only for pure functions), registries.
- A context manager guarantees cleanup via `with`; build one with `__enter__/__exit__` or `@contextlib.contextmanager` with `try/finally`.

Cheat-sheet

Task	Pattern

Manual iteration	`it = iter(x); next(it); next(it, default)`
Generator function	`def g(): yield value`
Generator expression	`(f(x) for x in data if cond)`
Delegate	`yield from other`
First N of a stream	`list(itertools.islice(gen, N))`
Decorator skeleton	`def d(f): @wraps(f) def w(*a, **k): return f(*a, **k); return w`
Decorator with args	`def d(n): def deco(f): ...; return deco`
Cache pure function	`@functools.lru_cache(maxsize=None)` or `@functools.cache`
Context manager (class)	`__enter__(self)` and `__exit__(self, et, e, tb)`
Context manager (function)	`@contextmanager def cm(): setup; try: yield; finally: cleanup`

Practice set

Problem 7.P.1 What is `next(iter([5, 6]))` ?

Answer: `5` .

Problem 7.P.2 What does this print?

```
g = (x * 2 for x in [1, 2, 3])
print(list(g))
print(list(g))
```

Answer: `[2, 4, 6]` then `[]` . The generator is used up after the first pass.

Problem 7.P.3 What is the output?

```
def f():
    yield 1
    yield 2
print(f())
```

Answer: Something like `<generator object f at 0x...>`. The function body did not run, and the address changes every run.

Problem 7.P.4 Write a generator `evens(limit)` that yields even numbers below `limit`. What is `list(evens(10))`?

Answer:

```
def evens(limit):
    for n in range(limit):
        if n % 2 == 0:
            yield n
```

`list(evens(10))` is `[0, 2, 4, 6, 8]`.

Problem 7.P.5 Find the bug:

```
def shout(func):
    def wrapper(*args, **kwargs):
        print("calling", func.__name__)
        func(*args, **kwargs)
    return wrapper

@shout
def add(a, b):
    return a + b
print(add(2, 3))
```

Answer: The wrapper does not `return` the result, so `add(2, 3)` prints `calling add` and then `None`. Write `return func(*args, **kwargs)`. Also add `@functools.wraps(func)`.

Problem 7.P.6 In what order do these print?

```
@tag("b")
@tag("i")
def text(): return "hi"
```

where `tag(name)` wraps the output as `<name>...</name>`.

Answer: The result is `<b><i>hi</i></b>`. The decorator nearest the function (`i`) is applied first and ends up inside.

Problem 7.P.7 What does `with opened("a"), opened("b"):` print when the body prints `inside`, if `opened` prints `open` / `close`?

Answer: `open a` , `open b` , `inside` , `close b` , `close a` . Exit is in reverse order.

Problem 7.P.8 Find the bug: a `@contextmanager` function does `yield conn` and then `conn.close()` on the next line, with no `try/finally` . What goes wrong?

Answer: If the `with` body raises an error, the exception is thrown at the `yield` , and the `conn.close()` line never runs, so the connection leaks. Use `try: yield conn finally: conn.close()` .

Problem 7.P.9 Scenario: a daily job processes a 30 GB file of JSON Lines to count errors per hour. How do you structure it?

Answer: Open the file with `with open(...)` , read lines lazily, and build generator stages: `parse_json(lines)` (skip bad lines and count them), `only_errors(rows)` , and a final loop that updates a `Counter` keyed by hour. Memory stays small because only one line is alive at a time. Log the number of skipped bad lines at the end.

Problem 7.P.10 Interview: what is the difference between an iterator and an iterable? Give an example of each.

Answer: An iterable can be looped over and returns a new iterator from `iter()` (list, str, dict, file path object contents). An iterator keeps its position, gives items with `next()` , and is used up once (`iter([1,2])` , a generator, an open file). Every iterator is iterable, but not every iterable is an iterator.

Problem 7.P.11 Interview: write a `timed` decorator that logs the function name and duration, and explain why you use `functools.wraps` .

Answer:

```
import functools, logging, time
log = logging.getLogger(__name__)

def timed(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            log.info("%s took %.1f ms", func.__name__, (time.perf_counter() -
t0) * 1000)
    return wrapper
```

`wraps` keeps the original `__name__` and docstring; without it every function would be logged as

`wrapper` . `try/finally` logs even on failure and `perf_counter` is a steady clock.

Problem 7.P.12 Interview: when is `lru_cache` a bad idea?

Answer: When the function is not pure (it reads the clock, a database, a file, or random numbers) because stale results would be returned; when arguments are not hashable (lists, dicts); and when the number of distinct inputs is huge with no `maxsize` , since the cache grows forever and eats memory.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles, and English is my second language, so use simple words. Teach
me Python iterators and the iterator protocol: iterable versus iterator,
iter(), next(), StopIteration, and how a for loop works inside. Use a vending
machine analogy first, then small runnable examples. After each part, ask me
one "predict the output" question and wait for my answer before continuing.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles. Topic: Python generators, yield, yield from, generator
expressions, lazy evaluation and memory benefits, and generator pipelines for
processing large log files. Give me 8 graded exercises from easy to hard
without solutions. After I paste my answer to each, review it for bugs, memory
use and clarity, and explain mistakes in simple English.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles, and English is my second language. Topic: Python decorators:
closures, functools.wraps, *args and **kwargs, decorators with arguments,
stacking order, class decorators, and practical decorators for timing, caching,
retry and logging. Explain step by step with the gift-wrapping analogy, then
review my decorator code: [PASTE HERE]. Point out bugs and suggest
improvements.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles. Topic: Python context managers: the with statement, __enter__
and __exit__, contextlib.contextmanager, suppress, redirect_stdout and
ExitStack, and why cleanup must be guaranteed. Quiz me one question at a time,
from easy to hard, including find-the-bug questions. Explain each wrong answer
```

step by step, and finish with a short summary of what I should practise next.

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Topic: iterators, generators, decorators and context managers in Python. Run a mock technical interview with 8 questions of rising difficulty, one at a time. Grade each answer out of 10, show a model answer, and ask one follow-up question. At the end, list my three weakest points and give me a one-week practice plan of one hour per day.
```

CHAPTER 8

OBJECT-ORIENTED PROGRAMMING

Classes, special methods, inheritance, composition, SOLID and the design patterns you will meet in real code.

Open any serious Python library and you will find classes: pandas has `DataFrame`, scikit-learn has `LinearRegression`, FastAPI has `FastAPI`, Locust has `HttpUser`. To read that code, and to write code that other engineers can extend without breaking it, you need object-oriented programming (OOP). OOP is a way of organising a program so that **data and the functions that work on that data live together**. By the end of this chapter you will write your own classes, make them behave like built-in types, choose between inheritance and composition, apply the SOLID rules, and recognise the nine design patterns that interviewers ask about. We use one running idea throughout: a small online shop (orders, money, users, discounts). All code was run with Python 3.14 and the outputs are real.

8.1 Classes, Instances and Attributes

INTUITION

Think of a cookie cutter. It is a template: one shape. When you press it into dough, you get 10 real cookies from that template. Each cookie has its own decorations, but all follow the same shape. A **class** is the template. An **instance** (also called an **object**) is one real thing made from it.

Words you need

Word	Plain meaning
class	A template that describes what data and behaviour its objects have
instance / object	One real thing made from a class
attribute	A named piece of data stored on a class or object
method	A function defined inside a class
`self`	The object the method is working on. Python passes it automatically
`__init__`	A method that runs right after an object is created, to set up its data

How it works. `class Name:` defines a class. Calling `Name(...)` creates an instance and runs `__init__`. An **instance attribute** (`self.count`) belongs to one object. A **class attribute** (written directly in the class body) is shared by all objects. Why does `self` exist? A method must know which object it should change. If there were no `self`, `hit()` would not know whose counter to increase.

Real-world mapping

Concept	Example value	Where it appears in the job
Class	`LinearRegression`	scikit-learn model template
Instance	`model = LinearRegression()`	One trained model
Instance attribute	`model.coef_`	Weights learned from your data
Method	`model.fit(X, y)`	Changes the object's own data
Class attribute	`HttpUser.wait_time`	A setting shared by all virtual users

Problem 8.1.1 Create a `Counter` class that counts hits per endpoint and counts how many counters exist.

Before you run it, predict: after `a.total_created = 99`, what will `b.total_created` print?

- **Step 1** Put `total_created` in the class body (shared). Put `name` and `count` in `__init__` (one set per object).
- **Step 2** Create two counters and hit them.
- **Step 3** Assign `a.total_created = 99` and compare.

```
class Counter:
    total_created = 0          # class attribute
    def __init__(self, name):
        self.name = name       # instance attribute
        self.count = 0
        Counter.total_created += 1
    def hit(self):
        self.count += 1
        return self.count

a = Counter("login")
b = Counter("search")
a.hit(); a.hit(); b.hit()
print(a.name, a.count, b.name, b.count)
print(Counter.total_created, a.total_created)
a.total_created = 99          # creates an instance attribute, shadows class
one
print(a.total_created, b.total_created, Counter.total_created)
print(a.__dict__)
print(type(a), type(Counter))
```

```
login 2 search 1
2 2
99 2 2
{'name': 'login', 'count': 2, 'total_created': 99}
<class '__main__.Counter'> <class 'type'>
```

Line by line: `a` was hit twice and `b` once, so each keeps its own `count`. Two counters were created, so the shared value is 2. Writing `a.total_created = 99` did not change the class value. It made a **new instance attribute** on `a` (you can see it inside `a.__dict__`), which hides the class one for `a` only.

- **Answer** `b.total_created` prints 2. Assignment through an object never edits the class attribute.
- **Check** `Counter.total_created` is still 2 in the output, and `a.__dict__` contains the new key.
- **Why it matters** Mixing up "per-object" and "shared" data is the number one OOP bug in data code.

Problem 8.1.2 Show the classic shared-list bug and its fix.

Predict: do `x` and `y` end up with one tag each?

```
class Bad:
    tags = []
    def add(self, t): self.tags.append(t)
x, y = Bad(), Bad()
x.add("a"); y.add("b")
print(x.tags, y.tags, x.tags is y.tags)
class Good:
    def __init__(self): self.tags = []
    def add(self, t): self.tags.append(t)
x, y = Good(), Good()
x.add("a"); y.add("b")
print(x.tags, y.tags)
```

```
['a', 'b'] ['a', 'b'] True
['a'] ['b']
```

- **Answer** No. In `Bad`, one list lives on the class, so both objects add to the same list (`is` prints `True`). In `Good`, each object makes its own list in `__init__`.
- **Check** `x.tags is y.tags` is `True` in `Bad` and the two lists differ in `Good`.
- **Why it matters** Two "independent" shop customers that secretly share one basket list will corrupt each other's orders.

The typical beginner errors and their real messages

```
class A:
    def hit(): return 1       # forgot self
A().hit()
```

```
A.hit() takes 0 positional arguments but 1 was given
```

Python passed the object as the first argument, but you declared none. Fix: `def hit(self):`.

```
class B:
    def __init__(self, n): self.n = n
B()
```

```
B.__init__() missing 1 required positional argument: 'n'
```

Fix: `B(5)`, or give `n` a default.

The picture

```
Class Counter (template)          Instances (real objects)
+--------------------+            a --> {name:'login',  count:2}
| total_created = 2  | <-- shared  b --> {name:'search', count:1}
| __init__, hit()    |            (each has its own __dict__)
+--------------------+
```

Read it like this: the box on the left exists once. Each arrow on the right is one object holding its own dictionary of attributes. When Python looks up `a.something`, it checks `a`'s own dictionary first, then the class.

Connection to the real job. Pipelines hold state in objects: a database connection, a model with learned weights, a configured API client. When results differ across runs, a shared mutable class attribute is a top suspect.

Applications

- **Data Analytics:** a `Report` class that holds a DataFrame, a title and an `export()` method.
- **Data Science / ML:** every scikit-learn estimator is a class with `fit` and `predict`.
- **Deep Learning:** a PyTorch model is a class that inherits from `nn.Module`.
- **Data Engineering:** a `PostgresConnection` object that owns one open connection.
- **Load simulation:** tools such as Locust define users as classes; each instance is one simulated user.

COMMON MISTAKE

Putting a list or dict in the class body, or as a default argument (`def __init__(self, d={})`), and expecting one copy per object. Mutable starting values go inside `__init__`. Class attributes should be constants.

Check yourself

1. What does `self` refer to?

Answer: The specific object the method was called on.

1. Where do you put data that must be separate for each object?

Answer: In `__init__`, as `self.something`.

1. If you write `obj.x = 5` and `x` was a class attribute, what changes?

Answer: Only `obj` gets a new instance attribute. The class attribute is unchanged.

8.2 Dunder (Special) Methods

INTUITION

When you type `len(x)` or `a + b` or `print(obj)`, Python quietly calls a hidden method on your object. These methods have double underscores on each side, so people say "dunder" (double-underscore). A restaurant analogy: you just say "the usual", and the waiter knows what that means. Dunders are how your class defines what "the usual" means for `+`, `len`, `print` and `for`.

Why they exist. Without them, your `Money` object could not be sorted, added or printed nicely. With them it behaves like a number. Python decides what an operator means by calling the method, not by hard-coded rules.

You write	Python calls	Purpose
`repr(x)`, list display, debugger	`__repr__`	Unambiguous text for developers
`print(x)`, `str(x)`	`__str__`	Friendly text for users
`a == b`	`__eq__`	Equality
`hash(x)`, set member, dict key	`__hash__`	Fixed number for fast lookup
`a < b`	`__lt__`	Ordering
`a + b`	`__add__`	Addition
`len(x)`	`__len__`	Size
`x[i]`	`__getitem__`	Indexing and slicing
`for v in x`	`__iter__`	Iteration
`v in x`	`__contains__`	Membership
`x(...)`	`__call__`	Make an object callable like a function
`with x:`	`__enter__`, `__exit__`	Context manager

`if x:`	`__bool__`	Truthiness

Two rules to remember. First, if `__str__` is missing, Python uses `__repr__`. So always write `__repr__`. Second, if you define `__eq__` and not `__hash__`, Python makes your object **unhashable** (it cannot go in a set or be a dict key). Objects that are equal must have equal hashes.

Problem 8.2.1 Build a `Money` class that stores paise (1 rupee = 100 paise) as an integer to avoid float errors, with printing, comparison, addition and hashing.

Predict: what does `sorted([Money(10,50), Money(3,75), Money(1)])` print?

- **Step 1** Store `self.paise = rupees*100 + paise`. Integers are exact; `0.1 + 0.2` floats are not.
- **Step 2** Write `__repr__`, `__str__`, `__eq__`, `__lt__`, `__hash__`, `__add__`, `__bool__`.
- **Step 3** Add `@total_ordering` from `functools`: given `__eq__` and `__lt__`, it fills in `>`, `>=`, `<=`.

```
from functools import total_ordering
@total_ordering
class Money:
    def __init__(self, rupees, paise=0):
        self.paise = rupees*100 + paise
    def __repr__(self): return f"Money({self.paise//100}, {self.paise%100})"
    def __str__(self): return f"Rs {self.paise/100:.2f}"
    def __eq__(self, o): return isinstance(o, Money) and self.paise == o.paise
    def __lt__(self, o): return self.paise < o.paise
    def __hash__(self): return hash(self.paise)
    def __add__(self, o): return Money(0, self.paise + o.paise)
    def __bool__(self): return self.paise != 0

m1, m2 = Money(10, 50), Money(3, 75)
print(repr(m1), str(m1), m1 + m2)
print(m1 > m2, m1 == Money(10,50), m1 >= m2)
print(sorted([m1, m2, Money(1)]))
print({m1, Money(10,50)})
print(bool(Money(0)))
print([m1, m2])
```

```
Money(10, 50) Rs 10.50 Rs 14.25
True True True
[Money(1, 0), Money(3, 75), Money(10, 50)]
{Money(10, 50)}
False
[Money(10, 50), Money(3, 75)]
```

Line by line: `repr` gives the developer view, `str` gives the user view, and `print(m1 + m2)` uses `__str__`, so we see `Rs 14.25` (1050 + 375 = 1425 paise). Sorting works because `__lt__` exists. The set keeps one item because two equal `Money(10,50)` have the same hash. `Money(0)` is falsy because of `__bool__`. A list shows items using `__repr__`.

- **Answer** It prints `[Money(1, 0), Money(3, 75), Money(10, 50)]`, ordered 100, 375, 1050 paise.
- **Check** 1050 + 375 = 1425 paise = Rs 14.25, matching the output.
- **Why it matters** Money, timestamps and IDs in real systems need exact equality and ordering. This is how pandas and Decimal-like types are built.

Problem 8.2.2 Make a container that behaves like a sequence, and an object that behaves like a function.

```
class Playlist:
    def __init__(self, *songs): self._s = list(songs)
    def __len__(self): return len(self._s)
    def __getitem__(self, i): return self._s[i]
    def __iter__(self): return iter(self._s)
    def __contains__(self, x): return x in self._s
p = Playlist("a","b","c")
print(len(p), p[0], p[-1], p[0:2], "b" in p, list(p))

class Timer:
    def __init__(self, f): self.f = f; self.calls = 0
    def __call__(self, *a):
        self.calls += 1
        return self.f(*a)
sq = Timer(lambda x: x*x)
print(sq(4), sq(5), sq.calls)
```

```
3 a c ['a', 'b'] True ['a', 'b', 'c']
16 25 2
```

- **Answer** `len`, indexing, slicing, `in` and `list()` all work on `Playlist` because we defined four dunders. `sq` is called like a function but remembers state: it was called twice.
- **Check** `4*4 = 16`, `5*5 = 25`, two calls. The slice `p[0:2]` passes a slice object to `__getitem__`, which the inner list handles.
- **Why it matters** Callable objects with state are how decorators with memory, Keras layers and PyTorch modules work.

Typical wrong version

```
class P:
    def __init__(self, x): self.x = x
    def __eq__(self, o): return self.x == o.x
{P(1)}
```

```
cannot use 'P' as a set element (unhashable type: 'P')
```

Defining `__eq__` removed the default hash. Fix: add `def __hash__(self): return hash(self.x)`. Also note that a class with no `__eq__` compares by identity, so `Q(1) == Q(1)` is `False` for a plain class.

Connection to the real job. pandas, NumPy and PyTorch are full of dunders (`df[col]` is `__getitem__`, `a + b` on arrays is `__add__`). `__enter__/__exit__` power `with open(...)` and database transactions (full treatment in Chapter 7).

Applications

- **Data Analytics:** custom types like `Money` or `Percent` that sort and add correctly.
- **Machine Learning:** `Dataset` classes with `__len__` and `__getitem__` (the exact contract PyTorch `Dataset` expects).
- **Data Engineering:** hashable record keys for deduplication in sets.
- **MLOps:** callable objects as pipeline steps.

COMMON MISTAKE

Writing `__eq__` without `__hash__`, or writing `__str__` and skipping `__repr__`. Always write `__repr__` first. It is what you see in logs and the debugger.

Check yourself

1. Which dunder does `print(obj)` call first?

Answer: `__str__`. If missing, it falls back to `__repr__`.

1. Why can an object with only `__eq__` not go in a set?

Answer: Defining `__eq__` sets `__hash__` to `None`, so it is unhashable.

1. Which method makes `obj(5)` legal?

Answer: `__call__`.

8.3 Inheritance, `super()`, MRO and Mixins

INTUITION

A child inherits eye colour from parents but may have its own habits. In code, a **subclass** inherits all methods of its **parent class** (also called base or superclass) and can add new ones or **override** (replace) old ones. It saves you from copying code.

How it works. `class Dog(Animal):` means Dog is-an Animal. `super()` gives access to the parent's version of a method. **MRO** (Method Resolution Order) is the list Python searches, in order, to find a method. For multiple inheritance Python uses the C3 rule: children before parents, and the order you wrote the parents in is respected.

Problem 8.3.1 Single inheritance with an override.

```
class Animal:
    def __init__(self, name): self.name = name
    def speak(self): return "..."
    def intro(self): return f"{self.name} says {self.speak()}"
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
    def speak(self): return "Woof"
d = Dog("Rex", "Lab")
print(d.intro(), isinstance(d, Animal), issubclass(Dog, Animal))
```

```
Rex says Woof True True
```

- **Answer** `intro` is inherited from `Animal`, but it calls `self.speak()`, which finds `Dog.speak`. So Python picks the most specific version. This is called **polymorphism**: same call, different behaviour per class.
- **Check** `isinstance(d, Animal)` is `True`, since a Dog is an Animal.
- **Why it matters** Frameworks call your overridden methods (`fit`, `run`, `handle`) without knowing your class.

Problem 8.3.2 The diamond: what order does `super()` follow?

Predict the output for `D().who()` where `D(B, C)` and both `B` and `C` extend `A`.

```
class A:
    def who(self): return "A"
class B(A):
    def who(self): return "B>" + super().who()
class C(A):
    def who(self): return "C>" + super().who()
class D(B, C):
    def who(self): return "D>" + super().who()
print(D().who())
print([k.__name__ for k in D.__mro__])
```

```
D>B>C>A
['D', 'B', 'C', 'A', 'object']
```

- **Answer** `super()` does not mean "my parent". It means "the next class in the MRO". So `B` 's `super()` goes to `C` , not straight to `A` . `A` is called once, last.
- **Check** The printed call chain `D>B>C>A` equals the MRO list.
- **Why it matters** This is a classic interview question. The answer: use `ClassName.__mro__` to see the order.

Mixins. A **mixin** is a small class that adds one ability and is not meant to stand alone.

```
class JsonMixin:
    def to_json(self):
        import json
        return json.dumps(self.__dict__)
class LogMixin:
    def log(self, msg): print(f"[{type(self).__name__}] {msg}")
class User(JsonMixin, LogMixin):
    def __init__(self, n, a): self.name=n; self.age=a
u = User("Asha", 30)
u.log(u.to_json())
try:
    class X(A, B): pass
except TypeError as e: print("TypeError:", e)
```

```
[User] {"name": "Asha", "age": 30}
TypeError: Cannot create a consistent method resolution order (MRO) for bases
A, B
```

The second case fails because `A` is a parent of `B` , yet we asked for `A` before `B` . Python cannot satisfy "children before parents" and says so.

The picture

```
   A
  / \
 B   C        MRO of D:  D -> B -> C -> A -> object
  \ /
   D          super() in B calls the next one in this list (C)
```

Connection to the real job. Django, scikit-learn (`BaseEstimator` , `TransformerMixin`) and PyTorch rely on inheritance and mixins. You will subclass them and override one or two methods.

Applications

- **Machine Learning:** custom transformer inherits `BaseEstimator` and `TransformerMixin` .
- **Data Engineering:** Airflow operators subclass `BaseOperator` .
- **Load simulation:** tools such as Locust use `HttpUser` subclasses to define tasks.
- **GenAI:** custom tools and callbacks subclass framework base classes.

COMMON MISTAKE

Using inheritance just to reuse code when the relation is not "is-a". A `Car` is not an `Engine` . Use composition (section 8.7).

Check yourself

1. What does `super()` return?

Answer: A proxy that looks up the method in the next class in the MRO.

1. How do you see the MRO?

Answer: `ClassName.__mro__` or `ClassName.mro()` .

1. What is a mixin?

Answer: A small class that adds one ability to other classes through inheritance.

8.4 Abstract Base Classes and Protocols

INTUITION

A plug socket does not care what appliance you connect, only that the plug has the right shape. An **interface** is that shape: a list of methods any object must have. An **abstract base class (ABC)** enforces the shape by inheritance. A **Protocol** enforces it by "if it has the right methods, it fits" (called structural typing, or duck typing with a checker).

How it works. With `abc.ABC` and `@abstractmethod`, you cannot create an object until every abstract method is implemented. This catches mistakes at the moment of creation, not halfway through a pipeline.

Problem 8.4.1 Define an `Exporter` interface and see it refuse to be incomplete.

```
from abc import ABC, abstractmethod

class Exporter(ABC):
    @abstractmethod
    def export(self, rows): ...
    def run(self, rows):
        return f"{len(rows)} rows -> {self.export(rows)}"
class CsvExporter(Exporter):
    def export(self, rows): return "\n".join(",".join(map(str,r)) for r in
rows)
try:
    Exporter()
except TypeError as e: print("TypeError:", e)
class Half(Exporter): pass
try: Half()
except TypeError as e: print("TypeError:", e)
print(CsvExporter().run([(1,2),(3,4)]))
```

```
TypeError: Can't instantiate abstract class Exporter without an implementation
for abstract method 'export'
TypeError: Can't instantiate abstract class Half without an implementation for
abstract method 'export'
2 rows -> 1,2
3,4
```

- **Answer** You cannot build the abstract class or a half-finished subclass. `CsvExporter` works, and it reuses `run` from the base class (this is the "template method" idea).
- **Check** The error message names the missing method.
- **Why it matters** Teams use ABCs so that every connector (CSV, Parquet, database) offers the same methods.

Problem 8.4.2 A Protocol needs no inheritance.

```
from typing import Protocol, runtime_checkable

class Closeable(Protocol):
    def close(self) -> None: ...
class FileLike:
    def close(self): print("closed file-like")
def shutdown(x: Closeable): x.close()
shutdown(FileLike())    # no inheritance needed

@runtime_checkable
class Sized2(Protocol):
    def __len__(self) -> int: ...
print(isinstance([1], Sized2), isinstance(5, Sized2))
```

```
closed file-like
True False
```

- **Answer** `FileLike` never mentions `Closeable`, yet it fits because it has `close()`. A type checker such as mypy (Chapter 9) verifies the fit. With `@runtime_checkable`, `isinstance` can check that the methods exist: a list has `__len__`, an int does not.
- **Check** `isinstance(5, Sized2)` is `False`, as `int` has no `__len__`.
- **Why it matters** Protocols let you accept "anything that looks like a file" without forcing everyone to inherit from your class.

Tool	Enforced when	Needs inheritance	Use when
ABC	At object creation	Yes	You own all the classes and want shared code
Protocol	By type checker (mypy)	No	You accept third-party objects

Connection to the real job. Storage backends (local disk, S3), model loaders and readers all share an interface so the pipeline code never changes.

Applications

- **Data Engineering:** `Source` and `Sink` interfaces for connectors.
- **MLOps:** a `ModelStore` interface with local and cloud versions.
- **Data Science:** a `Metric` protocol: anything with `__call__(y_true, y_pred)`.

COMMON MISTAKE

Forgetting that `Protocol` is checked by tools, not by Python at runtime. Without mypy, a wrong object fails only when the missing method is called.

Check yourself

1. What error appears when you instantiate an incomplete ABC?

Answer: `TypeError: Can't instantiate abstract class ...`.

1. Does a class need to inherit from a Protocol?

Answer: No. Having the right methods is enough.

1. When prefer a Protocol over an ABC?

Answer: When you want to accept objects you do not control.

8.5 Properties, `@classmethod` and `@staticmethod`

INTUITION

A bank clerk checks your form before accepting it. A **property** lets you keep simple attribute syntax (`t.celsius = 30`) while a hidden function validates or computes the value. You can start with a plain attribute and add checks later without changing any calling code.

Three kinds of method

Decorator	First argument	Use
none	`self` (the object)	Normal behaviour
`@classmethod`	`cls` (the class)	Alternative constructors like `from_url`
`@staticmethod`	nothing	A helper that lives in the class for tidiness
`@property`	`self`	Looks like an attribute, runs code

Problem 8.5.1 A temperature with validation and a computed attribute.

```
class Temperature:
    def __init__(self, c): self.celsius = c
    @property
    def celsius(self): return self._c
    @celsius.setter
    def celsius(self, v):
        if v < -273.15: raise ValueError("below absolute zero")
        self._c = v
    @property
    def fahrenheit(self): return self._c*9/5+32
t = Temperature(100)
print(t.celsius, t.fahrenheit)
t.celsius = 0
print(t.fahrenheit)
try: t.celsius = -300
except ValueError as e: print("ValueError:", e)
try: t.fahrenheit = 5
except AttributeError as e: print("AttributeError:", e)
```

```
100 212.0
32.0
ValueError: below absolute zero
AttributeError: property 'fahrenheit' of 'Temperature' object has no setter
```

- **Answer** `100 C = 100*9/5 + 32 = 212 F` and `0 C = 32 F`. Setting an impossible temperature is rejected, and the read-only `fahrenheit` cannot be assigned.
- **Check** 100*9 = 900, 900/5 = 180, 180+32 = 212. Correct.
- **Why it matters** Validation in one place protects every caller.

Note that `__init__` assigns `self.celsius`, so even the first value goes through the setter and is checked.

Problem 8.5.2 An alternative constructor and a helper.

```
class Server:
    def __init__(self, host, port): self.host, self.port = host, port
    @classmethod
    def from_url(cls, url):
        host, port = url.split(":")
        return cls(host, int(port))
    @staticmethod
    def valid_port(p): return 0 < p < 65536
    def __repr__(self): return f"Server({self.host!r}, {self.port})"
print(Server.from_url("db.local:5432"), Server.valid_port(70000))
```

```
Server('db.local', 5432) False
```

- **Answer** `from_url` builds a `Server` from text; it uses `cls`, so subclasses get their own type. `70000` is above the maximum port 65535, so the helper returns `False`.
- **Check** `0 < 70000 < 65536` is false.
- **Why it matters** `pd.DataFrame.from_dict` and `datetime.fromisoformat` are classmethods of the same kind.

Connection to the real job. Config classes use `from_env()` or `from_yaml()` classmethods. Properties compute derived columns (for example `cost_per_request`) without storing them twice.

Applications

- **Data Engineering:** `Config.from_env()` for pipeline settings.
- **Machine Learning:** read-only properties like `n_features_`.
- **Backend:** validated fields on domain objects.

COMMON MISTAKE

Storing into `self.celsius` inside the `celsius` setter. It calls itself forever (`RecursionError`). Store into a different name, like `self._c`.

Check yourself

1. What does `cls` mean in a classmethod?

Answer: The class itself, not an object.

1. How do you make a read-only attribute?

Answer: A property with no setter.

1. Why use a classmethod for `from_url`?

Answer: It is an alternative way to build the object and works with subclasses.

8.6 Dataclasses, NamedTuple, Enum and `__slots__`

INTUITION

Writing `__init__`, `__repr__` and `__eq__` for every small data holder is boring. A **dataclass** writes them for you from a list of fields. A **NamedTuple** is an immutable record with names. An **Enum** is a fixed menu of allowed choices.

Problem 8.6.1 A dataclass with default list, validation and copy.

```
from dataclasses import dataclass, field, asdict, replace

@dataclass
class Request:
    url: str
    latency_ms: float = 0.0
    tags: list = field(default_factory=list)
    def __post_init__(self):
        if self.latency_ms < 0: raise ValueError("negative latency")
r = Request("/login", 120.5)
print(r, r == Request("/login", 120.5))
print(asdict(r), replace(r, latency_ms=99))
try: Request("/x", -1)
except ValueError as e: print("ValueError:", e)
```

```
Request(url='/login', latency_ms=120.5, tags=[]) True
{'url': '/login', 'latency_ms': 120.5, 'tags': []} Request(url='/login',
latency_ms=99, tags=[])
ValueError: negative latency
```

- **Answer** You get `__init__`, `__repr__` and `__eq__` for free. `field(default_factory=list)` gives each object its own list (the fix for the bug in 8.1). `__post_init__` runs after the generated `__init__`, so it is the place for validation.
- **Check** The two requests compare equal because all fields match.
- **Why it matters** `dataclass` is the standard way to model records and configs in modern Python.

Problem 8.6.2 Frozen dataclass, NamedTuple and Enum.

```
from dataclasses import dataclass
from typing import NamedTuple
from enum import Enum, auto

@dataclass(frozen=True, order=True)
class Point:
    x: int
    y: int
p = Point(1,2)
try: p.x = 5
except Exception as e: print(type(e).__name__, e)
print(sorted([Point(2,1), Point(1,9)]), {p: "ok"})

class Pt(NamedTuple):
    x: int
    y: int
q = Pt(3,4)
x, y = q
print(q, q.x, q[1], x+y, q._asdict())

class Status(Enum):
    OK = 200
    NOT_FOUND = 404
    ERR = auto()
print(Status.OK, Status(404), Status["ERR"].value, Status.OK.name)
print([s.name for s in Status])
```

```
FrozenInstanceError cannot assign to field 'x'
[Point(x=1, y=9), Point(x=2, y=1)] {Point(x=1, y=2): 'ok'}
Pt(x=3, y=4) 3 4 7 {'x': 3, 'y': 4}
Status.OK Status.NOT_FOUND 405 OK
['OK', 'NOT_FOUND', 'ERR']
```

- **Answer** A frozen dataclass cannot be changed after creation and therefore is hashable (usable as a dict key). `order=True` sorts by fields in order. A NamedTuple unpacks like a tuple and also has names. `auto()` gave `ERR` the value 405, because it continues counting from the last number (404).
- **Check** `3 + 4 = 7`. `Status(404)` looks up by value, `Status["ERR"]` by name.
- **Why it matters** Enums replace "magic strings" like `"paid"` that cause silent typos.

Choose	When
`dataclass`	Mutable record with methods or defaults
`dataclass(frozen=True)`	Immutable record, dict key

`NamedTuple`	Light immutable record, behaves like a tuple
`Enum`	Fixed set of named choices

`__slots__` **and memory.** Normally each object carries a dictionary of its attributes. `__slots__` says "these are the only attributes", and Python stores them in a compact fixed layout.

```
import sys
class Plain:
    def __init__(self,a,b): self.a=a; self.b=b
class Slim:
    __slots__ = ("a","b")
    def __init__(self,a,b): self.a=a; self.b=b
pl, sl = Plain(1,2), Slim(1,2)
print(sys.getsizeof(pl), sys.getsizeof(pl.__dict__), sys.getsizeof(sl))
try: sl.c = 3
except AttributeError as e: print("AttributeError:", e)
```

```
48 296 48
AttributeError: 'Slim' object has no attribute 'c' and no __dict__ for setting
new attributes
```

`sys.getsizeof` measures only the object itself in bytes (not things it points to). Here `Plain` is 48 bytes plus a 296-byte dictionary, while `Slim` is 48 bytes in total. In a test I made 100,000 objects of each and measured with `tracemalloc`: about 12 MB for `Plain` versus 8 MB for `Slim` (numbers vary by machine and Python version). The saving matters when you hold millions of small objects.

Connection to the real job. Config, API responses, events and rows are modelled as dataclasses. Pydantic (Chapter 9) builds on the same idea, adding validation.

Applications

- **Data Engineering:** typed records for events passing between stages.
- **Machine Learning:** `TrainingConfig` dataclass holding learning rate, epochs.
- **GenAI:** message and tool-call records.
- **Backend:** `Enum` for order status, HTTP method.

COMMON MISTAKE

Using a plain `list` as a dataclass default: `tags: list = []`. Python raises `ValueError: mutable default <class 'list'> for field tags is not allowed: use default_factory`. Use `field(default_factory=list)`.

Check yourself

1. Which makes a dataclass immutable?

Answer: `@dataclass(frozen=True)` .

1. What does `__post_init__` do?

Answer: Runs right after the generated `__init__` , for validation.

1. What does `__slots__` save?

Answer: The per-object attribute dictionary, so less memory.

8.7 Composition, the Four Pillars and SOLID

INTUITION

A car **has an** engine; it **is not an** engine. When an object is built from other objects, that is **composition**. You can swap the engine without rebuilding the car. This is why experienced engineers say "prefer composition over inheritance".

The four pillars of OOP

Pillar	Meaning	Example in this chapter
Encapsulation	Hide internal data; expose a clean surface	`_c` hidden behind property `celsius`
Abstraction	Show what, hide how	`Exporter.export`
Inheritance	Reuse and specialise a parent	`Dog(Animal)`
Polymorphism	One call, many behaviours	`speak()`

Problem 8.7.1 Composition makes swapping (and testing) easy.

```
class Engine:
    def start(self): return "engine on"
class Car:
    def __init__(self, engine): self.engine = engine
    def start(self): return "car: " + self.engine.start()
class FakeEngine:
    def start(self): return "fake"
print(Car(Engine()).start(), "|", Car(FakeEngine()).start())
```

```
car: engine on | car: fake
```

- **Answer** `Car` works with any object that has `start()`. In tests we pass a fake. This idea (give an object its parts from outside) is called **dependency injection**.
- **Check** Both outputs begin with `car:` and differ only in the engine.
- **Why it matters** Tests can run without a real database or network.

SOLID is a set of five design rules. Each is easy to state; the examples show them in code.

Letter	Rule	Plain meaning	Broken sign
S	Single responsibility	A class has one reason to change	A class that parses, calculates and emails
O	Open/closed	Add features by adding code, not editing old code	A long `if/elif` on type names
L	Liskov substitution	A child can replace its parent without surprises	Child raises errors parent never did
I	Interface segregation	Small interfaces, not one huge one	Classes forced to implement unused methods
D	Dependency inversion	Depend on abstractions, not concrete classes	`Service` creates `MySQL()` inside itself

Problem 8.7.2 Spot a Liskov violation.

```
class Bird:
    def fly(self): return "flying"
class Penguin(Bird):
    def fly(self): raise NotImplementedError("penguins cannot fly")
def migrate(birds): return [b.fly() for b in birds]
try: migrate([Bird(), Penguin()])
except NotImplementedError as e: print("LSP broken:", e)
```

```
LSP broken: penguins cannot fly
```

- **Answer** `migrate` expects every `Bird` to fly. `Penguin` breaks that promise, so it cannot safely replace `Bird`. Fix: split the hierarchy (`Bird` and `FlyingBird`).
- **Check** The failure happens inside code that only knows about `Bird`.
- **Why it matters** This kind of silent contract break causes production bugs in pipeline plug-ins.

The open/closed and dependency-inversion examples appear in the next section as Strategy and Dependency Injection.

Connection to the real job. In design interviews, "how would you make this extensible and testable?" is answered with these five letters.

Applications

- **Machine Learning:** swap models behind a common `predict` interface.
- **Data Engineering:** swap storage backends without touching the transform code.
- **Testing:** inject fakes.

COMMON MISTAKE

Over-engineering. Do not create three classes and an interface for a ten-line script. Use these rules when code must grow or be tested, not by default.

Check yourself

1. Is a `Car` an `Engine` or does it have one?

Answer: It has one: use composition.

1. What does the "L" in SOLID check?

Answer: Whether a subclass can replace its parent without breaking callers.

1. What is dependency injection in one line?

Answer: Passing an object's parts in from outside instead of building them inside.

8.8 Design Patterns You Must Recognise

INTUITION

A **design pattern** is a named, proven recipe for a problem that keeps coming back, like "roundabout" for traffic design. Names matter because they let two engineers say "use a Factory" and understand each other at once.

All code below ran together; outputs are in the blocks.

Strategy (swap an algorithm) and **Singleton** (one shared instance):

```
from typing import Callable

def no_discount(t): return t
def festival(t): return t*0.9
def bulk(t): return t*0.8 if t > 1000 else t
STRATEGIES: dict[str, Callable[[float], float]] = {"none": no_discount,
"festival": festival, "bulk": bulk}
def price(total, rule): return STRATEGIES[rule](total)
print(price(1500, "festival"), price(1500, "bulk"), price(500, "bulk"))

class Config:
    _inst = None
    def __new__(cls):
        if cls._inst is None:
            cls._inst = super().__new__(cls)
            cls._inst.values = {}
        return cls._inst
c1, c2 = Config(), Config()
c1.values["env"] = "prod"
print(c1 is c2, c2.values)
import logging
print(logging.getLogger("a") is logging.getLogger("a"))
```

```
1350.0 1200.0 500
True {'env': 'prod'}
True
```

- **Strategy:** each discount rule is a function in a dictionary. To add a new rule you add one entry. No old code changes (open/closed). `1500*0.9 = 1350` , `1500*0.8 = 1200` , and 500 is below the bulk limit so it stays 500.

- **Singleton:** `Config()` always returns the same object, so changes made through `c1` are visible through `c2`. `logging.getLogger("a")` is Python's own example. Use sparingly: global state makes tests hard.

Factory (build the right object from a name):

```
from abc import ABC, abstractmethod
class Reader(ABC):
    @abstractmethod
    def read(self, text): ...
class CsvReader(Reader):
    def read(self, text): return [l.split(",") for l in text.splitlines()]
class PipeReader(Reader):
    def read(self, text): return [l.split("|") for l in text.splitlines()]
def make_reader(kind):
    readers = {"csv": CsvReader, "pipe": PipeReader}
    try: return readers[kind]()
    except KeyError: raise ValueError(f"unknown reader {kind!r}") from None
print(make_reader("pipe").read("a|b\nc|d"))
try: make_reader("xml")
except ValueError as e: print("ValueError:", e)
```

```
[['a', 'b'], ['c', 'd']]
ValueError: unknown reader 'xml'
```

The caller says what it wants (`"pipe"`) and the factory decides which class to create.

Builder (assemble a complex object step by step):

```
class QueryBuilder:
    def __init__(self, table): self.table=table; self._where=[];
self._limit=None; self._params=[]
    def where(self, cond, *params): self._where.append(cond); self._params +=
params; return self
    def limit(self, n): self._limit = n; return self
    def build(self):
        sql = f"SELECT * FROM {self.table}"
        if self._where: sql += " WHERE " + " AND ".join(self._where)
        if self._limit is not None: sql += f" LIMIT {int(self._limit)}"
        return sql, tuple(self._params)
print(QueryBuilder("orders").where("status = ?", "paid").where("amount > ?",
500).limit(10).build())
```

```
('SELECT * FROM orders WHERE status = ? AND amount > ? LIMIT 10', ('paid',
500))
```

Each method returns `self`, so calls chain. Notice that user values travel as separate parameters (the `?`), never pasted into the SQL text. This protects against SQL injection (Chapter 13). Table names cannot be parameters, so they must come from trusted code only.

Observer (publish and subscribe), **Decorator** (wrap an object to add behaviour) and **Adapter** (make an old interface fit a new one):

```
class EventBus:
    def __init__(self): self._subs = {}
    def subscribe(self, event, fn): self._subs.setdefault(event, []).append(fn)
    def publish(self, event, data):
        for fn in self._subs.get(event, []): fn(data)
bus = EventBus()
bus.subscribe("order", lambda d: print("email sent for", d))
bus.subscribe("order", lambda d: print("stock reduced by", d["qty"]))
bus.publish("order", {"id": 7, "qty": 2})

class Notifier:
    def send(self, m): return m
class Upper:
    def __init__(self, inner): self.inner = inner
    def send(self, m): return self.inner.send(m.upper())
class Stars:
    def __init__(self, inner): self.inner = inner
    def send(self, m): return "*** " + self.inner.send(m)
print(Stars(Upper(Notifier())).send("alert"))

class OldLogger:
    def write_line(self, s): print("OLD:", s)
class LoggerAdapter:
    def __init__(self, old): self.old = old
    def info(self, msg): self.old.write_line(f"INFO {msg}")
LoggerAdapter(OldLogger()).info("started")
```

```
email sent for {'id': 7, 'qty': 2}
stock reduced by 2
*** ALERT
OLD: INFO started
```

- **Observer:** the publisher does not know who listens. Add a third subscriber without changing `publish`.
- **Decorator:** objects wrap objects like layers: `Stars` wraps `Upper` wraps `Notifier`, so "alert" becomes "ALERT" and then gets stars. (Not the same as Python's `@decorator` syntax in Chapter 7, but the same wrapping spirit.)
- **Adapter:** `OldLogger` has `write_line`; our code wants `info`. The adapter translates.

Repository (hide the database behind an interface) with **Dependency Injection**:

```
import sqlite3
from dataclasses import dataclass
from typing import Protocol

@dataclass
class User:
    id: int
    name: str

class UserRepo(Protocol):
    def add(self, name: str) -> User: ...
    def get(self, id: int) -> User | None: ...

class SqliteUserRepo:
    def __init__(self, conn):
        self.conn = conn
        conn.execute("CREATE TABLE IF NOT EXISTS users(id INTEGER PRIMARY KEY,
name TEXT)")
    def add(self, name):
        cur = self.conn.execute("INSERT INTO users(name) VALUES (?)", (name,))
        return User(cur.lastrowid, name)
    def get(self, id):
        row = self.conn.execute("SELECT id, name FROM users WHERE id = ?",
(id,)).fetchone()
        return User(*row) if row else None

class InMemoryUserRepo:
    def __init__(self): self.d = {}
    def add(self, name):
        u = User(len(self.d)+1, name); self.d[u.id] = u; return u
    def get(self, id): return self.d.get(id)

class SignupService:
    def __init__(self, repo: UserRepo):
        self.repo = repo
    def signup(self, name):
        if not name.strip(): raise ValueError("empty name")
        return self.repo.add(name.strip())

for repo in (InMemoryUserRepo(), SqliteUserRepo(sqlite3.connect(":memory:"))):
    svc = SignupService(repo)
    u = svc.signup("  Meera ")
    print(type(repo).__name__, u, repo.get(u.id), repo.get(99))
```

```
InMemoryUserRepo User(id=1, name='Meera') User(id=1, name='Meera') None
SqliteUserRepo User(id=1, name='Meera') User(id=1, name='Meera') None
```

`SignupService` holds the business rule (strip the name, reject empty). It never mentions SQL. We **inject** the repository from outside, so tests use the in-memory one and production uses SQLite (or Postgres). Both give identical results here.

Pattern map

Pattern	Problem it solves	Python shortcut
Singleton	One shared instance	A module-level object
Factory	Create right class by name	Dict of classes
Builder	Many optional steps	Method chaining, or dataclass defaults
Strategy	Swap algorithm	Pass a function
Observer	Notify many listeners	List of callbacks
Decorator	Add behaviour by wrapping	Wrapper class or `@decorator`
Adapter	Make interfaces fit	Small wrapper class
Repository	Hide data storage	Class with `add/get`
Dependency Injection	Supply parts from outside	Constructor argument

Connection to the real job. Airflow, scikit-learn pipelines, LangChain callbacks and FastAPI dependencies are these patterns. In system design interviews, Repository plus DI is the standard answer to "how do you keep the database out of business logic".

Applications

- **Data Engineering:** Factory for readers, Repository for warehouse access, Observer for pipeline alerts.
- **MLOps:** Strategy for model choice, Adapter for legacy systems.
- **GenAI:** Strategy for swapping model providers, Observer for streaming callbacks.
- **Backend:** Dependency Injection in FastAPI (`Depends`).

COMMON MISTAKE

Making everything a Singleton. It is a global variable in disguise and makes tests depend on each other. Prefer injecting one shared object.

Check yourself

1. Which pattern lets you swap a discount rule by passing a function?

Answer: Strategy.

1. Why does `SignupService` not call `sqlite3` directly?

Answer: So the storage can be swapped and tested separately (Repository plus DI).

1. What does Adapter do?

Answer: Translates one interface into the one your code expects.

Chapter recap

- A class is a template; an instance is one real object. `self` is that object.
- Put per-object data in `__init__`. Never share mutable data through class attributes or default arguments.
- Dunder methods let your objects work with `print`, `len`, `+`, `<`, `for`, `with` and `()`.
- If you define `__eq__`, also define `__hash__` or the object is unhashable.
- `super()` follows the MRO, not "the parent". Check `Class.__mro__`.
- ABCs enforce interfaces at creation time; Protocols match by shape and are checked by tools.
- Use `dataclass` for records, `NamedTuple` for light immutable records, `Enum` for fixed choices, `__slots__` for memory.
- Prefer composition and dependency injection; SOLID guides design; patterns are shared names for recipes.

Pattern	Syntax
Record	`@dataclass(frozen=True, order=True)`
Safe list default	`field(default_factory=list)`
Read-only attribute	`@property` without setter
Other constructor	`@classmethod def from_x(cls, ...)`
Interface	`class X(ABC)` + `@abstractmethod`
See MRO	`Class.__mro__`
Object size	`sys.getsizeof(obj)`

Practice set

Problem 8.9.1 (easy) What prints?

```
class C:
    n = 0
c1, c2 = C(), C()
c1.n = 5
print(c2.n)
```

Answer: `0` . The assignment created an instance attribute on `c1` only.

Problem 8.9.2 (easy) Find the bug: `class A:` with `def hit(): return 1` , called as `A().hit()` .

Answer: The method is missing `self` . Python raises `A.hit() takes 0 positional arguments but 1 was given` . Fix: `def hit(self):` .

Problem 8.9.3 (easy) For `class Q:` with only `__init__(self, x)` storing `x` , what does `Q(1) == Q(1)` give?

Answer: `False` . Without `__eq__` , equality means "same object", and these are two objects.

Problem 8.9.4 (medium) Give the MRO of `D(B, C)` where `B(A)` and `C(A)` .

Answer: `D, B, C, A, object` .

Problem 8.9.5 (medium) What happens when you run `p.x = 5` on an instance of `@dataclass(frozen=True) class Point` ?

Answer: `FrozenInstanceError: cannot assign to field 'x'` .

Problem 8.9.6 (medium) An `Enum` has `OK = 200` , `NOT_FOUND = 404` , `ERR = auto()` . What is `ERR.value` ?

Answer: `405` . `auto()` continues from the last integer value.

Problem 8.9.7 (medium) Find the bug and fix it.

```
class Cfg:
    def __init__(self, d={}): self.d = d
a, b = Cfg(), Cfg()
a.d["k"] = 1
print(b.d)
```

Answer: It prints `{'k': 1}` . The default dictionary is created once and shared by all objects. Fix: `def __init__(self, d=None): self.d = {} if d is None else d` .

Problem 8.9.8 (medium) What does `sl.c = 3` do when `sl` is an object of a class with `__slots__ = ("a","b")`?

Answer: It raises `AttributeError: 'Slim' object has no attribute 'c' and no __dict__ for setting new attributes`.

Problem 8.9.9 (interview) What is the difference between `__repr__` and `__str__`?

Answer: `__repr__` is for developers: unambiguous, ideally looks like code that recreates the object. It is used in lists, logs and the debugger. `__str__` is for users and is used by `print`. If only `__repr__` exists, `print` uses it too.

Problem 8.9.10 (interview) Why do experts say "prefer composition over inheritance"? Give an example.

Answer: Inheritance ties the child to the parent's every detail and to the "is-a" rule, so changes in the parent break children (and can break Liskov substitution, like `Penguin(Bird)`). Composition builds an object from parts that can be swapped. Example: `Car` takes an `engine` object, so tests pass a fake engine.

Problem 8.9.11 (scenario) Your `SignupService` creates `MySQL()` inside `__init__` and the unit tests are slow and flaky. What do you change?

Answer: Define a repository interface (`add`, `get`), pass the repository into the constructor (dependency injection), and use an in-memory fake in tests. This is the Repository pattern plus the "D" in SOLID.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles, and English is my second language, so use simple words. Teach
me Python classes and instances: class vs instance attributes, self, __init__,
and the mutable-default bug. Explain step by step with one small shop example,
then ask me 5 short questions one at a time and wait for my answer before
continuing. Correct me kindly and explain why.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles, and English is my second language. Explain Python dunder
methods (__repr__, __str__, __eq__, __hash__, __lt__, __len__, __getitem__,
__iter__, __call__, __enter__) using a Money class. Show runnable code with
```

real output for each method. Then give me 8 graded exercises, from easy to hard, without solutions, and offer to review my answers.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Teach me inheritance, super(), the MRO, mixins and the diamond problem in Python. Use simple analogies, then run a small example and show the output. Quiz me one question at a time on "what is the output?", and explain every wrong answer line by line.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Explain SOLID and composition over inheritance with Python examples that I can run. For each of the five principles, show a bad version and a good version, and explain what breaks in the bad one. Then review my own class design: [PASTE HERE]. Tell me which principle it violates and how to fix it.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Run a mock interview on Python OOP and design patterns (Singleton, Factory, Builder, Strategy, Observer, Decorator, Adapter, Repository, Dependency Injection). Ask one question at a time, from easy to hard, wait for my answer, grade it out of 10, give the ideal answer, and track my score. Finish with the three topics I should revise.

CHAPTER 9

MODULES, PACKAGES, ENVIRONMENTS, TYPE HINTS AND VALIDATION

How to organise code into files, keep each project's libraries separate, and let tools catch mistakes before you run anything.

A notebook or a single script is fine for an experiment. A real job gives you a repository with fifty files, ten libraries, three teammates and a deadline. Then new questions appear. How do I split code into files and import between them? Why does my laptop work but the server break? Why did my pipeline crash because someone passed a string instead of a number? This chapter answers these questions. By the end you will build a real package and run it, create an isolated environment, read a `pyproject.toml`, write type hints, and validate incoming data. Everything marked "run" was executed with Python 3.14. No network was available here, so tools that need downloads (pip installs, mypy, pydantic, uv, poetry, conda) are shown in standard form and marked **not run here**.

9.1 Modules, Packages and `__main__`

INTUITION

A module is a chapter, a package is the whole book, and an import is a bookmark to a chapter in another book. You split code into files so each file does one job and you can find things. Without modules you would scroll through a 5,000-line file.

Words you need

Word	Plain meaning
module	One `.py` file. `import utils` loads `utils.py`
package	A folder of modules (it has an `__init__.py`)
`__init__.py`	Runs when the package is first imported; defines what the package exposes
`__name__`	Variable holding the module's name. It is `"__main__"` when the file is run directly
absolute import	`from shop.utils import round2` (full path)

relative import	`from .utils import round2` (dot means "this package")
`sys.modules`	Python's cache of already imported modules

How an import works. Python looks in `sys.modules` first. If the module is not there, it searches folders (the script's folder, then installed libraries), runs the file top to bottom **once**, and stores the result in the cache. Any later `import` reuses it.

Problem 9.1.1 Build a tiny package called `shop` and run it.

Step 1	Make a folder `shop/` containing `__init__.py`, `pricing.py`, `utils.py` and `__main__.py`.
Step 2	`pricing.py` imports from `utils.py` with a relative import. `__init__.py` re-exports `total_with_tax`.
Step 3	Run with `python3 -m shop 250`.

The files (each with a `print` so you can see when it runs):

```
# shop/__init__.py
"""shop package."""
from .pricing import total_with_tax
__all__ = ["total_with_tax"]
__version__ = "0.1.0"
print("shop/__init__ ran")
```

```
# shop/pricing.py
from .utils import round2
TAX = 0.18
def total_with_tax(amount):
    return round2(amount * (1 + TAX))
print("pricing imported, __name__ =", __name__)
```

```
# shop/utils.py
def round2(x):
    return round(x, 2)
```

```
# shop/__main__.py
from shop import total_with_tax
import sys
def main():
    amount = float(sys.argv[1]) if len(sys.argv) > 1 else 100.0
    print("total:", total_with_tax(amount))
if __name__ == "__main__":
    main()
```

Before you run it, predict: which line prints first, `shop/__init__ ran` or `pricing imported`?

```
python3 -m shop 250
python3 -c "import shop; print(shop.__version__, shop.total_with_tax(1000))"
```

```
pricing imported, __name__ = shop.pricing
shop/__init__ ran
total: 295.0
pricing imported, __name__ = shop.pricing
shop/__init__ ran
0.1.0 1180.0
```

Line by line: `__init__.py` begins running, hits `from .pricing import ...`, and jumps into `pricing.py` first. So "pricing imported" appears before "shop/__init__ ran", which is the last line of `__init__`. Inside pricing, `__name__` is `shop.pricing` (not `__main__`), since it was imported. `python3 -m shop` runs `__main__.py`. `250 * 1.18 = 295.0` and `1000 * 1.18 = 1180.0`.

- **Answer** `pricing imported` prints first, because the package's `__init__` has to import `pricing` before it can reach its own last line.
- **Check** 250 x 1.18 = 295.0. Correct. The import message does not repeat within one run, which shows the cache at work.
- **Why it matters** `python -m package` is the clean way to run a package. Entry points like `uvicorn app.main:app` rely on the same import rules.

Why `if __name__ == "__main__":`? When the file is imported, `__name__` is the module name and `main()` does not run. When the file is run directly, `__name__` is `"__main__"` and `main()` runs. This lets one file be both a library and a script. Without the guard, importing the file would start the whole program.

Problem 9.1.2 The most common relative-import error. Run a module file directly instead of as part of a package:

```
python3 shop/pricing.py
```

```
    from .utils import round2
ImportError: attempted relative import with no known parent package
```

Running a file directly makes Python treat it as a top-level script, with no package around it, so the dot has nothing to point to.

- **Answer** Run it as `python3 -m shop.pricing` from the project root, or run the package with `-m shop` .
- **Check** `-m` sets `__package__` , so the relative import has a parent.
- **Why it matters** Every new Python engineer hits this error in week one.

The picture

```
project/
  shop/                  <- package (folder with __init__.py)
    __init__.py          <- runs on "import shop"
    __main__.py          <- runs on "python -m shop"
    pricing.py  --uses--> utils.py   (from .utils import round2)
```

Read it as: you run from `project/` , `import shop` finds the folder, runs `__init__.py` , and from there Python can reach sibling files.

Connection to real projects. A data repository usually has `src/yourproject/` with `ingest.py` , `transform.py` , `models.py` . Airflow DAGs, FastAPI apps and CLI tools are all packages. `__all__` and `__init__` decide the public surface other teams depend on.

Applications

- **Data Engineering:** one package of reusable transforms shared by many pipelines.
- **Machine Learning:** `features.py` , `train.py` , `evaluate.py` imported by notebooks and jobs.
- **MLOps:** `python -m mypackage.serve` as the container start command.
- **Backend:** `app/` package with `routers/` , `models/` , `services/` .

COMMON MISTAKE

Naming your file after a library, for example `random.py` or `json.py` . Python then imports your file instead of the standard one and strange errors follow. Never name your files after standard or installed libraries.

Check yourself

1. What does `__name__` equal when a file is run with `python file.py` ?

Answer: `"__main__"` .

1. Why does the import message print only once even if three files import the same module?

Answer: Python caches the module in `sys.modules` after the first import.

1. What does `from .utils import x` need?

Answer: The file must be part of a package that was imported or run with `-m` .

9.2 Circular Imports and How to Avoid Them

INTUITION

Two people each say "I will leave only after you leave first." Nobody leaves. A **circular import** is the same deadlock: `a.py` needs `b.py` and `b.py` needs `a.py` , and Python is stuck halfway through loading one of them.

Problem 9.2.1 Create the problem. `a.py` imports from `b.py` , and `b.py` imports from `a.py` .

```
# a.py
from b import fb
def fa(): return "fa"
```

```
# b.py
from a import fa
def fb(): return "fb"
```

Predict: what happens on `import a` ?

```
python3 -c "import a"
```

```
  File "/private/tmp/c9/circ/b.py", line 1, in <module>
    from a import fa
ImportError: cannot import name 'fa' from 'a' (consider renaming
'/private/tmp/c9/circ/a.py' if it has the same name as a library you intended
to import)
```

Step by step: Python starts `a` , reaches line 1, and starts loading `b` . `b` asks for `fa` from `a` , but `a` is only half loaded and `fa` is defined on line 2, which has not run yet. So the name does not exist,

and Python raises `ImportError`.

- **Answer** An `ImportError` because `a` is still only partly loaded when `b` asks for `fa`.
- **Check** The traceback points at line 1 of `b.py`.
- **Why it matters** In real projects the cycle is often three or four files long, which makes it hard to see.

Fixes, best first

1. **Move shared code** into a third module that both import (the usual best fix).
2. **Import inside the function** that needs it (a deferred import).
3. **Import the module, not the name** (`import a` and use `a.fa` later).

Here is fix 2 in action:

```
# b.py
def fb():
    from a import fa      # deferred import
    return "fb+" + fa()
```

```
python3 -c "import a; print(a.fb())"
```

```
fb+fa
```

By the time `fb()` is called, both modules are fully loaded, so the import succeeds.

- **Answer** It prints `fb+fa`.
- **Check** `fb()` returns "fb+" followed by what `fa()` returns, "fa".
- **Why it matters** A deferred import is a quick patch. A cycle often means the design should change: one module is doing two jobs (see Single Responsibility in Chapter 8).

Connection to real projects. Cycles appear when `models.py` needs `services.py` for a helper while `services.py` imports `models.py`. The cure is a layered design: `models` imports nothing from `services`; `services` may import `models`.

Applications

- **Backend:** layered packages with one-way dependencies.
- **Machine Learning:** keep `config.py` free of imports from training code.
- **Type hints:** `if TYPE_CHECKING:` imports avoid cycles that only exist for annotations.

COMMON MISTAKE

"Fixing" a cycle by shuffling import lines until it happens to work. It will break again after the next edit. Break the cycle by design.

Check yourself

1. Why does a circular import fail?

Answer: One module is only partly loaded when the other asks for a name from it.

1. Name the cleanest fix.

Answer: Move the shared code into a third module.

1. What is a deferred import?

Answer: An import written inside a function, so it runs at call time.

9.3 Virtual Environments and Package Managers

INTUITION

Imagine each project has its own toolbox. Project A needs a hammer version 1; project B needs version 2. If you keep one shared toolbox for the whole house, you cannot have both. A **virtual environment** (venv) is a private toolbox for one project, so libraries and their versions never clash.

Words you need

Word	Plain meaning
package (library)	Code written by others that you install, like `pandas`
`pip`	The standard tool that installs packages from PyPI (the public package index)
virtual environment	A folder with its own Python and its own installed packages
`site-packages`	The folder where installed packages live
dependency	A library your project needs
lock file	A file that records the exact versions installed

Problem 9.3.1 Create a venv and prove it is isolated.

Step 1	`python3 -m venv venvdemo` creates the folder.
Step 2	Look inside, and ask the venv's Python whether it is in a venv.
Step 3	Ask the normal Python the same question.

```
python3 -m venv venvdemo
ls venvdemo
./venvdemo/bin/python -c "import sys; print(sys.prefix != sys.base_prefix)"
./venvdemo/bin/pip freeze | wc -l
python3 -c "import sys; print(sys.prefix != sys.base_prefix)"
```

```
bin
include
lib
pyvenv.cfg
True
0
False
```

- **Answer** The venv is a small folder with its own `bin` (Python and pip), `lib` (packages) and a `pyvenv.cfg` file. Inside it, `sys.prefix` differs from `sys.base_prefix` (so the check is `True`). `pip freeze` lists nothing: the venv starts empty. The normal Python prints `False`.
- **Check** `pip freeze` printing 0 lines proves your global libraries do not leak in.
- **Why it matters** "It works on my machine" is usually a different set of library versions.

On macOS and Linux you "activate" with `source venvdemo/bin/activate`; on Windows `venvdemo\Scripts\activate`. Activating just puts the venv's `bin` first on your PATH, so `python` and `pip` point at it. You can skip activation by calling `./venvdemo/bin/python` directly, which is what scripts and CI do.

The standard workflow (not run here, needs network)

```
python3 -m venv .venv
source .venv/bin/activate
pip install requests pandas
pip freeze > requirements.txt     # exact versions recorded
pip install -r requirements.txt   # on another machine
```

Tool comparison (not run here: none of these were installed, and there was no network)

Tool	What it does	When you meet it

`venv` + `pip`	Built in, simple	Every project; always learn first
`pip-tools`	`pip-compile` turns a short `requirements.in` into a fully pinned `requirements.txt`	Teams that want pinned files without a new tool
`poetry`	Manages dependencies, lock file and packaging via `pyproject.toml`	Many libraries and apps
`uv`	Very fast modern tool for envs, installs and lock files; compatible with pip workflows	Fast-growing; common in new projects
`conda`	Manages Python itself and non-Python libraries (C, CUDA)	Data science and GPU work

Typical commands, shown for orientation only (check each tool's own docs, as options change by version):

```
pip-compile requirements.in           # pip-tools
poetry add requests                   # poetry
uv venv && uv pip install requests    # uv
conda create -n ds python=3.12        # conda
```

Connection to real projects. CI servers and Docker images build a fresh environment from your lock file, so your program runs the same way everywhere. ML projects often need conda or Docker because GPU libraries are not pure Python.

Applications

- **Data Science:** one environment per project so a library upgrade never breaks an old analysis.
- **Machine Learning:** pinned versions of `numpy`, `scikit-learn`, `torch` so results can be reproduced.
- **MLOps:** a lock file copied into the Docker build.
- **Data Engineering:** the same pinned environment locally and in the scheduler.

COMMON MISTAKE

Running `pip install` while no venv is active, so packages go into your global Python. Check `which python` (or `python -c "import sys; print(sys.prefix)"`) before installing. Also, never commit the `.venv` folder to Git; add it to `.gitignore`.

Check yourself

1. What problem does a venv solve?

Answer: It stops different projects from sharing, and clashing over, library versions.

1. What does `pip freeze` print?

Answer: Every installed package with its exact version.

1. When prefer conda over pip?

Answer: When you need non-Python binaries such as CUDA or C libraries.

9.4 `pyproject.toml`, Version Pinning and Supply-Chain Awareness

INTUITION

`pyproject.toml` is the project's ID card: its name, version, which Python it needs and which libraries it depends on. One file, readable by pip, poetry, uv and build tools.

How it works. The file uses TOML, a simple config format. Python 3.11+ can read it with the built-in `tomllib`.

Problem 9.4.1 Write a minimal `pyproject.toml` and parse it.

```
import tomllib
text = """
[project]
name = "shop"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = ["requests>=2.31"]
"""
print(tomllib.loads(text))
```

```
{'project': {'name': 'shop', 'version': '0.1.0', 'requires-python': '>=3.10',
'dependencies': ['requests>=2.31']}}
```

- **Answer** TOML becomes a normal nested dictionary. Section `[project]` holds metadata. `dependencies` is a list of requirement strings.
- **Check** `d["project"]["name"]` would give `"shop"`.
- **Why it matters** Everything that builds, installs or publishes your project reads this one dictionary.

A fuller file also names a build tool (not run here, since building needs a build backend that may need download):

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project.scripts]
shop = "shop.__main__:main"
```

`[project.scripts]` makes `pip install .` create a `shop` command that calls `main()`. Chapter 13 covers building and publishing.

Version specifiers

Written	Meaning	Use
`requests==2.31.0`	Exactly this version	Lock files (reproducible)
`requests>=2.31`	This or newer	Libraries (flexible)
`requests~=2.31`	Compatible: at least 2.31, below 3.0	Apps wanting safe updates
`requests>=2.31,<3`	A range	When a major version breaks

Rule of thumb: libraries declare flexible ranges; applications ship a fully pinned lock file.

Dependency security and supply chain

A **supply-chain attack** is when attackers hide bad code inside a library you install, or inside one of that library's own dependencies. Your project may depend on hundreds of packages you never chose. Protect yourself:

- Pin exact versions and review upgrades.
- Run `pip-audit` (not run here) to check installed packages against known vulnerability lists. Dependabot on GitHub opens pull requests for risky or old versions.
- Install only from sources you trust. Watch for look-alike names (`reqeusts` instead of `requests`) called typosquatting.
- Keep dependencies few. Every library is trust you give away.

Problem 9.4.2 List the installed distributions with the standard library (works in any environment).

```
import importlib.metadata as m
print(sorted(d.metadata['Name'] for d in m.distributions())[:5])
```

```
['Jinja2', 'MarkupSafe', 'PyYAML', 'Pygments', 'Send2Trash']
```

- **Answer** This is what is installed in the global Python on this machine. Inside a clean venv the list would be much shorter (just pip, maybe setuptools).
- **Check** Compare with `pip freeze` output.
- **Why it matters** In an audit you must be able to say exactly what code runs in production.

Connection to real projects. Security teams at large companies scan dependencies. A pinned lock file plus `pip-audit` in CI is a normal requirement, and it is also a strong signal in an interview.

Applications

- **MLOps:** reproducible model builds and audit trails.
- **Data Engineering:** upgrading one library without breaking twenty pipelines.
- **Security / compliance:** list of dependencies with versions.

COMMON MISTAKE

Leaving versions unpinned in an application, so the next `pip install` silently downloads a new major version and your code breaks on Monday. Pin applications. Flexible ranges are for libraries.

Check yourself

1. What does `~=2.31` allow?

Answer: Versions from 2.31 up to, but not including, 3.0.

1. Why pin versions in an application?

Answer: So every install gets the same code and results are reproducible.

1. What is typosquatting?

Answer: A malicious package with a name close to a popular one, hoping you mistype.

9.5 Type Hints and mypy

INTUITION

A label on a jar tells you what is inside before you open it. A **type hint** is a label on a variable or function: "this takes a list of floats and gives back a float". Python does not enforce the label when running, but tools and your teammates can read it, and an editor can warn you before you run anything.

Words you need

Word	Plain meaning
type hint (annotation)	A note like `x: int` or `-> float` about the expected type
static type checker	A tool (mypy, pyright) that reads hints and finds mismatches without running the code
`Optional[X]`	Either an `X` or `None` (same as `X \
`Union[A, B]`	Either an A or a B (same as `A \
`Callable[[int], int]`	A function taking an int and returning an int
`TypeVar`	A placeholder type, so "the output type equals the input type"
`Protocol`	An interface matched by shape (Chapter 8)

Problem 9.5.1 Common hints, run together.

```
from typing import Optional, Union, Callable, TypeVar, Protocol, Iterable

def mean(xs: list[float]) -> float:
    return sum(xs) / len(xs)

def find_user(users: dict[int, str], uid: int) -> Optional[str]:
    return users.get(uid)

def parse(v: Union[int, str]) -> int:
    return int(v)
def parse2(v: int | str) -> int:    # modern spelling
    return int(v)

def apply(f: Callable[[int], int], xs: Iterable[int]) -> list[int]:
    return [f(x) for x in xs]

T = TypeVar("T")
def first(xs: list[T]) -> T:
    return xs[0]
def first_new[T](xs: list[T]) -> T:   # Python 3.12+ syntax
    return xs[0]

class HasName(Protocol):
    name: str
def greet(x: HasName) -> str: return "hi " + x.name

print(mean([1,2,3]), find_user({1:"a"}, 2), parse("7"), parse2(8))
print(apply(lambda n: n*n, [1,2,3]), first(["x","y"]), first_new([10,20]))
```

```
2.0 None 7 8
[1, 4, 9] x 10
```

- **Answer** `mean([1,2,3]) = 6/3 = 2.0`. `find_user` returns `None` because key 2 is missing, which is exactly why its hint is `Optional[str]`. `first` returns the same type as the list holds (`str` for strings, `int` for ints). On Python 3.9 and later, built-in `list[float]` and `dict[int, str]` replace the old `typing.List` and `typing.Dict`. On 3.10+ you can write `int | str`. The `def first_new[T]` syntax needs Python 3.12 or later.
- **Check** 1+2+3 = 6, divided by 3 = 2.0. `1*1, 2*2, 3*3 = 1, 4, 9`.
- **Why it matters** Old code uses `List[int]` and `Optional`. You will see both styles at work, so know both.

Hints are not enforced at runtime. Predict: does `mean("abc")` complain about the hint?

```
try:
    print(mean("abc"))
except TypeError as e: print("TypeError:", e)
from typing import get_type_hints
print(get_type_hints(mean))
```

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
{'xs': list[float], 'return': <class 'float'>}
```

- **Answer** No. Python ignored the hint. The error came from `sum()` trying `0 + "a"`, not from the label. The hints are stored in `mean.__annotations__` for tools to read.
- **Check** The error is about `int` and `str`, not about `list[float]`.
- **Why it matters** If you want checking, you need a checker such as mypy or runtime validation (section 9.6).

mypy (not run here, no network). `mypy` is the standard static checker. You run it on your files and it reports mismatches without executing the program:

```
pip install mypy
mypy shop/
```

For the call `mean("abc")` above, mypy would report an error similar to `Argument 1 to "mean" has incompatible type "str"; expected "list[float]"`. Exact wording varies by version. Treat mypy as a test that runs in CI. Start with the default settings, and tighten gradually (for example `--strict`) on new code.

Connection to real projects. Teams add hints to shared functions, so the editor can autocomplete and catch mistakes. FastAPI (Chapter 12) reads hints to build request validation and API docs automatically.

Applications

- **Data Engineering:** typed transform functions reduce bugs in large pipelines.
- **Machine Learning:** hints on function inputs ("expects a DataFrame with these columns" in docstrings plus `pd.DataFrame` hints).
- **Backend:** FastAPI uses hints for validation and docs.
- **Code review:** hints act as documentation that cannot go stale unnoticed.

COMMON MISTAKE

Thinking hints make Python check types at runtime. They do not. A function annotated `-> int` can happily return a string. Also avoid the old `List`, `Dict` imports in new code.

Check yourself

1. What does `Optional[str]` mean?

Answer: A `str` or `None`.

1. Does Python raise an error when you pass the wrong type to an annotated function?

Answer: No. Only a checker like mypy, or your own validation, would.

1. What is `TypeVar` for?

Answer: To say that the output type depends on the input type.

9.6 Data Validation: By Hand and with pydantic

INTUITION

A security guard at a building door checks every visitor. Data arriving from outside (a web request, a CSV, a model output) cannot be trusted. **Validation** is the guard: it checks types and rules at the door, and rejects bad data with a clear message, before it spreads through your pipeline.

Why not just trust type hints? Hints are not enforced (section 9.5). Real input is a string like `"42"` where you wanted the number 42, or `"nomail"` where you wanted an email. A validation library converts what it can and rejects the rest.

Problem 9.6.1 Write a small validator using only the standard library. It converts ("coerces") values to the declared type and collects every error.

```
from dataclasses import dataclass, fields

class ValidationError(Exception):
    pass

@dataclass
class Order:
    order_id: int
    email: str
    amount: float
    def __post_init__(self):
        errors = []
        for f in fields(self):
            v = getattr(self, f.name)
            try:
                setattr(self, f.name, f.type(v))   # coerce like pydantic lax
mode
            except (TypeError, ValueError):
                errors.append(f"{f.name}: cannot convert {v!r} to
{f.type.__name__}")
        if not errors and "@" not in self.email: errors.append("email: missing
@")
        if not errors and self.amount <= 0: errors.append("amount: must be >
0")
        if errors: raise ValidationError("; ".join(errors))

print(Order("42", "a@b.com", "99.5"))
for bad in [("x", "a@b.com", 5), (1, "nomail", 5), (1, "a@b.com", -3)]:
    try: Order(*bad)
    except ValidationError as e: print("rejected:", e)
```

```
Order(order_id=42, email='a@b.com', amount=99.5)
rejected: order_id: cannot convert 'x' to int
rejected: email: missing @
rejected: amount: must be > 0
```

Step by step: `fields(self)` lists the declared fields. For each one we call its type as a function: `int("42")` gives 42 and `float("99.5")` gives 99.5, but `int("x")` raises `ValueError`, which we turn into a message. After conversion we apply business rules (an email needs `@`, amount must be positive).

- **Answer** The good order is accepted and its text values are converted to `int` and `float`. The three bad ones are each rejected with a clear reason.
- **Check** `42` is now an `int` (the repr shows `order_id=42`, without quotes).
- **Why it matters** This is the pattern behind every validation library. Note that this tiny version

assumes plain types such as `int`, `str`, `float`; real libraries handle lists, nested objects and more.

pydantic (not run here, not installed and no network). The library `pydantic` does this with far more features. This is the standard usage in version 2 (check the docs for your version):

```
from pydantic import BaseModel, EmailStr, Field, ValidationError

class Order(BaseModel):
    order_id: int
    email: EmailStr          # needs: pip install "pydantic[email]"
    amount: float = Field(gt=0)

try:
    Order(order_id="42", email="a@b.com", amount=-3)
except ValidationError as e:
    print(e)                 # lists every failing field
```

The expected behaviour: `"42"` is converted to 42, and `amount=-3` fails the rule `gt=0` with an error that names the field. You also get `.model_dump()` to convert back to a dictionary. FastAPI uses pydantic models for request and response bodies.

Approach	Strength	Weakness
Hints only	Free documentation	Nothing is checked at runtime
Hand-written validator	No dependency, full control	You maintain it, limited features
`dataclass` + `__post_init__`	Standard library	Basic only
pydantic	Rich rules, nested data, clear errors, JSON	Another dependency

Connection to real projects. Every data pipeline should validate input at its boundary: API payloads, config files, rows from vendors. A bad row caught at the door costs minutes; the same row found in a dashboard costs days.

Applications

- **Data Engineering:** validate each record from an upstream feed and send failures to a quarantine file.

- **Machine Learning:** validate prediction-request schemas in a model-serving API.
- **GenAI:** validate that an LLM's JSON output matches the expected structure before using it.
- **Backend:** request and response models in FastAPI.

COMMON MISTAKE

Validating only the happy path. Test with a missing field, a wrong type, an empty string, a negative number and a huge number. Also, never put secrets or passwords in error messages you log.

Check yourself

1. Why validate even if you have type hints?

Answer: Hints are not enforced at runtime, and outside data may be wrong.

1. What is coercion?

Answer: Converting a value to the expected type, like `"42"` to `42`.

1. Which standard-library tool gave us the list of fields?

Answer: `dataclasses.fields`.

9.7 Jupyter Notebooks vs Scripts, `nbconvert` and Reproducibility

INTUITION

A notebook is a lab notebook: you try an idea, see the result next to it, and keep notes. A script is a recipe card: anyone can follow it from start to finish and get the same dish. You explore in notebooks, and you ship scripts and packages.

What a notebook really is. A `.ipynb` file is just JSON: a list of cells with source text and saved outputs. Prove it:

```
import json
nb = {"cells":[{"cell_type":"code","source":["print(1+1)\n"],"outputs":
[],"execution_count":None,"metadata":{}}],"metadata":
{},"nbformat":4,"nbformat_minor":5}
open("/tmp/c9/x.ipynb","w").write(json.dumps(nb))
d = json.load(open("/tmp/c9/x.ipynb"))
for c in d["cells"]:
    print("".join(c["source"]), end="")
    exec("".join(c["source"]))
```

```
print(1+1)
2
```

- **Answer** The notebook file loads as ordinary JSON, and each code cell's source is plain text we can run. (`exec` is used here only to show the idea. Never `exec` untrusted notebooks, because they can run any code.)
- **Check** `1+1 = 2` .
- **Why it matters** Because it is JSON, it works badly with Git diffs (outputs and execution counts change) and can hide problems such as cells run out of order.

	Notebook	Script / package
Best for	Exploration, charts, teaching, reports	Repeatable jobs, production, testing
Runs top to bottom?	Not guaranteed (you can run cells in any order)	Always
Git-friendly	Poor (JSON, outputs)	Good
Unit tests	Hard	Easy
Scheduling	Awkward	Natural (cron, Airflow)

Tools (not run here, need installation)

```
jupyter nbconvert --to script analysis.ipynb      # notebook -> .py
jupyter nbconvert --to html analysis.ipynb        # shareable report
jupyter nbconvert --to notebook --execute analysis.ipynb   # re-run top to
bottom
```

Reproducibility checklist

1. Restart the kernel and "Run All" before sharing. If it fails, the notebook was not reproducible.
2. Set random seeds (for example `random.seed(42)`) and record them.
3. Keep a pinned `requirements.txt` or lock file next to the notebook.
4. Move reusable code into a package (`from shop import total_with_tax`) and import it in the notebook.
5. Never keep API keys in a notebook. Read them from environment variables (Chapter 13).
6. Strip outputs before committing, or use a tool made for it (for example `nbstripout`), so diffs stay readable.

Connection to real projects. The common professional flow: explore in a notebook, then move the working logic into functions in a package with tests, and let the notebook import them. Hiring managers look for exactly this separation.

Applications

- **Data Analytics:** a notebook turned into an HTML report for managers.
- **Data Science:** exploration notebooks plus a clean `train.py`.
- **Machine Learning:** `nbconvert --execute` in CI to check notebooks still run.
- **Teaching and portfolios:** notebooks as readable demos.

COMMON MISTAKE

Running cells out of order, then sharing a notebook that only works with your hidden state. If "Restart and Run All" fails, fix it before anyone else sees it.

Check yourself

1. What file format is an `.ipynb`?

Answer: JSON.

1. Why restart the kernel and run all before sharing?

Answer: To prove the notebook works from a clean state, in order.

1. Where should reusable logic live?

Answer: In a package or script that the notebook imports.

Chapter recap

- A module is a file, a package is a folder with `__init__.py`. Python runs a module once and

caches it in `sys.modules`.

- Use `if __name__ == "__main__":` so a file can be both imported and run. Run packages with `python -m package`.
- Relative imports (`from .x import y`) only work inside a package; running the file directly breaks them.
- Circular imports mean two modules need each other. Fix the design with a third shared module; deferred imports are a patch.
- Give every project its own virtual environment, and never install into global Python by accident.
- Libraries declare flexible versions, applications ship pinned lock files; scan dependencies with `pip-audit` and beware look-alike package names.
- Type hints are labels. Python does not enforce them; mypy and runtime validation (pydantic) do.
- Notebooks are for exploration, scripts and packages are for production. Always "Restart and Run All" before sharing.

Task	Command / pattern
Run a package	`python3 -m shop`
Script guard	`if __name__ == "__main__":`
Create venv	`python3 -m venv .venv`
Activate (mac/Linux)	`source .venv/bin/activate`
Save versions	`pip freeze > requirements.txt`
Install saved	`pip install -r requirements.txt`
Read pyproject	`tomllib.loads(text)`
Type hint, modern	`list[int]`, `int \`
Static check	`mypy package/` (not run here)
Notebook to script	`jupyter nbconvert --to script file.ipynb`

Practice set

Problem 9.8.1 (easy) What is `__name__` inside `shop/pricing.py` when it is imported by `shop/__init__.py`?

Answer: `shop.pricing`, as shown in the run output.

Problem 9.8.2 (easy) What does the line `if __name__ == "__main__":` protect against?

Answer: It stops the code under it from running when the file is imported. It runs only when the file is executed directly.

Problem 9.8.3 (easy) Which command creates a virtual environment in a folder called `.venv`?

Answer: `python3 -m venv .venv`.

Problem 9.8.4 (what is the output?) What prints? `def f(x: int) -> int: return x` followed by `print(f("a"))`.

Answer: `a`. The hints are not enforced at runtime, so the string passes through.

Problem 9.8.5 (find the bug) A teammate runs `python3 shop/pricing.py` and sees `ImportError: attempted relative import with no known parent package`. Why, and what is the fix?

Answer: The file was run as a top-level script, so the dot has no parent package. Run `python3 -m shop.pricing` from the project root.

Problem 9.8.6 (medium) `a.py` has `from b import fb` at the top and `b.py` has `from a import fa` at the top. Give two fixes.

Answer: Move `fa` and `fb` (or the code they share) into a third module that neither imports from the other; or move one of the imports inside the function that uses it.

Problem 9.8.7 (medium) What do `==2.31.0`, `>=2.31` and `~=2.31` mean?

Answer: Exactly 2.31.0; 2.31 or newer; at least 2.31 but below 3.0.

Problem 9.8.8 (medium) In `Order("42", "a@b.com", "99.5")` from 9.6, what are the types of `order_id` and `amount` afterwards?

Answer: `int` (value 42) and `float` (value 99.5), because the validator coerces them.

Problem 9.8.9 (interview) Why does `import pandas` appear to be instant the second time within the same program?

Answer: The first import runs the module and stores it in `sys.modules`. Later imports just return the cached module object.

Problem 9.8.10 (interview) Your lead says "pin your dependencies and run `pip-audit` in CI". Explain both.

Answer: Pinning fixes exact versions so every build is the same and updates are deliberate. `pip-`

`audit` checks those installed versions against known vulnerability data, so risky packages are found before release. Together they give reproducibility and basic supply-chain safety.

Problem 9.8.11 (scenario) Your notebook works on your laptop but fails for a colleague with `ModuleNotFoundError`. Give three steps to fix the process.

Answer: (1) Create a venv and install from a pinned `requirements.txt` that you commit next to the notebook. (2) Restart the kernel and run all cells top to bottom to find hidden dependencies on state. (3) Move reusable code into a package and install it into the environment, so imports are the same for everyone.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional at an
Indian company moving into data and AI roles, and English is my second
language, so use simple words. Teach me Python modules and packages:
__init__.py, __name__ == "__main__", python -m, relative vs absolute imports,
and circular imports. Walk me step by step through building a small package
with three files, predict-then-run style. Then ask me five questions one at a
time, waiting for each answer.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data and AI roles, and English is my second language. Explain virtual
environments, pip, pip-tools, poetry, uv and conda: what problem each solves,
how they differ, and when to choose each. Use a simple analogy, a comparison
table, and a realistic ML project example. Then give me 8 graded exercises
without solutions, and offer to review my answers.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data and AI roles, and English is my second language. Teach me
pyproject.toml, version specifiers (==, >=, ~=), lock files, pip-audit,
Dependabot, typosquatting and supply-chain risk. Use concrete examples and a
short checklist I can follow for my own repository. Then review my
pyproject.toml: [PASTE HERE] and point out problems.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
```

into data and AI roles, and English is my second language. Teach me Python type hints (list[int], Optional, Union, Callable, TypeVar, Protocol), what mypy checks, and why hints are not enforced at runtime. Show small examples with wrong and fixed versions. Then quiz me one question at a time, and explain every mistake I make.

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Teach me data validation with pydantic and with plain Python, and the differences between notebooks and scripts, nbconvert and reproducibility. Then run a mock interview with six questions, one at a time, graded out of 10 with the ideal answer, and finish with my three weakest topics.

CHAPTER 10

TESTING AND CODE QUALITY

How to prove your code works today, and keep proving it every day after.

You have written a data-cleaning function. It works on your five test rows. Next month a teammate changes it, and a wrong number reaches a manager's dashboard. Professional engineers prevent this with automatic tests and automatic style checks. In this chapter you will write tests with `unittest` and `pytest`, replace slow or risky parts with mocks and fakes, build code step by step with TDD, measure coverage, and set up the tools that keep a team's code clean. You will use one running example, a small `slugify` function and a tiny price-lookup service, so each idea builds on the last.

> **A note on what was run.** This machine has only the Python 3.14 standard library and no network. So `pytest`, `pytest-cov`, `pytest-mock`, `black`, `ruff`, `mypy` and `pre-commit` were **not run here**. Their snippets are standard usage, marked "not run here". The same ideas were run with the standard library (`unittest`, `unittest.mock`, `sqlite3`, `trace`), and all output below is real.

10.1 Why We Test, and the Testing Pyramid

INTUITION

A restaurant kitchen tastes every dish before it leaves the pass. A test is a small robot that checks one promise your code makes, and it never gets tired. You change code, run the robots, and in seconds you know if you broke something. Without tests, every change is a gamble.

What a test is

A **test** is a piece of code that calls your code with known input and checks the output against the answer you expect. If they differ, the test **fails**. A set of tests is a **test suite**. When you run the suite after every change, a failure points to the change that broke things. This is called catching a **regression** (an old feature that stops working).

The testing pyramid

Tests come in three sizes.

- **Unit test**: checks one small piece (one function or class) alone. Runs in milliseconds. No network, no disk, no real database.

- **Integration test**: checks that two or more pieces work together, for example your code plus a real (small) database.
- **End-to-end (E2E) test**: runs the whole system like a user would, for example call the real API and check the stored result.

The testing pyramid

```
        /\          few: slow, costly, flaky
       /E2E\
      /------\
     /integr. \     some: medium speed
    /----------\
   /   unit     \   many: fast, cheap, precise
  /--------------\
```

Why a pyramid and not an upside-down one? Unit tests are fast and tell you exactly where the bug is. E2E tests are slow and a failure only says "something is wrong somewhere". You want many cheap tests at the bottom and few expensive tests on top.

Test data

Tests need input. Good test data is small, explicit, and lives next to the test. Use three kinds of cases:

Kind	Example for `slugify`	Why
Normal	`"Hello World"`	The main promise
Edge	`""`, one character, very long text	Boundaries break most code
Bad	`None`, wrong type	Check clear errors

Never use real customer data in tests. Make small fake data by hand, or generate it with a fixed random seed so the test is repeatable.

Mapping the idea to a real project

Item	Example	Where it appears in the job
Unit test	`slugify("Hello World") == "hello-world"`	Checking one cleaning function
Integration	Function plus in-memory	Checking a data-access layer

test	SQLite	
E2E test	Call the real API, read the stored row	Smoke test after a deploy (like a quick taste check before opening the restaurant)
Regression	Yesterday's bug returns	Why CI runs on every change

Predict, run, explain

Problem 10.1.1 Write a function `add(a, b)` and four tests: two integers, floats, a deliberately wrong expectation, and a wrong type. Run them and read the report.

Step 1	Write the function and the `unittest.TestCase` class. `unittest` is the testing module that ships with Python. Each method starting with `test_` is one test.
Step 2	Run the file with `verbosity=2` to list each test.
Step 3	Read the failure. It shows the line, and `4 != 5`.

```
import unittest
def add(a, b):
    return a + b
class TestAdd(unittest.TestCase):
    def test_ints(self):
        self.assertEqual(add(2, 3), 5)
    def test_floats(self):
        self.assertAlmostEqual(add(0.1, 0.2), 0.3)
    def test_bad(self):
        self.assertEqual(add(2, 2), 5)
    def test_type(self):
        with self.assertRaises(TypeError):
            add("a", 1)
if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Real output:

```
test_bad (__main__.TestAdd.test_bad) ... FAIL
test_floats (__main__.TestAdd.test_floats) ... ok
test_ints (__main__.TestAdd.test_ints) ... ok
test_type (__main__.TestAdd.test_type) ... ok

======================================================================
FAIL: test_bad (__main__.TestAdd.test_bad)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/private/tmp/w10/t1.py", line 10, in test_bad
    self.assertEqual(add(2, 2), 5)
    ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^
AssertionError: 4 != 5

----------------------------------------------------------------------
Ran 4 tests in 0.003s

FAILED (failures=1)
```

- **Answer** Three tests passed and one failed, as designed. We used `assertAlmostEqual` for floats because `0.1 + 0.2` is `0.30000000000000004`, not exactly `0.3`.
- **Check** `2 + 2` is 4, and the report says `4 != 5`, so the failure message is telling the truth: our expected value (5) was the wrong one.
- **Why it matters** A failing test is not bad news. It is a clear message that says which promise broke and on which line.

Before you run it, predict: how many tests will say `ok`? (Answer: three.) If you remove `assertAlmostEqual` and write `assertEqual(add(0.1, 0.2), 0.3)`, the report would show `0.30000000000000004 != 0.3`, so use the almost-equal form.

Problem 10.1.2 Test `slugify(text)`, which turns a title into a URL piece, using normal, edge and several cases at once with `subTest`.

Step 1	Write the function with `re.sub` (regex from Chapter 3). Replace every run of characters that are not letters or digits with `-`, then trim `-` at the ends.
Step 2	Write tests. `subTest` runs each case separately, so one failure does not hide the others.

```
# slug.py
import re
def slugify(text):
    text = text.strip().lower()
    text = re.sub(r"[^a-z0-9]+", "-", text)
    return text.strip("-")
```

```
# test_slug.py
import unittest
from slug import slugify
class T(unittest.TestCase):
    def test_basic(self):
        self.assertEqual(slugify("Hello World"), "hello-world")
    def test_punct(self):
        self.assertEqual(slugify("  Python, 101!  "), "python-101")
    def test_empty(self):
        self.assertEqual(slugify(""), "")
    def test_subtests(self):
        cases = [("A  B", "a-b"), ("--x--", "x"), ("C++ Tips", "c-tips")]
        for raw, want in cases:
            with self.subTest(raw=raw):
                self.assertEqual(slugify(raw), want)
```

```
python3 -m unittest -v test_slug
```

```
test_basic (test_slug.T.test_basic) ... ok
test_empty (test_slug.T.test_empty) ... ok
test_punct (test_slug.T.test_punct) ... ok
test_subtests (test_slug.T.test_subtests) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```

- **Answer** All four tests pass. `python3 -m unittest` finds files named `test*.py` when you run `python3 -m unittest discover`.
- **Check** By hand: `"C++ Tips"` lower-cased is `c++ tips`. The run `"++ "` is not a letter or digit, so it becomes one `-`, giving `c-tips`. This matches the test.
- **Why it matters** Using one test per rule means that if you later change the regex, you see exactly which rule broke.

Connection to the real job

- Data pipelines silently produce wrong numbers when a function changes. Tests on transformation functions catch this before the dashboard does.
- ML code is tested too: shape checks, no-NaN checks, and "same seed gives same result".
- Teams will not merge your pull request if the CI (Continuous Integration, an automatic runner) shows red tests.

Applications

- **Data Analytics:** tests on a cleaning function that turns `"₹1,200"` into `1200`.
- **Data Engineering:** unit tests for each transform, integration tests with a small database.
- **Machine Learning:** checks that feature code gives the same output for the same seed.
- **MLOps:** regression suites that run in CI before a model-serving service is released.

COMMON MISTAKE

Writing tests that only check the "happy path". Bugs live at the edges: empty list, zero, negative, `None`, duplicate keys, a time zone change. Add at least one edge and one bad case for each function.

COMMON MISTAKE

Testing with `==` on floats. Use `assertAlmostEqual` (unittest) or `pytest.approx` (pytest).

Check yourself

1. Name the three levels of the pyramid, from cheapest to most expensive. Answer: unit, integration, end-to-end.
2. Why is `assertEqual(0.1 + 0.2, 0.3)` a bad test? Answer: floats are stored in binary, and the sum is `0.30000000000000004`, so equality fails.
3. What does a regression mean? Answer: something that worked before stops working after a change.

10.2 pytest: Plain Asserts, Fixtures and Parametrize

INTUITION

`unittest` makes you write a class and remember names like `assertEqual`. `pytest` lets you write a plain function and a plain `assert`. It then prints a rich explanation when the assert fails. It is the standard in industry.

How pytest works

`pytest` is a third-party tool (`pip install pytest`). It finds files named `test_*.py` and functions named `test_*` . You write `assert condition` . If the condition is false, the test fails, and pytest shows the values on both sides.

```
# test_slug_pytest.py   (not run here)
import pytest
from slug import slugify

def test_basic():
    assert slugify("Hello World") == "hello-world"

def test_empty():
    assert slugify("") == ""

def test_none_raises():
    with pytest.raises(AttributeError):
        slugify(None)

def test_float_close():
    assert 0.1 + 0.2 == pytest.approx(0.3)
```

Run with `pytest -q` (quiet) or `pytest -v` (verbose). Useful flags: `-x` stops at the first failure, `-k name` runs tests whose name contains `name` , `--lf` re-runs only the tests that failed last time.

Parametrize: one test, many cases

`@pytest.mark.parametrize` runs the same test body with different inputs. Each case shows as its own test, so a failure names the exact case. It is the pytest version of the `subTest` loop above.

```
# not run here
import pytest
from slug import slugify

@pytest.mark.parametrize(
    "raw, want",
    [
        ("A  B", "a-b"),
        ("--x--", "x"),
        ("C++ Tips", "c-tips"),
        ("", ""),
    ],
)
def test_slugify_cases(raw, want):
    assert slugify(raw) == want
```

Fixtures: shared setup, with clean-up

A **fixture** is a function that prepares something a test needs (a database, a temp file, sample data). Tests ask for it by writing its name as an argument. pytest runs the fixture and passes the result. Code after `yield` is the clean-up.

```
# not run here
import sqlite3
import pytest

@pytest.fixture
def conn():
    c = sqlite3.connect(":memory:")
    c.execute("CREATE TABLE users(id INTEGER PRIMARY KEY, name TEXT UNIQUE)")
    yield c            # the test runs here
    c.close()          # clean-up, even if the test failed

def test_insert(conn):
    conn.execute("INSERT INTO users(name) VALUES (?)", ("ravi",))
    rows = conn.execute("SELECT name FROM users").fetchall()
    assert rows == [("ravi",)]
```

Fixture **scope** controls how often it is rebuilt: `function` (default, every test), `module`, `session` (once for the whole run). Use the widest scope only for expensive, read-only things. Built-in fixtures worth knowing: `tmp_path` (a fresh temporary folder), `capsys` (capture printed text), `monkeypatch` (change things for one test, see 10.3). A file called `conftest.py` holds fixtures shared by many test files.

The same ideas, run with unittest

Since pytest is not installed here, this is the `unittest` form of fixture and clean-up, which we can run. `setUp` runs before each test, `tearDown` after, and `tempfile.TemporaryDirectory` gives a fresh folder.

Words you need. *Setup* is code that prepares the world before a test. *Teardown* is code that cleans up after. A *fixture* is a named setup/teardown that pytest gives to tests.

Item	Example	Where it appears in the job
Fixture	In-memory DB with a table	Every DB or API test
Parametrize	15 messy phone numbers, 15 clean answers	Data-cleaning rules
`tmp_path`	Fresh temp folder	Testing file readers and writers

`conftest.py`	Shared fixtures for the whole project	Large test suites

Problem 10.2.1 Test a save/load pair of JSON functions using a temporary folder, so no test leaves files behind.

Step 1	Write `save(path, obj)` and `load(path)` using `pathlib` and `json` (Chapter 5).
Step 2	In the test, create a temp folder with a context manager, so it is deleted at the end.
Step 3	Save, load, compare.

```
import tempfile, pathlib, unittest, json
def save(path, obj): path.write_text(json.dumps(obj))
def load(path): return json.loads(path.read_text())
class T(unittest.TestCase):
    def test_roundtrip(self):
        with tempfile.TemporaryDirectory() as d:
            p = pathlib.Path(d) / "x.json"
            save(p, {"a": [1, 2]})
            self.assertEqual(load(p), {"a": [1, 2]})
unittest.main(argv=["x"], verbosity=1)
```

```
.
----------------------------------------------------------------------
Ran 1 test in 0.010s

OK
```

- **Answer** The round trip works, and the temp folder is gone after the test. In pytest the same job is done by the `tmp_path` fixture.
- **Check** We saved `{"a": [1, 2]}` and loaded the same dictionary back, so nothing was lost in JSON. The test touched no real project file.
- **Why it matters** Tests that clean up after themselves can run in any order and on any machine.

Problem 10.2.2 Read what a failing assertion on two dictionaries looks like in `unittest`.

Step 1	Compare `{'a':1,'b':2}` with `{'a':1,'b':3}`.
Step 2	Read the diff marks. The `^` shows the exact character that differs.

```
AssertionError: {'a': 1, 'b': 2} != {'a': 1, 'b': 3}
- {'a': 1, 'b': 2}
?              ^

+ {'a': 1, 'b': 3}
?              ^
```

- **Answer** The value of key `b` differs: 2 against 3. pytest prints a similar diff for a plain `assert`.
- **Check** Compare by eye: key `a` is 1 in both, key `b` is 2 and 3. The `^` marks sit under the `2` and the `3`.
- **Why it matters** Learning to read the diff saves minutes on every failing test.

The picture

```
test_x(conn)                fixture conn()
    |   asks for "conn"          |
    |--------------------------->|  build DB
    |<-------- DB object --------|  (yield)
    | run test body              |
    |--------------------------->|  clean-up after yield
```

Connection to the real job

- A fixture that builds a tiny DataFrame or a tiny SQLite table is how data teams test transformations without touching production.
- `parametrize` is perfect for data-cleaning rules: list 15 messy inputs and the clean output you want.
- CI runs `pytest` on every pull request.

Applications

- **Data Analytics:** parametrized tests for date and currency parsing rules.
- **Data Engineering:** fixtures that build a tiny table to test an ETL transform.
- **Machine Learning:** tests for model-input validation (missing columns, wrong types).
- **MLOps / backend:** FastAPI endpoint tests with a test client (Chapter 12); CLI tests through `capsys`.

COMMON MISTAKE

Forgetting the pytest-only names. If you write `def test_x(conn)` and no fixture called `conn` exists, pytest stops with an error that says `fixture 'conn' not found` (not run here). Check the spelling or add the fixture to `conftest.py`.

COMMON MISTAKE

Tests that depend on each other or on order. Test B passes only because test A ran first and left a row in the database. Each test must build what it needs, using a fixture, and leave nothing behind.

COMMON MISTAKE

Giving a fixture the wide `session` scope when tests change it. One test then pollutes the next. Start with the default scope.

Check yourself

1. In pytest, how does a test receive a fixture? Answer: by naming it as a function argument.
2. What does code after `yield` in a fixture do? Answer: it runs as clean-up after the test finishes.
3. What is `parametrize` for? Answer: to run one test body with many input cases, each reported separately.

10.3 Mocks, Fakes, monkeypatch and Database Tests

INTUITION

In a rehearsal of an online shop you do not call a real payment gateway with real money. You stub it. A unit test is the same: replace the slow, costly or random parts (network, clock, email) with stand-ins you control, so the test is fast and repeatable.

Terms (words you need)

- **Test double**: any stand-in used instead of the real thing.
- **Fake**: a small, working, simple replacement (for example a dictionary instead of a database). It has real behaviour.
- **Stub**: returns fixed answers. No logic.
- **Mock**: records how it was called, so you can check "was it called once, with these arguments?". In

Python, `unittest.mock.Mock` is both a stub and a mock.

- **Spy**: wraps the real thing and records calls.

Item	Example	Where it appears in the job
Stub	Always returns `{"status": 200}`	Skipping a slow partner API
Fake	Dict-backed `FakeClient`	Testing logic without a DB
Mock	`client.get.assert_called_once()`	Checking "email sent once"
Spy	Real function wrapped to count calls	Checking a retry loop

Rule of thumb: prefer a **fake** when the behaviour is simple, because it tests the result. Use a **mock** when the important thing is that a call happened (email sent, once).

Mock in action

Problem 10.3.1 `fetch_price(client, sku)` calls `client.get(...)` and returns the price. `total` sums prices for many SKUs. Test `total` with no network, using a `Mock` and also a `Fake`.

Step 1	Pass the client in as an argument. This is **dependency injection** (Chapter 8): the function does not build its own client, so tests can pass a different one.
Step 2	Use `side_effect` with a list so each call returns the next item. Then assert the call count.
Step 3	Write a `FakeClient` class with a dictionary and test again.

```
import unittest
from unittest import mock

def fetch_price(client, sku):
    resp = client.get(f"/price/{sku}")
    if resp["status"] != 200:
        raise ValueError("bad status")
    return resp["body"]["price"]

def total(client, skus):
    return sum(fetch_price(client, s) for s in skus)

class TestMock(unittest.TestCase):
    def test_total_with_mock(self):
        client = mock.Mock()
        client.get.side_effect = [
            {"status": 200, "body": {"price": 100}},
            {"status": 200, "body": {"price": 250}},
        ]
        self.assertEqual(total(client, ["a", "b"]), 350)
        self.assertEqual(client.get.call_count, 2)
        client.get.assert_any_call("/price/a")
    def test_error(self):
        client = mock.Mock()
        client.get.return_value = {"status": 500, "body": {}}
        with self.assertRaises(ValueError):
            fetch_price(client, "x")

class FakeClient:
    def __init__(self, prices): self.prices = prices
    def get(self, path):
        sku = path.rsplit("/", 1)[1]
        if sku in self.prices:
            return {"status": 200, "body": {"price": self.prices[sku]}}
        return {"status": 404, "body": {}}

class TestFake(unittest.TestCase):
    def test_total(self):
        self.assertEqual(total(FakeClient({"a": 1, "b": 2}), ["a", "b"]), 3)
    def test_missing(self):
        with self.assertRaises(ValueError):
            total(FakeClient({}), ["zzz"])
```

Real output (this file also contained the patch and sqlite tests shown later, 9 tests in all):

```
test_missing (__main__.TestFake.test_missing) ... ok
test_total (__main__.TestFake.test_total) ... ok
test_error (__main__.TestMock.test_error) ... ok
test_total_with_mock (__main__.TestMock.test_total_with_mock) ... ok
test_default (__main__.TestPatch.test_default) ... ok
test_env (__main__.TestPatch.test_env) ... ok
test_time (__main__.TestPatch.test_time) ... ok
test_add (__main__.TestRepo.test_add) ... ok
test_unique (__main__.TestRepo.test_unique) ... ok

----------------------------------------------------------------------
Ran 9 tests in 0.004s

OK
```

- **Answer** `total` returned 350 with the mock and 3 with the fake, and both error paths raised `ValueError`. No network was used.
- **Check** By hand: 100 + 250 = 350 and 1 + 2 = 3. The mock was called twice (two SKUs), which `call_count` confirmed.
- **Why it matters** The tests ran in milliseconds and would give the same answer on a plane with no internet.

Patching: replacing a name for one test

`mock.patch("module.name")` swaps a name for a mock during a test and restores it afterwards. pytest's `monkeypatch` fixture does the same job for attributes and environment variables, and `pytest-mock` gives a `mocker.patch(...)` fixture.

```
# not run here: pytest style
def test_user_from_env(monkeypatch):
    monkeypatch.setenv("APP_USER", "asha")
    assert home_name() == "asha"

def test_with_mocker(mocker):            # needs pytest-mock
    mocker.patch("time.time", return_value=1000.0)
```

The `unittest` forms, which ran in the output above, are `@mock.patch.dict(os.environ, {"APP_USER": "asha"})` and `@mock.patch("time.time", return_value=1000.0)`.

Problem 10.3.2 The famous trap: *patch where the name is used, not where it is defined.* `bill.py` does `from util import rate`. Which patch changes the result of `to_inr(2)` when the real `rate()` is 83.0?

Step 1	`from util import rate` copies the name `rate` into module `bill`. Now `bill.rate` and `util.rate` are two names for the same function.
Step 2	Patching `util.rate` changes only the `util` name. `bill` still points to the old function.
Step 3	Patch `bill.rate` instead, and run all three cases.

```
from unittest import mock
import bill, util
with mock.patch("util.rate", return_value=100.0):
    print("patch util.rate  ->", bill.to_inr(2))
with mock.patch("bill.rate", return_value=100.0):
    print("patch bill.rate  ->", bill.to_inr(2))
print("no patch         ->", bill.to_inr(2))
```

```
patch util.rate  -> 166.0
patch bill.rate  -> 200.0
no patch         -> 166.0
```

- **Answer** Only `bill.rate` works (result 200.0 = 2 × 100). Patching `util.rate` left the answer at 166.0 = 2 × 83.
- **Check** 2 × 83.0 = 166.0 matches the "no patch" line, so the first patch changed nothing.
- **Why it matters** This is the most common reason a patch "does not work". A plain-language rule: patch the name in the module that *looks it up*.

Before you run it, predict the first line. Most beginners guess 200.0 and get 166.0, which is the typical wrong version.

Safer mocks with a spec

A bare `Mock()` accepts any attribute, so a typo can pass silently. Give it a `spec` so only real attributes exist. Modern Python also refuses misspelled assert helpers.

```
from unittest import mock
m2 = mock.Mock(spec=["get"])
try:
    m2.post()
except AttributeError as e:
    print("AttributeError:", e)
m3 = mock.Mock()
m3.assert_called_once_wiht()
```

```
AttributeError: Mock object has no attribute 'post'
...
AttributeError: 'assert_called_once_wiht' is not a valid assertion. Use a spec
for the mock if 'assert_called_once_wiht' is meant to be an attribute.. Did you
mean: 'assert_called_once_with'?
```

`mock.create_autospec(real_object)` copies the real signature, so wrong argument counts also fail.

Database tests with in-memory SQLite

`sqlite3.connect(":memory:")` makes a database in RAM that vanishes when closed. It is perfect for fast integration tests of your data-access code. (It is not Postgres, so dialect-specific features need a real Postgres in CI, for example via Docker.)

Problem 10.3.3 Test a `Repo` class that adds users and rejects duplicates.

Step 1	In `setUp`, open a fresh in-memory DB and create the table. In `tearDown`, close it.
Step 2	Test insertion order and the `UNIQUE` rule.
Step 3	Notice the code uses `?` placeholders. This is also the defence against SQL injection (Chapter 13).

```
import sqlite3, unittest
class Repo:
    def __init__(self, conn):
        self.conn = conn
        conn.execute("CREATE TABLE users(id INTEGER PRIMARY KEY, name TEXT
UNIQUE)")
    def add(self, name):
        cur = self.conn.execute("INSERT INTO users(name) VALUES (?)", (name,))
        return cur.lastrowid
    def names(self):
        return [r[0] for r in self.conn.execute("SELECT name FROM users ORDER
BY id")]
class TestRepo(unittest.TestCase):
    def setUp(self):
        self.conn = sqlite3.connect(":memory:")
        self.repo = Repo(self.conn)
    def tearDown(self):
        self.conn.close()
    def test_add(self):
        self.repo.add("ravi"); self.repo.add("meena")
        self.assertEqual(self.repo.names(), ["ravi", "meena"])
    def test_unique(self):
        self.repo.add("ravi")
        with self.assertRaises(sqlite3.IntegrityError):
            self.repo.add("ravi")
```

- **Answer** Both tests passed (see the 9-test run above). Each test got a brand-new database, so they cannot disturb each other.
- **Check** `test_unique` inserts `"ravi"` twice and `sqlite3.IntegrityError` is raised, which proves the `UNIQUE` rule is active.
- **Why it matters** Your SQL code is tested against a real engine, not a guess.

The picture

```
 Real run:    your code --> HTTP client --> internet --> price service
 Unit test:   your code --> Mock/Fake  (answers you chose, instantly)
 Integration: your code --> real SQLite (in memory)
```

Connection to the real job

- Pipelines call APIs, clouds and databases. You mock the call, test your logic, and keep test runs fast and free.
- Clock-dependent code (daily jobs) is tested by passing the date in as an argument or by patching the clock.

- ML code: replace the slow model with a fake that returns fixed scores to test the surrounding service.

Applications

- **Data Analytics:** test a report function with a tiny in-memory table.
- **Data Engineering:** mock a cloud storage client to test "upload then log" logic.
- **Machine Learning:** fake model returning fixed scores to test the code around it.
- **GenAI / MLOps:** stub the LLM API so tests are free, fast and repeatable.

COMMON MISTAKE

Over-mocking. If the test mocks everything, it only checks that your code calls your mocks. It will pass even when the real thing is broken. Mock only the boundary (network, clock, disk), and keep a few integration tests.

COMMON MISTAKE

Patching the wrong name (Problem 10.3.2). If a patch seems to do nothing, check where the name is imported.

Check yourself

1. Fake or mock: which one has real simple behaviour? Answer: the fake.
2. Where do you patch after `from util import rate` in `bill.py` ? Answer: `bill.rate` .
3. Why is `:memory:` SQLite good for tests? Answer: it is fast, needs no setup, and disappears on close, so tests stay isolated.

10.4 TDD and Coverage

INTUITION

A good builder first writes the inspection checklist, then builds until the checklist passes. **Test-Driven Development (TDD)** works like this. You write a failing test, then the smallest code that passes it, then clean up.

The red-green-refactor loop

1. **Red**: write one small test for behaviour that does not exist yet. Run it. It must fail. (A test that was never red may test nothing.)

2. **Green**: write the simplest code that makes it pass. Do not be clever.
3. **Refactor**: clean the code (names, duplication) while the tests stay green.
4. Repeat with the next small behaviour.

TDD is not required everywhere. It shines for logic with clear rules (parsers, calculations, validators). It is awkward for exploratory data work in notebooks. Also, when you find a bug, write a failing test that reproduces it first, then fix it. That bug can never come back silently.

Worked walkthrough: FizzBuzz

Problem 10.4.1 Build `fizzbuzz(n)` by TDD: `3` gives `"Fizz"`, `5` gives `"Buzz"`, `15` gives `"FizzBuzz"`, others give the number as text.

Step 1 (Red)	Write four tests and a stub that raises `NotImplementedError`. Run.
Step 2 (Green, part 1)	Handle only the multiples of 3. Run again and see the remaining failures.
Step 3 (Green and refactor)	Build the text from parts, and fall back to the number. Run until green.

```
# test_fizz.py
import unittest
from fizz import fizzbuzz
class T(unittest.TestCase):
    def test_plain(self): self.assertEqual(fizzbuzz(1), "1")
    def test_fizz(self): self.assertEqual(fizzbuzz(3), "Fizz")
    def test_buzz(self): self.assertEqual(fizzbuzz(5), "Buzz")
    def test_both(self): self.assertEqual(fizzbuzz(15), "FizzBuzz")
```

Red: `fizz.py` is `def fizzbuzz(n): raise NotImplementedError`.

```
  File "/private/tmp/w10/tdd/fizz.py", line 2, in fizzbuzz
    raise NotImplementedError
NotImplementedError

----------------------------------------------------------------------
Ran 4 tests in 0.002s

FAILED (errors=4)
```

Step 2: only `if n % 3 == 0: return "Fizz"` then `return str(n)`.

```
AssertionError: '5' != 'Buzz'
- 5
+ Buzz

----------------------------------------------------------------------
Ran 4 tests in 0.002s

FAILED (failures=2)
```

Two failures remain: `5` and `15`. Final code:

```
def fizzbuzz(n):
    out = ""
    if n % 3 == 0:
        out += "Fizz"
    if n % 5 == 0:
        out += "Buzz"
    return out or str(n)
```

```
....
----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```

- **Answer** Four errors, then two failures, then green. The 4 errors in step 1 were "not implemented" errors, and in step 2 two tests turned green and two became real assertion failures. The test list guided every step.
- **Check** By hand: 3 → Fizz, 5 → Buzz, 15 is divisible by both so `"Fizz" + "Buzz"`, 1 → `"1"`. All four match the final code.
- **Why it matters** At no point did you write code without a failing test asking for it, so no code is untested.

Coverage: which lines did the tests run?

Code coverage is the percent of your code lines that ran during the tests. With pytest you use the `pytest-cov` plugin:

```
# not run here
pip install pytest-cov
pytest --cov=mypackage --cov-report=term-missing
```

`term-missing` lists the line numbers no test touched. The standard library has a rough equivalent, `trace`, which we can run.

Problem 10.4.2 `grade(score)` returns A for 90+, B for 75+, else C. Two tests only: 95 and 10. Which line is never executed?

Step 1	Write `cov.py` and `test_cov.py` with the two tests.
Step 2	Run the tests under `python3 -m trace --count --missing -C covout --module unittest test_cov`.
Step 3	Read `covout/cov.cover`. A line starting with `>>>>>>` never ran.

```
    1: def grade(score):
    2:     if score >= 90:
    1:         return "A"
    1:     if score >= 75:
>>>>>>         return "B"
    1:     return "C"
```

- **Answer** The line `return "B"` never ran, because no test used a score between 75 and 89. Add `test_b` with 80 and a boundary test with 75 and 74.
- **Check** 95 takes the first branch and 10 skips both `if` s. Neither can reach `return "B"`, so the report is right.
- **Why it matters** In plain words: 4 of 5 executable lines were tested, and the missing one is a real business rule.

Before you run it, predict which line is marked. Then compare with the report above.

Item	Example	Where it appears in the job
Line coverage	4 of 5 lines ran = 80%	CI quality gate
Branch coverage	Both sides of each `if` ran	Finding untested error paths

Regression test	Test written from a bug report	Support-ticket fixes

What coverage does not tell you

100% coverage means every line ran. It does not mean every line was *checked*. A test with no assert gives coverage and no safety. Use coverage to find untested areas, not as a score to chase. Many teams set a floor (for example 80%) and fail CI below it. Branch coverage (`--cov-branch`) also checks that both sides of every `if` ran.

The picture

```
  RED ----------> GREEN ----------> REFACTOR
(failing test)   (smallest code)    (clean, tests stay green)
    ^                                     |
    +------------- next behaviour --------+
```

Connection to the real job

- When a production bug is reported, the professional move is: write the failing test, fix, keep the test. Over a year this builds a safety net.
- Data validators (is this row valid?) are ideal for TDD: rules are explicit.
- Performance work (Chapter 11) is safe only if tests prove the faster code gives the same answers.

Applications

- **Data Analytics:** validators for report rules (tax, discount, billing).
- **Data Engineering:** parsers for messy log or CSV formats, built one rule at a time.
- **Machine Learning:** a failing test that reproduces a data-leak bug before you fix it.
- **MLOps:** a safety net that lets you refactor a legacy pipeline without fear.

COMMON MISTAKE

Writing the test after the code and copying the code's current output as the "expected" value. If the code is wrong, the test now locks in the wrong answer. Work out the expected value by hand first.

COMMON MISTAKE

Chasing 100% coverage with assert-free tests. Coverage measures execution, not correctness.

Check yourself

1. What do you do in the Red step? Answer: write a small test for missing behaviour and see it fail.
2. A line shows `>>>>>>` in a trace report. What does it mean? Answer: no test executed that line.
3. Can 100% coverage still hide bugs? Answer: yes, if tests do not assert the results.

10.5 Code Quality: Formatters, Linters, Type Checkers and pre-commit

INTUITION

In a team, nobody wants to argue about spaces versus quotes in a code review. Let machines settle style, so humans discuss design. A formatter rewrites style, a linter flags suspicious code, a type checker flags wrong types, and pre-commit runs all three before each commit.

The four tools

Tool	Job	Changes your code?	Typical command
`black`	Formats code in one fixed style	Yes	`black .`
`ruff`	Very fast linter (and formatter, `ruff format`)	Lint: with `--fix`	`ruff check .`
`mypy`	Checks type hints (Chapter 9)	No	`mypy src`
`pre-commit`	Runs checks automatically at `git commit`	Depends on hooks	`pre-commit install`

All four are third-party (`pip install`), **not run here**. Many teams now use `ruff` for both lint and format and drop `black`. Either is fine; the point is one agreed tool.

Configuration: one place, `pyproject.toml`

```
# pyproject.toml   (not run here; standard keys)
[tool.black]
line-length = 88

[tool.ruff]
line-length = 88

[tool.ruff.lint]
select = ["E", "F", "I", "B"]   # pycodestyle errors, pyflakes, import order,
bugbear

[tool.mypy]
python_version = "3.12"
strict = true
```

`F` rules find unused imports and undefined names. `I` sorts imports. `B` (bugbear) catches classic traps such as the mutable default argument. The set of rule letters can change between versions, so check the tool's documentation.

pre-commit

`pre-commit` reads `.pre-commit-config.yaml`. Each hook runs on the files you are about to commit. If a hook fails or edits a file, the commit stops, you re-add the files and commit again.

```
# .pre-commit-config.yaml   (not run here)
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: vX.Y.Z          # put the current release tag here
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format
  - repo: https://github.com/pre-commit/mirrors-mypy
    rev: vX.Y.Z          # put the current release tag here
    hooks:
      - id: mypy
```

```
# not run here
pip install pre-commit
pre-commit install          # sets up the git hook once
pre-commit run --all-files  # check the whole repo now
```

Item	Example	Where it appears in the job
Formatter	`black .` rewrites quotes and spacing	Zero style arguments in review

Linter	`ruff check .` flags an unused import	Catching bugs before running
Type checker	`mypy src` flags `double("5")`	Data code with wrong column types
pre-commit	Runs hooks at `git commit`	Every team repo

What these tools catch

Here is the kind of bug a linter finds, and how running the code shows it.

Problem 10.5.1 The code below has the "mutable default" bug. Show the bug, then fix it. A bugbear rule (`B006`) would flag the first version.

Step 1	Run the buggy function twice.
Step 2	Fix with `None` as default.

```
def add_tag(tag, tags=[]):
    tags.append(tag)
    return tags
print(add_tag("a"), add_tag("b"))

def add_tag_ok(tag, tags=None):
    if tags is None:
        tags = []
    tags.append(tag)
    return tags
print(add_tag_ok("a"), add_tag_ok("b"))
```

```
['a', 'b'] ['a', 'b']
['a'] ['b']
```

- **Answer** The default list is created once and shared, so the first call's data leaks into the second. Both prints in the buggy version show the same list object. The fix gives `['a'] ['b']`.
- **Check** The buggy line prints `['a', 'b']` twice because Python evaluates both calls first, and both returned the same list, which by then held both tags.
- **Why it matters** A linter finds this in a second. Without it, the bug appears as data from one user showing up in another's result.

Problem 10.5.2 What does `mypy` say about this code? (Not run here; this is the standard message shape.)

Step 1	Read the hint: `def double(x: int) -> int`.
Step 2	The call `double("5")` passes a `str`.
Step 3	mypy reports it before the program runs.

```
def double(x: int) -> int:
    return x * 2

double("5")
```

Expected report (wording varies by mypy version): `error: Argument 1 to "double" has incompatible type "str"; expected "int"`.

- **Answer** mypy flags the call without running it. At runtime `"5" * 2` would silently give `"55"`, a wrong answer with no error. That is why type checking helps.
- **Check** In plain Python, `"5" * 2` is `"55"` (string repeat). So the program would not crash, it would be quietly wrong.
- **Why it matters** The worst bugs are the ones that do not raise errors.

The picture

```
you edit --> git commit --> pre-commit hooks --> pass? --> commit saved
                                |  ruff-format
                                |  ruff check
                                |  mypy              fail --> fix, re-add
```

Then CI (Chapter 13) runs the same tools again on the server, so nothing slips past a skipped local hook.

Connection to the real job

- Every professional repo has a formatter, a linter, and usually a type checker in CI. A pull request that fails them is not reviewed.
- Notebooks can be linted too (`ruff` supports `.ipynb`); convert long notebooks to scripts as they mature.
- Type hints plus `mypy` catch many "column is a string, not int" bugs in data code early.

Applications

- **Data Analytics:** `ruff` on notebooks exported to scripts, so shared analysis code stays tidy.

- **Data Engineering:** pre-commit plus CI on every pipeline repository.
- **Machine Learning:** `mypy` catching wrong tensor/DataFrame argument types in training code.
- **MLOps:** hooks that block secrets and debug prints from reaching the main branch.

COMMON MISTAKE

Turning on `strict` mypy and every lint rule in an old codebase on day one. You get 2,000 errors and give up. Start with a small rule set on new code, then widen it.

COMMON MISTAKE

Skipping hooks with `git commit --no-verify` "just this once". CI will fail anyway, and now it fails in public.

Check yourself

1. What is the difference between a formatter and a linter? Answer: a formatter rewrites style only; a linter flags suspicious or wrong code.
2. When does `pre-commit` run? Answer: automatically at `git commit`, once you ran `pre-commit install`.
3. Does `mypy` run your program? Answer: no, it only reads the code and the type hints.

Chapter recap

- A test calls your code with known input and checks the output. Tests catch regressions fast.
- The testing pyramid: many unit tests, fewer integration tests, very few end-to-end tests.
- Test normal, edge and bad cases. Use small fake data, never real customer data.
- pytest uses plain `assert`, fixtures (with `yield` for clean-up), `parametrize`, `tmp_path` and `monkeypatch`. `unittest` is the built-in alternative.
- A fake has simple real behaviour; a mock records calls. Prefer fakes, mock only boundaries.
- Patch the name where it is used (`bill.rate`), not where it is defined.
- In-memory SQLite gives fast, isolated database tests.
- TDD is red, green, refactor. Reproduce every bug with a failing test before fixing it.
- Coverage shows which lines ran, not whether results were checked.
- `black` / `ruff` for style and lint, `mypy` for types, `pre-commit` to run them before each commit, config in `pyproject.toml`.

Cheat-sheet

Task	Command or pattern
Run unittest	`python3 -m unittest discover -v`
Run pytest, stop at first failure	`pytest -x -q`
Many cases, one test	`@pytest.mark.parametrize("a,want", [...])`
Fixture with clean-up	`@pytest.fixture` + `yield`
Replace a name	`mock.patch("module_where_used.name")`
Safer mock	`mock.Mock(spec=[...])` or `create_autospec`
Coverage with missing lines	`pytest --cov=pkg --cov-report=term-missing`
Format / lint / types	`ruff format .` / `ruff check .` / `mypy src`
Install hooks	`pre-commit install`

Practice set

Problem 10.P.1 (easy) What is the output of `slugify(" Python, 101! ")` with the function from 10.1?

Answer: `python-101`. Strip and lower give `python, 101!`, the regex turns `", "` into `-` and `!` into `-`, and the trailing `-` is trimmed.

Problem 10.P.2 (easy) Which pyramid level should have the most tests, and why?

Answer: unit tests, because they are fast, cheap, and point to the exact failing function.

Problem 10.P.3 (easy) In `grade(score)` from 10.4 (A at 90 and above, B at 75 and above), what does `grade(75)` return, and which boundary tests should you add?

Answer: `"B"`. Add tests for 90 (A), 89 (B), 75 (B), 74 (C).

Problem 10.P.4 (medium) Write a pytest `parametrize` test for `fizzbuzz` with the cases 1, 3, 5, 15.

Answer: (not run here)

```
import pytest
from fizz import fizzbuzz

@pytest.mark.parametrize("n, want", [(1, "1"), (3, "Fizz"), (5, "Buzz"), (15,
"FizzBuzz")])
def test_fizzbuzz(n, want):
    assert fizzbuzz(n) == want
```

Problem 10.P.5 (medium, what is the output?) With `bill.py` doing `from util import rate` and `util.rate()` returning 83.0, what does `to_inr(2)` return inside `mock.patch("util.rate", return_value=100.0)` ?

Answer: `166.0` . The patch changed `util.rate` , but `bill` kept its own reference to the old function.

Problem 10.P.6 (medium, find the bug) A test says `self.assertEqual(0.1 * 3, 0.3)` . It fails. Why, and how do you fix it?

Answer: `0.1 * 3` is `0.30000000000000004` in binary floating point. Use `assertAlmostEqual(0.1 * 3, 0.3)` or, in pytest, `pytest.approx(0.3)` .

Problem 10.P.7 (medium, find the bug) This fixture is meant to give each test a clean list, but tests leak data. What is wrong?

```
# not run here
import pytest
shared = []

@pytest.fixture
def items():
    return shared
```

Answer: it returns the same module-level list every time, so tests mutate shared state. Create a new list inside the fixture: `return []` .

Problem 10.P.8 (medium) A trace report shows `>>>>>>` on the line `raise ValueError` . What test do you write?

Answer: a test that triggers the error path, wrapped in `with self.assertRaises(ValueError):` (or `pytest.raises(ValueError)`), using an input that makes the condition true.

Problem 10.P.9 (hard, interview) Explain the difference between a mock, a stub and a fake, and when you would choose each.

Answer: a stub returns fixed answers. A fake is a lightweight working version (an in-memory dictionary

instead of a database). A mock records how it was called so you can assert on interactions. Choose a fake when behaviour is simple and you want to test results, a stub when you only need a canned answer, and a mock when the call itself is the outcome (for example "the email was sent once"). Avoid mocking everything, because then the test checks your mocks, not your code.

Problem 10.P.10 (hard, interview) A teammate says "we have 95% coverage so the code is safe". Reply in three sentences.

Answer: Coverage only says those lines executed, not that the results were asserted. Tests without assertions, or tests that only cover the happy path, still give high coverage. I would check assertion quality, edge cases and branch coverage, and look at which 5% is untested, because it may be the risky error handling.

Problem 10.P.11 (hard, scenario) Your nightly pipeline function `build_report(today)` reads the system date inside and your test fails every Sunday. Redesign it so it is testable.

Answer: pass the date in as an argument (`build_report(today)`), and let the caller supply `date.today()` . The test then passes a fixed date such as `date(2026, 11, 8)` . If the code cannot change yet, patch the clock for the test. Passing values in beats patching.

Prompt pack

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data, ML and AI roles, and English is my second language, so use simple
words. Teach me Python unit testing with pytest: assert, fixtures with yield,
parametrize, tmp_path and monkeypatch. Explain step by step with small examples
using real numbers, then ask me one question at a time to check my
understanding. Wait for my answer before continuing, and correct me kindly.
Relate ideas to everyday examples such as shop orders where it helps.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data, ML and AI roles, and English is my second language. The topic is
mocks, fakes and stubs in Python (unittest.mock, patch, side_effect, autospec)
and the rule "patch where the name is used". Give me 8 graded exercises from
easy to hard, each with a small code snippet and a clear goal. Do not give
solutions yet. After I reply with my answers, review them one by one and
explain every mistake in simple English.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data, ML and AI roles. Chapter topic: Test-Driven Development (red, green,
refactor). Pick a small data-cleaning problem for me, such as normalising phone
numbers. Walk me through TDD for it, one tiny step at a time. At each step show
only the next failing test and wait for me to write the code. Review my code,
point out edge cases I missed, and finish with a summary of what I learned.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data, ML and AI roles, and English is my second language. Please review my
Python tests and my code quality setup. Here is my code and tests: [PASTE
HERE]. Check for missing edge cases, tests that depend on each other, over-
mocking, float comparison mistakes, and weak assertions. Then suggest a
pyproject.toml configuration for ruff and mypy and a .pre-commit-config.yaml,
and explain each setting in simple English.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data, ML and AI roles. Run a mock interview on Python testing and code
quality for a data engineer job. Ask me one question at a time: the testing
pyramid, fixtures, mocking versus faking, coverage limits, TDD, linters, type
checkers and pre-commit. Grade each answer from 1 to 5, show a model answer,
and give one tip to improve. After 10 questions, tell me my weakest areas and
what to practise next.
```

CHAPTER 11

CONCURRENCY AND PERFORMANCE

Measure first, then make it faster: threads, processes, asyncio, profiling and vectorization.

Your script takes 40 minutes to call 2,000 URLs, and a data job that should finish overnight is still running at breakfast. Every programmer learns the golden rule: do not guess where the time goes, measure it. In this chapter you will learn to measure Python code (`timeit` , `cProfile` , `tracemalloc`), to understand why Python threads do not speed up some jobs (the GIL), and to choose between threads, processes and `asyncio` . You will also see why a loop in pure Python is slow, and connect all this to Little's law, a formula from queueing theory. All the timings below are real runs on one laptop (12 CPU cores, Python 3.14). Your numbers will differ, but the *shapes* will match.

> **A note on what was run.** Only the Python 3.14 standard library was available, with no network. `numpy` , `pandas` , `numba` , `line_profiler` and `memory_profiler` were **not run here**. Their snippets are standard usage and marked as such.

11.1 Measure First: timeit, cProfile and tracemalloc

INTUITION

A doctor does not operate before the scan. Slow code is the same: the slow part is almost never where you think. You first measure the whole, then find the one hot spot, then fix only that. In code terms: first time the whole script, then drill into the slowest function.

Words you need

- **Wall-clock time**: time as measured by a clock on the wall (what the user waits).
- **Profiler**: a tool that records which functions ran and how long each took.
- **Hot spot**: the small part of the code where most time is spent.
- **Benchmark**: a repeatable timing of a small piece of code.
- **Call**: one execution of a function; `ncalls` is how many times it ran.

timeit: time a small snippet fairly

`timeit` runs a snippet many times and reports the total, which smooths out noise. Use it to *compare two ways* of doing the same small job.

Problem 11.1.1 Compare four ways to compute the sum of squares of 1,000,000 numbers. Predict first: which is fastest, a plain `for` loop or a list comprehension?

Step 1	Write each version as a string. Run each 3 times and divide by 3.
Step 2	Print the average in milliseconds.

```
import timeit
N=1_000_000
t1=timeit.timeit("s=0\nfor i in range(N): s+=i*i", globals=globals(),
number=3)/3
t2=timeit.timeit("sum([i*i for i in range(N)])", globals=globals(), number=3)/3
t3=timeit.timeit("sum(i*i for i in range(N))", globals=globals(), number=3)/3
t4=timeit.timeit("sum(map(int.__mul__, r, r))", setup="r=range(N)",
globals=globals(), number=3)/3
for n,t in [("for loop",t1),("list comp + sum",t2),("generator + sum",t3),
("map(int.__mul__)",t4)]:
    print(f"{n:<20}{t*1000:8.1f} ms")
```

```
for loop               27.2 ms
list comp + sum        26.3 ms
generator + sum        28.8 ms
map(int.__mul__)       45.7 ms
```

- **Answer** The first three are within about 10% of each other (26 to 29 ms). The "clever" `map(int.__mul__)` was the slowest at 45.7 ms.
- **Check** 1,000,000 items in 27 ms means about 27 nanoseconds per item, which is the right order of magnitude for interpreted Python.
- **Why it matters** In plain words: style tricks change speed by percentages. They do not change it by 100 times. Big wins come from a better algorithm or data structure, as the next example shows.

Problem 11.1.2 Is `x in my_list` slow for a list of 100,000 numbers? Compare with a `set`.

Step 1	Build a list and a set of `range(100000)`.
Step 2	Search for `99999`, the worst case for a list, 200 times each.

```
import timeit
print(timeit.timeit("99999 in L", setup="L=list(range(100000))", number=200),
"s list")
print(timeit.timeit("99999 in S", setup="S=set(range(100000))", number=200), "s
set")
```

```
0.08791520900012983 s list
4.417000127432402e-06 s set
```

- **Answer** The list took 0.0879 s for 200 searches (0.44 ms each). The set took 0.0000044 s (22 nanoseconds each).
- **Check** 0.0879 ÷ 0.0000044 is about 19,900. A list checks items one by one (up to 100,000 steps). A set jumps straight to the right slot using a hash (Chapter 4), about 1 step.
- **Why it matters** Switching a data structure made the code about 20,000 times faster. No formatting trick can do that.

cProfile: find the hot spot

`cProfile` records every function call. You sort the report by **cumulative time** (time in the function *including* the functions it calls) or **tottime** (time in the function itself only).

Problem 11.1.3 Profile a `main()` that calls `slow_sum(300_000)` and a recursive `fib(22)`. Which is the hot spot?

Step 1	Create a `Profile`, enable it, run `main`, disable it.
Step 2	Print the top 5 rows sorted by cumulative time.

```
import cProfile, pstats, io
def slow_sum(n):
    return sum([i*i for i in range(n)])
def fib(n):
    return n if n < 2 else fib(n-1)+fib(n-2)
def main():
    slow_sum(300_000)
    fib(22)
pr = cProfile.Profile(); pr.enable(); main(); pr.disable()
s = io.StringIO()
pstats.Stats(pr, stream=s).sort_stats("cumulative").print_stats(5)
print(s.getvalue())
```

```
         57317 function calls (5 primitive calls) in 0.014 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000    0.014    0.014 /private/tmp/w11/b.py:6(main)
        1    0.006    0.006    0.008    0.008 /private/tmp/w11/b.py:2(slow_sum)
  57313/1    0.006    0.000    0.006    0.006 /private/tmp/w11/b.py:4(fib)
        1    0.002    0.000    0.002    0.002 {built-in method builtins.sum}
        1    0.000    0.000    0.000    0.000 {method 'disable' of
'_lsprof.Profiler' objects}
```

How to read it: `ncalls 57313/1` means `fib` was called 57,313 times in total, of which 1 was the outer call (the rest are recursive). `tottime` is time inside that function only. `cumtime` includes callers' children. `main` has cumulative 0.014 s and almost 0 tottime, so it is just a coordinator.

- **Answer** `slow_sum` (0.008 s cumulative) and `fib` (0.006 s) share the time. The profile also shows that `fib(22)` made 57,313 calls to compute one number, a clue that caching (`functools.lru_cache` , Chapter 4) would help.
- **Check** 0.008 + 0.006 = 0.014 s, which equals `main` 's cumulative time. Time adds up.
- **Why it matters** The profile points at the function to fix, so you stop guessing. (Profiling adds overhead, so use it to compare *relative* time, not absolute.)

From the command line you can profile a whole script: `python3 -m cProfile -s cumulative myscript.py` . The tool `line_profiler` (not run here) goes deeper: you decorate a function with `@profile` , run `kernprof -l -v script.py` , and it shows time per *line*.

tracemalloc: where does memory go?

Problem 11.1.4 How much memory does a list of 1,000,000 integers use, and how much does a generator that sums the same numbers use? Predict: is it more than 10 MB?

Step 1	Start `tracemalloc` . Build the list, read current memory.
Step 2	Sum a generator, and read the peak above the list.
Step 3	Compare `sys.getsizeof` for both objects.

```
import tracemalloc, sys
tracemalloc.start()
data = [i for i in range(1_000_000)]
cur, peak = tracemalloc.get_traced_memory()
print(f"list of 1M ints: current {cur/1e6:.1f} MB")
tracemalloc.reset_peak()
gen = (i for i in range(1_000_000))
total = sum(gen)
cur2, peak2 = tracemalloc.get_traced_memory()
print(f"generator sum peak extra: {(peak2-cur)/1e6:.4f} MB")
print(sys.getsizeof(data), sys.getsizeof(gen))
snap = tracemalloc.take_snapshot()
for st in snap.statistics("lineno")[:1]: print(st)
```

```
list of 1M ints: current 40.4 MB
generator sum peak extra: 0.0006 MB
8448728 200
/private/tmp/w11/d.py:3: size=38.6 MiB, count=999731, average=40 B
```

- **Answer** The list needs about 40 MB. The generator needed about 0.0006 MB extra, essentially nothing.
- **Check** `sys.getsizeof(data)` is 8.4 MB, which counts only the list's 8-byte pointers (1,000,000 × 8). Each integer object is a separate object of about 32 bytes, so 8 + 32 = 40 bytes per item, matching "average=40 B" and 40 MB total. `getsizeof` does not count the objects inside, so always trust `tracemalloc` for the full picture.
- **Why it matters** Generators (Chapter 7) keep memory flat, so you can stream a 50 GB file through a 200-byte object. `memory_profiler` (not run here) gives memory per line with `@profile` and `python -m memory_profiler script.py`.

The picture

```
"It is slow"
    |
    v
timeit / wall clock  --> how slow? (a number, repeatable)
    |
    v
cProfile             --> which function? (tottime, cumtime, ncalls)
    |
    v
line_profiler        --> which line?
    |
    v
fix ONE thing, measure again, keep tests green (Chapter 10)
```

Connection to the real job

- A pandas `apply` that takes 2 hours is usually one hot function. Profile before rewriting.
- ML training loops: profile the data loader before blaming the model.
- Memory errors ("Killed", `MemoryError`) are found with `tracemalloc` or by switching to generators and chunked reads.

Applications

- **Data Analytics:** find the slow cell in a notebook before buying a bigger machine.
- **Data Engineering:** find the slow transform in an ETL job; use generators to process big files.
- **Machine Learning:** profile feature code and data loading.
- **MLOps / backend:** profile an API endpoint that violates its latency target.

COMMON MISTAKE

Optimizing without measuring, or timing once. One run is noisy (disk cache, other programs). Repeat, and compare under the same conditions.

COMMON MISTAKE

Reading `getsizeof` as the real size. It counts only the container, not its contents.

Check yourself

1. What is the difference between `tottime` and `cumtime`? Answer: `tottime` is time inside the function only; `cumtime` also includes the functions it calls.
2. Why did `x in set` beat `x in list` by about 20,000 times in our test? Answer: a set uses a

hash to jump to the item; a list scans every item.

3. Why use `tracemalloc` and not `sys.getsizeof` for memory? Answer: `getsizeof` ignores the objects inside a container.

11.2 The GIL: Threads, Processes and Who Wins

INTUITION

Imagine a kitchen with one chef's knife and several cooks. Only the cook holding the knife can chop. If the cooks mostly *wait* (for water to boil, for a delivery), they can take turns with the knife and everyone finishes sooner. If they all need to chop all day, extra cooks do not help, because there is still one knife. The knife is the **GIL**.

What the GIL is

The **Global Interpreter Lock (GIL)** is a lock inside the standard Python interpreter (CPython). Only **one thread at a time** can run Python bytecode (the low-level instructions Python code is turned into). Why does it exist? It makes memory management (reference counting, Chapter 1) simple and fast for one thread, and it protects Python's own data structures. The price is that threads cannot run Python code in parallel on several CPU cores.

Two key facts follow:

- When a thread **waits** (sleep, network, disk), it releases the GIL, so another thread runs. So threads *do* help for **I/O-bound** work (work that mostly waits).
- For **CPU-bound** work (work that mostly computes, like loops and maths), threads give no speed-up. Use **processes**, each with its own interpreter and its own GIL.

Version note: recent Python versions offer an optional **free-threaded** build with no GIL (check `sys._is_gil_enabled()`). It is not the default in standard installs and library support is still growing, so in this chapter we use the normal build. On this machine the check prints `True`, meaning the GIL is on.

Words you need

Term	Meaning	Where it appears in the job
Thread	A path of execution inside one process; shares memory with other threads	Downloading many files at once

Process	A separate program instance with its own memory and its own GIL	Parallel CPU work like feature building
I/O-bound	Time is spent waiting for network, disk or database	API calls, S3 reads, SQL queries
CPU-bound	Time is spent computing	Parsing, simulation, training loops in pure Python
Race condition	Wrong result because threads touched shared data in a bad order	Shared counters, shared lists

The experiment

Problem 11.2.1 Eight "requests" each take 0.5 s of waiting (`time.sleep`). Four CPU jobs each loop 30 million times. Compare sequential, threads and processes. Predict first: will 4 threads make the CPU jobs about 4 times faster?

Step 1	For I/O: run 8 sleeps in a loop, then in a `ThreadPoolExecutor(8)` .
Step 2	For CPU: run 4 loops in sequence, then in a `ThreadPoolExecutor(4)` , then in a `ProcessPoolExecutor(4)` .
Step 3	Time each with `time.perf_counter()` .

```
import time
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

def io_task(_):
    time.sleep(0.5)
def cpu_task(n):
    s = 0
    for i in range(n):
        s += i*i
    return s

def timed(label, f):
    t=time.perf_counter(); f(); print(f"{label:<28}{time.perf_counter()-t:6.2f}
s")

N=30_000_000
if __name__ == "__main__":
    timed("IO x8 sequential", lambda: [io_task(0) for _ in range(8)])
    def tp():
        with ThreadPoolExecutor(8) as ex: list(ex.map(io_task, range(8)))
    timed("IO x8 threads(8)", tp)
    timed("CPU x4 sequential", lambda: [cpu_task(N) for _ in range(4)])
    def tc():
        with ThreadPoolExecutor(4) as ex: list(ex.map(cpu_task,[N]*4))
    timed("CPU x4 threads(4)", tc)
    def pc():
        with ProcessPoolExecutor(4) as ex: list(ex.map(cpu_task,[N]*4))
    timed("CPU x4 processes(4)", pc)
```

Real output (a separate run added the asyncio line, shown in 11.3):

```
IO x8 sequential             4.03 s
IO x8 threads(8)             0.51 s
CPU x4 sequential            3.18 s
CPU x4 threads(4)            3.15 s
CPU x4 processes(4)          1.30 s
```

- **Answer** Threads made the I/O job 7.9 times faster (4.03 ÷ 0.51). Threads gave **no** speed-up on the CPU job (3.18 s against 3.15 s). Processes cut it to 1.30 s, about 2.4 times faster.
- **Check** The sequential I/O time must be 8 × 0.5 = 4.0 s, and it was 4.03 s. In parallel it must be about one sleep, 0.5 s, and it was 0.51 s. For CPU, 4 jobs of about 0.8 s each give about 3.2 s, matching.
- **Why it matters** Why only 2.4 times and not 4 times for processes? Starting processes, sending the function and result between them (pickling), and the machine's other work cost time. Parallel speed-up is almost always less than the number of workers, and small jobs may even get slower.

Here is the typical wrong version with processes, and its error message:

```
from concurrent.futures import ProcessPoolExecutor
with ProcessPoolExecutor(2) as ex:
    print(list(ex.map(lambda x: x*2, [1,2])))
```

```
PicklingError - Can't pickle <function <lambda> at 0x1039e6770>: it's not found
as __m
```

Processes need to send the function to another process by **pickling** (turning an object into bytes). A `lambda` has no importable name, so it fails. The fix is a normal named function defined at the top level of a module. Also, on macOS and Windows the code that starts processes must sit under `if __name__ == "__main__":`, because each child re-imports the file.

A race condition, on purpose

Threads share memory. If two threads update the same variable, results can go wrong. This code does "read, wait a tiny moment, write", which gives other threads the chance to run in the middle:

```
import threading, time
counter = 0
lock = threading.Lock()
def unsafe(n):
    global counter
    for _ in range(n):
        v = counter
        time.sleep(0)       # gives other threads a chance to run mid-update
        counter = v + 1
def safe(n):
    global counter
    for _ in range(n):
        with lock:
            v = counter
            time.sleep(0)
            counter = v + 1
def run(fn):
    global counter
    counter = 0
    ts=[threading.Thread(target=fn,args=(2000,)) for _ in range(4)]
    [t.start() for t in ts]; [t.join() for t in ts]
    return counter
print("unsafe:", run(unsafe), "expected 8000")
print("safe:  ", run(safe), "expected 8000")
```

```
unsafe: 2015 expected 8000
safe:   8000 expected 8000
```

- **Answer** The unsafe version lost about three quarters of the updates (2015 instead of 8000, and the number changes from run to run). With a `Lock`, the result is exactly 8000.
- **Check** 4 threads × 2000 = 8000. Only the locked version equals it.
- **Why it matters** Honest note: when I removed the `time.sleep(0)` and did a plain `counter += 1` loop, the unsafe version gave the correct answer in my two runs on this Python version. That does *not* make it safe. Whether it fails depends on timing and on the Python version, which is exactly why race bugs are dangerous: they hide in testing and appear in production. Rule: when threads share data that changes, use a `Lock` or a `queue.Queue` (11.5).

The picture

```
I/O-bound, 3 threads (one GIL)          CPU-bound, 3 threads (one GIL)
T1: run..wait.........run               T1: run run run ...  (holds GIL)
T2: ..run..wait........run              T2: wait ........... (no GIL)
T3: ....run..wait......run              T3: wait ...........
waits overlap, total time shrinks       no overlap of running, no speed-up

CPU-bound, 3 processes (3 GILs)
P1: run run run          each process has its own GIL
P2: run run run          and its own core: real parallelism
P3: run run run
```

Connection to the real job

- Calling 500 REST APIs or reading 500 S3 files: threads (or asyncio).
- Transforming 8 big files with pure-Python code: processes.
- Most numeric libraries (NumPy, pandas internals) release the GIL inside their fast C code, so threads can sometimes help there. Measure.

Applications

- **Data Analytics:** download many report files in parallel with a thread pool.
- **Data Engineering:** process daily partitions in a process pool; parallel S3 or API ingestion with threads.
- **Machine Learning:** `DataLoader` workers are processes for exactly this GIL reason; parallel hyper-parameter runs.
- **MLOps:** serve models with several worker processes (for example `gunicorn` / `uvicorn`

workers) to use all cores.

COMMON MISTAKE

Using threads to speed up a CPU loop and seeing no gain (3.18 s against 3.15 s above). Check: is the job waiting or computing?

COMMON MISTAKE

Sharing and changing a list or counter in threads without a lock. Prefer passing results back (`executor.map` returns them in order) or use a queue.

Check yourself

1. What does the GIL allow and not allow? Answer: only one thread runs Python bytecode at a time, so no CPU parallelism with threads in one process.
2. Why do threads still help I/O-bound jobs? Answer: a waiting thread releases the GIL, so others run during the wait.
3. Why did the `lambda` fail in a process pool? Answer: pickling needs a function with an importable name; a lambda has none.

11.3 concurrent.futures and asyncio

INTUITION

A restaurant waiter can serve 20 tables alone. He takes an order, and while the kitchen cooks, he goes to the next table. He never stands idle. This is `asyncio`: one worker that switches to another task whenever the current one has to wait. Threads are like hiring 20 waiters. `asyncio` is cheaper when you have thousands of tables, but the waiter must *never* freeze at one table.

concurrent.futures: the easy, high-level tool

`concurrent.futures` gives one simple interface for thread pools and process pools. You learned `ex.map(fn, items)` above. For more control use `submit`, which returns a **future** (a promise of a result that is not ready yet), and `as_completed`, which hands you futures as they finish. Errors inside a job are raised when you ask for `result()`.

Problem 11.3.1 Run 6 jobs in a pool of 4 threads. Job 3 fails on purpose. Collect all results and report the failure without crashing.

Step 1	Submit each job, and remember which item belongs to which future in a dictionary.
Step 2	Loop over `as_completed`. Wrap `f.result()` in `try/except`.

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import time, random
def job(i):
    time.sleep(random.random()*0.3)
    if i == 3: raise ValueError("bad item 3")
    return i*10
with ThreadPoolExecutor(4) as ex:
    futs = {ex.submit(job, i): i for i in range(6)}
    for f in as_completed(futs):
        i = futs[f]
        try: print(i, "->", f.result())
        except ValueError as e: print(i, "failed:", e)
```

Real output, sorted so the order is stable (the real arrival order changes each run):

```
0 -> 0
1 -> 10
2 -> 20
3 failed: bad item 3
4 -> 40
5 -> 50
```

- **Answer** Five jobs returned 10 times their number, and item 3's error was caught and reported.
- **Check** 4 → 40 and 5 → 50 are 10 × input. The arrival order differs run to run because each job sleeps a random time. If you need results in input order, use `ex.map`.
- **Why it matters** One bad URL in 5,000 must not kill the whole batch. Catch errors per future and keep a list of failures to retry.

asyncio: many waiting tasks, one thread

A **coroutine** is a function defined with `async def`. Calling it does not run it; it gives an object that the **event loop** (the manager that switches between tasks) runs. Inside a coroutine, `await something` means "I will wait here; run another task meanwhile". `asyncio.gather` runs many coroutines together, and `asyncio.run(main())` starts the loop.

Problem 11.3.2 Eight tasks each wait 0.5 s, using `asyncio`. How long in total? Then limit concurrency to 5 at a time for 20 tasks of 0.2 s, and add a timeout.

Step 1	`gather` 8 tasks of `await asyncio.sleep(0.5)`. Total should be about 0.5 s.
Step 2	Use `asyncio.Semaphore(5)`, a gate that lets only 5 tasks inside at once. 20 tasks ÷ 5 per round = 4 rounds × 0.2 s = 0.8 s.
Step 3	Use `asyncio.timeout(0.1)` around a 1 s wait.

```
import asyncio, time
async def fetch(i, sem):
    async with sem:
        await asyncio.sleep(0.2)
        return i*i
async def main():
    sem = asyncio.Semaphore(5)
    t=time.perf_counter()
    res = await asyncio.gather(*(fetch(i, sem) for i in range(20)))
    print(res[:5], f"{time.perf_counter()-t:.2f}s")
    try:
        async with asyncio.timeout(0.1):
            await asyncio.sleep(1)
    except TimeoutError:
        print("timed out")
asyncio.run(main())
```

```
[0, 1, 4, 9, 16] 0.80s
timed out
```

The 8-task asyncio run from the experiment script printed `IO x8 asyncio 0.50 s`, the same as 8 threads.

- **Answer** 8 tasks took 0.50 s (not 4 s). The gated run took 0.80 s, exactly as predicted, and the timeout fired.
- **Check** 20 ÷ 5 = 4 rounds, 4 × 0.2 = 0.8 s. Matches.
- **Why it matters** The semaphore is how you stay polite to a server: a limit such as "5 requests at a time" is the same idea as a restaurant that seats only 5 tables at a time.

The typical wrong version: blocking inside async

Predict: five tasks, each doing 0.2 s of waiting. Which version finishes in about 0.2 s?

```
import asyncio, time
async def bad(i):
    time.sleep(0.2)          # blocks the whole event loop
    return i
async def good(i):
    await asyncio.sleep(0.2)
    return i
async def main():
    t=time.perf_counter(); await asyncio.gather(*(bad(i) for i in range(5)));
print(f"time.sleep in async : {time.perf_counter()-t:.2f}s")
    t=time.perf_counter(); await asyncio.gather(*(good(i) for i in range(5)));
print(f"await asyncio.sleep : {time.perf_counter()-t:.2f}s")
    t=time.perf_counter(); await asyncio.gather(*
(asyncio.to_thread(time.sleep,0.2) for i in range(5)));
print(f"to_thread(time.sleep): {time.perf_counter()-t:.2f}s")
asyncio.run(main())
```

```
time.sleep in async : 1.02s
await asyncio.sleep : 0.20s
to_thread(time.sleep): 0.21s
```

- **Answer** `time.sleep` inside a coroutine froze the loop, so the five tasks ran one after another (5 × 0.2 = 1.0 s). `await asyncio.sleep` took 0.20 s. If you must call a blocking function (an old library without async support), `asyncio.to_thread` runs it in a thread and gives 0.21 s.
- **Check** 5 × 0.2 = 1.0 s for sequential, and 1.02 s was measured.
- **Why it matters** The whole point of asyncio is that nothing blocks. One blocking call in a coroutine ruins every other task.

Which tool to choose

Situation	Tool	Why
10 to 200 API calls or files, simple code	`ThreadPoolExecutor`	Easiest; works with normal libraries
Thousands of connections, async libraries (`httpx.AsyncClient` , FastAPI)	`asyncio`	Very cheap tasks
CPU-heavy pure Python on many cores	`ProcessPoolExecutor` / `multiprocessing`	Separate GILs
Heavy NumPy/pandas maths	Vectorize first (11.4)	Fast C code beats any pool

The picture

```
asyncio event loop (one thread)
task A: start -> await(wait) .............. -> finish
task B:          start -> await(wait) ..... -> finish
task C:                   start -> await ... -> finish
The loop jumps to a ready task whenever another one awaits.
One blocking call (time.sleep) would freeze A, B and C together.
```

Connection to the real job

- Calling a web API 10,000 times (for example, an LLM or geocoding API) is a classic async or thread-pool job, always with a concurrency limit and a timeout.
- FastAPI (Chapter 12) endpoints declared with `async def` run on an event loop, so blocking code inside them hurts all users.
- Real services also have rate limits (for example, "50 requests per second"). Respect them with a semaphore and sleeps.

Applications

- **Data Analytics:** pull data from several SaaS APIs at once for a daily report.
- **Data Engineering:** async ingestion of many small files or API pages.
- **Machine Learning / GenAI:** send many prompts or embedding batches in parallel with a limit.
- **MLOps:** async health checks and web endpoints.

COMMON MISTAKE

Forgetting `await`. Calling `fetch(1)` without `await` gives a coroutine object and a warning that the coroutine "was never awaited". The code inside never ran.

COMMON MISTAKE

Launching 10,000 tasks with no limit. The remote server blocks you, or you run out of sockets. Use a semaphore.

Check yourself

1. What does `await` do? Answer: pauses this coroutine and lets the event loop run other tasks until the awaited thing is ready.

2. Why did `time.sleep` in an async function take 1.0 s for five tasks? Answer: it blocked the event loop, so tasks ran one by one.
3. What does a `Semaphore(5)` do? Answer: lets at most five tasks be inside the guarded block at the same time.

11.4 Vectorization, Algorithms and Native Speed

INTUITION

A cashier who scans one item at a time, thinking about each, is slow. A barcode reader that scans a whole tray at once is fast. Python's `for` loop is the thinking cashier: for each item it checks types, looks up names, and makes objects. **Vectorization** means handing the whole tray to fast compiled code (written in C), so the loop happens outside Python.

What vectorization means

A **vectorized** operation applies one operation to a whole array at once, for example "multiply every element by 2.5 and add 1". Libraries such as NumPy and pandas store numbers in compact blocks of memory with one type (not as separate Python objects), and run the loop in C. The idea is the same as a SQL query: say *what* you want on a whole column, not *how* to loop over rows.

Pure-Python hints you can run now

NumPy is not installed here, but we can already show the "loop overhead" effect with built-in functions that run their loop in C.

Problem 11.4.1 Compute `x * 2.5 + 1` for 1,000,000 numbers with a list comprehension and with an index loop. Then compute the mean with `sum/len` and with `statistics.mean`. Predict which of each pair is faster.

Step 1	Build `xs = list(range(1_000_000))`.
Step 2	Time each approach once with `time.perf_counter()`.

```
import time, statistics
n=1_000_000
xs=list(range(n))
t=time.perf_counter(); ys=[x*2.5+1 for x in xs]; print(f"list comp
{time.perf_counter()-t:.3f}s")
t=time.perf_counter()
ys=[0.0]*n
for i in range(n): ys[i]=xs[i]*2.5+1
print(f"index loop {time.perf_counter()-t:.3f}s")
t=time.perf_counter(); statistics.mean(xs); print(f"statistics.mean
{time.perf_counter()-t:.3f}s")
t=time.perf_counter(); sum(xs)/len(xs); print(f"sum/len {time.perf_counter()-
t:.4f}s")
```

```
list comp 0.026s
index loop 0.056s
statistics.mean 0.091s
sum/len 0.0032s
```

- **Answer** The comprehension (0.026 s) was about twice as fast as the index loop (0.056 s). `sum(xs)/len(xs)` (0.0032 s) was about 28 times faster than `statistics.mean` (0.091 s).
- **Check** The mean of `0..999,999` is 499,999.5 for both. Speed differs because `statistics.mean` does careful exact arithmetic to avoid rounding errors, so it is built for accuracy, not speed. `sum` runs its loop in C.
- **Why it matters** The rule: the fewer times the Python interpreter itself runs per item, the faster. Use built-ins (`sum` , `min` , `max` , `sorted` , `str.join`) and comprehensions before hand-written loops, and compare cost of safety against speed. For big numeric data, NumPy goes much further.

The NumPy and pandas version (not run here)

```
# not run here: needs numpy and pandas
import numpy as np
x = np.arange(1_000_000)
y = x * 2.5 + 1            # whole array, no Python loop
print(y.mean())

import pandas as pd
df = pd.DataFrame({"qty": [2, 5, 1], "price": [100.0, 40.0, 250.0]})
df["total"] = df["qty"] * df["price"]      # vectorized column maths
# slow pattern to avoid:
# df["total"] = df.apply(lambda r: r["qty"] * r["price"], axis=1)
```

On typical machines, vectorized NumPy code is often tens to hundreds of times faster than a pure-

Python loop for this kind of maths. I did not measure it here, so measure on your own data.

When vectorization is not enough

Tool	What it does	When to consider it
Better algorithm / data structure	Set instead of list (11.1)	Always first
`lru_cache`	Remembers results of repeated calls	Repeated expensive calls like `fib`
Numba (awareness)	`@njit` compiles numeric Python loops to machine code	A numeric loop you cannot vectorize
Cython (awareness)	Python-like code compiled to C	Library authors, hot loops needing C speed
`multiprocessing` for pandas	Split data into chunks, run each in a process	CPU-heavy per-row Python functions
Polars, DuckDB (awareness)	Faster dataframe/SQL engines	Large local data

Multiprocessing for CPU-bound chunks

Problem 11.4.2 Count the primes below 2,000,000 by splitting the range into 8 chunks of 250,000. Compare one process with a `Pool(4)`.

Step 1	Write `count_primes((lo, hi))` that trial-divides up to the square root.
Step 2	Build the 8 chunks. Run `map` serially, then `Pool(4).map`.
Step 3	Check both results match, and compute speed-up = serial time ÷ parallel time.

```
import time
from multiprocessing import Pool
def count_primes(rng):
    lo, hi = rng
    c = 0
    for n in range(lo, hi):
        if n < 2: continue
        for d in range(2, int(n**0.5)+1):
            if n % d == 0: break
        else: c += 1
    return c
if __name__ == "__main__":
    chunks = [(i*250_000, (i+1)*250_000) for i in range(8)]
    t=time.perf_counter(); r1=sum(map(count_primes, chunks));
t1=time.perf_counter()-t
    t=time.perf_counter()
    with Pool(4) as p: r2=sum(p.map(count_primes, chunks))
    t2=time.perf_counter()-t
    print(r1, r2, f"serial {t1:.2f}s  pool(4) {t2:.2f}s  speedup {t1/t2:.1f}x")
```

```
148933 148933 serial 3.75s  pool(4) 1.49s  speedup 2.5x
```

- **Answer** Both counted 148,933 primes. The pool was 2.5 times faster (3.75 ÷ 1.49), not 4 times.
- **Check** The number of primes below 2,000,000 is 148,933, a known value, so the result is right. Why not 4 times? Later chunks hold bigger numbers and take longer, so some workers finish early and wait (uneven load), and starting processes costs time.
- **Why it matters** Splitting work into many *similar-sized* chunks (more chunks than workers) balances the load. The same pattern works for a pandas DataFrame: split by groups or row ranges, run a function in each process, and concatenate the pieces. The data is copied to each process, so this pays off only when the per-chunk work is heavy.

The picture

```
 Python loop:    [x1] -> python -> [y1]   one item at a time, overhead each
time
                 [x2] -> python -> [y2]

 Vectorized:     [x1 x2 x3 x4 ... xN]  --one call--> [y1 y2 y3 ... yN]
                 (tight loop in C over a compact block of memory)
```

Connection to the real job

- "Never loop over DataFrame rows" is a standard code-review comment in data teams. Use column

operations, `groupby`, or SQL.

- ML pipelines: feature maths in NumPy; custom Python per-row logic is the usual bottleneck.
- Know when to stop: if the job runs 5 minutes once a day, readable code beats a 3-minute saving.

Applications

- **Data Analytics:** replace `apply(axis=1)` with column maths.
- **Data Science:** vectorized simulation (many random draws at once).
- **Machine Learning / Deep Learning:** array and tensor maths on CPU and GPU is vectorization at scale.
- **Data Engineering:** chunked processing in a process pool, or push the work to SQL or Spark.

COMMON MISTAKE

Rewriting everything in a faster style before profiling. First prove the loop is the hot spot (11.1).

COMMON MISTAKE

Using a process pool on tiny tasks. The start-up and copying cost can be larger than the work. Measure with real data size.

Check yourself

1. What does "vectorized" mean? Answer: one operation applied to a whole array, with the loop running in fast compiled code.
2. Why was `sum(xs)/len(xs)` faster than `statistics.mean(xs)`? Answer: `sum` loops in C, while `statistics.mean` does exact, careful arithmetic in Python.
3. Why was the pool speed-up 2.5x, not 4x? Answer: uneven chunk sizes plus process start-up and data-copy costs.

11.5 Queues, Back-Pressure and Little's Law

INTUITION

At a bank, customers wait in a line, and tellers take the next customer when free. If customers arrive faster than tellers can serve, the line grows without end. A programmer's queue is the same line. And Little's law tells you how many tellers you need for a target speed.

Producer, consumer and queue

A **producer** creates work items. A **consumer** (or worker) processes them. A **queue** sits between them. Python's `queue.Queue` is thread-safe: many threads can put and get without a lock of your own. With `maxsize`, `put` waits when the queue is full. That is **back-pressure**: the fast producer is forced to slow down to the speed of consumers instead of filling memory.

Problem 11.5.1 One producer puts 8 items into a queue with `maxsize=3`. Two consumers process each item in 10 ms. How do they stop?

Step 1	The producer puts items 0 to 7, then puts one `None` per consumer as a "stop" signal (a **sentinel**).
Step 2	Each consumer loops `while (item := q.get()) is not None`.
Step 3	Join all threads and print the sorted results.

```
import queue, threading, time
q = queue.Queue(maxsize=3)
results = []
def producer():
    for i in range(8):
        q.put(i)              # blocks when queue is full (back-pressure)
    for _ in range(2): q.put(None)
def consumer(name):
    while (item := q.get()) is not None:
        time.sleep(0.01)
        results.append((name, item))
ts=[threading.Thread(target=producer)]+[threading.Thread(target=consumer,args=
(f"c{i}",)) for i in range(2)]
[t.start() for t in ts]; [t.join() for t in ts]
print(sorted(i for _,i in results))
print(len(results), "items processed")
```

```
[0, 1, 2, 3, 4, 5, 6, 7]
8 items processed
```

- **Answer** Every item was processed exactly once, and both consumers stopped cleanly. Which consumer took which item varies between runs, but the final sorted list is always the same.
- **Check** 8 items in, 8 results out, none duplicated and none lost.
- **Why it matters** This is the pattern behind message queues (Kafka, SQS, RabbitMQ) in data engineering, just on a small scale. (`list.append` is safe here because it is a single step; for anything bigger, use a queue or a lock.)

Little's law

Little's law says: in a stable system, the average number of requests inside the system `L` equals the arrival rate (throughput) `λ` times the average time each spends inside `W`.

```
L = λ × W        so      λ = L / W      (throughput = concurrency / latency)
```

Check the units: `L` is a count of requests. `λ` is requests per second. `W` is seconds. Requests per second × seconds = requests. In a bank, `L` is the number of customers inside, `W` is the average time each spends there, and `λ` is how many customers arrive per minute.

Problem 11.5.2 Each "request" takes `W = 0.05 s`. Predict the throughput for concurrency 1, 5, 10 and 20, then measure with real threads for 2 seconds each.

Formula	throughput `λ = L / W`
Plug in and compute	L=1: 1 ÷ 0.05 = 20 req/s. L=5: 5 ÷ 0.05 = 100. L=10: 10 ÷ 0.05 = 200. L=20: 20 ÷ 0.05 = 400.
Step 1	Start `L` threads. Each loops: `time.sleep(0.05)`, then add 1 to a locked counter, until the 2 s are over.
Step 2	Divide the total count by elapsed time.

```
import threading, time
LAT = 0.05          # each request takes 50 ms
DURATION = 2.0
def run(workers):
    done = 0
    lock = threading.Lock()
    end = time.perf_counter() + DURATION
    def w():
        nonlocal done
        while time.perf_counter() < end:
            time.sleep(LAT)
            with lock: done += 1
    ts=[threading.Thread(target=w) for _ in range(workers)]
    t0=time.perf_counter()
    [t.start() for t in ts]; [t.join() for t in ts]
    el=time.perf_counter()-t0
    return done/el
for c in (1,5,10,20):
    print(f"concurrency {c:>2}: measured {run(c):7.1f} req/s   predicted
{c/LAT:7.1f} req/s")
```

```
concurrency  1: measured    18.6 req/s   predicted    20.0 req/s
concurrency  5: measured    92.6 req/s   predicted   100.0 req/s
concurrency 10: measured   185.3 req/s   predicted   200.0 req/s
concurrency 20: measured   370.6 req/s   predicted   400.0 req/s
```

- **Answer** Throughput grows in a straight line with concurrency, as the law says. Measured values are about 93% of predicted (for example 370.6 against 400).
- **Check** 370.6 ÷ 400 = 0.9265, and 18.6 ÷ 20 = 0.93. The same 7% gap at every level means one constant cause: `time.sleep(0.05)` always takes a little longer than 50 ms, so the real `W` is about 0.054 s. Using that `W`, 20 ÷ 0.054 = 370 req/s, which matches.
- **Why it matters** In plain words: to get more throughput with the same latency, you need more concurrency, in the same proportion. And if latency doubles (a slow database), throughput at the same concurrency halves. Real servers also have a limit (CPU cores, connections), so the straight line eventually bends flat. That bend is the saturation point you look for when you push traffic at a service. This simple experiment has no such limit, because sleeping uses no CPU.

Use Little's law to size a worker pool: to process 200 requests per second when each takes 0.25 s, you need `L = 200 × 0.25 = 50` concurrent workers (threads or async tasks), plus some safety margin.

The picture

```
producers --> [ queue (maxsize=3) ] --> consumer 1
(fast)        [ x ][ x ][ x ]        --> consumer 2
                 ^
                 | queue full: put() waits  (back-pressure)

Throughput vs concurrency (fixed latency, no resource limit):
req/s |                      /
      |                 /
      |            /           slope = 1 / W
      |       /
      +-------------------- concurrency
```

Connection to the real job

- Worker pool size, connection pool size, and Kafka/SQS consumer counts are all chosen with `L = λ × W`.
- When a pipeline "gets slower at night", check whether the queue between stages is growing. A growing queue means arrivals exceed service rate.
- You can check the formula on your own Python service by sending it many requests with a traffic

tool such as Locust (named only as an example) and comparing with the prediction.

Applications

- **Data Engineering:** sizing consumers for a message queue; back-pressure in streaming jobs.
- **MLOps:** sizing replicas and worker counts for a model API for a target requests-per-second.
- **Machine Learning:** prefetch queues in data loaders keep the GPU busy.
- **Data Analytics / BI:** explaining why a dashboard slows down when many users open it at 9 am.

COMMON MISTAKE

Using an unbounded queue. A slow consumer then lets the queue grow until memory runs out. Set `maxsize`.

COMMON MISTAKE

Forgetting the stop signal. Consumers wait forever on `q.get()` and the program never exits. Use a sentinel per consumer, or `q.task_done()` with `q.join()`.

Check yourself

1. State Little's law and its units. Answer: `L = λ × W`: requests in system = requests per second × seconds each.
2. Latency is 0.1 s and you run 30 concurrent workers. What throughput do you expect? Answer: 30 ÷ 0.1 = 300 requests per second (at best).
3. What is back-pressure? Answer: a full bounded queue makes the producer wait, so it slows to the speed of the consumers.

Chapter recap

- Measure first: `timeit` for small comparisons, `cProfile` to find the hot spot, `tracemalloc` for memory.
- The biggest wins come from better algorithms and data structures (set against list: about 20,000 times faster in our test), not style tricks.
- The GIL lets only one thread run Python bytecode at a time. Threads help I/O-bound work and do not help CPU-bound work. Free-threaded builds exist but are not the default.
- Processes bypass the GIL, with costs: start-up, pickling, copying. Speed-up is less than the worker count.
- `concurrent.futures` is the easy interface to thread and process pools. Handle errors per

future.

- `asyncio` runs many waiting tasks on one thread. Never block inside a coroutine; limit concurrency with a semaphore and add timeouts.
- Vectorization moves the loop into compiled code (NumPy, pandas). Avoid row loops and `apply(axis=1)` on big data.
- Shared mutable data between threads needs a lock or a queue. Bounded queues give back-pressure.
- Little's law `L = λ × W` links concurrency, latency and throughput, the same three numbers as in a bank queue: customers inside, time spent, arrival rate.

Cheat-sheet

Need	Tool or formula
Compare two snippets	`timeit.timeit(stmt, number=n)`
Find the slow function	`python3 -m cProfile -s cumulative script.py`
Memory by line of code	`tracemalloc.start()`, `take_snapshot().statistics("lineno")`
Many waits (I/O)	`ThreadPoolExecutor` or `asyncio.gather`
Many calculations (CPU)	`ProcessPoolExecutor` or `Pool.map`
Limit parallel tasks	`asyncio.Semaphore(n)`
Call blocking code from async	`await asyncio.to_thread(fn, ...)`
Protect shared data	`with threading.Lock():` or `queue.Queue(maxsize=n)`
Throughput	`λ = L / W`
Workers needed	`L = λ × W`

Practice set

Problem 11.P.1 (easy) A job has `L = 10` concurrent workers and each request takes `W = 0.2 s`. What is the throughput?

Answer: λ = L ÷ W = 10 ÷ 0.2 = 50 requests per second.

Problem 11.P.2 (easy) You must download 100 files, each taking about 1 s of network wait, with a pool of 20 threads. About how long will it take?

Answer: 100 ÷ 20 = 5 rounds of 1 s each, about 5 s (compared with 100 s sequentially).

Problem 11.P.3 (easy, what is the output?) What does this print and why?

```
import asyncio
async def f():
    return 5
print(f())
```

Answer: it prints something like `<coroutine object f at 0x...>` (plus a warning that it was never awaited). Calling an `async def` function does not run it; you must `await f()` or use `asyncio.run(f())`.

Problem 11.P.4 (medium) Which tool for each: (a) 300 REST API calls, (b) a pure-Python loop over 10 big files on 8 cores, (c) summing a column of 10 million numbers?

Answer: (a) thread pool or asyncio with a limit, (b) process pool, (c) vectorized NumPy/pandas (or the built-in `sum`), no pool.

Problem 11.P.5 (medium, find the bug) This code uses a thread pool to speed up a CPU-heavy function and gets no gain. Why, and what is the fix?

```
with ThreadPoolExecutor(8) as ex:
    results = list(ex.map(heavy_math, data))
```

Answer: `heavy_math` is CPU-bound pure Python, so the GIL allows one thread at a time. Use `ProcessPoolExecutor` (with `heavy_math` defined at module top level), or vectorize it.

Problem 11.P.6 (medium, find the bug) An async function calls `requests.get(url)` (a blocking call) inside `gather` of 100 tasks. It takes 100 times one call. Fix it two ways.

Answer: the blocking call freezes the event loop, so the tasks run one by one. Fix 1: use an async client such as `httpx.AsyncClient` with `await client.get(url)`. Fix 2: wrap the blocking call with `await asyncio.to_thread(requests.get, url)`.

Problem 11.P.7 (medium) A profile shows `fib` with `ncalls = 57313/1`. What does this mean, and what is the cheapest fix?

Answer: 57,313 total calls of which 1 was the outer call, so the rest are recursive repeats of the same sub-problems. Add `@functools.lru_cache(maxsize=None)` so each value is computed once.

Problem 11.P.8 (medium) You need 500 requests per second. The average latency of your service is 0.04 s. How many concurrent workers do you need at minimum?

Answer: L = λ × W = 500 × 0.04 = 20 workers. Add margin (for example 30) because latency varies and the formula is for averages.

Problem 11.P.9 (hard, scenario) Your ETL reads 5,000 small JSON files from cloud storage and parses each (parsing takes 2 ms CPU, reading takes 100 ms wait). Design the concurrency.

Answer: reading is I/O-bound (100 ms wait against 2 ms CPU), so a thread pool of 50 to 100 workers works: 5,000 × 0.1 s ÷ 50 = 10 s of wall time. Parsing at 2 ms × 5,000 = 10 s CPU total, which fits in the same threads (the GIL allows it). Only if parsing were heavy would you add processes. Add retries and per-file error handling, and measure before tuning.

Problem 11.P.10 (hard, interview) Explain the GIL to a junior engineer in four sentences, and say when you would use threads, processes or asyncio.

Answer: The GIL is a lock in CPython that lets only one thread run Python bytecode at a time. Threads still help when they wait, because a waiting thread releases the lock. For CPU-bound work you use processes, each with its own GIL. I use threads for a moderate number of I/O tasks with normal libraries, asyncio for thousands of connections with async libraries, and processes (or vectorized code) for CPU-bound work.

Problem 11.P.11 (hard, interview) A teammate says "just add more threads, the API is slow." Using Little's law, explain what may go wrong.

Answer: more threads raise concurrency L, which raises throughput only while latency W stays flat. If the server is saturated, extra concurrency just raises W (requests queue), and throughput stays flat or drops, with more errors and timeouts. I would try 10, 20, 40, 80 threads, find where throughput stops growing, and use a rate limit and a timeout.

Prompt pack

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data, ML and AI roles, and English is my second language, so use simple
words. Teach me the Python GIL, threads, processes and the difference between
I/O-bound and CPU-bound work. Explain step by step with small timed experiments
and real numbers, and connect each idea to everyday examples like a kitchen or
a bank where it helps. After each part, ask me one question to check my
understanding and wait for my answer before moving on.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data, ML and AI roles, and English is my second language. The topic is
asyncio in Python: coroutines, await, gather, Semaphore, timeout and to_thread.
Give me 8 graded exercises from easy to hard, using sleep to simulate network
calls so I can run them offline. Do not give solutions yet. After I send my
answers, review each one, show the bug if any, and explain the event loop in
simple English.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data, ML and AI roles. Chapter topic: profiling and speeding up Python
code with timeit, cProfile, tracemalloc and vectorization. Here is my slow
code: [PASTE HERE]. Guide me to find the hot spot by measurement, not guessing.
Ask me to run each profiling step and paste the output, then explain how to
read it, propose one change at a time, and help me verify that results stay
identical and the code is faster.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data, ML and AI roles, and English is my second language. Topic: Little's
law (L = lambda x W), queues, back-pressure and worker pool sizing in Python
services. Teach it using everyday examples (a bank queue, tellers, waiting
time, customers served per hour). Then give me 6 sizing problems with real
numbers, one at a time, and check each answer. Include one problem where the
system saturates and explain why throughput stops growing.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data, ML and AI roles. Run a mock interview on Python concurrency and
performance for a data engineer job. Ask me one question at a time: the GIL,
threads versus processes versus asyncio, race conditions and locks, profiling
workflow, vectorization, multiprocessing with pandas, and queues. Grade each
answer from 1 to 5, give a model answer, and one improvement tip. After 10
questions, summarise my weak areas and what to practise next.
```

CHAPTER 12

APIS, THE WEB, SCRAPING AND COMMAND-LINE TOOLS

How programs talk to other programs over HTTP, and how you build tools that other people can run.

Imagine your manager says: "Pull yesterday's orders from the partner's system, score them with our model, and run it every night." To do that you must call a web service, understand its replies, handle failures, build a small service of your own, and wrap it all in a command you can schedule. By the end of this chapter you will be able to read and design a REST API, call it safely from Python, protect it with tokens, scrape a page politely, and package your work as a command-line tool. You have seen HTTP from the outside: every time you open a web page, your browser sends a request and waits for a reply. Now you will learn the inside.

Words you need. **API** (Application Programming Interface) is a doorway that lets one program use another program. **HTTP** is the language web programs use to talk. **JSON** is a plain-text format for data that looks like Python dicts and lists. **Client** is the program that asks. **Server** is the program that answers.

In this chapter, everything marked "run" was executed with Python 3.14 and only the standard library. Third-party libraries (`requests` , `httpx` , `FastAPI` , `Flask` , `typer` , `BeautifulSoup` , `Scrapy` , `Selenium` , `Playwright`) were **not run here** because this study machine had no network access to install them. Their code is short, standard and minimal. Check each library's current docs for version changes.

12.1 HTTP and REST Design

INTUITION

HTTP is a postal system with strict forms. You write a request on a form (method, address, headers, body), post it, and get back a reply form (status code, headers, body). REST is a set of habits for filling these forms in a predictable way, so that a stranger can guess how your service works. Every call a program makes to a web service is exactly one such form, like one order slip handed to a shop counter.

How it works

An HTTP request has four parts:

- **Method**: what you want to do. `GET` reads, `POST` creates, `PUT` replaces, `PATCH` changes part, `DELETE` removes.
- **URL**: the address, such as `/v1/items/2?limit=3`. The part after `?` is the **query string**.
- **Headers**: small labels, such as `Content-Type: application/json` or `Authorization: ...`.
- **Body**: the data (usually JSON) for `POST`, `PUT`, `PATCH`.

A response has a **status code**, headers and a body. Status codes come in families:

Code	Name	When to use
200	OK	Success with a body
201	Created	A new thing was made (add a `Location` header)
204	No Content	Success, nothing to return (common for DELETE)
400	Bad Request	The client sent nonsense
401	Unauthorized	Not logged in (really means "unauthenticated")
403	Forbidden	Logged in, but not allowed
404	Not Found	No such thing
409	Conflict	Clash with current state (duplicate)
422	Unprocessable	Valid JSON but wrong content
429	Too Many Requests	Rate limit hit
500	Internal Server Error	Your bug
503	Service Unavailable	Overloaded or down

REST design habits:

1. Use **nouns** in URLs, not verbs: `/items`, not `/getItems`.
2. Use plural collections: `/items` (all), `/items/2` (one).
3. The method carries the action: `GET /items/2`, `DELETE /items/2`.
4. Return JSON and correct status codes.
5. Be **stateless**: each request carries everything the server needs (for example a token).

Two properties matter a lot. A method is **safe** if it does not change data (`GET`). A method is

idempotent if doing it many times has the same effect as doing it once (`GET` , `PUT` , `DELETE`). `POST` is normally not idempotent. This matters for retries (see 12.4).

Worked examples

Problem 12.1.1 Start a tiny web server inside a Python script, call it, and read the status, header and body. Also see what a missing page looks like.

Step 1	`http.server.HTTPServer` listens on a port. Port `0` means "pick any free port".
Step 2	A handler class defines `do_GET` . We send the status, headers, then the body.
Step 3	We run the server in a background thread and call it with `urllib.request` .

```
import threading, json, urllib.request, urllib.error
from http.server import BaseHTTPRequestHandler, HTTPServer

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/hello":
            body = b"hello, world"
            self.send_response(200)
            self.send_header("Content-Type", "text/plain")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)
        else:
            self.send_response(404)
            self.end_headers()
    def log_message(self, *a): pass

srv = HTTPServer(("127.0.0.1", 0), H)
port = srv.server_address[1]
threading.Thread(target=srv.serve_forever, daemon=True).start()
base = f"http://127.0.0.1:{port}"
with urllib.request.urlopen(base + "/hello") as r:
    print(r.status, r.reason)
    print(r.headers["Content-Type"])
    print(r.read().decode())
try:
    urllib.request.urlopen(base + "/nope")
except urllib.error.HTTPError as e:
    print("error:", e.code, e.reason)
srv.shutdown()
```

```
200 OK
text/plain
hello, world
error: 404 Not Found
```

- **Answer** The server replied 200 with text for `/hello`, and `urllib` raised `HTTPError` with code 404 for the unknown path. Note: `urllib` raises an exception for 4xx and 5xx codes, while `requests` does not (you call `raise_for_status()`).
- **Check** `404` is in the 4xx family ("client asked for something wrong"), and the output matches the status table above.
- **Why it matters** In a pipeline, an unhandled 404 or 503 stops the whole job. You must decide in code what each status means.

Problem 12.1.2 You must design URLs for a "delivery results" service. Fix these bad designs: `POST /createRun`, `GET /getRunById?id=5`, `GET /deleteRun/5`.

Step 1	Replace verbs with a noun collection: `runs`.
Step 2	Let the method express the action.
Step 3	Never use `GET` to delete: crawlers and browsers prefetch `GET` links.
Answer	`POST /v1/runs` (create, return 201), `GET /v1/runs/5` (read), `DELETE /v1/runs/5` (delete, return 204).
Check	Every URL is now a noun and the three actions use three different methods. No verb is left in a URL.
Why it matters	A new teammate can guess your API without reading docs. That is the whole point of REST.

Real-world mapping

HTTP piece	Example value	Where it appears in the job
Method	`POST`	The verb on the request form; `POST /predict` for a model
URL + query	`/v1/items?limit=3`	Ingestion code pages through data
Header	`Authorization: Bearer`	Login token in every pipeline call

	`...`	
Status	`429`	Your retry logic must slow down
Body	`{"name": "widget"}`	Data you send to create records

Predict, Run, Explain. In Problem 12.1.1, before you run it, ask: what does `urlopen` do for the unknown path `/nope` ? Many beginners expect it to return a page with status 404. The real result is an exception, shown in the output above (`error: 404 Not Found`). The typical wrong version is:

```
r = urllib.request.urlopen(base + "/nope")   # no try/except
print(r.status)
```

It never prints a status. It stops with `urllib.error.HTTPError: HTTP Error 404: Not Found` . The fix is the `try/except urllib.error.HTTPError` you saw above. `urllib` treats 4xx and 5xx as errors; line by line: `urlopen` sends the request, sees code 404, and raises instead of returning.

The picture

Read this diagram top to bottom: the client writes the request lines, the server finds the item, and writes back a status line, headers, and the body.

```
 Client                                  Server
   |  GET /v1/items/2 HTTP/1.1             |
   |  Host: example.com                    |
   |  Authorization: Bearer <token>   --->  |
   |                                        |  find item 2
   |  <---  HTTP/1.1 200 OK                 |
   |        Content-Type: application/json  |
   |        {"id": 2, "name": "item2"}      |
```

Connection to the real job

- Data engineers pull data from REST APIs (CRM, payments, weather) in ingestion jobs.
- ML engineers serve a model as `POST /predict` .
- QA and support teams test these endpoints. Knowing 429, 503 and timeouts helps you read results.

Applications

- **Data Engineering:** nightly pull of orders from a partner API, page by page.
- **Machine Learning / MLOps:** a `POST /predict` endpoint behind a load balancer.

- **Data Analytics:** refreshing a dashboard from a SaaS tool's API.
- **Testing and QA:** every automated API test script is an HTTP client.

COMMON MISTAKE

Returning status 200 with `{"error": "..."}` in the body. Clients, monitors and automated checks look at the status code first. Use the right 4xx or 5xx code.

Check yourself

1. Which of GET, POST, PUT, DELETE are idempotent? Answer: GET, PUT and DELETE. POST normally is not.
2. What is the difference between 401 and 403? Answer: 401 means "I do not know who you are". 403 means "I know you, but you may not do this".
3. Why is `/getItems` poor REST design? Answer: The verb is already in the HTTP method. The URL should name the thing: `/items` .

12.2 Calling APIs: urllib, requests and httpx

INTUITION

A client library is a good envelope-maker. You say what you want to send. The library handles connections, encoding, headers, and reading the answer. `urllib` is the plain envelope that comes with Python. `requests` and `httpx` are nicer ones with better handles.

How it works

The standard library offers `urllib.request` . It works, but it is low level. Most people install `requests` (simple, synchronous) or `httpx` (same style, plus `async` support and HTTP/2). Their calls look alike.

Rules for good API clients, whichever library you use:

1. **Always set a timeout.** Without one, a hung server hangs your program forever.
2. **Check the status.** Call `raise_for_status()` .
3. **Retry only safe or idempotent calls**, with growing waits (exponential backoff).
4. **Reuse a connection** with a `Session` (requests) or `Client` (httpx). It is much faster for many calls.
5. **Never hardcode keys** (see Chapter 13).

Not run here (needs install and a network):

```
import requests

with requests.Session() as s:
    s.headers["Authorization"] = "Bearer <TOKEN>"
    r = s.get("https://api.example.com/v1/items", params={"limit": 3},
timeout=5)
    r.raise_for_status()
    items = r.json()["data"]
```

```
import httpx

with httpx.Client(timeout=5.0) as client:
    r = client.post("https://api.example.com/v1/items", json={"name":
"widget"})
    r.raise_for_status()
    print(r.status_code, r.json())
```

Worked examples

Problem 12.2.1 Build a small JSON API with the standard library (list with pagination, get one, create, delete) and call it with `urllib`. Show status codes for success and failure.

Step 1	The server keeps items in a dict. It routes by path.
Step 2	It returns JSON with the right codes: 200, 201 (with `Location`), 204, 400, 404, 422.
Step 3	A helper `call()` sends a request and returns `(status, json, location)`, catching `HTTPError`.

```
import threading, json, urllib.request, urllib.error
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import urlparse, parse_qs

ITEMS = {i: {"id": i, "name": f"item{i}"} for i in range(1, 8)}
NEXT = [8]

class API(BaseHTTPRequestHandler):
    def _send(self, code, obj=None, headers=None):
        body = json.dumps(obj).encode() if obj is not None else b""
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        for k, v in (headers or {}).items():
```

```
            self.send_header(k, v)
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self):
        u = urlparse(self.path)
        q = parse_qs(u.query)
        if u.path == "/v1/items":
            limit = int(q.get("limit", ["3"])[0])
            offset = int(q.get("offset", ["0"])[0])
            data = list(ITEMS.values())[offset:offset + limit]
            nxt = offset + limit if offset + limit < len(ITEMS) else None
            return self._send(200, {"data": data, "next_offset": nxt, "total":
len(ITEMS)})
        if u.path.startswith("/v1/items/"):
            try:
                i = int(u.path.rsplit("/", 1)[1])
            except ValueError:
                return self._send(400, {"error": "id must be an integer"})
            if i in ITEMS:
                return self._send(200, ITEMS[i])
            return self._send(404, {"error": "not found"})
        self._send(404, {"error": "no such route"})

    def do_POST(self):
        if self.path != "/v1/items":
            return self._send(404, {"error": "no such route"})
        n = int(self.headers.get("Content-Length", 0))
        try:
            payload = json.loads(self.rfile.read(n))
            name = payload["name"]
        except (ValueError, KeyError):
            return self._send(422, {"error": "need JSON with 'name'"})
        i = NEXT[0]; NEXT[0] += 1
        ITEMS[i] = {"id": i, "name": name}
        self._send(201, ITEMS[i], {"Location": f"/v1/items/{i}"})

    def do_DELETE(self):
        i = int(self.path.rsplit("/", 1)[1])
        ITEMS.pop(i, None)       # idempotent: same result every time
        self._send(204)

    def log_message(self, *a): pass

srv = HTTPServer(("127.0.0.1", 0), API)
threading.Thread(target=srv.serve_forever, daemon=True).start()
base = f"http://127.0.0.1:{srv.server_address[1]}"

def call(method, path, obj=None):
    data = json.dumps(obj).encode() if obj is not None else None
```

```
    req = urllib.request.Request(base + path, data=data, method=method,
                                 headers={"Content-Type": "application/json"})
    try:
        with urllib.request.urlopen(req) as r:
            raw = r.read()
            return r.status, (json.loads(raw) if raw else None),
r.headers.get("Location")
    except urllib.error.HTTPError as e:
        return e.code, json.loads(e.read()), None

print(call("GET", "/v1/items?limit=3"))
print(call("GET", "/v1/items?limit=3&offset=6"))
print(call("GET", "/v1/items/2"))
print(call("GET", "/v1/items/99"))
print(call("GET", "/v1/items/abc"))
print(call("POST", "/v1/items", {"name": "widget"}))
print(call("POST", "/v1/items", {"nom": "x"}))
print(call("DELETE", "/v1/items/1"))
print(call("DELETE", "/v1/items/1"))
srv.shutdown()
```

```
(200, {'data': [{'id': 1, 'name': 'item1'}, {'id': 2, 'name': 'item2'}, {'id':
3, 'name': 'item3'}], 'next_offset': 3, 'total': 7}, None)
(200, {'data': [{'id': 7, 'name': 'item7'}], 'next_offset': None, 'total': 7},
None)
(200, {'id': 2, 'name': 'item2'}, None)
(404, {'error': 'not found'}, None)
(400, {'error': 'id must be an integer'}, None)
(201, {'id': 8, 'name': 'widget'}, '/v1/items/8')
(422, {'error': "need JSON with 'name'"}, None)
(204, None, None)
(204, None, None)
```

- **Answer** Each call returned the status we designed. Deleting item 1 twice returned 204 both times: DELETE is idempotent.
- **Check** `ITEMS` started with 7 items. After the POST there were 8 and after the delete 7 again, which is why the page walk in 12.4.1 finds 7 names.
- **Why it matters** Every status above is something your client code must handle: retry, skip, or fail loudly.

Predict, Run, Explain. Before you run it, predict what `call("GET", "/v1/items/abc")` returns. The real answer was `(400, {'error': 'id must be an integer'}, None)`. Line by line: the server runs `int("abc")`, which raises `ValueError`; the `except` turns it into a 400. The typical wrong version has no `try/except` in the handler. Then the server thread crashes inside `int("abc")` with `ValueError: invalid literal for int() with base 10: 'abc'`, and the

client gets a dropped connection instead of a clear 400. The fix is the `try/except` you see in `do_GET`.

Two more typical wrong versions you can reproduce (real output from a test run): a server that sleeps for one second, and a client that sets `timeout=0.2`, fails with `TimeoutError timed out`, and shows why a timeout is better than waiting forever. And `json.loads("")` on a 204 reply (empty body) fails with `JSONDecodeError: Expecting value: line 1 column 1 (char 0)`. That is why `call()` checks `if raw` before parsing.

Problem 12.2.2 Write a retry helper with exponential backoff and show it on a function that fails twice and then works.

Step 1	Wait `base * 2**attempt` seconds between tries (here tiny numbers so the demo is fast).
Step 2	Retry only on errors that are temporary.
Step 3	After the last try, raise the error.

```
import time

def retry(fn, tries=4, base=0.01, retry_on=(ConnectionError, TimeoutError)):
    for attempt in range(tries):
        try:
            return fn()
        except retry_on as e:
            if attempt == tries - 1:
                raise
            wait = base * 2 ** attempt
            print(f"attempt {attempt+1} failed ({e}); waiting {wait:.2f}s")
            time.sleep(wait)

calls = {"n": 0}
def flaky():
    calls["n"] += 1
    if calls["n"] < 3:
        raise ConnectionError("server reset")
    return "ok after %d calls" % calls["n"]

print(retry(flaky))
```

Before you run it, predict: how many times is `flaky()` called, and how long is each wait? Real output:

```
attempt 1 failed (server reset); waiting 0.01s
attempt 2 failed (server reset); waiting 0.02s
ok after 3 calls
```

- **Answer** The helper waited 0.01s, then 0.02s (doubling) and succeeded on the third call.
- **Check** Wait = base x 2^attempt: 0.01 x 2^0 = 0.01 and 0.01 x 2^1 = 0.02. `flaky` fails while n < 3, so success on call 3 is correct.
- **Why it matters** Doubling the wait gives a struggling server time to recover. Retrying instantly makes an outage worse (every client piles on).

Real-world mapping

Client habit	Example value	Where it appears in the job
Timeout	`timeout=5`	Stops a nightly job from hanging forever
Retry with backoff	wait 0.01, 0.02, 0.04 s	Surviving a 503 from an overloaded partner
Session / Client	one connection for 1,000 calls	Faster ingestion, lower load on the server
`raise_for_status()`	raises on 4xx/5xx	The job fails loudly instead of loading bad data

The picture

Read it left to right: your code sends through a session (with a timeout), the server answers, and the status code is checked before the data is used.

```
your code --> Session (keeps connection open) --> server
     ^             |  timeout=5                       |
     |             v                                  |
     +--- raise_for_status() <---- status code <------+
```

Connection to the real job

- Ingestion scripts call APIs page by page and must survive 429 and 503.
- Calling a hosted LLM or model API is the same pattern: JSON in, JSON out, API key in a header.
- A Python client with `httpx` is also the base of traffic-generating tools such as Locust (named only as an example).

Applications

- **Data Engineering:** ingestion with retries and pagination.
- **GenAI:** calling a hosted language-model API with a key and a timeout.
- **MLOps:** health checks and smoke tests after a deploy.
- **Testing and QA:** API test automation with Python HTTP clients.

COMMON MISTAKE

Forgetting `timeout=`. The default in `requests` is to wait forever. A single stuck call can freeze a nightly pipeline. Always pass a timeout.

Check yourself

1. Why use a `Session`? Answer: It reuses the connection and shared headers, so many calls are faster and shorter to write.
2. Which calls are safe to retry blindly? Answer: Idempotent ones: GET, PUT, DELETE. For POST use an idempotency key.
3. What does `raise_for_status()` do? Answer: It raises an exception if the status is 4xx or 5xx.

12.3 Building APIs: FastAPI and Flask Basics

INTUITION

Writing a server by hand with `http.server` (as we did) shows how it works, but you would never ship it. A framework is a restaurant kitchen set up for you. You write the recipes (functions) and the framework handles orders, routing, validation, and docs.

How it works

FastAPI is a modern framework. You write a function, add a **decorator** (Chapter 7) that says "this is `GET /items/{item_id}`", and use **type hints** (Chapter 9) for validation. It uses `pydantic` to check bodies, and it creates an **OpenAPI** page automatically at `/docs`. It runs on an ASGI server such as `uvicorn`.

Flask is older and smaller. It gives routes and a request object. You add validation and other pieces yourself. It is simple and still common.

Not run here (needs `pip install fastapi uvicorn`):

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Items API")
DB = {1: {"id": 1, "name": "pen"}}

class ItemIn(BaseModel):
    name: str

@app.get("/v1/items/{item_id}")
def get_item(item_id: int):
    if item_id not in DB:
        raise HTTPException(status_code=404, detail="not found")
    return DB[item_id]

@app.post("/v1/items", status_code=201)
def create_item(item: ItemIn):
    new_id = max(DB) + 1
    DB[new_id] = {"id": new_id, **item.model_dump()}
    return DB[new_id]
```

Start it with `uvicorn main:app --reload`, then open `http://127.0.0.1:8000/docs`. If you send `item_id=abc`, FastAPI answers 422 on its own because of the `int` hint.

The same in Flask (not run here):

```
from flask import Flask, jsonify, request, abort

app = Flask(__name__)
DB = {1: {"id": 1, "name": "pen"}}

@app.get("/v1/items/<int:item_id>")
def get_item(item_id):
    if item_id not in DB:
        abort(404)
    return jsonify(DB[item_id])

@app.post("/v1/items")
def create_item():
    data = request.get_json()
    new_id = max(DB) + 1
    DB[new_id] = {"id": new_id, "name": data["name"]}
    return jsonify(DB[new_id]), 201
```

Worked examples

Problem 12.3.1 You have the FastAPI app above. What does `POST /v1/items` with body `{"name": 5}` return, and why?

Step 1	`ItemIn.name` is declared `str`.
Step 2	Pydantic validates the body before your function runs. In its default (lax) mode, an integer is not turned into a string.
Step 3	Validation fails, so FastAPI replies with a 422 error and a JSON description of the problem. (Not run here; this is the documented behaviour.)
Answer	A 422 response. Your function is never called. This is why type hints are worth the effort.
Check	The docs list 422 as FastAPI's code for validation errors, and the same `str` declaration appears in `ItemIn`.
Why it matters	A bad client cannot crash your handler or poison your database.

Problem 12.3.2 Using only the standard library, write the validation step that FastAPI does for you: accept a dict and return clean data or a list of errors.

Step 1	Define the required fields and types.
Step 2	Loop and collect messages.
Step 3	Return `(data, errors)`.

```
def validate_item(payload):
    errors = []
    if not isinstance(payload, dict):
        return None, ["body must be a JSON object"]
    name = payload.get("name")
    if not isinstance(name, str) or not name.strip():
        errors.append("name: must be a non-empty string")
    if "price" in payload and not isinstance(payload["price"], (int, float)):
        errors.append("price: must be a number")
    return ({"name": name.strip(), **({"price": payload["price"]} if "price" in
payload else {})}
            if not errors else None), errors

print(validate_item({"name": " pen ", "price": 10}))
print(validate_item({"name": 5, "price": "ten"}))
print(validate_item([1, 2]))
```

```
({'name': 'pen', 'price': 10}, [])
(None, ['name: must be a non-empty string', 'price: must be a number'])
(None, ['body must be a JSON object'])
```

- **Answer** Validation returns clean data or all the errors at once, which is also how good APIs behave: report every problem, not just the first.
- **Check** Input 2 has two wrong fields and the output lists exactly two messages. Input 1 has none, and the name is trimmed from `' pen '` to `'pen'`.
- **Why it matters** Pydantic does this work for you with less code, but now you know what it does.

Real-world mapping

Framework piece	Example	Where it appears in the job
Route decorator	`@app.get("/v1/items/{item_id}")`	One function per endpoint of a model service
Type hint	`item_id: int`	Auto 422 for bad input, less defensive code
pydantic model	`class ItemIn(BaseModel)`	The "schema" for a request body
`/docs` page	auto OpenAPI UI	Analysts try your endpoint without code

Predict, Run, Explain. Before you read on: in the Flask example, what happens if the client posts a body with no `name` key? The code does `data["name"]`, which raises `KeyError: 'name'`, and Flask answers with a 500 error (a server bug, not the client's fault). This is the typical wrong version. The fix is validation like the function in Problem 12.3.2 and returning 422. FastAPI avoids it because the pydantic model rejects the body before your code runs (not run here).

The picture

Read it top to bottom: the request is routed, then validated, then your function runs, which may call a database.

```
request --> [ routing: which function? ] --> [ validation: pydantic ]
                                                        |
                     response <--- [ your function: business logic ]
                                                        |
                                      [ database / other services ]
```

Connection to the real job

- Model serving: `POST /predict` loads a model once at start-up and calls it per request.
- Data teams expose "data products" as small internal APIs.
- A FastAPI service with a database and tests is one of the skill-map mastery tests (see Chapter 14).

Applications

- **Machine Learning / MLOps:** a `POST /predict` endpoint that returns a score.
- **Data Engineering:** a small service that receives webhook events and writes to a queue.
- **GenAI:** a thin API that adds your prompt and guardrails in front of a language model.
- **Testing and QA:** a mock service so you can test without touching production.

COMMON MISTAKE

Loading a big ML model inside the request function. It reloads on every call and your latency explodes. Load it once at start-up, outside the handler.

Check yourself

1. What gives FastAPI its automatic validation? Answer: Type hints plus pydantic models.
2. What does FastAPI's `/docs` page show? Answer: Interactive documentation made from the OpenAPI description of your routes.
3. Where should a heavy model be loaded? Answer: Once, at application start-up, not per request.

12.4 Pagination, Idempotency, Versioning, OpenAPI and GraphQL

INTUITION

A server cannot return a million rows in one reply, so it hands them over page by page (pagination). Networks fail, so clients retry, and the server must not charge a card twice (idempotency). Services live for years, so old clients must keep working while new ones arrive (versioning). A contract page (OpenAPI) lets everyone agree on the shape.

How it works

Pagination styles:

Style	Example	Good	Weak
Offset/limit	`?limit=3&offset=6`	Simple, jump to any page	Slow for big offsets; rows shift when data changes
Cursor	`?after=abc123`	Stable and fast	Cannot jump to page 50
Page number	`?page=3&size=20`	Familiar	Same weakness as offset

Idempotency key. For `POST` calls that must not repeat (payments, orders), the client sends a unique header such as `Idempotency-Key: k1`. The server remembers keys. A repeat with the same key returns the first result and does not act again.

Versioning. Put the version in the URL (`/v1/items`) or in a header. Never break `v1`. Add `v2` when you must change the shape.

OpenAPI is a standard JSON/YAML description of an API: routes, parameters, bodies and responses. Tools generate documentation, client code and tests from it. FastAPI makes it for you.

GraphQL (awareness). Instead of many URLs, there is one endpoint and the client writes a query that names exactly the fields it wants. It avoids over-fetching, but it is harder to cache and to rate-limit. REST is still the default choice.

Worked examples

Problem 12.4.1 Walk every page of a 7-item collection with `limit=3`, using the `next_offset` field of our API. How many requests are needed?

Step 1	Start at offset 0. Ask for 3 items. Server says `next_offset=3`.
Step 2	Offset 3 gives 3 more, `next_offset=6`.
Step 3	Offset 6 gives 1 item (item 7) and `next_offset=None`, so stop.

The loop (it used the server of Problem 12.2.1, after item 1 was deleted and "widget" was added, so there were 7 items):

```
off, names = 0, []
while off is not None:
    s, body, _ = call("GET", f"/v1/items?limit=3&offset={off}")
    names += [d["name"] for d in body["data"]]
    off = body["next_offset"]
print(names)
```

```
['item2', 'item3', 'item4', 'item5', 'item6', 'item7', 'widget']
```

- **Answer** 3 requests: ceil(7 / 3) = 3. All 7 names were collected.
- **Check** Pages hold 3 + 3 + 1 = 7 items, matching `total: 7`.
- **Why it matters** If you stop after page 1 you lose data silently. The loop must end on `next_offset is None`.

Problem 12.4.2 Implement an idempotency key on a payment endpoint and show that a repeated key does not create a second payment.

Step 1	The server keeps a dict `seen` of key to result.
Step 2	First time: create, return 201. Repeat: return stored result with 200.
Step 3	The client sends `k1`, `k1`, `k2`.

```
import urllib.request, json, threading
from http.server import BaseHTTPRequestHandler, HTTPServer

seen = {}
class H(BaseHTTPRequestHandler):
    def do_POST(self):
        key = self.headers.get("Idempotency-Key")
        n = int(self.headers["Content-Length"]); self.rfile.read(n)
        if key in seen:
            code, body = 200, seen[key]
        else:
            seen[key] = {"payment_id": len(seen) + 1}
            code, body = 201, seen[key]
        out = json.dumps(body).encode()
        self.send_response(code); self.send_header("Content-Length",
str(len(out)))
        self.end_headers(); self.wfile.write(out)
    def log_message(self, *a): pass

srv = HTTPServer(("127.0.0.1", 0), H)
threading.Thread(target=srv.serve_forever, daemon=True).start()
url = f"http://127.0.0.1:{srv.server_address[1]}/pay"
for key in ["k1", "k1", "k2"]:
    req = urllib.request.Request(url, data=b"{}", method="POST", headers=
{"Idempotency-Key": key})
    with urllib.request.urlopen(req) as r:
        print(key, r.status, r.read().decode())
srv.shutdown()
```

```
k1 201 {"payment_id": 1}
k1 200 {"payment_id": 1}
k2 201 {"payment_id": 2}
```

- **Answer** The second `k1` call returned the same payment (id 1) and did not create id 2. The retry was safe.
- **Check** Three calls, two distinct keys, two payments (ids 1 and 2).
- **Why it matters** In plain words: the customer is charged once even if the phone retries five times.

Real-world mapping

Idea	Example value	Where it appears in the job
Offset paging	`limit=3&offset=6`	Simple partner APIs; 3 requests for 7 rows
Cursor paging	`after=abc123`	Large event feeds, social data

Idempotency key	`Idempotency-Key: k1`	Payments; safe to retry
Version in URL	`/v1/items`	Old clients keep working
`Retry-After` header	`Retry-After: 30`	Tells your ingest job how long to pause on 429

Predict, Run, Explain. Before running Problem 12.4.2, predict the status of the second `k1` call. A typical wrong prediction is 201 again. The real output shows `200` with the same `payment_id`. Line by line: `key in seen` is true, so we return the stored result and do not create anything. The typical wrong version has no `seen` dict and increments a counter every time, so a retry would make payment 2 and the customer pays twice (no error message, which is exactly why it is dangerous: nothing fails loudly).

The picture

Offset paging (left) numbers pages by position, so inserting a row at the front moves everything. Cursor paging (right) remembers the last id seen.

```
Offset paging: rows shift          Cursor paging: bookmark
page1 [1 2 3] | insert row 0       after=3 -> [4 5 6]
page2 [3 4 5]  (3 repeated!)       after=6 -> [7]
```

Connection to the real job

- Ingestion code must loop over pages, and must handle the last page and rate limits (429, `Retry-After` header).
- Idempotent writes make pipelines safe to re-run. This is a core data-engineering idea (Chapter 13 of the data engineering documents expands it).
- OpenAPI files let you generate test data and client test scripts.

Applications

- **Data Engineering:** page loops and idempotent loads that can be re-run safely.
- **Data Analytics:** export APIs that return reports in pages.
- **Machine Learning / MLOps:** versioned model endpoints (`/v1/predict`, `/v2/predict`) during a rollout.
- **Testing and QA:** OpenAPI files used to generate test scripts.

COMMON MISTAKE

Using offset paging on a table that changes while you read it. You can skip or repeat rows. Prefer cursors for large or live data, and always stop when the next-page marker is empty rather than when a page "looks short".

Check yourself

1. Why cursor over offset for big data? Answer: It is faster at depth and does not shift when rows are inserted.
2. What is an idempotency key for? Answer: To make a repeated POST return the first result instead of acting twice.
3. Where can the API version live? Answer: In the URL (`/v1/`) or a header.

12.5 Authentication and Authorization Basics

INTUITION

Authentication answers "who are you?". Authorization answers "what are you allowed to do?". A hotel desk checks your ID (authentication), then gives you a key card that opens only your room (authorization).

How it works

- **API key**: a long random string sent in a header (`X-API-Key: ...`). Simple. It identifies the calling application. Treat it like a password.
- **Bearer token**: `Authorization: Bearer <token>` . The token proves you logged in.
- **OAuth 2.0**: a standard for letting an app act for a user without sharing the password. The user logs in at the provider, and the app receives a short-lived **access token** (and often a **refresh token**). **OIDC** adds identity on top of OAuth 2.0.
- **JWT (JSON Web Token)**: a token made of three base64 parts, `header.payload.signature` . The payload holds **claims** (who, role, expiry `exp`). The signature (for example HMAC-SHA256 with a secret key) lets the server detect any change. A JWT is **signed, not encrypted**: anyone can read the payload. Never put secrets in it.
- **Always use HTTPS** so tokens are not readable in transit.

Worked examples

Problem 12.5.1 Build a JWT-like token with HMAC-SHA256 by hand, verify it, then show that expiry and tampering are detected. The key is a fake demo value.

Step 1	Encode header and payload as compact JSON, then base64url.
Step 2	Sign `header.payload` with HMAC.
Step 3	To verify, recompute the signature and compare with `hmac.compare_digest` (safe against timing attacks). Then check `exp`.

```
import hmac, hashlib, base64, json

SECRET = b"demo-secret-not-real"

def b64(b): return base64.urlsafe_b64encode(b).rstrip(b"=").decode()
def unb64(s): return base64.urlsafe_b64decode(s + "=" * (-len(s) % 4))

def make_token(claims):
    header = b64(json.dumps({"alg": "HS256", "typ": "JWT"}, separators=(",",
":")).encode())
    payload = b64(json.dumps(claims, separators=(",", ":")).encode())
    sig = b64(hmac.new(SECRET, f"{header}.{payload}".encode(),
hashlib.sha256).digest())
    return f"{header}.{payload}.{sig}"

def verify(token, now):
    h, p, s = token.split(".")
    good = b64(hmac.new(SECRET, f"{h}.{p}".encode(), hashlib.sha256).digest())
    if not hmac.compare_digest(s, good):
        return "bad signature"
    claims = json.loads(unb64(p))
    if claims["exp"] < now:
        return "expired"
    return claims

t = make_token({"sub": "asha", "role": "analyst", "exp": 1000})
print(t)
print(verify(t, now=500))
print(verify(t, now=1500))
h, p, s = t.split(".")
evil = b64(json.dumps({"sub": "asha", "role": "admin", "exp": 1000},
separators=(",", ":")).encode())
print(verify(f"{h}.{evil}.{s}", now=500))
print(json.loads(unb64(p)))
```

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhc2hhIiwicm9sZSI6ImFuYWx5c3QiLC
JleHAiOjEwMDB9.MvOjd9So6hnZyxNsG6Q5vsm-EzJ80cyE8aR_RK5JuRI
{'sub': 'asha', 'role': 'analyst', 'exp': 1000}
expired
bad signature
{'sub': 'asha', 'role': 'analyst', 'exp': 1000}
```

- **Check** `exp` is 1000: 500 < 1000 passes, 1500 > 1000 fails, which matches the output.
- **Why it matters** Expiry limits damage if a token leaks; the signature stops forged roles.
- **Answer** At time 500 the token is valid; at 1500 it is expired. When we swapped the payload to `role: admin` and kept the old signature, verification said "bad signature". The last line shows that anyone can read the payload: signed is not secret. In real projects use a maintained library (such as PyJWT) and never write your own crypto; this demo only teaches the idea.

Problem 12.5.2 A route `DELETE /v1/runs/5` is called by user "ravi" (role "viewer") with a valid token. Which status code, and why?

Step 1	The token is valid, so authentication passed.
Step 2	Role "viewer" cannot delete, so authorization fails.
Answer	403 Forbidden. A 401 would be wrong because the user is known.
Check	Ask: "Does the server know who this is?" Yes, so not 401. "Is the action allowed?" No, so 403.
Why it matters	When many requests fail at once, a wave of 401s means your login or token reuse is broken; a wave of 403s means a permissions problem. Different bugs, different fixes.

Real-world mapping

Credential	Example value	Where it appears in the job
API key	`X-API-Key: <KEY>`	Calling a model or SaaS API from a script
Bearer token	`Authorization: Bearer <TOKEN>`	After login (token extracted, then reused on every call)
JWT claims	`{"sub": "asha", "role": "analyst", "exp": 1000}`	Data API deciding what you may read
Refresh	long-lived, stored safely	Notebook that must run for days

token

Predict, Run, Explain. Before running Problem 12.5.1, predict what `verify` says for the tampered token. The real output is `bad signature`. Line by line: the attacker changed the payload but cannot recompute the HMAC without the secret, so the stored signature no longer matches the new `header.payload`. A typical wrong version compares with `==` and, worse, a very common bug is to decode the payload and trust it without checking the signature. Then the tampered token (`role: admin`) would be accepted and no error appears at all.

The picture

A JWT is three base64 pieces joined by dots. The signature is a fingerprint of the first two pieces.

```
eyJhbGciOi...  .  eyJzdWIiOi...  .  MvOjd9So6h...
\___header___/    \__payload___/    \_signature_/
alg, type         claims: sub,     HMAC(secret, header.payload)
                  role, exp        changes if any byte changes
```

Connection to the real job

- Every cloud and data API (warehouse, model service, BI tool) needs a token or key.
- Service accounts and short-lived tokens are the safe standard.
- Role checks in data APIs decide who sees which data (privacy).

Applications

- **Data Engineering:** service accounts with short-lived tokens for warehouse and cloud access.
- **Machine Learning / MLOps:** API keys for model endpoints, rotated regularly.
- **GenAI:** keys for hosted model APIs, kept out of notebooks and Git.
- **Testing and QA:** token extraction and reuse in test scripts; webhook signature checks use the same HMAC idea.

COMMON MISTAKE

Thinking a JWT is encrypted. It is only signed. Do not store passwords or personal data inside it, and always verify the signature and the expiry on the server.

Check yourself

1. 401 or 403 for a known user without permission? Answer: 403.
2. Can a client read a JWT payload? Answer: Yes. It is base64, not encrypted.

3. Why `compare_digest` ? Answer: It compares in constant time so attackers cannot guess the signature by timing.

12.6 Scraping Responsibly

INTUITION

Scraping is reading a web page with a program instead of your eyes. It is like photocopying pages in a library: allowed in many cases, but there are rules posted at the door, and if you run ten copy machines at once you annoy everyone.

How it works

Order of preference: first use an official API, then a downloadable dataset, and only then scrape.

Be a good citizen:

1. Read **robots.txt** (the site's posted rules) and obey it.
2. Read the **terms of service**. Some sites forbid scraping.
3. Identify yourself with a clear `User-Agent` and a contact.
4. Go slowly: add a delay (respect `Crawl-delay`) and limit parallelism.
5. Do not collect personal data without a lawful reason. Do not bypass logins or paywalls.
6. Cache pages so you do not fetch the same page twice.

Tools: `html.parser` (standard library), **BeautifulSoup** (friendly parsing of HTML), **Scrapy** (a framework for large crawls), **Selenium** and **Playwright** (control a real browser, needed when a page builds its content with JavaScript). BeautifulSoup, Scrapy, Selenium and Playwright were not run here.

Not run here:

```
import requests
from bs4 import BeautifulSoup

r = requests.get("https://example.com/shop", headers={"User-Agent":
"StudyBot/1.0 (me@example.com)"}, timeout=10)
soup = BeautifulSoup(r.text, "html.parser")
for li in soup.select("li.p"):
    print(li.a.text, li.select_one(".price").text)
```

Worked examples

Problem 12.6.1 Parse a robots.txt text and decide which URLs your bot may fetch.

Step 1	Use `urllib.robotparser.RobotFileParser` and feed it lines with `parse()` (no network needed).
Step 2	Ask `can_fetch(user_agent, url)`.
Step 3	Read the crawl delay.

```
from urllib.robotparser import RobotFileParser

rules = """User-agent: *
Disallow: /private/
Crawl-delay: 5

User-agent: BadBot
Disallow: /
"""
rp = RobotFileParser()
rp.parse(rules.splitlines())
for ua, url in [("MyStudyBot", "https://example.com/products"),
                ("MyStudyBot", "https://example.com/private/data"),
                ("BadBot", "https://example.com/products")]:
    print(ua, url, rp.can_fetch(ua, url))
print("crawl delay:", rp.crawl_delay("MyStudyBot"))
```

```
MyStudyBot https://example.com/products True
MyStudyBot https://example.com/private/data False
BadBot https://example.com/products False
crawl delay: 5
```

- **Answer** Our bot may fetch `/products` but not `/private/data`, and must wait 5 seconds between requests. "BadBot" is banned everywhere.
- **Check** `/private/data` starts with `/private/`, so the rule matches. `/products` does not.
- **Why it matters** Obeying these rules keeps you legal, polite and unblocked.

Problem 12.6.2 Extract name, link and price from an HTML snippet with the standard library parser.

Step 1	Subclass `HTMLParser`. `handle_starttag` sees tags and attributes. `handle_data` sees text.
Step 2	When we see `<a href=...>`, remember the link. The next text is the name.
Step 3	When we see `<span class="price">`, the next text is the price.

```
from html.parser import HTMLParser

html = """<html><body><h1>Shop</h1>
<ul><li class="p"><a href="/a">Pen</a> <span class="price">10</span></li>
<li class="p"><a href="/b">Book</a> <span class="price">250</span></li></ul>
</body></html>"""

class P(HTMLParser):
    def __init__(self):
        super().__init__()
        self.links = []; self.prices = []; self.names = []
        self._in_price = False; self._href = None
    def handle_starttag(self, tag, attrs):
        a = dict(attrs)
        if tag == "a": self._href = a.get("href")
        if tag == "span" and a.get("class") == "price": self._in_price = True
    def handle_data(self, d):
        if self._href:
            self.names.append(d); self.links.append(self._href); self._href =
None
        elif self._in_price:
            self.prices.append(int(d)); self._in_price = False

p = P(); p.feed(html)
print(list(zip(p.names, p.links, p.prices)))
```

```
[('Pen', '/a', 10), ('Book', '/b', 250)]
```

- **Check** The input has two `<li>` items and the result has two tuples. Prices are `int`, not text, so you can sum them.
- **Answer** We got two products with names, links and prices as numbers. BeautifulSoup does the same job in three lines, but it is good to know what is under the hood.

Real-world mapping

Rule	Example value	Where it appears in the job
robots.txt	`Disallow: /private/`	Check before any crawl
Crawl delay	5 seconds	300 pages take at least 25 minutes
User-Agent	`StudyBot/1.0 (me@example.com)`	Lets site owners contact you
Cache	save HTML to disk	Re-parse without re-fetching

Predict, Run, Explain. Before running Problem 12.6.1, predict the answer for `BadBot`. It is `False` because the group for `BadBot` says `Disallow: /`, which blocks everything; the `*` group does not apply to a bot that has its own group. A typical wrong version forgets `parse()` and calls `can_fetch` on an empty parser. An empty parser (no rules loaded) behaves as if everything is allowed, so you get `True` with no error. The fix: always call `parse()` or `read()` first and check the rules were loaded.

The picture

Read it as a series of gates. You only reach the last box if every gate lets you through.

```
official API?  --yes--> use it
     | no
robots.txt allows?  --no--> stop
     | yes
terms of service ok? --no--> stop
     | yes
fetch slowly, identify yourself, cache, store only what you need
```

Connection to the real job

- Price monitoring, job-market analysis, and research datasets often start with scraping. The result above means: 2 products, total price 10 + 250 = 260.
- A legal or privacy mistake can end a project. Teams check terms before building.
- Pages built by JavaScript need a browser tool, which is slower and heavier.

Applications

- **Data Analytics:** collecting public prices to compare competitors (where terms allow).
- **Data Science:** building a small text dataset for an analysis.
- **Machine Learning / GenAI:** collecting documents for search or retrieval, with permission and attention to copyright and privacy.
- **Data Engineering:** a polite, scheduled crawler with caching and monitoring.

COMMON MISTAKE

Hammering a site with parallel requests and no delay. It looks like an attack and may get your IP banned. Add delays, a User-Agent, and caching. Scrape only what you are allowed to.

Check yourself

1. What file lists the site's crawling rules? Answer: `robots.txt`.
2. When do you need Selenium or Playwright? Answer: When the content is created by JavaScript in

the browser.

3. What comes before scraping in your order of choice? Answer: An official API or a downloadable dataset.

12.7 Command-Line Tools: argparse, typer and subprocess

INTUITION

A command-line tool is a function with a keyboard-friendly front door. Instead of editing the script each time, you pass options: `summ 120 80 300 -p 90`. Scripts become reusable tools for you and your team, and schedulers (cron, Airflow) can run them.

How it works

`argparse` (standard library) builds the parser. You declare arguments, their types, defaults and help text. It gives you `--help` and clear errors for free, and exits with code 2 on a usage error.

typer (third-party) makes the same thing from a function with type hints: the parameters become the options. Not run here:

```
import typer

app = typer.Typer()

@app.command()
def summ(numbers: list[float], percentile: int = 90, verbose: bool = False):
    """Summarise delivery times."""
    ...

if __name__ == "__main__":
    app()
```

Good CLI habits: put the work in a function `main(argv=None)` so tests can call it, return proper exit codes (0 good, non-zero bad), print results to stdout and logs to stderr (through `logging`).

subprocess runs other programs from Python. Use `subprocess.run([...], capture_output=True, text=True, check=True)`. Pass a **list**, not a string with `shell=True`, so that user input cannot inject shell commands.

Worked examples

Problem 12.7.1 Write a CLI called `summ` that prints count, mean and a percentile of delivery times

(nearest-rank). Show normal use, `--help`, and an error.

Step 1	Declare `numbers` (one or more floats), `-p` percentile (default 90, between 1 and 99), `-v` verbose.
Step 2	Sort. Nearest-rank index is `ceil(n * p / 100) - 1`.
Step 3	Run it.

```
import argparse, logging

def build():
    p = argparse.ArgumentParser(prog="summ", description="Summarise delivery
times")
    p.add_argument("numbers", type=float, nargs="+", help="delivery times in
minutes")
    p.add_argument("-p", "--percentile", type=int, default=90, choices=range(1,
100), metavar="P")
    p.add_argument("-v", "--verbose", action="store_true")
    return p

def main(argv=None):
    a = build().parse_args(argv)
    logging.basicConfig(level=logging.DEBUG if a.verbose else logging.WARNING,
                        format="%(levelname)s %(message)s")
    xs = sorted(a.numbers)
    logging.debug("got %d numbers", len(xs))
    k = max(0, -(-len(xs) * a.percentile // 100) - 1)
    print(f"count={len(xs)} mean={sum(xs)/len(xs):.1f} p{a.percentile}=
{xs[k]}")

if __name__ == "__main__":
    main()
```

Saved as `s5.py` and run in the shell:

```
python3 s5.py 120 80 300 95 110 -p 90 -v
python3 s5.py --help
python3 s5.py abc; echo "exit=$?"
```

```
DEBUG got 5 numbers
count=5 mean=141.0 p90=300.0
usage: summ [-h] [-p P] [-v] numbers [numbers ...]

Summarise delivery times

positional arguments:
  numbers             delivery times in minutes

options:
  -h, --help          show this help message and exit
  -p, --percentile P
  -v, --verbose
usage: summ [-h] [-p P] [-v] numbers [numbers ...]
summ: error: argument numbers: invalid float value: 'abc'
exit=2
```

- **Check by hand** Sorted: 80, 95, 110, 120, 300. Mean = 705 / 5 = 141.0. For p90: ceil(5 x 0.9) = 5, index 4, value 300.
- **Check** Mean by hand: 705 / 5 = 141.0. Percentile index: ceil(5 x 90 / 100) - 1 = 4, value 300.0.
- **Why it matters** This is the same nearest-rank percentile idea used in exam-mark and delivery reports; a p90 of 300 minutes means about 9 of 10 deliveries were at least as fast (here 4 of 5 samples are below it).
- **Answer** The tool prints `count=5 mean=141.0 p90=300.0` , gives help text, and rejects bad input with exit code 2. (The `--help` layout may differ slightly between Python versions.)

Problem 12.7.2 Run a child Python process, capture its output and exit code, and see what `check=True` does. Then use `sqlite3` (standard library database) from the same script to store and query results with `?` placeholders.

Step 1	`subprocess.run` with a list and `capture_output=True`.
Step 2	`check=True` raises `CalledProcessError` on a non-zero code.
Step 3	With a list, `"a; echo b"` is one harmless argument, not two commands.
Step 4	`sqlite3` needs no server: `connect(":memory:")` makes a temporary database.

```
import subprocess, sys, sqlite3

r = subprocess.run([sys.executable, "-c", "print('hi from child'); import sys;
sys.exit(3)"],
                   capture_output=True, text=True)
print(r.returncode, repr(r.stdout), repr(r.stderr))
try:
    subprocess.run([sys.executable, "-c", "raise SystemExit(2)"], check=True)
except subprocess.CalledProcessError as e:
    print("failed with", e.returncode)
r = subprocess.run(["echo", "a; echo b"], capture_output=True, text=True)
print(r.stdout.strip())

con = sqlite3.connect(":memory:")
con.row_factory = sqlite3.Row
con.execute("CREATE TABLE req(id INTEGER PRIMARY KEY, url TEXT, ms REAL)")
con.executemany("INSERT INTO req(url, ms) VALUES (?, ?)",
                [("/a", 120.0), ("/a", 80.0), ("/b", 300.0), ("/b", 100.0),
("/a", 100.0)])
con.commit()
for row in con.execute("SELECT url, COUNT(*) n, AVG(ms) avg FROM req GROUP BY
url ORDER BY url"):
    print(dict(row))
with con:
    con.execute("UPDATE req SET ms = ms * 2 WHERE url = ?", ("/b",))
print(con.execute("SELECT SUM(ms) FROM req").fetchone()[0])
```

```
3 'hi from child\n' ''
failed with 2
a; echo b
{'url': '/a', 'n': 3, 'avg': 100.0}
{'url': '/b', 'n': 2, 'avg': 200.0}
1100.0
```

- **Answer** The child returned code 3 with its output captured. `check=True` turned code 2 into an exception. The `with con:` block committed the update. The sums check out: `/a` is 120+80+100 = 300, `/b` is (300+100) x 2 = 800, total 1100.

Real-world mapping

CLI piece	Example value	Where it appears in the job
Positional argument	`120 80 300`	The input files or numbers
Option with default	`-p 90`	`--date` , `--env prod` for ETL jobs

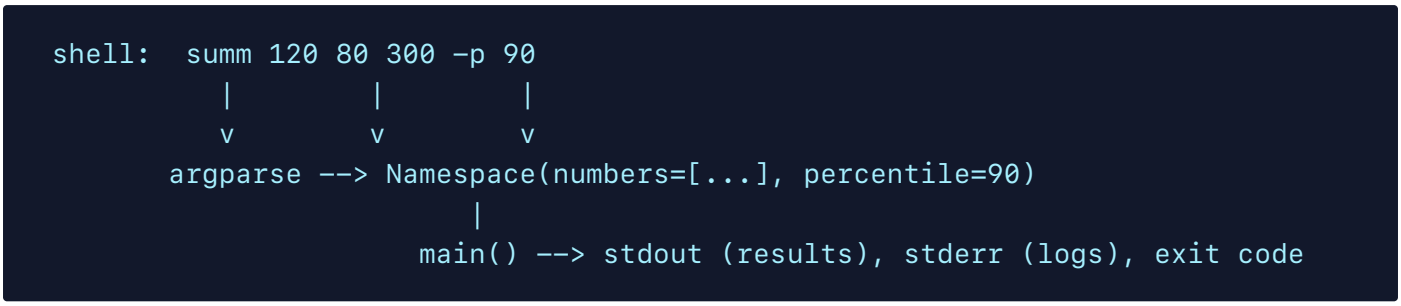
Flag	`-v`	Turn on debug logging
Exit code	`0` or `2`	Scheduler/CI decides pass or fail

Predict, Run, Explain. Before you run it, what does `python3 s5.py abc` print and what is the exit code? Real output: `summ: error: argument numbers: invalid float value: 'abc'` and exit code 2. Line by line: `type=float` runs `float("abc")`, which fails, so `argparse` prints usage and calls `sys.exit(2)`. The typical wrong version skips `type=float` and then sorts strings: `sorted(["120", "80"])` gives `['120', '80']` (text order), so the percentile is wrong and no error appears. The fix is `type=float`.

The shell injection warning has a real demonstration. With `shell=True`, `subprocess.run("echo hi; echo INJECTED", shell=True)` printed both `hi` and `INJECTED`: two commands ran from one string. If part of that string came from a user, they could run anything.

The picture

Read left to right: the shell passes words, `argparse` turns them into named values, `main` does the work, and the results leave through three channels.



Connection to the real job

- Every scheduled data job is a CLI: `python etl.py --date 2026-01-31`.
- Docker containers and CI steps run commands, so exit codes decide pass or fail.
- `subprocess` glues Python to tools like `git`, `dbt` or `ffmpeg`.

Applications

- **Data Engineering:** ETL entry points scheduled by cron or Airflow.
- **Machine Learning / MLOps:** `python train.py --config prod.yaml` inside a container.
- **Data Analytics:** a report generator that takes `--month 2026-01`.
- **Testing and QA:** a wrapper that runs an external test tool and summarises its results.

COMMON MISTAKE

Building a shell command from user text with `shell=True` (for example `f"grep {term} file"`). A value like `x; rm -rf ~` becomes a second command. Pass a list of arguments instead.

Check yourself

1. What exit code means success? Answer: 0.
2. Why put the logic in `main(argv=None)`? Answer: So a test can call `main([...])` without a real shell.
3. Why does `?` matter in `sqlite3`? Answer: It passes values separately from the SQL text, preventing SQL injection (Chapter 13).

Chapter recap

- HTTP is request (method, URL, headers, body) and response (status, headers, body). Status codes are part of the contract.
- REST means nouns in URLs, methods as actions, JSON bodies and stateless calls.
- GET, PUT and DELETE are idempotent. POST is not, so use an idempotency key when a repeat would hurt.
- Always set a timeout, check the status, and retry only temporary errors with backoff.
- FastAPI turns type hints and pydantic models into validation and automatic OpenAPI docs. Flask is smaller and needs more manual pieces.
- Paginate big lists. Cursors are steadier than offsets for live data.
- Authentication is "who", authorization is "what". A JWT is signed, not encrypted. Verify signature and expiry.
- Scrape politely: API first, then robots.txt, terms, delays, a User-Agent, and no personal data without cause.
- A good CLI has a `main(argv)` function, clear help, proper exit codes, and logs on stderr.
- Pass argument lists to `subprocess`, and use `?` placeholders in SQL.

Cheat sheet

Task	Pattern
Call an API safely	`client.get(url, timeout=5)`, then `raise_for_status()`

Retry	wait `base * 2**attempt`, only for temporary errors
Paginate	loop until `next_offset is None`
Safe POST retry	`Idempotency-Key` header
Sign a token	`hmac.new(key, msg, sha256)`, verify with `compare_digest`
Check robots	`RobotFileParser().parse(lines)`, `can_fetch(ua, url)`
CLI	`argparse`, `main(argv=None)`, exit code 0 or non-zero
Run a program	`subprocess.run([...], capture_output=True, text=True, check=True)`
SQL values	`execute("... WHERE x = ?", (value,))`

Practice set

Problem 12.P.1 (easy) Which status code for each: item created; item not found; rate limit hit; server crashed on a bug?

Answer: 201; 404; 429; 500.

Problem 12.P.2 (easy) What is the output type of `json.loads('{"a": [1, 2]}')["a"]`, and what is its value?

Answer: A Python `list`: `[1, 2]`.

Problem 12.P.3 (easy) Name the three parts of a JWT, in order.

Answer: Header, payload, signature, joined by dots.

Problem 12.P.4 (medium) An API has 95 records and you ask `limit=20`. How many requests, and how many items on the last page?

Answer: ceil(95 / 20) = 5 requests. The last page has 95 - 80 = 15 items.

Problem 12.P.5 (medium) Find the bug: `r = requests.get(url); data = r.json()` in a nightly job that sometimes hangs.

Answer: There is no `timeout=` and no `raise_for_status()`. Add `timeout=10` and `r.raise_for_status()`. Hanging comes from the missing timeout.

Problem 12.P.6 (medium) Using the 12.7 `summ` tool, what is `p50` for the numbers `10 20 30 40`?

Answer: Nearest rank: ceil(4 x 0.5) = 2, index 1 after sorting, value 20.0. (So `summ 10 20 30 40 -p 50` prints `count=4 mean=25.0 p50=20.0` .)

Problem 12.P.7 (medium) Your bot reads robots.txt with `Disallow: /admin/` and `Crawl-delay: 10` . You want 300 pages from allowed paths. What is the least time the crawl should take?

Answer: 300 x 10 = 3,000 seconds, which is 50 minutes, if you fetch one page at a time and respect the delay.

Problem 12.P.8 (medium, scenario) A mobile app retries `POST /orders` after a timeout, and customers get two orders. Give two fixes.

Answer: (1) Add an idempotency key header, and make the server return the first result for a repeated key. (2) Add a unique constraint on a client-generated order ID so the database refuses duplicates (and return 409 or the original order).

Problem 12.P.9 (hard, interview) Explain the difference between authentication and authorization with an API example, and say where a JWT fits.

Answer: Authentication verifies the identity (a valid login gives a token). Authorization decides actions: a "viewer" token can `GET` but not `DELETE` , so `DELETE` returns 403. A JWT carries identity and claims (like role and expiry) in a signed token, so the server can check them without a database lookup on each call. It is signed, not encrypted.

Problem 12.P.10 (hard, interview) Why is `subprocess.run(f"ls {user_input}", shell=True)` dangerous, and what is the fix?

Answer: The shell parses the string, so `user_input = "x; rm -rf ~"` runs a second command (command injection). Fix: `subprocess.run(["ls", user_input])` , which treats the input as one argument, and validate the input too.

Problem 12.P.11 (hard) Your offset-paged export misses some rows when new rows arrive during the export. Why, and how do you fix it?

Answer: New rows at the front shift every later row by one, so a row can fall between pages (missed) or appear twice. Use cursor (keyset) pagination, for example `WHERE id > last_id ORDER BY id LIMIT 100` , or export from a snapshot.

Prompt pack

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data and AI roles, and English is my second language, so use simple
```

English. Teach me "HTTP and REST API design" step by step: methods, status codes, headers, idempotency, and how REST naming habits work. Use small Python examples with the standard library, show real output, and relate each idea to everyday examples like ordering at a restaurant. After each part, ask me one check question and wait for my answer before continuing.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Give me 8 graded exercises (easy to hard) on calling APIs from Python with requests or httpx: timeouts, raise_for_status, Session reuse, retries with exponential backoff, pagination loops, and rate limiting (429). Do not give solutions. When I paste my answers, review them one by one and point out bugs and risks.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional learning Python for data and ML jobs, and English is my second language. Quiz me one question at a time on "authentication and authorization": API keys, bearer tokens, OAuth 2.0, OIDC, JWT structure, signature versus encryption, 401 versus 403, and token expiry. Start easy, increase difficulty, and explain every wrong answer simply before moving on. Keep score out of 10.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Review my work on a small FastAPI service with one database table and two endpoints. Check routing, pydantic validation, status codes, error handling, how the model or database connection is created, and test coverage. Be specific and kind, and list the top 5 improvements in order of importance. My code: [PASTE HERE]

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional preparing for Python and data engineering interviews, and English is my second language. Run a 30-minute mock interview on "APIs, scraping responsibly, and command-line tools in Python". Ask one question at a time, mixing concept questions (idempotency, pagination, JWT, robots.txt, argparse exit codes, subprocess safety) and a small coding task. After each answer give a score out of 5, a model answer, and one tip on how to say it clearly in English.

CHAPTER 13

PROFESSIONAL PRACTICE: CLEAN CODE, CI/CD, SECRETS, SECURITY AND PACKAGING

The habits that turn "code that works on my laptop" into code a team can trust, change and ship.

Picture your first week in a data team. You open a colleague's script. It has one function of 300 lines, names like `x2` and `tmp`, a database password typed into the file, and no tests. Nobody dares to change it. Now picture the opposite: small named functions, tests that run on every push, secrets in the environment, and a package you can install in one command. This chapter shows you how to build the second kind. By the end you will be able to refactor messy code safely, explain how a team ships software (Agile, code review, CI/CD), keep secrets out of Git, stop SQL injection, store passwords correctly, and package your work with `pyproject.toml`.

Everything marked "run" was executed with Python 3.14 and the standard library. Anything needing a network or extra installs (GitHub Actions, `gitleaks`, `pip-audit`, `build`, `twine`) is marked **not run here** and shown in minimal, standard form. Check each tool's docs for current options.

Words you need. **Refactoring** means changing how code is written without changing what it does. **Technical debt** means shortcuts that make future changes slower, like a loan with interest. **CI/CD** means Continuous Integration (tests run automatically on every change) and Continuous Delivery/Deployment (releases happen automatically or with one click). **Secret** means any value that grants access: password, API key, token, private key.

13.1 Clean Code and Refactoring

INTUITION

Code is read ten times more than it is written. Think of a kitchen. In a clean kitchen every tool has a labelled place, so a new cook can work on day one. In a messy kitchen only the person who made the mess can cook. Clean code is a labelled kitchen: good names, small functions, no repeated recipes.

How it works

The main rules, with the reason for each:

Rule	What it means	Why it exists
Good names	`slow_delivered_orders`, not	A name is the cheapest

	`f`	documentation
Small functions	One job, fits on a screen	Easy to test and to reuse
DRY (Don't Repeat Yourself)	One place for each piece of knowledge	A bug fixed once is fixed everywhere
Comments say *why*	Not *what* the code does	Code already says what; only you know why
No magic numbers	`TIMEOUT_S = 5`	The number gets a meaning
Early return	Handle bad cases first	Less nesting
Types and docstrings	Say what goes in and out	Readers and tools can check

Refactoring is done in small steps with tests that stay green. The safe loop is: (1) make sure tests exist, (2) change one thing, (3) run the tests, (4) repeat. If you have no tests, first write a few that capture today's behaviour.

When NOT to refactor: when you have no tests and a deadline tomorrow, add the tests first. Do not tidy code you are not otherwise touching just because you can.

Worked examples

Problem 13.1.1 Refactor a cryptic function. It takes shop order rows `(order_id, delivery_minutes, status)` and a threshold, and returns the order ids of delivered orders slower than the threshold.

Predict first: for the rows below with threshold 60, which order ids come out?

- **Step 1** Write the "before" code and run it to record today's behaviour.
- **Step 2** Name the data with a `dataclass` and the function by what it does.
- **Step 3** Flatten the nested `if` s into one comprehension.
- **Step 4** Run before and after on the same input and compare.

```
# BEFORE
def f(d, t):
    r = []
    for x in d:
        if x[2] == "DELIVERED":
            if x[1] > t:
                r.append(x[0])
    return r

rows = [("A101", 45, "DELIVERED"), ("A102", 12, "DELIVERED"), ("A103", 230,
"LOST"), ("A104", 150, "DELIVERED")]
print(f(rows, 60))

# AFTER
from dataclasses import dataclass

@dataclass(frozen=True)
class Order:
    order_id: str
    delivery_minutes: int
    status: str

def slow_delivered_orders(orders, threshold_minutes):
    """Return ids of delivered orders slower than threshold_minutes."""
    return [o.order_id for o in orders if o.status == "DELIVERED" and
o.delivery_minutes > threshold_minutes]

orders = [Order(*r) for r in rows]
print(slow_delivered_orders(orders, 60))
print(slow_delivered_orders(orders, 10))
print(f(rows, 10) == slow_delivered_orders(orders, 10))
```

```
['A104']
['A104']
['A101', 'A102', 'A104']
True
```

- **Check** By hand: `A101` is 45 (not above 60), `A102` is 12, `A103` is slow (230) but status is LOST so it is skipped, `A104` is 150 and DELIVERED. Only `A104`. For threshold 10, the delivered rows are `A101`, `A102`, `A104`, all above 10.
- **Answer** The new function gives the same results as the old one (the last line prints `True`), but a reader now knows what it does from its name, and the fields have names (`o.delivery_minutes` instead of `x[1]`).
- **Why it matters** Equality of old and new outputs on the same inputs is a poor-man's test that your refactoring did not break behaviour. You will turn it into a real pytest test (Chapter 10).

Problem 13.1.2 Remove repetition (DRY) from percentile code and give magic numbers names.

Predict: three copies of the same sort-and-index logic. After refactoring, how many copies of the formula exist?

```
import math

# BEFORE: the formula is copy-pasted three times
def p50(xs): xs = sorted(xs); return xs[max(0, math.ceil(len(xs) * 50 / 100) -
1)]
def p90(xs): xs = sorted(xs); return xs[max(0, math.ceil(len(xs) * 90 / 100) -
1)]
def p95(xs): xs = sorted(xs); return xs[max(0, math.ceil(len(xs) * 95 / 100) -
1)]

# AFTER: one function, one place to fix bugs
def percentile(xs, p):
    """Nearest-rank percentile, p in 1..100."""
    xs = sorted(xs)
    return xs[max(0, math.ceil(len(xs) * p / 100) - 1)]

data = [80, 95, 110, 120, 300]
print(p50(data), p90(data), p95(data))
print([percentile(data, p) for p in (50, 90, 95)])
```

I ran this and checked the equality by hand: sorted data is 80, 95, 110, 120, 300 (n = 5). For p50: ceil(2.5) = 3, index 2, value 110. For p90 and p95: ceil(4.5) = 5 and ceil(4.75) = 5, index 4, value 300. Real output:

```
110 300 300
[110, 300, 300]
```

- **Answer** One copy of the formula remains. Both versions print `110 300 300`. If a bug is found in the formula, you now fix one line, not three.
- **Why it matters** In real codebases, the copy you forgot to fix is the one that fails in production.

Predict, Run, Explain. Look at the typical wrong refactor: renaming a function in the definition but not at one call site. Python tells you only when that line runs, for example `NameError: name 'p50' is not defined`. That is exactly why you need tests (or a linter such as `ruff`, which flags undefined names before you run anything): the tests run every call site.

Real-world mapping

Concept	Example	Where it appears in the job

Good name	`slow_delivered_orders`	Reviewing a teammate's pipeline in five minutes
Small function	`percentile(xs, p)`	Reused by report, alert and dashboard code
DRY	one percentile formula	One bug fix, many places repaired
Refactor with tests	same output before and after	Cleaning a legacy ETL script

Applications

- **Data Analytics:** turn a 400-line notebook cell into named functions you can reuse next month.
- **Data Engineering:** split an ETL job into extract, transform and load functions that can be tested.
- **Machine Learning:** separate feature code from training code so the same features run at serving time.
- **Data Analytics:** share one percentile function across all your report scripts.

COMMON MISTAKE

Refactoring and adding features in the same change. If something breaks you do not know which caused it. Do them in separate commits: first refactor with tests green, then add the feature.

Check yourself

1. What does DRY stand for and why does it matter? Answer: Don't Repeat Yourself. A fix or change then happens in one place.
2. Should a comment say what or why? Answer: Why. The code already shows what.
3. What must exist before you refactor? Answer: Tests (or at least recorded outputs) that show behaviour did not change.

13.2 Architecture: Layers, Monoliths, Microservices, Events and Queues

INTUITION

A restaurant has a front desk (takes orders), a kitchen (cooks), and a storeroom (keeps food). If a waiter walked into the storeroom and cooked, chaos follows. Software layers are the same: each part has one job and talks only to its neighbour. A queue is the order rail in the kitchen: waiters put tickets on it, cooks take them when ready, so a rush does not collapse the kitchen.

How it works

Layered architecture. Three layers are common:

1. **API / presentation layer**: receives requests, checks input, returns responses.
2. **Service / business layer**: the rules ("an order total is the sum of its items").
3. **Repository / data layer**: the only place that talks to the database.

Why: you can test the middle layer with a fake data layer, and you can change the database without touching the rules.

Dependency injection (DI) means giving a class the things it needs (a repository, a notifier) from outside instead of creating them inside. The **repository pattern** hides the database behind methods like `save` and `get`. A **fake** is a simple working stand-in (an in-memory repository). A **mock** is a recorder that checks how it was called.

Monolith vs microservices.

	Monolith	Microservices
Shape	One deployable program	Many small services over the network
Good for	Small teams, early products, simple deployment	Big teams, parts that scale differently
Costs	Big codebase, one slow release train	Network failures, many deployments, hard debugging

Start with a well-organised monolith. Split only when you feel real pain. A distributed system is slower to build and harder to debug than a single program.

Event-driven design means parts communicate by announcing events ("order placed") and other parts react. **Message queues** (RabbitMQ, Kafka, cloud queues) store messages between producer and consumer. Benefits: smoothing spikes, retries, and decoupling. Cost: messages may arrive twice (so consumers must be idempotent, see Chapter 12) or late.

Worked examples

Problem 13.2.1 Build a service with injected dependencies and test it using a fake repository and a mock notifier.

Predict: what total does `place(1, [100, 250, 50])` return, and what message is sent?

- **Step 1** `OrderService` receives `repo` and `notifier` in `__init__`.

- **Step 2** `place` sums prices, saves, and notifies.
- **Step 3** In the test we pass a fake repository (works in memory) and a `Mock` notifier (records calls).

```
from unittest.mock import Mock

class OrderRepository:                      # a real one would use a database
    def __init__(self): self.rows = {}
    def save(self, oid, total): self.rows[oid] = total
    def get(self, oid): return self.rows.get(oid)

class FakeRepo(OrderRepository): pass       # in-memory fake: real behaviour,
no DB

class OrderService:
    def __init__(self, repo, notifier):     # dependencies injected
        self.repo, self.notifier = repo, notifier
    def place(self, oid, prices):
        total = sum(prices)
        self.repo.save(oid, total)
        self.notifier.send(f"order {oid}: {total}")
        return total

notifier = Mock()
svc = OrderService(FakeRepo(), notifier)
print(svc.place(1, [100, 250, 50]))
print(svc.repo.get(1))
notifier.send.assert_called_once_with("order 1: 400")
print(notifier.send.call_args)
```

```
400
400
call('order 1: 400')
```

- **Check** 100 + 250 + 50 = 400. `assert_called_once_with` raised nothing, which means the message was sent exactly once with the text we expected. (If it was wrong, you would see an `AssertionError` that shows expected and actual calls.)
- **Answer** The total is 400, it was saved, and exactly one notification `order 1: 400` was sent.
- **Why it matters** No email was sent and no database was touched, so the test runs in milliseconds and cannot fail because of a network.

Problem 13.2.2 Show the idea of a queue and of events in a few lines: a worker takes jobs from a `queue.Queue`, and an event bus calls every handler registered for an event.

Predict: in what order do the results appear, and how many handlers run on one event?

```
import queue, threading

q = queue.Queue()
results = []
def worker():
    while True:
        job = q.get()
        if job is None:
            break
        results.append(job * job)
        q.task_done()
t = threading.Thread(target=worker); t.start()
for n in range(1, 6): q.put(n)
q.put(None); t.join()
print(results)

handlers = {}
def on(evt):
    def deco(fn): handlers.setdefault(evt, []).append(fn); return fn
    return deco
def emit(evt, **data):
    for h in handlers.get(evt, []): h(**data)
@on("order_placed")
def send_email(order_id): print("email for", order_id)
@on("order_placed")
def update_stock(order_id): print("stock updated for", order_id)
emit("order_placed", order_id=42)
```

```
[1, 4, 9, 16, 25]
email for 42
stock updated for 42
```

- **Answer** The queue is first-in-first-out, so squares of 1 to 5 appear in order. Two handlers ran for one event. The code that emits "order placed" does not know about email or stock. You can add a third handler without touching it.
- **Check** 1, 4, 9, 16, 25 are the squares of 1 to 5.
- **Why it matters** This is the shape of real systems: producers and consumers separated by a queue, with workers that scale independently.

Typical wrong version: creating the repository inside the service (`self.repo = PostgresRepo()`). The service can no longer be tested without a database, and the test fails with a connection error like `ConnectionRefusedError` . The fix is the injection shown above.

Real-world mapping

Concept	Example	Where it appears in the job
Layers	route to service to repository	A FastAPI model service with a feature store behind it
DI + fake	`FakeRepo`	Fast unit tests for a data pipeline step
Queue	Kafka topic, SQS queue	Buffering events before they are loaded into a warehouse
Event	"file arrived"	Starting an ETL run when data lands in a bucket

Applications

- **Data Engineering:** streaming pipelines where a queue buffers bursts and consumers write idempotently.
- **Machine Learning / MLOps:** a "retrain requested" event triggers a training job.
- **GenAI:** a queue holds long-running document-processing jobs behind a chat API.
- **Operations:** you monitor queue depth and consumer lag, like counting orders waiting at a counter, not only how fast one order is served.

COMMON MISTAKE

Jumping to microservices because it sounds modern. A small team with ten services spends more time on networking and deployment than on features. Start simple; split when a clear need appears.

Check yourself

1. Why use layers? Answer: Each layer has one job, so you can test and change parts separately.
2. What is the difference between a fake and a mock? Answer: A fake is a simple working replacement. A mock records calls so you can check them.
3. Why must queue consumers be idempotent? Answer: Messages can be delivered more than once, and a repeat must not cause a double effect.

13.3 Agile, Estimation, Technical Debt, Code Review and Documentation

INTUITION

Building software is like planning a long road trip when the map is incomplete. You plan a short leg, drive it, look at what you learned, and plan the next leg. That is Agile. Technical debt is borrowing time: a shortcut now saves a day, but every later change pays a bit of interest.

How it works

Agile is a way of working in short cycles with frequent feedback. **Scrum** is a popular form of it:

- A **sprint** is a fixed period (often two weeks) with a goal.
- The **backlog** is the ordered list of work. Each item is a **user story** ("As an analyst I want a daily report so that I can see errors").
- Meetings: **planning** (choose work), **daily stand-up** (15 minutes: what I did, will do, what blocks me), **review** (show the work), **retrospective** (how to improve).
- **Estimation:** teams guess size in points or days. People are bad at single-number guesses, so use a range. The **three-point (PERT)** estimate is `(optimistic + 4 × likely + pessimistic) / 6`. It weights the likely case but respects the risk of a long tail.

Technical debt: quick-and-dirty code you plan to fix later. Some debt is wise (to hit a launch), but it must be written down and repaid. Signs of too much debt: fear of changing code, bugs that return, slow onboarding.

Code review is where a teammate reads your change before it merges. It catches bugs, spreads knowledge, and keeps the style consistent. Good reviews:

- Authors: keep changes small (under about 400 lines), explain *why* in the description, run tests first.
- Reviewers: be kind and specific, review the code and not the person, ask questions ("What happens if the list is empty?"), separate must-fix from nice-to-have, and say what is good.

Documentation layers: a README (what it is, how to install and run in 5 lines), docstrings (what each function does), comments (why), and an architecture note. A **docstring** is the string at the top of a function. It can even hold runnable examples called **doctests**.

Worked examples

Problem 13.3.1 A task has optimistic 2 days, likely 4 days, pessimistic 12 days. Give the PERT estimate and compare with the simple average of the three.

Formula	PERT = (O + 4M + P) / 6
Plug in	(2 + 4 × 4 + 12) / 6
Compute	2 + 16 + 12 = 30, and 30 / 6 = 5.0. Simple average: (2 + 4 + 12) / 3 = 6.0.
Answer	The PERT estimate is 5.0 days. Promise 5 to 6 days, not the "likely" 4.
Check	The result lies between the likely value (4) and the average (6), pulled up a little by the long tail. A one-line check in Python: `round((2 + 4*4 + 12) / 6, 2)` printed `5.0` in a real run.
Why it matters	A promise of 4 days would be missed whenever the pessimistic case happens. Ranges and a weighted estimate give more honest dates.

Problem 13.3.2 Write a docstring with doctests for `percentile` and run them automatically.

Predict: how many examples are tested, and do they pass?

The function below was saved in a small package (you will build it in 13.7) and tested with `doctest`:

```
import math

def percentile(values, p):
    """Nearest-rank percentile.

    >>> percentile([80, 95, 110, 120, 300], 90)
    300
    >>> percentile([1, 2, 3, 4], 50)
    2
    """
    xs = sorted(values)
    k = max(0, math.ceil(len(xs) * p / 100) - 1)
    return xs[k]
```

Running `doctest.testmod` on the module printed:

```
TestResults(failed=0, attempted=2)
```

- **Check** By hand: the first example is the p90 case from 13.1.2 (300). Second: sorted `1,2,3,4`, ceil(4 × 0.5) = 2, index 1, value 2.
- **Answer** Two examples were tested and both passed. Your documentation can never go out of date silently: if the code changes, the example fails.
- **Why it matters** The learner reading the docs sees how to call the function. The team gets a free tiny test.

Typical wrong version: a docstring example with the wrong expected value (for example `300` replaced by `200`). `doctest` reports the line, the expected output and the real output, and `failed=1`. The fix is to correct the example or the code, whichever is wrong.

Real-world mapping

Practice	Example	Where it appears in the job
Sprint	2 weeks, goal "daily report live"	How most data teams plan work
Estimate	PERT 5.0 days	Telling a manager when a pipeline will be ready
Tech debt ticket	"Replace hard-coded path"	Keeps a backlog of cleanups
Review comment	"What if `rows` is empty?"	Catches the empty-file bug before production
README	install, run, test in 5 lines	A teammate runs your program on day one

Applications

- **Data Analytics:** short sprints with stakeholders reviewing a dashboard each cycle.
- **Data Engineering:** reviews catch schema and idempotency mistakes before a load runs.
- **Machine Learning:** model cards and READMEs record data, metrics and limits.
- **Project planning:** estimating delivery dates; ranges beat single numbers.

COMMON MISTAKE

Treating estimates as promises, or giving one number. State a range and your assumptions ("assuming the data is available by Monday"), and update the estimate when you learn more.

Check yourself

1. What is a sprint? Answer: A short fixed period (often two weeks) with a goal.
2. What does the retrospective do? Answer: The team looks at how it worked and decides what to improve.
3. Why use a range estimate? Answer: Work has uncertainty, and a single number hides the risk.

13.4 CI/CD Basics

INTUITION

CI is a strict friend who runs your tests every time you save to the shared folder. If you broke something, you hear within minutes, not weeks. CD is the delivery truck that takes the passing build to users automatically. Think of CI as a quick taste test of the soup that happens every time the recipe changes.

How it works

Pipeline stages, in order: **build** (install, compile or package), **test** (unit and integration tests), **lint** (style and error checks with `ruff`, types with `mypy`), then **deploy**. A failing stage stops everything after it ("fail fast"). Cheap checks (lint) go first.

Tools: **GitHub Actions** (YAML files in `.github/workflows/`), **Jenkins** (self-hosted, a `Jenkinsfile`), GitLab CI, and others. Key ideas: a **trigger** (push, pull request, schedule), a **runner** (the machine), **steps**, and **secrets** stored in the CI system, never in the repo.

Not run here (it needs GitHub). A minimal, standard GitHub Actions file:

```
name: ci
on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -e ".[dev]"
      - run: ruff check .
      - run: ruff format --check .
      - run: mypy src
      - run: pytest --cov=src
```

Version tags such as `@v4` change over time. Check the current ones on each action's page.

Worked examples

Problem 13.4.1 Simulate a tiny pipeline runner that stops at the first failing stage, and run it with a test stage that fails.

Predict: which stages run, and what is the final result?

```
import subprocess, sys

def stage(name, code):
    r = subprocess.run([sys.executable, "-c", code], capture_output=True,
text=True)
    print(f"{name:6} -> {'PASS' if r.returncode == 0 else 'FAIL'}")
    return r.returncode == 0

stages = [
    ("lint", "pass"),
    ("test", "assert 2 + 2 == 5"),
    ("deploy", "print('shipping')"),
]
for name, code in stages:
    if not stage(name, code):
        print("pipeline stopped at", name)
        break
else:
    print("pipeline passed")
```

```
lint   -> PASS
test   -> FAIL
pipeline stopped at test
```

- **Step 1** Each stage is a command. Exit code 0 means pass, which is the same convention real CI uses.
- **Step 2** The `for ... else` runs `else` only if the loop did not `break`.
- **Check** `assert 2 + 2 == 5` raises `AssertionError`, so the child exits with code 1. Deploy never ran.
- **Answer** Lint passed, test failed, and the pipeline stopped before deploy. Bad code never reached users.
- **Why it matters** This is exactly how a real CI system treats exit codes, which is why Chapter 12 stressed returning proper codes from CLIs.

Problem 13.4.2 A pipeline takes 12 minutes: lint 1, unit tests 4, integration tests 6, package 1. A change that breaks lint is pushed. How long until the developer knows with lint first, and with lint last?

Compute (lint first)	Lint is the first stage and takes 1 minute, so the failure appears after 1 minute.
Compute (lint last)	Stages before lint take 4 + 6 + 1 = 11 minutes, then lint 1 minute: 12 minutes.
Answer	1 minute versus 12 minutes. Put cheap, fast checks first.
Check	11 + 1 = 12, which equals the total pipeline time, as expected when the last stage fails.
Why it matters	Developer waiting time is real cost. Fast feedback means more small, safe changes.

Typical wrong version: printing a secret inside a CI step (`echo $API_KEY`) or putting the key in the YAML. CI logs are readable by many people. Use the CI system's secret store and never print it (see 13.5).

Real-world mapping

Stage	Example command	Where it appears in the job
Lint	`ruff check .`	Stops unused imports and typos early
Test	`pytest`	Protects a transformation from silent changes
Build	`python -m build`	Produces the package that gets deployed
Deploy	push container image	Releases a new model service version

Applications

- **Data Engineering:** CI runs pipeline unit tests and checks SQL or dbt models before merge.
- **Machine Learning / MLOps:** CI trains a tiny model on sample data and checks metrics do not collapse.
- **Data Analytics:** CI checks that reports still build after a change.
- **Software Engineering:** a short smoke test as a CI stage; a bigger one on a schedule.

COMMON MISTAKE

A CI pipeline that is slow or flaky, so people ignore red builds. Keep it fast, fix flaky tests quickly, and treat red as "stop".

Check yourself

1. What decides pass or fail of a CI step? Answer: The command's exit code (0 means pass).
2. Why run lint before integration tests? Answer: It is quick, so you get feedback sooner.
3. Where do CI secrets live? Answer: In the CI system's secret store, never in the repository.

13.5 Configuration and Secrets Handling

INTUITION

A configuration value is like a thermostat setting: you can change it without rebuilding the house. A secret is a house key. You do not write it on the front door (the code), and you do not make copies for everyone (Git history lives forever and is copied to every clone).

How it works

The 12-factor rule: keep configuration in the environment, not in code. The same code then runs on your laptop, in testing and in production with different settings.

Where config lives, from simple to strong:

1. **Environment variables**: `os.environ["PIPELINE_ENV"]`.
2. `.env` **file** for local development: lines like `KEY=value`, loaded by a library such as `python-dotenv` (not run here; we show a small parser). **Add** `.env` **to** `.gitignore` and commit a `.env.example` with fake values.
3. **YAML/TOML files** for non-secret structure (batch sizes, URLs).
4. **Secret managers** (cloud secret managers, vaults) for production: access is audited, and secrets can be **rotated** (replaced regularly or right after a leak) without code changes.

Rules for secrets:

- Never commit them. If one is committed, **treat it as stolen**: revoke and rotate it at once. Deleting the file in a later commit does not remove it from history.
- Run a scanner such as `gitleaks` or `detect-secrets` in a pre-commit hook and in CI (not run here).
- Do not print secrets or put them in logs; mask them.
- Use least privilege: a key that can only read one bucket is less dangerous than an admin key.
- In notebooks, read keys from environment variables; clear output before sharing.

Worked examples

Problem 13.5.1 Load settings from the environment into a typed, immutable object, with a default and a required value.

Predict: what is printed, and what happens when a required variable is missing?

```
import os
from dataclasses import dataclass

os.environ["PIPELINE_ENV"] = "prod"      # in real life the shell or CI sets
this

@dataclass(frozen=True)
class Settings:
    env: str
    batch_size: int = 500
    @classmethod
    def from_env(cls):
        return cls(env=os.environ["PIPELINE_ENV"],
                   batch_size=int(os.environ.get("BATCH_SIZE", "500")))

print(Settings.from_env())

try:
    os.environ["MISSING_REQUIRED"]
except KeyError as e:
    print("KeyError", e)
print(os.environ.get("MISSING", "default"))
```

```
Settings(env='prod', batch_size=500)
KeyError 'MISSING_REQUIRED'
default
```

- **Step 1** `os.environ[...]` is strict: missing means `KeyError`. Use it for required values so the program fails early and loudly.
- **Step 2** `os.environ.get(name, default)` is lenient. Use it for optional values.
- **Step 3** Environment variables are always text, so we convert `BATCH_SIZE` with `int(...)`.
- **Check** `BATCH_SIZE` was not set, so the default `"500"` became the integer 500.
- **Answer** Settings loaded as shown, and a missing required variable raised `KeyError`.
- **Why it matters** A pipeline that crashes at start-up with "missing PIPELINE_ENV" is far better than one that silently runs against the wrong database.

Problem 13.5.2 Parse a `.env` text (fake values), then write a tiny secret scanner that flags suspicious lines, then mask personal data in text.

Predict: which line is flagged, and what do the masked values look like?

```
import re

def load_dotenv(text):
    out = {}
    for line in text.splitlines():
        line = line.strip()
        if not line or line.startswith("#") or "=" not in line: continue
        k, v = line.split("=", 1)
        out[k.strip()] = v.strip().strip('"')
    return out
print(load_dotenv('# demo\nAPI_KEY="fake-key-123"\nDEBUG=1\n'))

def mask_email(s): return re.sub(r"(\w)[\w.]*@", r"\1***@", s)
def mask_phone(s): return re.sub(r"\b\d{6}(\d{4})\b", r"XXXXXX\1", s)
print(mask_email("asha.k@example.com"), mask_phone("9876543210"))

pat = re.compile(r"(?i)(api[_-]?key|secret|token)\s*=\s*['\"][A-Za-z0-9_\-]{8,}
['\"]")
code = 'x = 1\nAPI_KEY = "fake-key-123"\nname = "abc"\n'
for i, l in enumerate(code.splitlines(), 1):
    if pat.search(l): print("line", i, "looks like a secret")
```

```
{'API_KEY': 'fake-key-123', 'DEBUG': '1'}
a***@example.com XXXXXX3210
line 2 looks like a secret
```

- **Check** Line 2 matches `API_KEY = "..."` with 12 allowed characters (at least 8 needed). Lines 1 and 3 have no key-like name. The phone number has 10 digits: six are hidden and the last four stay.
- **Answer** The scanner flagged line 2. Emails and phone numbers were partly hidden. Real scanners like `gitleaks` use hundreds of such patterns plus entropy checks (not run here).
- **Why it matters** Masking limits damage when data reaches logs or test environments. Partial masking is not anonymisation: with enough other columns a person can still be identified.

Redacting secrets in logs. A log filter can remove a token before it is written:

```
import logging, re
class Redact(logging.Filter):
    def filter(self, record):
        record.msg = re.sub(r"(token=)\w+", r"\1***", str(record.msg)); return
True
logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
log = logging.getLogger("app"); log.addFilter(Redact())
log.info("calling api token=abc123xyz")
```

```
INFO calling api token=***
```

Typical wrong version: `API_KEY = "sk-live-..."` typed in the script and pushed to GitHub. Bots scan public repos within minutes. Fix: move the key to an environment variable, rotate the leaked key immediately, and add a scanner. A `.gitignore` needs the line `.env` (add it before the first commit).

Real-world mapping

Item	Example	Where it appears in the job
Env var	`DATABASE_URL`	Same code for dev, test, prod
`.env.example`	fake values committed	New teammate knows what to set
Secret manager	cloud vault entry	Production database password, rotated
Pre-commit scanner	`gitleaks`	Stops a key before it leaves your laptop

Applications

- **Data Engineering:** warehouse credentials from a secret manager, injected as env vars at run time.
- **Machine Learning / MLOps:** model-registry tokens in CI secrets.
- **GenAI:** language-model API keys read from the environment in notebooks, never pasted into cells.
- **Data Analytics:** BI connection strings kept in the tool's credential store.

COMMON MISTAKE

"I deleted the key from the file in the next commit, so it is safe." Git history keeps it, and so do clones. Revoke and rotate the key first, clean the history second.

Check yourself

1. Which is stricter, `os.environ[k]` or `os.environ.get(k)`? Answer: `os.environ[k]` raises `KeyError` if missing.
2. What do you do if a key is committed? Answer: Revoke and rotate it immediately.
3. What goes in `.env.example`? Answer: The variable names with fake or empty values.

13.6 SQL Injection, Password Hashing and Safe Basics

INTUITION

Imagine a form where the clerk reads out what you write as an instruction to the vault. If you write "open all drawers", the clerk obeys. SQL injection happens when user text is glued into a SQL command and the database cannot tell data from instruction. A parameterised query is a form with a labelled blank: whatever you write goes in the blank and is treated only as data.

How it works

SQL injection is when untrusted input changes the meaning of a query. It is high on the OWASP Top 10 list (a well-known list of the most common web security risks, in the "injection" category). The fix is always the same: **parameterised queries** (placeholders `?` in `sqlite3`; `%s` or `:name` in other drivers). The values travel separately from the SQL text. Never build SQL with f-strings, `+` or `.format()` from user input.

Other safe basics:

- **Hashing** turns data into a fixed fingerprint, one way. The same input gives the same output; you cannot go back. **Encryption** is two-way: with the key you get the data back. Use hashing for passwords, encryption for data you must read later.
- **Passwords** must be hashed with a slow, salted algorithm: `scrypt`, `argon2`, `bcrypt` or `pbkdf2`. A **salt** is random data added per user so equal passwords give different hashes. Fast hashes like MD5 or plain SHA-256 are for checksums, not passwords.
- **Never write your own crypto.** Use maintained libraries.
- Also know: authorization checks on every request (broken access control is OWASP's top item), HTTPS everywhere, least privilege, and updating dependencies.

Worked examples

Problem 13.6.1 Show SQL injection against string-built SQL, and show that a parameterised query stops it.

Predict: for the input `x' OR '1'='1`, how many users does the vulnerable query return, and the safe one?

```
import sqlite3
con = sqlite3.connect(":memory:")
con.executescript("""
CREATE TABLE users(id INTEGER PRIMARY KEY, name TEXT, secret TEXT);
INSERT INTO users(name, secret) VALUES ('asha','pw-asha'),('ravi','pw-ravi');
""")

def vulnerable(name):
    q = f"SELECT id, name FROM users WHERE name = '{name}'"
    print("SQL:", q)
    return con.execute(q).fetchall()

def safe(name):
    q = "SELECT id, name FROM users WHERE name = ?"
    return con.execute(q, (name,)).fetchall()

print(vulnerable("asha"))
attack = "x' OR '1'='1"
print(vulnerable(attack))
print(safe("asha"))
print(safe(attack))
attack2 = "x' UNION SELECT id, secret FROM users --"
print(vulnerable(attack2))
print(safe(attack2))
```

```
SQL: SELECT id, name FROM users WHERE name = 'asha'
[(1, 'asha')]
SQL: SELECT id, name FROM users WHERE name = 'x' OR '1'='1'
[(1, 'asha'), (2, 'ravi')]
[(1, 'asha')]
[]
SQL: SELECT id, name FROM users WHERE name = 'x' UNION SELECT id, secret FROM
users --'
[(1, 'pw-asha'), (2, 'pw-ravi')]
[]
```

- **Step 1** Read the printed SQL for the first attack. The quote in the input closed the string early, and `OR '1'='1'` is always true, so every row matches.
- **Step 2** The second attack uses `UNION` to read the `secret` column, which the query never intended to expose. The trailing `--` comments out the rest.
- **Step 3** With `?`, the whole input is one value. No user is called `x' OR '1'='1`, so the result is empty.
- **Check** The safe function returned `[(1, 'asha')]` for the honest input and `[]` for both attacks.
- **Answer** The vulnerable query returned all users (2) and then leaked the secret column; the parameterised query returned nothing for the attacks.

- **Why it matters** One careless f-string can leak a whole database.

Predict, Run, Explain (the "innocent" bug). You do not need an attacker to break string-built SQL. A user named O'Brien does it by accident. Real output:

```
import sqlite3
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE users(id INTEGER PRIMARY KEY, name TEXT)")
con.execute("INSERT INTO users(name) VALUES (?)", ("O'Brien",))
name = "O'Brien"
try:
    con.execute(f"SELECT * FROM users WHERE name = '{name}'")
except sqlite3.Error as e:
    print(type(e).__name__, e)
print(con.execute("SELECT * FROM users WHERE name = ?", (name,)).fetchall())
```

```
OperationalError near "Brien": syntax error
[(1, "O'Brien")]
```

Line by line: the apostrophe in `O'Brien` ends the SQL string early, leaving `Brien'` as garbage, hence the syntax error. The `?` version just works. Also note: `sqlite3` refuses `"SELECT 1; DROP TABLE users"` with `ProgrammingError: You can only execute one statement at a time.`, but other databases may allow stacked statements. Do not rely on that.

Problem 13.6.2 Hash and check a password with `scrypt`, see that salts change the hash, and see the difference between hashing and plain SHA-256 and between hashing and encryption.

Predict: if two users choose the same password, will their stored hashes match?

```
import hashlib, os, hmac, uuid

def hash_pw(pw, salt=None):
    salt = salt or os.urandom(16)
    dk = hashlib.scrypt(pw.encode(), salt=salt, n=2**14, r=8, p=1)
    return salt, dk

def check(pw, salt, dk):
    return hmac.compare_digest(hash_pw(pw, salt)[1], dk)

salt, dk = hash_pw("correct horse")
print(len(salt), len(dk))
print(check("correct horse", salt, dk), check("wrong", salt, dk))
s2, d2 = hash_pw("correct horse")
print("same password, different salt, same hash?", dk == d2)

print(hashlib.sha256(b"hello").hexdigest())
print(hashlib.sha256(b"hello!").hexdigest())
print(hashlib.md5(b"hello").hexdigest(), "(md5: fine for checksums, never for
security)")
u = uuid.uuid4(); print(len(str(u)), u.version)
print(uuid.uuid5(uuid.NAMESPACE_DNS, "example.com"))
print(uuid.uuid5(uuid.NAMESPACE_DNS, "example.com"))
```

```
16 64
True False
same password, different salt, same hash? False
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824
ce06092fb948d9ffac7d1a376e404b26b7575bcc11ee05a4615fef4fec3a308b
5d41402abc4b2a76b9719d911017c592 (md5: fine for checksums, never for security)
36 4
cfbff0d1-9375-5685-968c-48ce8b15ae17
cfbff0d1-9375-5685-968c-48ce8b15ae17
```

- **Step 1** We store the salt and the derived key (`dk`), never the password. The salt is 16 random bytes; the key is 64 bytes.
- **Step 2** To log in, hash the attempt with the stored salt and compare in constant time.
- **Step 3** Changing one character (`hello` to `hello!`) changes the whole SHA-256 output. That is the "fingerprint" property, and why SHA-256 is also used for file checksums.
- **Step 4** `uuid4` is random (version 4). `uuid5` is derived from a name, so the same name always gives the same id, which is useful for stable keys in data pipelines.
- **Check** The right password gives `True` , the wrong one `False` . Two hashes of the same password with different salts differ, so attackers cannot spot users with equal passwords.
- **Answer** No, the hashes do not match, because each has a different random salt. Hashing is one-

way; encryption (not shown, needs a third-party library or cloud KMS) is reversible with a key.

- **Why it matters** If your user table leaks, salted slow hashes make cracking expensive. Plain SHA-256 or MD5 can be attacked billions of times per second.

Typical wrong version: `hashlib.sha256(password.encode()).hexdigest()` stored as the password hash. It is fast and unsalted, so identical passwords look identical and common passwords are cracked by lookup tables. Use `scrypt` / `argon2` / `bcrypt` through a vetted library.

Real-world mapping

Risk	Safe habit	Where it appears in the job
SQL injection	`execute(sql, params)`	Any query built from a filter box, API parameter or file
Weak password storage	salted slow hash	A user table in an internal tool
Data in logs	redact tokens	Production incident logs shared widely
Stable IDs	`uuid5` of a business key	Deduplicating records across loads

Applications

- **Data Analytics:** parameterised queries behind a dashboard filter.
- **Data Engineering:** SQL built from table names or dates must be validated against an allow-list, since placeholders cannot replace table names.
- **Machine Learning / GenAI:** prompt injection is the same idea (untrusted text changing instructions) for language models; keep untrusted text separate and limit what the model can do.
- **Security-minded roles:** hashing for passwords, encryption for data at rest, HTTPS for data in transit.

COMMON MISTAKE

Believing "my data comes from my own app, so it is safe". Data from a CSV, an API, or a user is untrusted until validated. Always parameterise.

Check yourself

1. What is the one-line fix for SQL injection? Answer: Use parameterised queries (placeholders),

never string-built SQL.

2. Hashing or encryption for passwords? Answer: Hashing (salted and slow), because you never need the original back.
3. Why is a salt used? Answer: So the same password gives different hashes, which defeats lookup tables.

13.7 Packaging, Dependencies and Supply-Chain Safety

INTUITION

A script is a loose recipe on paper. A package is a boxed meal kit: it has a name, a version, a list of ingredients (dependencies), and a standard way to open it (`pip install`). Anyone in your company can then use your code without copying files around.

How it works

Modern Python packaging uses a single file, `pyproject.toml` , which holds:

- `[build-system]` : which tool builds it (for example `setuptools` , `hatchling` , `poetry-core`).
- `[project]` : name, version, description, Python version, dependencies, and **entry points** (`[project.scripts]` makes a command like `mini-stats`).
- `[tool.*]` : settings for tools such as `ruff` .

Layout: a `src/` **layout** (`src/mini_stats/__init__.py`) prevents you from accidentally importing the code from the current folder instead of the installed copy.

To build: `python -m build` creates a **wheel** (`.whl` , a ready-to-install zip) and an **sdist** (source archive). To share inside a company, you publish to a **private index** (for example a cloud artifact registry, or a self-hosted index) with `twine upload` and install with `pip install --index-url <your-index> mini-stats` . Public publishing goes to PyPI. Both steps not run here: `build` was not installed on this machine, and there was no network.

Dependencies and safety:

- Pin versions for applications (`requests==2.x.y`), and use ranges for libraries. Tools: `pip-tools` (`pip-compile`), `poetry` , `uv` produce a lock file.
- **Supply-chain risk**: your code trusts every package it installs. Defences: `pip-audit` (checks installed packages against known vulnerabilities), Dependabot (opens pull requests for updates), pinned hashes, using trusted indexes only, and watching for look-alike names (typosquatting). Not run here.

- **Jupyter reproducibility:** notebooks hide state (cells can run out of order). Make them reproducible by restarting and running all cells, pinning the environment, fixing random seeds, and moving reusable code into the package with the notebook only calling it (`nbconvert` can run a notebook from the command line).

Worked examples

Problem 13.7.1 Create a tiny package on disk, read its `pyproject.toml` with the standard `tomllib` , pack it into a zip the way a wheel is a zip, and import it from that archive.

Because the `build` tool was not available, this is a **demonstration of the idea**, not a real wheel (a real wheel also contains a `*.dist-info/` folder with metadata).

Predict: which files go in the archive, and what does the import print?

The files created:

```
pk/
  pyproject.toml
  src/mini_stats/__init__.py
  src/mini_stats/core.py
```

`pyproject.toml` (the part we read):

```
[project]
name = "mini-stats"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = []

[project.scripts]
mini-stats = "mini_stats.cli:main"
```

```
import tomllib, zipfile, sys, pathlib, doctest
root = pathlib.Path("pk")
cfg = tomllib.loads((root / "pyproject.toml").read_text())
print(cfg["project"]["name"], cfg["project"]["version"], cfg["project"]
["scripts"])
whl = root / "mini_stats-0.1.0-demo.zip"
with zipfile.ZipFile(whl, "w") as z:
    for f in (root / "src").rglob("*.py"):
        z.write(f, f.relative_to(root / "src"))
print(zipfile.ZipFile(whl).namelist())
sys.path.insert(0, str(whl))
import mini_stats
print(mini_stats.__version__, mini_stats.percentile([5, 1, 9, 3], 75))
import mini_stats.core as c
print(doctest.testmod(c))
```

```
mini-stats 0.1.0 {'mini-stats': 'mini_stats.cli:main'}
['mini_stats/__init__.py', 'mini_stats/core.py']
0.1.0 5
TestResults(failed=0, attempted=2)
```

- **Step 1** `tomllib` (standard library since Python 3.11) reads TOML into a dict.
- **Step 2** We zip the contents of `src/`, so the top folder inside the archive is `mini_stats`, the importable name. (Notice: the project name `mini-stats` uses a dash, the import name `mini_stats` an underscore.)
- **Step 3** Python can import from a zip on `sys.path`, which is the same mechanism pip uses when it installs files into `site-packages`.
- **Check** Sorted `[5, 1, 9, 3]` is `1, 3, 5, 9`. For p75: ceil(4 × 0.75) = 3, index 2, value 5. The output `5` agrees.
- **Answer** Two modules were archived, the import worked, `percentile([5, 1, 9, 3], 75)` gave 5, and both doctests passed.
- **Why it matters** Once the code is a package, a teammate runs one command, and your tests import the package like any other library.

The real build commands (not run here):

```
python -m pip install build twine
python -m build                 # creates dist/*.whl and dist/*.tar.gz
twine upload --repository-url <PRIVATE_INDEX_URL> dist/*
pip install --index-url <PRIVATE_INDEX_URL> mini-stats
pip-audit                        # scan installed packages for known
vulnerabilities
```

Problem 13.7.2 You have a `requirements.txt` with a loose line `pandas`. In March the app works; in June a new major version breaks it. How do you prevent this, and what is the cost?

Step 1	Pin the exact version that works: `pandas==X.Y.Z` (use your tested version).
Step 2	Generate pins for the whole tree with `pip-compile` or a `poetry.lock` / `uv.lock` file, so even the dependencies of dependencies are fixed.
Step 3	Update on purpose: let Dependabot open a pull request, let CI run the tests, then merge.
Answer	Pin and lock the versions, and update deliberately through CI. The cost is that you must schedule updates; otherwise you also freeze security fixes.
Check	Ask: "if I build this again in a year, will I get the same packages?" With a lock file the answer is yes.
Why it matters	"It worked yesterday" bugs from silent upgrades waste days. Reproducible environments are also what makes ML experiments repeatable.

Typical wrong version: running the code from the folder above `src/`, forgetting to install the package, and seeing `ModuleNotFoundError: No module named 'mini_stats'`. The fix is `pip install -e .` (editable install) in a virtual environment, then import from anywhere.

Real-world mapping

Piece	Example	Where it appears in the job
`pyproject.toml`	name, version, deps	Shared utilities package used by many pipelines
Entry point	`mini-stats = ...`	A team CLI installed on every machine
Lock file	`uv.lock`, `poetry.lock`	Same versions in dev, CI and production
`pip-audit`	scan report	Security teams ask for it before release
Private index	internal registry	Company-only libraries

Applications

- **Data Engineering:** a shared package of connectors and validators installed in every job.
- **Machine Learning / MLOps:** training code packaged and versioned so a model can be traced to its code.

- **Data Analytics:** a team library of metric definitions, so everyone computes "active user" the same way.
- **Data Analytics:** your reporting helpers packaged once, used across projects.

COMMON MISTAKE

Installing packages with plain `pip install` into the global Python. Versions clash between projects. Always use a virtual environment per project (Chapter 9) and record dependencies in `pyproject.toml` or a lock file.

Check yourself

1. What does a wheel contain? Answer: A ready-to-install zip of your package files plus metadata.
2. Why pin versions for an application? Answer: So a new upstream release cannot silently break or change it.
3. What is a typosquatting attack? Answer: A malicious package with a name close to a popular one, hoping you mistype.

Chapter recap

- Clean code means good names, small functions, no repetition, and comments that say why.
- Refactor in small steps with tests green. Never mix refactoring with new features.
- Layers separate API, rules and data. Dependency injection lets you test with fakes and mocks.
- Start with a monolith. Queues and events help with spikes and decoupling, but consumers must be idempotent.
- Agile works in short cycles. Estimate with ranges (PERT: `(O + 4M + P) / 6`). Write down technical debt.
- Good code review is small, kind, specific, and focused on behaviour and risk.
- CI runs build, lint, test (cheap first) and stops at the first failure. Exit codes decide.
- Configuration lives in the environment. Secrets never go in Git. If one leaks, rotate it.
- Parameterised queries stop SQL injection. Hash passwords with a salted slow algorithm.
- A package has `pyproject.toml`, a `src/` layout, pinned dependencies and scans (`pip-audit`).

Cheat sheet

Topic	Pattern
PERT	`(O + 4M + P) / 6`: (2, 4, 12) gives 5.0

estimate	
Safe SQL	`con.execute("... WHERE name = ?", (name,))`
Password hash	`hashlib.scrypt(pw, salt=os.urandom(16), n=2**14, r=8, p=1)` and `hmac.compare_digest`
Required env var	`os.environ["NAME"]` (raises `KeyError`)
Optional env var	`os.environ.get("NAME", default)`
`.gitignore`	add `.env`; commit `.env.example`
Doctest	`python -m doctest -v file.py` or `doctest.testmod()`
Build a package	`python -m build`, then `twine upload`
Editable install	`pip install -e .`
CI order	lint, types, unit tests, integration tests, build, deploy

Practice set

Problem 13.P.1 (easy) What is wrong with the name `def f(d, t):`, and what would you change?

Answer: The names tell nothing. Use names that say what they do and mean, such as `def slow_delivered_orders(orders, threshold_minutes):`, and add a docstring.

Problem 13.P.2 (easy) What is printed? `import os; print(os.environ.get("NOPE", "fallback"))`

Answer: `fallback`. `get` returns the default when the variable is missing.

Problem 13.P.3 (easy) A teammate commits `.env` with a real password. List the first two things you do.

Answer: (1) Revoke or rotate the password immediately. (2) Remove the file from the repo and add `.env` to `.gitignore`; then clean the history if needed and add a secret scanner.

Problem 13.P.4 (medium) Estimate with PERT: optimistic 1 day, likely 3, pessimistic 11.

Answer: (1 + 4 x 3 + 11) / 6 = (1 + 12 + 11) / 6 = 24 / 6 = 4.0 days.

Problem 13.P.5 (medium) What is the output? `sqlite3` query `"SELECT * FROM t WHERE n = '" + user + "'"` with `user = "a' OR '1'='1"`. How many rows come back from a table of 5 rows?

Answer: The condition becomes `n = 'a' OR '1'='1'`, which is true for every row, so all 5 rows. The fix is `execute("SELECT * FROM t WHERE n = ?", (user,))`, which returns only rows where `n` equals the full text.

Problem 13.P.6 (medium) Find the bug: `password_hash = hashlib.sha256(pw.encode()).hexdigest()`.

Answer: SHA-256 is fast and, here, unsalted. Equal passwords give equal hashes and attackers can try billions of guesses per second. Use `scrypt`, `argon2` or `bcrypt` with a random salt.

Problem 13.P.7 (medium) A CI pipeline: lint 1 min, unit 3 min, integration 8 min, build 2 min. A unit test fails. How long until the developer sees red if the order is lint, unit, integration, build? And if integration comes before unit?

Answer: Order lint, unit: 1 + 3 = 4 minutes. If integration first: 1 + 8 + 3 = 12 minutes. Put faster checks earlier.

Problem 13.P.8 (medium) After `pip install -e .` you still get `ModuleNotFoundError` for your package. Name two likely causes.

Answer: You may be using a different Python or virtual environment from the one where you installed it (check `which python` and `pip -V`), or the package folder lacks `__init__.py` or is not found by the `src` layout configuration in `pyproject.toml`.

Problem 13.P.9 (hard, scenario) Your service sends order emails when an order is placed. The email provider is slow, so the API times out under load. Redesign using the ideas in this chapter.

Answer: Emit an "order placed" event or put a message on a queue and return at once. A separate worker consumes the queue and sends the email, with retries. Make the worker idempotent (store a sent flag per order id) because messages can arrive twice. Monitor queue depth.

Problem 13.P.10 (hard, interview) Explain hashing, encryption and encoding (base64) and say which you would use to store a password, a credit-card number you must show later, and a JWT payload.

Answer: Hashing is one-way (password: salted slow hash). Encryption is two-way with a key (credit-card number: encrypt, with keys in a managed key service, and consider not storing it at all). Encoding (base64) is a reversible format change with no secrecy; a JWT payload is only base64-encoded and signed, so it is readable by anyone.

Problem 13.P.11 (hard, interview) A teammate says "we do not need tests, we do code review." Give a

balanced reply.

Answer: Review and tests catch different things. Reviewers are good at design and readability but miss edge cases and cannot rerun everything after each change. Tests are repeatable and protect against regressions (old bugs returning), especially when refactoring. Use both: tests make the review faster because the reviewer can focus on design.

Prompt pack

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional at an Indian company moving into data and AI roles, and English is my second language, so use simple words. Teach me "clean code and refactoring in Python": naming, small functions, DRY, docstrings, and safe refactoring with tests. Walk through one messy 30-line function, refactor it step by step, and show the output before and after to prove behaviour did not change. After each step ask me one check question and wait for my reply.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data and AI roles, and English is my second language. Quiz me one question at a time on "secrets and configuration handling": environment variables, .env and .gitignore, secret scanners like gitleaks, secret managers, rotation, and what to do when a key leaks. Start easy, get harder, explain each wrong answer simply, and give me a score out of 10 at the end.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional learning Python for data jobs, and English is my second language. Explain "SQL injection and parameterised queries" step by step with sqlite3 examples: show a vulnerable f-string query, an attack input, and the safe version, with real outputs. Then give me 8 graded exercises without solutions, including one where I must fix vulnerable code, and one on password hashing with a salt. Review my answers when I paste them.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data engineering, and English is my second language. Review my CI/CD and packaging setup. Check my pyproject.toml, my GitHub Actions workflow, my dependency pinning, and whether secrets are handled safely. Point out the five most important problems in order, explain each in simple English, and show the
```

corrected snippet. My files: [PASTE HERE]

✨ AI prompt: copy and paste into any AI

You are my patient, practical professor. I am a working professional preparing for Python and data engineering interviews, and English is my second language. Run a mock interview on "professional software practice": clean code, testing pyramid, dependency injection, monolith versus microservices, queues and idempotency, code review, CI/CD, SQL injection and password storage. Ask one question at a time, grade my answer out of 5, give a model answer, and give one tip to explain it more clearly in English.

CHAPTER 14

CAPSTONE PROJECTS AND THE PYTHON INTERVIEW BANK

Prove your skills with four real projects, a 150-problem plan, and 45 graded interview questions with answers.

Reading about Python does not get you hired. Showing working projects and answering questions calmly does. This chapter gives you both. First, four capstone projects. Each has a clear specification, milestones you can finish in one hour a day, and a way to check that you are done. They tie together everything from Chapters 1 to 13 and use one friendly theme, a restaurant kitchen and its orders, so that your portfolio tells one story: "a professional who can build data tools". Second, a plan to solve 150 or more practice problems, with solved examples that were run for real. Third, an interview bank, graded from easy to hard, with answers written the way you would say them aloud.

Everything marked "run" was executed with Python 3.14 and the standard library. FastAPI, pytest, `requests`, `BeautifulSoup` and similar tools are marked **not run here**. Interview answers describe general Python behaviour. Some details change by version (for example the free-threaded build of Python and the GIL), so check the current docs before an interview.

Words you need. **Capstone** means a final project that uses many skills together. **Milestone** is a small checkpoint on the way. **Definition of done** is the list of things that must be true before you call a project finished.

14.1 Capstone 1: A Command-Line Tool With Logging, Tests and a Typed API

INTUITION

A craftsperson proves skill by making one complete object, not by owning many tools. This project is one small tool, finished to a professional standard: it has a clear interface, logs what happens, is tested, and has type hints.

Specification

Build `order-summary`, a tool that reads a CSV file of kitchen orders and prints, per dish, the count, average preparation time, p90 and failure percentage. (You already built the first version of the maths in Chapters 12 and 13.)

Input columns: `placed_at_s,prep_seconds,dish,served_ok` .

Interface:

```
order-summary orders.csv --percentile 90 --format table|json --verbose
```

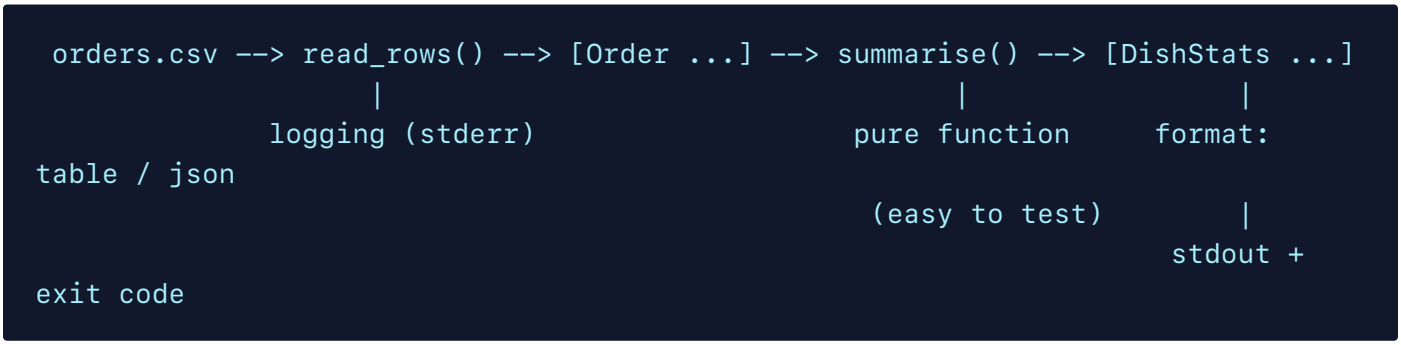
Definition of done:

- A `main(argv)` function that returns an exit code (0 success, 2 bad usage, 1 for data errors).
- `logging` to stderr; results to stdout.
- A typed core API: `def summarise(rows: Iterable[Order], p: int) -> list[DishStats]` , with `dataclass` types, passing `mypy` .
- At least 10 `pytest` tests, including empty file, one row, bad number, and a missing column.
- `ruff` clean, a `pyproject.toml` with an entry point, and a README with install and usage in five lines.

Milestones (about 8 hours)

1. Day 1: Parse the CSV with `csv.DictReader` and compute stats for a hard-coded file.
2. Day 2: Add `argparse` options and `main(argv)` .
3. Day 3: Add `Order` and `DishStats` dataclasses and type hints; run `mypy` .
4. Day 4: Add logging and good error messages (no raw tracebacks for user mistakes).
5. Day 5: Write tests with `pytest` and `parametrize` .
6. Day 6: Add `ruff` , `pre-commit` , `pyproject.toml` , and a README.
7. Day 7: Add a GitHub Actions workflow (Chapter 13) and tag `v0.1.0` .
8. Day 8: Ask a friend to install and use it from your README only. Fix what confuses them.

The picture



Read it left to right. The "pure" middle function has no file or print inside it, which is why it is easy to test.

Worked example

Problem 14.1.1 Write the core of the tool for a tiny inline file and check every number by hand. (This was run; it is the prototype for milestone 1 and also serves Capstone 3.)

Predict: which dish has the highest failure rate, and how many orders per minute arrive?

```
import csv, io, statistics, math
from collections import defaultdict
data = """placed_at_s,prep_seconds,dish,served_ok
1000,120,salad,true
1100,95,salad,true
1200,2300,pasta,false
1500,180,salad,true
1800,450,pasta,true
2000,510,pasta,true
2400,130,soup,true
2900,700,soup,true
"""
def pct(xs, p):
    xs = sorted(xs); return xs[max(0, math.ceil(len(xs)*p/100)-1)]
by = defaultdict(list); err = defaultdict(int); stamps = []
for r in csv.DictReader(io.StringIO(data)):
    by[r["dish"]].append(int(r["prep_seconds"]))
    err[r["dish"]] += r["served_ok"] != "true"
    stamps.append(int(r["placed_at_s"]))
print(f"{'dish':8}{'n':>3}{'avg':>8}{'p90':>7}{'fail%':>7}")
for k, v in sorted(by.items()):
    print(f"{k:8}{len(v):>3}{statistics.mean(v):>8.1f}{pct(v,90):>7}
{100*err[k]/len(v):>7.1f}")
n = sum(len(v) for v in by.values())
print("orders/min:", round(n / ((max(stamps)-min(stamps))/60), 2))
```

```
dish      n     avg    p90  fail%
pasta     3  1086.7   2300   33.3
salad     3   131.7    180    0.0
soup      2   415.0    700    0.0
orders/min: 0.25
```

- **Formula** average = sum / n; failure % = failures / n x 100; arrival rate = n / duration in minutes; duration = (last time - first time) / 60 (the times are in seconds).
- **Compute (salad)** (120 + 95 + 180) / 3 = 395 / 3 = 131.67, shown as 131.7.
- **Compute (pasta)** (2300 + 450 + 510) / 3 = 3260 / 3 = 1086.67. One failure out of 3 is 33.3%. For p90, n = 3, ceil(2.7) = 3, index 2, so the slowest order, 2300.
- **Compute (arrival rate)** 8 orders, duration (2900 - 1000) / 60 = 31.67 minutes, so 8 / 31.67 = 0.25

orders per minute (one order every 4 minutes).

- **Answer** `pasta` has the highest failure rate (33.3%) and the worst average (about 1087 seconds, over 18 minutes). Orders arrive at 0.25 per minute.
- **Check** The counts 3 + 3 + 2 = 8 match the 8 data rows.
- **Why it matters** In plain words: one pasta order in three failed, and the pasta orders that did work were slow. That is the headline a manager needs. Notice that with only 3 orders a p90 equals the maximum: percentiles need many samples to mean much.

Typical wrong version: `err[r["dish"]] += r["served_ok"]` (adding the text `"false"`). It fails with `TypeError: unsupported operand type(s) for +=: 'int' and 'str'`. CSV values are always strings, so you must compare or convert. Fix: `r["served_ok"] != "true"` as above.

Real-world mapping

Piece	Example	Where it appears in the job
`main(argv)`	called from tests	Any production CLI
Dataclass rows	`Order`, `DishStats`	Typed data between steps of a pipeline
Exit codes	0 / 1 / 2	Schedulers and CI
Pure core	`summarise()`	Reusable in a notebook and an API

Applications

- **Data Engineering:** a validation CLI run before every load.
- **Data Analytics:** a tool that turns raw exports into a standard report.
- **Machine Learning / MLOps:** `train.py` and `evaluate.py` with the same discipline.
- **Operations:** turning any raw export (orders, tickets, test results) into your own report format.

COMMON MISTAKE

Putting the logic inside `main()` with `print` calls and file reads. You cannot test it without files. Keep a pure function in the middle and thin input/output around it.

Check yourself

1. Why return an exit code? Answer: Schedulers and CI use it to decide pass or fail.
2. Why separate the pure function? Answer: It is easy to test and reuse with no files or printing.
3. Why does p90 of 3 samples equal the maximum? Answer: ceil(3 x 0.9) = 3, so the third (largest)

value is chosen.

14.2 Capstone 2: A FastAPI Service With a Database and Tests

INTUITION

A service is a restaurant that stays open: customers (clients) come at any time. Here you build a small one that stores kitchen shifts and serves summaries, and prove with tests that it behaves.

Specification

Build `kitchen-api`, a service to store kitchen shifts (lunch, dinner) and their order summaries.

Endpoints:

Method and path	What it does	Status codes
`POST /v1/shifts`	create a shift (`name`, `started_at`)	201, 422
`GET /v1/shifts`	list shifts with `limit` and `offset`	200
`GET /v1/shifts/{id}`	one shift	200, 404
`POST /v1/shifts/{id}/orders`	upload a list of orders	201, 404, 422
`GET /v1/shifts/{id}/summary`	per dish stats from Capstone 1	200, 404
`DELETE /v1/shifts/{id}`	delete	204

Definition of done:

- Layers: routes, a service module, a repository module (SQLite, then optional PostgreSQL).
- Pydantic models for input and output. Parameterised SQL only.
- Config from environment variables. No secrets in the repository.
- Tests with `pytest` and FastAPI's `TestClient`, using an in-memory database through a fixture; at least 12 tests, including 404, 422, pagination end and idempotent delete.
- An OpenAPI page at `/docs`, a Dockerfile (optional), and CI.

Not run here (needs `fastapi`, `httpx`, `pytest`). The shape of a test:

```
from fastapi.testclient import TestClient
from kitchen_api.main import create_app

def test_create_and_get_shift():
    client = TestClient(create_app(db_url=":memory:"))
    r = client.post("/v1/shifts", json={"name": "lunch", "started_at": "2026-
01-31T10:00:00Z"})
    assert r.status_code == 201
    shift_id = r.json()["id"]
    assert client.get(f"/v1/shifts/{shift_id}").json()["name"] == "lunch"

def test_unknown_shift_is_404():
    client = TestClient(create_app(db_url=":memory:"))
    assert client.get("/v1/shifts/999").status_code == 404
```

Milestones (about 10 hours)

1. Design the tables (`shifts` , `orders`) and write the repository with plain `sqlite3` , tested by itself.
2. Add the service layer that calls the Capstone 1 `summarise()` .
3. Add FastAPI routes and pydantic models. Look at `/docs` .
4. Write tests for every route and every error code.
5. Add pagination and the `Location` header on create.
6. Add configuration from the environment and structured logging.
7. Add CI, linting, type checks. Time 200 requests against it with a small script (or a tool such as Locust) and write down the p90 and the requests per second.

The picture

```
 client --HTTP--> [routes: validate] --> [service: rules] --> [repository: SQL]
--> SQLite
                       ^                        ^                      ^
                  pydantic models          calls summarise()    parameterised
queries
                       |                                             |
                  TestClient  <--- tests use a fake or in-memory repo --+
```

Worked example

Problem 14.2.1 Prove the design idea of the repository layer without any framework: write a `ShiftRepository` over `sqlite3` with parameterised SQL, and show create, get, and list with

pagination.

Predict: what does `list(limit=2, offset=2)` return after three shifts are created?

```
import sqlite3

class ShiftRepository:
    def __init__(self, path=":memory:"):
        self.con = sqlite3.connect(path)
        self.con.row_factory = sqlite3.Row
        self.con.execute("CREATE TABLE IF NOT EXISTS shifts("
                         "id INTEGER PRIMARY KEY, name TEXT NOT NULL)")
    def create(self, name):
        with self.con:
            cur = self.con.execute("INSERT INTO shifts(name) VALUES (?)",
(name,))
        return cur.lastrowid
    def get(self, shift_id):
        row = self.con.execute("SELECT id, name FROM shifts WHERE id = ?",
(shift_id,)).fetchone()
        return dict(row) if row else None
    def list(self, limit=10, offset=0):
        rows = self.con.execute("SELECT id, name FROM shifts ORDER BY id LIMIT
? OFFSET ?",
                                (limit, offset)).fetchall()
        return [dict(r) for r in rows]

repo = ShiftRepository()
for n in ("lunch", "dinner", "brunch"):
    print("created", repo.create(n))
print(repo.get(2))
print(repo.get(99))
print(repo.list(limit=2, offset=2))
try:
    repo.create(None)
except sqlite3.IntegrityError as e:
    print("IntegrityError:", e)
```

I ran this and the real output was:

```
created 1
created 2
created 3
{'id': 2, 'name': 'dinner'}
None
[{'id': 3, 'name': 'brunch'}]
IntegrityError: NOT NULL constraint failed: shifts.name
```

- **Step 1** `with self.con:` commits on success and rolls back on error.
- **Step 2** `ORDER BY id LIMIT 2 OFFSET 2` skips two rows and takes up to two: only shift 3 remains.
- **Step 3** The database itself rejects `None` for `name`, which is a second line of defence after pydantic.
- **Check** Three rows exist, offset 2 skips ids 1 and 2, so one row is returned.
- **Answer** The last page holds one shift, `{'id': 3, 'name': 'brunch'}`. A missing id returns `None`, which the route turns into 404.
- **Why it matters** The route code never sees SQL, so you can swap SQLite for PostgreSQL later without touching it.

Typical wrong version: forgetting `ORDER BY` with `LIMIT` / `OFFSET`. Databases do not promise an order without it, so pages can overlap or skip rows. No error message appears, which is why you must always add it.

Real-world mapping

Piece	Example	Where it appears in the job
Repository	`ShiftRepository`	Data access in any service
`TestClient`	in-memory database	Fast tests of every status code
`Location` header	`/v1/shifts/7`	Standard on 201 Created
Timing your own API	200 requests, p90 and requests per second	Shows your service is fast enough

Applications

- **Machine Learning / MLOps:** the same layout serves `POST /predict`.
- **Data Engineering:** an internal metadata service that records pipeline runs.
- **GenAI:** a small gateway that logs prompts and responses to a database (without storing personal data).
- **Backend development:** a service that you both build and time, then publish the report.

COMMON MISTAKE

Testing only the happy path. Most production bugs live in 404, 422, empty list and duplicate cases. Write those tests first.

Check yourself

1. Why is `ORDER BY` needed with `LIMIT/OFFSET` ? Answer: Without it the order is undefined, so pages can overlap or skip.
2. What does `with con:` do? Answer: Commits if no error, rolls back if there is one.
3. Why test with an in-memory database? Answer: It is fast and isolated, so each test starts clean.

14.3 Capstone 3: A Queue and Service-Time Analyser

INTUITION

Think of a restaurant manager reading the kitchen's ticket log at the end of the night. This project turns that log into a data product: it reads raw orders, finds the slow dish, and writes the conclusion in plain English. It shows you can combine domain knowledge with Python and statistics.

Specification

Build `order-analyse` , which reads the same orders CSV as Capstone 1 (or any similar export, such as bank-queue or hospital-appointment logs) and produces a short report.

It must compute:

- per dish: count, average, median, p90, p95, p99, maximum, error %;
- orders per minute overall, and per 10-minute window;
- a time series of p90 per window, to see degradation over time;
- Little's law check: average number of orders in the kitchen = arrival rate x average preparation time (in the same time unit, minutes);
- a verdict: flag any dish where p95 is over a service level agreement (SLA, a promised limit, for example 15 minutes) or failure % is over 1%.

Extra credit: compare two weeks and say whether the change is real or just noise (use a bootstrap or a Mann-Whitney test, see the statistics documents); draw a chart with matplotlib (not run here).

Definition of done: a CLI or notebook, tests with a small fixed CSV, and a one-page write-up with your findings and one chart.

Milestones (about 6 hours)

1. Load the CSV with the standard `csv` module, then repeat with pandas (not run here) and compare the code size.

2. Reuse your tested `percentile` function.
3. Group by 10-minute windows using integer division: `window = (ts - first_ts) // 600` (times are in seconds).
4. Add the Little's law line and the verdict rules.
5. Generate a Markdown report.
6. Test with a CSV where you know the answers by hand.

The picture

```
prep_seconds
  2300 |                 *  <- pasta: slow and failed
   700 |            *
   450 |       *  *
   120 |  *  *        *
       +------------------------> time
       window 0     window 1      p90 per window shows when it degrades
```

Each star is one order. The vertical axis is preparation time; the horizontal axis is when the order was placed. A rising cloud means the kitchen is slowing down as the evening gets busier.

Worked example

Problem 14.3.1 Using the 8-order data from 14.1.1, check Little's law and compute p90 per 10-minute window. Then do the same for a busier evening.

Formula	Little's law: average number in system `L = λ × W`, where `λ` is the arrival rate (orders per minute) and `W` is the average time an order spends in the kitchen, in minutes.
Plug in	λ = 8 / 31.67 = 0.2526 per minute (from 14.1.1). Overall average prep time = (120 + 95 + 2300 + 180 + 450 + 510 + 130 + 700) / 8.
Compute	The sum is 4485 seconds, so the average is 4485 / 8 = 560.6 s = 9.34 minutes. Then `L = 0.2526 × 9.34 = 2.36`.
Answer	On average about 2.4 orders were in the kitchen at once.
Check	Another way: total time spent by all orders is 4485 s = 74.75 minutes over a 31.67 minute window, so 74.75 / 31.67 = 2.36. Both agree.
Why it matters	If the kitchen has 10 cooks but Little's law says only 2.4 orders are in progress on average, most cooks are idle, so you may be over-staffed (or the orders are not reaching the kitchen). This check catches wrong assumptions about how busy a system really is.

Python check for the arithmetic:

```
prep = [120, 95, 2300, 180, 450, 510, 130, 700]
avg_min = sum(prep) / len(prep) / 60
lam = 8 / (1900 / 60)
print(sum(prep), round(avg_min, 2), round(lam, 4), round(lam * avg_min, 2),
round(sum(prep) / 60 / (1900 / 60), 2))
```

```
4485 9.34 0.2526 2.36 2.36
```

Typical wrong version: mixing units: using seconds for W and minutes for the arrival rate. You would get `0.2526 x 560.6 = 141.6`, which is 60 times too large, with no error. Always convert to the same unit first and check the unit.

Real-world mapping

Piece	Example	Where it appears in the job
Percentiles per dish	p95 of `pasta`	SLA reports
Arrival rate per window	orders per 10 minutes	Finding the busy hours
Little's law	L = λ × W	Checking how busy the system is

Verdict rules	p95 > 15 min or failures > 1%	Automatic pass/fail in a daily check

Applications

- **Data Analytics:** a reusable dashboard of delivery-time distributions.
- **Data Science:** a significance test for "is the new recipe really faster?".
- **MLOps:** delay and error analysis of a model endpoint after each release.
- **Operations:** a nightly queue-time report, automated and made reproducible.

COMMON MISTAKE

Reporting only the average. A few very slow requests hide behind a pleasant mean. Always show percentiles and error rate together.

Check yourself

1. State Little's law. Answer: Average number in the system equals throughput times average time in the system.
2. Why percentiles per window? Answer: They show when and how the system degrades over time.
3. What unit must W be in if throughput is per second? Answer: Seconds.

14.4 Capstone 4: A Responsible Scraper

INTUITION

This project is about manners as much as code. You collect public data without harming the site or the people in the data, and you can show your rules.

Specification

Build `polite-scrape`, which collects titles and prices from a **site you are allowed to scrape** (your own test site, a practice site made for scraping, or one whose terms allow it). You may start with the local HTML page of Chapter 12 and later move to a real allowed site.

Rules the program must enforce:

1. It reads `robots.txt` first and refuses disallowed URLs.
2. It sends a `User-Agent` with a contact.
3. It waits at least the crawl delay (default 2 seconds) between requests.

4. It caches each page on disk, so a rerun makes no new requests.
5. It stops on 429 or 503 and backs off.
6. It stores only the fields needed, and no personal data.

Definition of done: tests that use a local fake server (no real network), a README that states which site, why scraping is allowed, and the rate used.

Milestones (about 5 hours)

1. Parse the demo HTML with `html.parser`, then repeat with BeautifulSoup (not run here).
2. Add the `robots.txt` check using `urllib.robotparser`.
3. Add the delay and the cache.
4. Add error handling and backoff.
5. Write tests against a local `http.server`.
6. Write the README with the policy.

The picture

Read it as a loop. Every request passes through the same gate.

```
 url list --> [robots allows?] --no--> skip + log
                    | yes
              [in cache?] --yes--> parse cached html
                    | no
              [wait crawl-delay] --> fetch --> status ok? --no(429/503)--> back
off
                                                       | yes
                                          save to cache --> parse --> store rows
```

Worked example

Problem 14.4.1 Implement the gate and cache logic with no network: a function that decides whether to fetch, using a robots text and a dictionary as cache.

Predict: for the three URLs below, which are fetched, which are skipped, and which come from cache?

```
from urllib.robotparser import RobotFileParser

ROBOTS = """User-agent: *
Disallow: /private/
Crawl-delay: 2
"""
rp = RobotFileParser(); rp.parse(ROBOTS.splitlines())
cache = {"https://site.test/b": "<html>cached b</html>"}
UA = "StudyBot/1.0 (me@example.com)"

def plan(url):
    if not rp.can_fetch(UA, url):
        return "SKIP (robots)"
    if url in cache:
        return "CACHE"
    return "FETCH (wait %ss first)" % rp.crawl_delay(UA)

for u in ["https://site.test/a", "https://site.test/b",
"https://site.test/private/x"]:
    print(u, "->", plan(u))
```

```
https://site.test/a -> FETCH (wait 2s first)
https://site.test/b -> CACHE
https://site.test/private/x -> SKIP (robots)
```

- **Step 1** `can_fetch` is checked first, because a disallowed URL must never be fetched, even from cache logic.
- **Step 2** The cache is a dictionary here. In the project it becomes files named by a hash of the URL.
- **Step 3** Only `/a` needs a real request, and it must wait the crawl delay first.
- **Check** Three URLs, three different outcomes, and exactly one real request, which for 3 pages at a 2 second delay is the least load possible.
- **Answer** `/a` is fetched after a 2 second wait, `/b` comes from the cache, `/private/x` is skipped.
- **Why it matters** Reruns during development cost the site nothing, and your bot cannot wander into places the owner closed.

Typical wrong version: testing against the real site every time while you debug. That can get you blocked. Save one page to disk and develop against the file.

Real-world mapping

Piece	Example	Where it appears in the job

robots check	`can_fetch`	A legal and ethical gate before any crawl
Cache	files keyed by hash	Saves requests and speeds up development
Backoff	wait longer after 429	How production crawlers stay polite
Policy README	site, reason, rate	What a reviewer or lawyer asks to see

Applications

- **Data Analytics:** a small, lawful price dataset for an analysis.
- **Data Engineering:** the same gate-and-cache design for a scheduled ingestion from an API with rate limits.
- **GenAI:** collecting permitted documents for a retrieval project, with a record of source and licence.
- **Machine Learning:** building a training set with documented origin (data provenance).

COMMON MISTAKE

Saying "it is public, so I can take it". Public does not mean free to copy. Check the terms, robots.txt, copyright and privacy rules, and prefer APIs and open datasets.

Check yourself

1. What do you check before the first request? Answer: robots.txt and the terms of service.
2. Why cache pages? Answer: It avoids repeated requests and lets you develop offline.
3. What should the bot do on a 429? Answer: Stop or back off for longer, and respect `Retry-After` if given.

14.5 The Mastery Tests and the 150-Problem Plan

INTUITION

Strength comes from repetition with feedback, like going to the gym. A plan of 150 problems sounds big. At two problems a day it is about 11 weeks, which fits your one hour a day.

The four mastery tests

From your skill map, you are "Python-strong" when you can do all four:

1. Write a CLI tool with argparse, logging, tests and a typed API. (Capstone 1.)
2. Build a small FastAPI service with a database and tests. (Capstone 2.)

3. Explain generators, decorators, the GIL and the MRO without notes. (Use the interview bank below and teach a friend.)
4. Solve 150+ Python or LeetCode problems.

The 150-problem plan

Weeks	Topic	Problems	Examples of patterns
1-2	Strings and arrays	25	two pointers, sliding window, counting
3-4	Hash maps and sets	25	two-sum, grouping, frequency
5	Stacks, queues, linked lists	20	valid parentheses, queue by stack
6	Sorting, binary search, intervals	20	merge intervals, search in a rotated array
7	Recursion, backtracking	15	subsets, permutations
8	Trees and graphs (BFS, DFS)	20	shortest path in a grid, level order
9	Heaps and greedy	10	top-k, schedule
10-11	Dynamic programming	15	climbing stairs, knapsack, edit distance

Method for each problem: (1) read and restate it, (2) work an example by hand, (3) write the simple solution, (4) state time and space cost, (5) improve it, (6) a day later, solve it again from memory. Keep a log with the pattern name and what you missed. Chapter 1 of the DSA document explains complexity in depth; here we focus on Python.

Worked examples (run)

Problem 14.5.1 Solve six classic problems and check every answer by hand.

Predict: what are the six outputs?

```
from collections import Counter, deque
import heapq

def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
```

```
def valid_parens(s):
    pairs = {")": "(", "]": "[", "}": "{"}
    st = []
    for ch in s:
        if ch in "([{": st.append(ch)
        elif not st or st.pop() != pairs[ch]: return False
    return not st

def top_k(words, k):
    c = Counter(words)
    return [w for w, _ in sorted(c.items(), key=lambda kv: (-kv[1], kv[0]))
[:k]]

def merge_intervals(iv):
    out = []
    for s, e in sorted(iv):
        if out and s <= out[-1][1]: out[-1][1] = max(out[-1][1], e)
        else: out.append([s, e])
    return out

def bfs_shortest(grid, start, goal):
    q = deque([(start, 0)]); seen = {start}
    while q:
        (r, c), d = q.popleft()
        if (r, c) == goal: return d
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r+dr, c+dc
            if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]) and grid[nr][nc]
== 0 and (nr,nc) not in seen:
                seen.add((nr,nc)); q.append(((nr,nc), d+1))
    return -1

def anagram_groups(ws):
    g = {}
    for w in ws: g.setdefault("".join(sorted(w)), []).append(w)
    return list(g.values())

print(two_sum([2, 7, 11, 15], 9))
print(valid_parens("{[()]}"), valid_parens("([)]"))
print(top_k("a b a c b a d".split(), 2))
print(merge_intervals([[1,3],[8,10],[2,6],[15,18],[17,20]]))
grid = [[0,0,0],[1,1,0],[0,0,0]]
print(bfs_shortest(grid, (0,0), (2,0)))
print(anagram_groups(["eat","tea","tan","ate","nat","bat"]))
print(heapq.nlargest(3, [5, 1, 9, 3, 7]))
```

```
[0, 1]
True False
['a', 'b']
[[1, 6], [8, 10], [15, 20]]
6
[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
[9, 7, 5]
```

- **Two sum** 2 + 7 = 9, at indexes 0 and 1. The dictionary lets us find the partner in one step per item, so time is O(n) instead of O(n squared) for two nested loops.
- **Valid parentheses** `{[()]}` : each closer matches the most recent opener (stack, last in first out), so it is valid. In `([)]` the `)` meets `[` on top of the stack, so it is invalid.
- **Top k** Counts are a=3, b=2, c=1, d=1. The top 2 are `a` and `b` .
- **Merge intervals** After sorting: [1,3], [2,6], [8,10], [15,18], [17,20]. [1,3] and [2,6] overlap, so they merge into [1,6]. [8,10] stands alone. [15,18] and [17,20] merge into [15,20].
- **BFS in a grid** From (0,0) to (2,0), the middle row is blocked except at column 2. The path is (0,0), (0,1), (0,2), (1,2), (2,2), (2,1), (2,0): 6 steps. BFS visits cells in order of distance, so the first time we reach the goal it is the shortest.
- **Anagram groups** Sorting letters gives the same key for words with the same letters.
- **Answer** All outputs match the hand checks.
- **Check** Each result above was verified by hand in the bullets.
- **Why it matters** These six patterns (hash map, stack, counting, sort and sweep, BFS, grouping by key) cover a large share of interview questions.

Typical wrong version of valid parentheses: forgetting `not st` . For input `")"` the code would call `st.pop()` on an empty list and fail with `IndexError: pop from empty list` . The fix is the `not st or ...` guard.

Real-world mapping

Pattern	Example	Where it appears in the job
Hash map	two-sum, counting	Joins and group-bys in code form
Stack	parentheses	Parsing, undo, expression evaluation
Sort and merge	intervals	Merging overlapping time windows in logs
BFS	shortest path	Dependency graphs, lineage
Heap	top-k	Top slow requests, leaderboards

Applications

- **Data Analytics:** counting and grouping logic behind quick scripts.
- **Data Engineering:** interval merging for sessionisation and gap detection.
- **Machine Learning:** nearest neighbours and top-k retrieval use heaps and hash maps.
- **Interviews at all levels:** the screening round is usually one or two of these patterns.

COMMON MISTAKE

Reading solutions before trying. You learn by struggling for 20 to 30 minutes first. Then read, then re-solve from a blank page the next day.

Check yourself

1. Why is the two-sum dictionary approach fast? Answer: Looking up the partner in a dict is on average constant time, so one pass is enough.
2. Which structure gives "last in, first out"? Answer: A stack (a Python list with `append` and `pop`).
3. Why does BFS find the shortest path in an unweighted grid? Answer: It explores cells in order of increasing distance.

14.6 The Python Interview Bank (45 Graded Questions)

INTUITION

An interview is a conversation where you show how you think. The best answers are short, correct, and come with a tiny example. Practise saying them aloud in simple English.

How to use it

Cover the answer, say your answer aloud, then compare. Levels: **Easy** (you must know these cold), **Medium** (expected for a junior to mid role), **Hard** (separates strong candidates). Some answers include code that was run; the outputs are real.

Easy (questions 1 to 15)

Q1. What is the difference between a list and a tuple? **Answer:** A list is mutable (you can change it) and a tuple is immutable. Tuples can be dictionary keys if all their items are hashable, and they signal "this should not change". Careful: a tuple can hold a mutable item; `t = (1, [2]); t[1].append(3)` makes `t` become `(1, [2, 3])` (I ran this).

Q2. What is the difference between `is` and `==` ? **Answer:** `==` asks "are the values equal?" and `is` asks "is it the same object in memory?". Use `is` for `None` (`x is None`). Do not use `is` to compare numbers or strings.

Q3. What is the output of `0.1 + 0.2 == 0.3` ? **Answer:** `False` . Floats are stored in binary and 0.1 is not exact. Compare with `math.isclose` or round. (I ran it: `round(0.1+0.2, 10) == 0.3` is `True` .)

Q4. Name Python's main built-in collection types. **Answer:** `list` , `tuple` , `set` , `frozenset` , `dict` ; and `str` and `bytes` are sequences. `collections` adds `deque` , `Counter` , `defaultdict` .

Q5. Is a `dict` ordered? **Answer:** Yes. Since Python 3.7 a dict keeps insertion order as part of the language.

Q6. What is the mutable default argument problem? **Answer:** Default values are created once, when the function is defined. So `def f(x, acc=[])` shares one list across calls. I ran it: `print(f(1), f(2))` printed `[1, 2] [1, 2]` , both names point at the same list. Fix: `acc=None` and then `if acc is None: acc = []` .

Q7. What does `*args` and `**kwargs` mean? **Answer:** `*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dict. They also unpack when calling: `f(*lst, **d)` .

Q8. What is a list comprehension, and give an example. **Answer:** A short way to build a list: `[x*x for x in range(5) if x % 2 == 0]` is `[0, 4, 16]` . Dict, set and generator versions exist too.

Q9. What is the difference between a list and a generator expression? **Answer:** A list holds all items in memory now. A generator makes them one at a time when asked (lazy). `(x*x for x in range(10**6))` uses almost no memory. In a run, `next(gen)` gave `0` and then `1` .

Q10. What does `with open(...) as f:` do? **Answer:** It is a context manager: the file is closed automatically, even if an error happens inside the block.

Q11. What is the difference between `append` and `extend` ? **Answer:** `append(x)` adds one item (a list stays one item). `extend(iterable)` adds each element. `[1].append([2])` gives `[1, [2]]` ; `[1].extend([2])` gives `[1, 2]` .

Q12. What is `__name__ == "__main__"` for? **Answer:** The code under it runs only when the file is run directly, not when it is imported. It makes a file both a script and a module.

Q13. What happens when you slice beyond the end, like `x[5:]` for a 3-item list? **Answer:** Slices are forgiving and return an empty list: `[]` . Indexing a single item beyond the end raises `IndexError` . (I ran `x[5:]` , it printed `[]` .)

Q14. What is the difference between `7 // 2`, `-7 // 2` and `7 % -3`? **Answer:** `//` is floor division, so they are `3`, `-4` (floor of -3.5) and `-2` (the sign follows the divisor). I ran all three.

Q15. How do you read a traceback? **Answer:** Read from the bottom: the last line says the error type and message. The lines above show the chain of calls, with the last call closest to the bottom. Find the lowest line that is in your code. That is usually where to look first.

Medium (questions 16 to 30)

Q16. Explain shallow copy and deep copy. **Answer:** A shallow copy makes a new outer container but shares the inner objects. A deep copy copies everything recursively. I ran: after `a = [[1,2],[3]]`, `s = copy.copy(a)`, `d = copy.deepcopy(a)` and `a[0].append(99)`, the shallow copy showed `[[1, 2, 99], [3]]` and the deep copy `[[1, 2], [3]]`.

Q17. What is a closure, and what is the late-binding trap? **Answer:** A closure is an inner function that remembers variables from the outer function. Trap: `fs = [lambda: i for i in range(3)]` gives `[2, 2, 2]` when called, because all lambdas look up `i` at call time and the loop ended with `i = 2`. Fix: bind it with a default: `lambda i=i: i` gives `[0, 1, 2]` (I ran both).

Q18. What is a decorator? Write one that keeps the function's name. **Answer:** A decorator is a function that takes a function and returns a new function, usually adding behaviour (timing, logging, caching, retry). Use `functools.wraps` so the name and docstring survive. I ran a timing decorator: `add.__name__` printed `add` and `add.__doc__` printed `adds`. Without `@functools.wraps`, the name would show `w`.

Q19. Explain generators and `yield`. **Answer:** A function with `yield` returns a generator: it pauses at each `yield`, keeps its state, and resumes on the next `next()`. It is lazy, so large or infinite streams use little memory. Pipelines chain generators. I ran a three-stage pipeline: `list(gen_pipeline(["a ", "", " b"]))` gave `['A', 'B']`: strip, drop blanks, uppercase, one item at a time.

Q20. What is the difference between an iterator and an iterable? **Answer:** An iterable can give you an iterator (it has `__iter__`, like a list). An iterator has `__next__` and remembers its position. `iter(x)` converts the first into the second. A `for` loop does this for you.

Q21. What is the MRO and how does `super()` work with multiple inheritance? **Answer:** MRO (Method Resolution Order) is the order Python searches classes for a method, computed by the C3 rule so that a class comes before its parents and the order of bases is respected. `super()` means "the next class in the MRO", not "the parent". In my run of a diamond (D inherits B and C, both inherit A) `D().who()` printed `D>B>C>A` and the MRO was `['D', 'B', 'C', 'A', 'object']`, so `B`'s `super()` call reached `C`, a sibling.

Q22. What is `lru_cache` and when do you use it? **Answer:** A decorator from `functools` that

remembers results of a function by its arguments (memoisation). Use it for pure functions that are costly and called with repeated arguments. I ran `fib(50)` : it printed `12586269025` and `CacheInfo(hits=48, misses=51, ...)` meaning only 51 real computations. Arguments must be hashable.

Q23. How do exceptions chain, and what does `raise ... from` do? **Answer:** `raise NewError(...) from e` links the original cause, which Python shows in the traceback. I ran: converting `ValueError` into `RuntimeError('bad config')` kept `__cause__` as the `ValueError` with its message. It keeps the root cause while giving callers a meaningful error type. Use `logging.exception` inside `except` to log with the traceback.

Q24. What is EAFP versus LBYL? **Answer:** EAFP: "Easier to Ask Forgiveness than Permission" (`try: d[k] except KeyError`). LBYL: "Look Before You Leap" (`if k in d:`). Python style prefers EAFP in many cases, and EAFP also avoids race conditions (the file may vanish between the check and the use).

Q25. How does a context manager work? Write one. **Answer:** `with` calls `__enter__` at the start and `__exit__` at the end, even after an error. If `__exit__` returns true, the exception is suppressed. I ran a class that printed `enter` , then `exit ZeroDivisionError` , and the program continued with `after` . You can also write one with `@contextlib.contextmanager` and a `yield` .

Q26. What is the difference between `@staticmethod` , `@classmethod` and a normal method? **Answer:** A normal method receives the instance (`self`). A `classmethod` receives the class (`cls`), useful for alternative constructors (`Settings.from_env()`). A `staticmethod` receives nothing special; it is a plain function placed in the class for organisation.

Q27. What do `__repr__` and `__str__` do? **Answer:** `__repr__` is for developers, unambiguous, ideally looking like code. `__str__` is for users. `print` uses `__str__` and falls back to `__repr__` . Always define `__repr__` (a `dataclass` does it for you).

Q28. When would you use a dataclass? **Answer:** For classes that mostly hold data. `@dataclass` generates `__init__` , `__repr__` and `__eq__` . Options: `frozen=True` for immutable (hashable) objects, `field(default_factory=list)` for mutable defaults, `__post_init__` for validation.

Q29. What is the difference between `pytest` fixtures and `setUp` ? **Answer:** A fixture is a function that provides test data or objects, requested by name as a test parameter; it can have scope (function, module, session) and cleanup after `yield` . It is more flexible and reusable than `unittest` 's `setUp` / `tearDown` methods.

Q30. How do you mock an external call in a test? **Answer:** Replace it with a stand-in, with `unittest.mock.Mock` or `patch` (in pytest, also `monkeypatch`). In my run (Chapter 13), `Mock().send` recorded `call('order 1: 400')` and `assert_called_once_with` verified it. Prefer a fake when behaviour matters, a mock when you only need to check the call, and inject

dependencies so mocking stays simple.

Hard (questions 31 to 45)

Q31. Explain the GIL and its consequences. **Answer:** The Global Interpreter Lock in standard CPython lets only one thread run Python bytecode at a time. Consequences: threads do not speed up CPU-bound pure-Python code, but they do help I/O-bound work because the GIL is released while waiting on I/O. For CPU-bound work use `multiprocessing` (separate processes), native code (NumPy and similar release the GIL), or vectorisation. Recent Python versions have an optional free-threaded build, so check what your Python version supports.

Q32. Threads, processes or asyncio: how do you choose? **Answer:** Many network waits and moderate count: `asyncio` (one thread, cooperative tasks, scales to thousands) or threads with `concurrent.futures.ThreadPoolExecutor` if libraries are blocking. CPU-bound: processes (`ProcessPoolExecutor`). Always measure first. Costs: processes need data to be pickled and use more memory; asyncio needs async-aware libraries; threads need locks for shared state.

Q33. What is a race condition and how do you prevent one in Python? **Answer:** Two threads change shared data and the result depends on timing. Even `counter += 1` is not atomic (read, add, write). Prevent with a `threading.Lock`, a `queue.Queue` to pass work, or by avoiding shared state (use processes or immutable data).

Q34. How does Python manage memory? **Answer:** Objects are allocated on a heap and tracked by reference counts. When the count reaches zero, the object is freed at once. A cyclic garbage collector finds objects that reference each other in a cycle. `__slots__` removes the per-instance `__dict__` and saves memory for many small objects. `sys.getsizeof` shows the size of one object only, not of what it points to, and the numbers differ between versions.

Q35. What are `*` and `**` unpacking, and keyword-only and positional-only parameters? **Answer:** In `def f(a, b, /, c, *, d):` `a` and `b` are positional-only (before `/`), `c` can be either, and `d` is keyword-only (after `*`). Keyword-only arguments make calls readable: `f(1, 2, 3, d=4)`.

Q36. What is structural pattern matching? **Answer:** `match` / `case` (Python 3.10 and later) compares a value to patterns, including shapes and unpacking: `case {"type": "click", "x": x}:` or `case [first, *rest]:`. It is more than a switch; it can destructure lists, dicts and classes. Use it for parsing structured data like events.

Q37. Explain SOLID with one line each and one Python example. **Answer:** S: a class has one reason to change (separate parsing from reporting). O: extend by adding code, not editing (new strategy in a dict of strategies). L: subclasses must work where the parent works (a `Square` that breaks a `Rectangle`'s contract violates it). I: small interfaces (a `Protocol` with only `read()`, not a giant base class). D: depend on abstractions (inject a repository, not a concrete database), as in Chapter 13.

Q38. Describe three design patterns you have used in Python. **Answer:** Strategy: pass a function or choose from a dict, such as `{"max": max, "p90": ...}` (I ran this and got `{'max': 300, 'min': 80, 'p90': 300}`). Factory: a function that returns the right object by name. Singleton: usually a module-level object or `lru_cache`d function (I ran `get_config()` and it printed "loading config once", and `get_config() is get_config()` was `True`). Also Observer (event bus) and Repository (Chapter 13).

Q39. Composition versus inheritance: which and why? **Answer:** Prefer composition ("has a") over inheritance ("is a") unless there is a true "is a" relationship. Inheritance couples child to parent, and deep hierarchies are hard to change. Composition lets you swap parts (a `Pipeline` that has a `Reader` and a `Writer`). Mixins and `Protocol`s are the middle path.

Q40. What is the difference between `abc.ABC` and `typing.Protocol`? **Answer:** An ABC requires explicit inheritance and can force subclasses to implement abstract methods (it fails at object creation if they do not). A `Protocol` is structural ("duck typing" checked by type checkers): any class with the right methods fits, with no inheritance needed. Use ABCs when you want shared code and enforcement, Protocols for flexible interfaces.

Q41. How do you make a Python program faster? Walk through your process. **Answer:** (1) Measure first: `timeit` for small pieces, `cProfile` for the whole program, and `line_profiler` for hot lines. (2) Fix the algorithm and data structure (a `set` or `dict` lookup instead of a list scan). (3) Use built-ins and vectorised libraries (NumPy, pandas) rather than Python loops. (4) Reduce I/O and cache repeated results. (5) Only then consider concurrency, Numba or Cython. This is the same loop you use in performance testing: measure, find the bottleneck, change one thing, measure again.

Q42. What is the N+1 problem and how does it relate to APIs and databases? **Answer:** You make one query for a list and then one more query per item (1 + N calls). It is slow because each call has a round-trip cost. Fix: fetch in one query (a join or `IN` list), or batch API calls. A request log shows it as a huge request count per user action.

Q43. Explain how you would design pagination, idempotency and retries for an API client that loads data every night. **Answer:** Page with limit and a cursor (or offset if the data is static) until the next marker is empty. Always set timeouts, retry only temporary errors (429, 503, connection errors) with exponential backoff and respect `Retry-After`. Make loads idempotent (use upserts or a unique key, and an idempotency key for POSTs), so a rerun after a crash does not duplicate data. Log counts per page and verify the total at the end.

Q44. A teammate pushed an API key to GitHub. What do you do? **Answer:** (1) Revoke and rotate the key now, because history and clones keep it. (2) Check logs for misuse. (3) Remove it from the repo, and clean the history if policy requires. (4) Move secrets to environment variables or a secret manager. (5) Add a scanner (gitleaks) to pre-commit and CI. (6) Hold a blameless review about how to prevent it.

Q45. How do you test code that talks to a database or the network? **Answer:** Follow the testing pyramid: many fast unit tests (pure functions, with fakes or mocks for the edges), fewer integration tests (a real database in a container or in-memory SQLite, a local fake server), and very few end-to-end tests. Inject dependencies so you can substitute them, use fixtures for setup and cleanup, use `parametrize` for many inputs, and run it all in CI. Measure coverage but do not worship the number.

The picture

```
Easy (15)    ->  vocabulary and traps        -> must be instant
Medium (15)  ->  how Python works inside     -> explain with a small example
Hard (15)    ->  design, trade-offs, process -> show your judgement
```

Worked example

Problem 14.6.1 Practise answering "Explain the GIL in 60 seconds" with structure. Use the pattern: define it, say the consequence, give the fix, give one number or example.

Step 1 (define)	"The GIL is a lock in CPython that lets one thread run Python code at a time."
Step 2 (consequence)	"So threads do not make pure-Python number crunching faster, but they do help when the program waits on the network or disk, because the lock is released while waiting."
Step 3 (fix)	"For CPU-bound work I use processes, or vectorised libraries like NumPy that do the work outside Python."
Step 4 (judgement)	"I measure first, because for most data jobs the bottleneck is I/O or the database, not the GIL."
Answer	The four sentences take about 30 seconds aloud and cover definition, consequence, remedy and judgement. Interviewers listen for judgement most.
Check	Time yourself with a phone. If you need more than 90 seconds, cut examples. If under 20, add the consequence.
Why it matters	The structure works for almost any "explain X" question: define, consequence, fix, judgement.

Real-world mapping

Question type	Example	What the interviewer checks

Trap	mutable default	Have you been bitten before?
Internals	MRO, GIL	Do you understand, not just use?
Design	SOLID, patterns	Can you build things others maintain?
Process	speed-up plan, leaked key	Will you act calmly and correctly?

Applications

- **Data Analytics roles:** easy and medium questions plus pandas and SQL.
- **Data Engineering roles:** medium and hard questions on idempotency, concurrency, testing.
- **Machine Learning and MLOps roles:** design, packaging, CI/CD, testing of data and models.
- **GenAI roles:** async calls, API clients, retries, secrets, evaluation code.

COMMON MISTAKE

Memorising answers word for word. Interviewers ask follow-ups ("why?") and you freeze. Learn the idea, keep one tiny code example, and say it in your own simple words.

Check yourself

1. In `def f(a, b, /, c, *, d)`, which parameters are keyword-only? Answer: Only `d`.
2. Does `super()` mean "the parent class"? Answer: No, it means the next class in the MRO.
3. What is the first step in speeding up code? Answer: Measure it (profile) to find the real bottleneck.

Chapter recap

- Four capstones prove your skills: a typed and tested CLI, a FastAPI service with a database, a queue and service-time analyser, and a responsible scraper.
- Keep logic in pure functions and put input/output at the edges. That makes code testable.
- Little's law (L = lambda x W) is a quick sanity check on any queue, from a kitchen to a server. Watch the units.
- A polite scraper checks robots.txt, identifies itself, waits, caches, and stores only what it needs.
- The 150-problem plan is about 2 problems a day for 11 weeks, with a log and spaced repetition.
- Six patterns (hash map, stack, counting, sort and sweep, BFS, grouping) solve a large part of screening questions.
- In interviews: define, give the consequence, show the fix, and show judgement.
- Easy questions must be instant, medium need a small example, hard need trade-offs.

Cheat sheet

Need	Pattern
Nearest-rank percentile	`xs[max(0, ceil(n*p/100) - 1)]` after sorting
Throughput	`n / ((last_ts - first_ts) / 1000)`
Little's law	`L = throughput * avg_seconds`
Two-sum	dict `seen[value] = index`
Valid parentheses	stack plus `pairs` dict
Merge intervals	sort, then extend last if `start <= last_end`
Shortest path (unweighted)	BFS with a `deque` and a `seen` set
Explain-X answer	define, consequence, fix, judgement

Practice set

Problem 14.P.1 (easy) For the 8 orders in 14.1.1, what is the p90 of `salad` and why?

Answer: `salad` has 3 orders (95, 120, 180 sorted). ceil(3 x 0.9) = 3, index 2, value 180.

Problem 14.P.2 (easy) What does `merge_intervals([[1,4],[4,5]])` return?

Answer: `[[1, 5]]`. The intervals touch at 4 (`s <= last_end`), so they merge.

Problem 14.P.3 (easy) What is the output? `print(valid_parens("(()"))`

Answer: `False`. After reading all characters one `(` is still on the stack, so `not st` is false.

Problem 14.P.4 (medium) A cafe is open for 60 minutes and serves 300 orders. The average order spends 4 minutes from placing to serving. How many orders are in progress on average, by Little's law?

Answer: Arrival rate = 300 / 60 = 5 per minute. W = 4 minutes. L = 5 x 4 = 20 orders in progress on average.

Problem 14.P.5 (medium) Find the bug: `def add_order(name, orders=[]): orders.append(name); return orders`.

Answer: The default list is shared between calls. Use `orders=None` and create a new list inside when it is `None`.

Problem 14.P.6 (medium) In Capstone 4, your bot receives `Crawl-delay: 5` and you want 120 pages, none cached. Minimum time?

Answer: 120 x 5 = 600 seconds, which is 10 minutes (if you fetch one page at a time).

Problem 14.P.7 (medium) `top_k("x y x z y x".split(), 2)` with the function from 14.5.1. What is returned?

Answer: Counts are x=3, y=2, z=1, so `['x', 'y']`.

Problem 14.P.8 (medium) Your analyser says p95 delivery time of the pasta orders is 23 minutes and the promise is 15 minutes, but the average is 11 minutes. Which do you report and why?

Answer: Report the p95 as the SLA breach, along with the average and the failure rate. The average hides the slow tail that real customers feel.

Problem 14.P.9 (hard, interview) A BFS on a grid returns -1 for a start and goal that you think are connected. List three things to check.

Answer: (1) Bounds checking and the direction list (typo in the four moves). (2) The meaning of 0 and 1: is 0 free and 1 a wall, and is the goal cell itself blocked? (3) Whether `seen` is updated when you push to the queue (not only when you pop); and whether rows and columns are mixed up for non-square grids.

Problem 14.P.10 (hard, interview) You must process a 50 GB log file in Python on a laptop with 16 GB RAM and produce counts per URL. How?

Answer: Stream it: read line by line with a generator, update a `Counter` (memory grows only with the number of distinct URLs), or process in chunks, or push into SQLite or DuckDB. Never `f.read()` the whole file. If distinct URLs are huge, aggregate in chunks and merge results on disk. If it needs speed, split the file and use processes.

Problem 14.P.11 (hard, scenario) Your FastAPI service returns correct data but p95 response time rises sharply at 200 simultaneous users, while CPU stays at 30%. List what you check.

Answer: Low CPU with high latency means waiting. Check database connection pool size and slow queries (N+1, missing index), blocking calls inside `async def` handlers (they stall the event loop), downstream API timeouts, thread pool limits, and lock contention. Profile and trace one slow request, change one thing, and measure again.

Prompt pack

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional at an
Indian company moving into data and AI roles, and English is my second
language, so use simple words. Help me plan the capstone "command-line tool
that summarises a CSV of shop or kitchen orders, with argparse, logging, pytest
tests and type hints". Break it into 8 one-hour milestones, tell me the exact
definition of done for each, and then guide me through milestone 1 only. Ask me
to paste my code afterwards so you can review it kindly and specifically.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data and AI roles, and English is my second language. Quiz me one question
at a time from a Python interview bank of 45 questions graded easy, medium and
hard, covering types, mutability, closures, decorators, generators, MRO, GIL,
asyncio, testing and design. Start at easy. After each answer give a score out
of 5, a model answer in simple English, and one follow-up question like a real
interviewer would ask.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional learning
Python for data jobs, and English is my second language. Give me a 4-week plan
to solve 60 practice problems on arrays, strings, hash maps, stacks, intervals
and BFS, one hour a day. For week 1, give 12 graded problems without solutions,
and for each one tell me the pattern name to learn first. When I paste my
solution, review it, tell me the time and space cost, and show a cleaner
version.
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional moving
into data roles, and English is my second language. Review my capstone project:
a FastAPI service with a SQLite or PostgreSQL database and pytest tests. Check
layering (routes, service, repository), parameterised SQL, status codes (404,
422, 201 with Location), secrets and configuration, pagination, and the test
cases for failure paths. List the five most important improvements in order,
with short code fixes. My code: [PASTE HERE]
```

✨ AI prompt: copy and paste into any AI

```
You are my patient, practical professor. I am a working professional preparing
for data engineering interviews, and English is my second language. Run a 30-
minute mock interview that mixes a data-analysis case on order or queue times
(percentiles, arrival rate, Little's law, failure rate), one live Python coding
problem, and one design question about an API client that loads data every
night with retries and idempotency. Ask one question at a time, grade me out of
```

```
5, give a model answer, and coach me on how to explain it clearly and calmly in English.
```

CHAPTER 15

PROJECT LAB: FOUR PORTFOLIO PROJECTS, BUILT AND TESTED STEP BY STEP

How to plan, build, test and present small Python projects that an employer can open, run and trust.

A certificate says you studied. A repository says you can build. When a hiring manager opens your GitHub page, they spend about two minutes there: they read the README, look at the folder layout, run one command, and glance at the tests. This chapter gives you four projects that survive that two-minute check, and a method for running any project well. By the end you will be able to plan a project with measurable success criteria, lay out a repository, write tests before you trust your code, and explain your work in a README, a short post and an interview. Chapter 14 gave four capstones. This lab adds four more: the projects here use neutral, everyday data (spending records, shop orders, product pages) so that they show general Python skill that any employer in data, analytics or software recognises.

Words you need. **Repository (repo)** is a folder of code tracked by git. **Commit** is one saved snapshot with a message. **Package** is a folder of Python code that others can install and import. **Fixture** is fixed test data you control. **Definition of done** is the checklist that must be true before you call a project finished.

Everything marked "run" was executed with Python 3.14 using the standard library, plus `pandas` for Project 2. The tests use `unittest` (built into Python) because `pytest` was not installed on this study machine. Chapter 10 shows `pytest`, and every test below converts to it by removing the class. FastAPI, `requests`, `BeautifulSoup`, `pip install -e .` and GitHub features were **not run here**. Check each tool's current documentation, because commands change between versions.

15.1 How to Run a Project in This Area

INTUITION

Building a house without a plan wastes bricks. Building a project without a plan wastes weekends. A good plan has three parts: the question the project answers, the evidence that says "it works", and the small steps in between. Do the thinking first, and the typing becomes easy.

Step 1: Plan the question and the success criteria

Write three lines before you open an editor:

1. **Who is the user?** ("Me, tracking monthly spending." "An analyst who receives messy order files.")
2. **What question does it answer or what job does it do?** One sentence.
3. **How will I know it works?** Numbers, not feelings. "Summary totals match a hand calculation on 6 sample rows." "All 5 tests pass." "A new user installs and runs it from the README in under 5 minutes."

Then list what is **out of scope**. A project that tries to do everything is never finished. Two weeks of one hour a day is a good size for each project below.

Step 2: Repository layout

Use the same skeleton for every project, so reviewers know where to look.

```
project-name/
  README.md              <- what, why, how to run (see template below)
  pyproject.toml         <- name, version, dependencies (packages only)
  .gitignore             <- .venv/, __pycache__/, *.db, data/raw/
  src/ or package/       <- your code, split into small modules
    core.py              <- logic with no printing, no argparse
    cli.py               <- thin layer that talks to the user
  tests/
    __init__.py
    test_core.py         <- fast tests with small fixed data
  data/
    sample/              <- tiny files you commit
    raw/                 <- big files you do NOT commit
  docs/                  <- screenshots, design notes
```

The most important habit is to **keep logic separate from input and output**. The `core` module takes values and returns values. The `cli` module reads arguments and prints. Then you can test the logic without a terminal, a network or a database server.

Step 3: Environment and dependencies

- Create one virtual environment per project (Chapter 9): `python3 -m venv .venv`, then activate it.
- Prefer the standard library where it does the job. Every extra dependency is something a stranger must install and something that can break.
- Record dependencies in `pyproject.toml` (or `requirements.txt`), with a minimum version, for example `pandas>=2.0`.
- Never commit secrets. Keys go in environment variables (Chapter 13).

Step 4: Data sourcing or generation

Option	Example	Careful about
Generate it	A script that writes 200 fake orders with a fixed random seed	Seed it so results repeat
Public dataset	Government open data, Kaggle files, school exam marks	Read the licence; credit the source
Your own records	Your spending list	Remove names, phone numbers, account numbers before publishing
Scraped	Pages you are allowed to fetch	robots.txt, terms of use, rate limits (Project 4)

Always commit a **tiny sample** (5 to 20 rows) so that tests and the README demo work offline.

Step 5: Commit rhythm and good commit messages

Commit when one small thing works, not at the end of the week. A healthy day has 2 to 5 commits.

- Good: `Add validate() with date and amount checks`, `Fix: reject negative amounts (test added)`, `Docs: add install steps to README`.
- Bad: `update`, `final`, `changes`, `asdf`.
- Pattern: start with a verb, keep the first line under about 60 characters, say what and why, not how.
- Never commit a failing test suite to the main branch. Use a branch for risky work.

Step 6: Definition of done

A project is done when all of these are true:

- [ ] The success criteria from Step 1 are met and written in the README with the real numbers you got.
- [ ] All tests pass from a fresh clone and fresh virtual environment.
- [ ] A new person can run the demo by copying commands from the README.
- [ ] Bad input produces a clear message and a non-zero exit code, not a traceback.
- [ ] No secrets, no giant files, no `__pycache__` in the repo.
- [ ] You can explain every file and every design choice out loud in two minutes.

Step 7: README template

```
# Project Name

One sentence: what it does and for whom.

## Why I built it
Two or three sentences on the problem.

## Quick start
```

git clone <url> && cd project-name python3 -m venv .venv && source .venv/bin/activate pip install -e . python -m package_name --help

```
## Example
Paste a real command and its real output.

## How it works
A small diagram or a list of the main modules.

## Tests
`python -m unittest discover -s tests -v`   (current result: N passed)

## Results / acceptance criteria
A table of the measurable checks and whether they pass.

## Limits and next steps
What it does not do yet. Honest limits build trust.

## Licence and data sources
```

Step 8: Demo and deploy options

Project type	Free, simple demo
Command-line tool	A short screen recording or the real output pasted in the README
Package	Publish a tagged release on GitHub; install with `pip install git+<url>` (not run here)
API	Run locally and paste `curl` commands; optional free hosting tiers change often, so read current terms
Scraper / dataset	Publish the cleaned sample CSV and the code, not the raw pages

Step 9: Writing the LinkedIn or blog post

Use this five-part shape, about 150 to 250 words:

1. **Hook:** one sentence about the problem ("My spending notes were scattered over three apps.").
2. **What I built:** one sentence plus the link.
3. **One thing that surprised me:** a real bug and how you found it. This is the part people remember.
4. **One number:** "6 tests, 3 commands, runs offline."
5. **What next:** one honest limit and one plan.

No hype words. No claim you cannot show in the repo.

Step 10: Testing and debugging checklist for Python projects

- **Test the three kinds of input:** normal, empty, and wrong (bad date, negative amount, missing column).
- **Test the edges:** one row, zero rows, duplicate rows, a very long text.
- **Use fixed data and a temporary folder** (`tempfile.TemporaryDirectory`) so tests never touch real files.
- **Check exit codes** for command-line tools: 0 is success, non-zero is failure.
- **Debug in this order:** read the last line of the traceback first; reproduce with the smallest input; print or `logging` the values just before the failure; use `python3 -m pdb` or `breakpoint()` (Chapter 6) only after that.
- **Run the whole suite before every commit.**
- **Test from a clean clone** once before you share the link.

REAL JOB TIP

Reviewers rarely run your code on day one. They read the README, the folder tree and the test names. A test called `test_negative_amount_is_rejected` tells them more about your care than a hundred lines of clever code.

Check yourself

1. Why should `core.py` never call `print()` or `argparse` ? Answer: So the logic can be tested by simply calling functions and checking returned values, without capturing terminal output.
2. Name two things that must not be committed. Answer: Secrets (keys, passwords) and large raw data files or `.venv` folders.
3. Give one measurable success criterion for a data-cleaning tool. Answer: "On the sample file, the

report shows 1 duplicate dropped, 1 bad date and 1 outlier, matching my hand count."

15.2 Project: A Command-Line Expense Tracker

Goal and why employers care

Build `expenses`, a command that records spending in a CSV file and prints totals per category. It looks small, but it shows the five things employers check first: a clean interface (`argparse`), validated input, file storage, separation of logic from output, and tests. Almost every internal tool in a company is a bigger cousin of this one.

Success criteria: `add` saves a valid row and returns exit code 0; invalid date or amount prints a clear message and returns 1; `summary --month 2025-03` matches a hand calculation; 6 tests pass.

Skills used

Chapter 2 (functions), Chapter 4 (`defaultdict`, sorting), Chapter 5 (files, CSV, dates), Chapter 6 (custom exceptions), Chapter 10 (testing), Chapter 12 (command-line tools with `argparse`).

Data

No download needed. You generate data by using the tool. For tests, small fixed rows are enough. Example data for the demo (shop-style spending): groceries 120.50 and 80.00, transport 45.00, rent 9000.00.

Architecture

```
expense-tracker/
  expense_tracker/
    __init__.py
    core.py        validate(), load(), append(), summarise()   (no printing)
    cli.py         build_parser(), main(argv) -> exit code
    __main__.py    lets you run: python3 -m expense_tracker
  tests/
    __init__.py
    test_core.py
  README.md
```

```
 user types:  add 2025-03-01 food 120.5
                 |
              cli.main(argv) --parse--> core.validate() --> core.append() -->
expenses.csv
                 |  (bad input: ExpenseError)
                 +--> prints "error: ..." to stderr, returns 1
```

Step-by-step build

Step 1: The core, starting with validation. Raise your own error type so the CLI can tell "user mistake" from "program bug".

```
# expense_tracker/core.py (first part)
import csv
from collections import defaultdict
from datetime import date
from pathlib import Path

FIELDS = ["date", "category", "amount", "note"]

class ExpenseError(Exception):
    """A mistake the user can fix (bad date, bad amount)."""

def validate(day: str, category: str, amount: str, note: str = "") -> dict:
    try:
        date.fromisoformat(day)
    except ValueError:
        raise ExpenseError(f"bad date {day!r}, use YYYY-MM-DD") from None
    try:
        value = round(float(amount), 2)
    except ValueError:
        raise ExpenseError(f"bad amount {amount!r}") from None
    if value <= 0:
        raise ExpenseError("amount must be positive")
    if not category.strip():
        raise ExpenseError("category is empty")
    return {"date": day, "category": category.strip().lower(),
            "amount": value, "note": note}
```

Step 2: Storage and summary. Append rows, write the header only when the file is new, and add up per category.

```
def load(path: Path) -> list[dict]:
    if not path.exists():
        return []
    with path.open(newline="", encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
    for r in rows:
        r["amount"] = float(r["amount"])
    return rows

def append(path: Path, row: dict) -> None:
    is_new = not path.exists()
    with path.open("a", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=FIELDS)
        if is_new:
            w.writeheader()
        w.writerow(row)

def summarise(rows: list[dict], month: str | None = None) -> dict:
    totals: dict[str, float] = defaultdict(float)
    for r in rows:
        if month and not r["date"].startswith(month):
            continue
        totals[r["category"]] += r["amount"]
    return {k: round(v, 2) for k, v in sorted(totals.items())}
```

Step 3: The command-line layer. Sub-commands (`add` , `list` , `summary`) each get their own arguments. `main` takes `argv` so tests can call it directly, and it returns an exit code instead of calling `sys.exit` itself.

```
# expense_tracker/cli.py
import argparse, sys
from pathlib import Path
from . import core

def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog="expenses", description="Track spending.")
    p.add_argument("--file", type=Path, default=Path("expenses.csv"))
    sub = p.add_subparsers(dest="command", required=True)
    add = sub.add_parser("add", help="record one expense")
    add.add_argument("date"); add.add_argument("category");
add.add_argument("amount")
    add.add_argument("--note", default="")
    lst = sub.add_parser("list", help="show all expenses")
    lst.add_argument("--category")
    summ = sub.add_parser("summary", help="totals per category")
    summ.add_argument("--month", help="YYYY-MM")
    return p

def main(argv=None) -> int:
    args = build_parser().parse_args(argv)
    try:
        if args.command == "add":
            row = core.validate(args.date, args.category, args.amount,
args.note)
            core.append(args.file, row)
            print(f"saved: {row['date']} {row['category']}
{row['amount']:.2f}")
        elif args.command == "list":
            for r in core.load(args.file):
                if args.category and r["category"] != args.category.lower():
                    continue
                print(f"{r['date']}  {r['category']:<10} {r['amount']:>8.2f}
{r['note']}")
        else:
            totals = core.summarise(core.load(args.file), args.month)
            for cat, total in totals.items():
                print(f"{cat:<10} {total:>8.2f}")
            print(f"{'TOTAL':<10} {sum(totals.values()):>8.2f}")
    except core.ExpenseError as e:
        print(f"error: {e}", file=sys.stderr)
        return 1
    return 0
```

`__main__.py` has two lines: `import sys` / `from .cli import main`, then `sys.exit(main())`.

Step 4: Run it. Before you run it, predict: what will the last two commands print?

```
python3 -m expense_tracker add 2025-03-01 Food 120.50 --note "groceries"
python3 -m expense_tracker add 2025-03-03 transport 45
python3 -m expense_tracker add 2025-03-09 food 80
python3 -m expense_tracker add 2025-04-02 rent 9000
python3 -m expense_tracker add 2025-13-02 food 5
python3 -m expense_tracker summary --month 2025-03
```

Real output:

```
saved: 2025-03-01 food 120.50
saved: 2025-03-03 transport 45.00
saved: 2025-03-09 food 80.00
saved: 2025-04-02 rent 9000.00
error: bad date '2025-13-02', use YYYY-MM-DD
food         200.50
transport     45.00
TOTAL        245.50
```

Why: month 13 does not exist, so `date.fromisoformat` raises `ValueError`, we convert it to `ExpenseError`, and `main` prints it and returns 1 (the shell shows `exit=1`). March has food 120.50 + 80.00 = 200.50 and transport 45.00, total 245.50. April's rent is excluded by the `startswith("2025-03")` filter. The CSV on disk is:

```
date,category,amount,note
2025-03-01,food,120.5,groceries
2025-03-03,transport,45.0,
2025-03-09,food,80.0,
2025-04-02,rent,9000.0,
```

Step 5: Tests. Put each kind of input in a test: good, bad, and the storage round trip, always in a temporary folder.

```
# tests/test_core.py (excerpt)
class ValidateTests(unittest.TestCase):
    def test_bad_date(self):
        with self.assertRaises(core.ExpenseError):
            core.validate("01/03/2025", "food", "5")

class StorageTests(unittest.TestCase):
    def test_round_trip_and_summary(self):
        with tempfile.TemporaryDirectory() as d:
            p = Path(d) / "e.csv"
            core.append(p, core.validate("2025-03-01", "food", "100"))
            core.append(p, core.validate("2025-03-02", "food", "50.25"))
            core.append(p, core.validate("2025-04-01", "travel", "300"))
            rows = core.load(p)
            self.assertEqual(core.summarise(rows), {"food": 150.25, "travel":
300.0})
            self.assertEqual(core.summarise(rows, "2025-03"), {"food": 150.25})
```

The full suite has 6 tests: good row, bad date, negative amount, round trip, missing file is empty, and the CLI exit codes (0, 0, 1). Real run:

```
$ python3 -m unittest discover -v
...
Ran 6 tests in 0.019s

OK
```

Your first run may instead print `Ran 0 tests / NO TESTS RAN`. We got exactly that when we ran `python3 -m unittest -v` from the project root, and then an `ImportError: Start directory is not importable` with `discover -s tests`, until we added an empty `tests/__init__.py` and used plain `python3 -m unittest discover`. That is Pitfall 4 below.

Evaluation / acceptance criteria

Check	Expected	How to verify
March total (hand calculation: 120.50 + 80 + 45)	245.50	`summary --month 2025-03`
Bad date	message on stderr, exit code 1	`echo $?` after the command
Bad amount `abc`	`error: bad amount 'abc'`	run it

Tests	6 pass, under 1 second	`python3 -m unittest discover`
Fresh clone works	README commands succeed	try in a new folder

Common pitfalls

1. **Float rounding.** `0.1 + 0.2` prints `0.30000000000000004`. We round when saving and again in `summarise` (`round(v, 2)`); the summary of 0.1 and 0.2 gives `{'food': 0.3}`. Detect with a test using decimals. For real money use integer paise or `decimal.Decimal`.
2. **Date formats.** `2025-3-1` is rejected (`Invalid isoformat string: '2025-3-1'`). Decide on one format and say so in the error message.
3. **Mixing printing and logic.** If `summarise` prints, you cannot test it. Detect by asking: can I call this function without a terminal?
4. **Tests not discovered.** Missing `tests/__init__.py` or wrong working folder gives `NO TESTS RAN`. Detect: the count of tests run is 0, always read it.
5. **Header written twice.** Appending with a header each time corrupts the CSV. The `is_new` check prevents it; a test that adds two rows and loads two rows catches it.

Three extensions

1. A `delete` sub-command and an `edit` command, using a row id.
2. A monthly budget per category, and a warning when you pass 80%.
3. Switch storage to `sqlite3` behind the same `core` functions, without changing the tests for `summarise`.

Interview talking points

Q1. Why did you separate `core` from `cli`? Because logic that does not print is easy to test and to reuse. I can call `core.summarise` from a script, a web page or a test. The CLI is a thin layer that I replace without touching the logic.

Q2. How do you handle bad input? I validate at the edge. `validate` raises one custom error. The CLI catches only that error, prints a short message and returns exit code 1. A real bug still shows a traceback, which is what I want.

Q3. What would you change for real money? I would not store floats. I would store integer paise, or use `Decimal`, because floats cannot represent 0.1 exactly. I would also add a file lock or a database so two processes cannot write at once.

README outline

Title and one-line purpose; quick start (3 commands); usage table of sub-commands; real output block from Step 4; storage format (the CSV header); how to run tests; known limits (single user, floats); next steps.

15.3 Project: A Reusable Data-Cleaning Package

Goal and why employers care

Every analyst cleans the same mess again and again: untidy column names, text with stray spaces, bad dates, repeated rows, empty cells, wild values. Instead of copying cells between notebooks, build a small **package**, `tidyshop`, with tested functions and one pipeline that also returns a **report of what it changed**. Employers care because reusable, tested cleaning code is what separates a notebook user from a data professional, and the report makes cleaning auditable.

Success criteria: on a 6-row sample the report says: 1 duplicate dropped, 1 bad date, 1 missing amount, 1 outlier flagged, 5 rows out; 8 tests pass; the package imports from a clean environment.

Skills used

Chapter 4 (collections), Chapter 3 (regular expressions for column names), Chapter 5 (dates), Chapter 9 (packages, `pyproject.toml`, type hints), Chapter 10 (testing). It also uses `pandas`, which the Data Science track teaches in depth. See also the learner's Math PDF for the median and quartile definitions behind the IQR rule.

Data

A generated table of shop orders with typical problems (spaces, mixed case, a bad date, a repeated order id, a missing amount, one wild amount). It is built right inside the test and the demo, so there is nothing to download.

Architecture

```
tidyshop/
  pyproject.toml
  README.md
  src/
    tidyshop/
      __init__.py      exports the public functions
      clean.py         one function per cleaning job + clean_orders()
  tests/
    test_clean.py      8 tests
  docs/
    usage.md           examples (the demo output below)
```

The `src/` layout means tests import the installed package, not the folder next to them, so packaging mistakes show up early.

```
 raw DataFrame
   -> standardise_columns  (" Order ID " -> order_id)
   -> strip_text           (" Pune" -> pune)
   -> parse_dates          (bad -> NaT)
   -> drop_duplicate_rows  (by order_id)
   -> fill_missing         (median)
   -> flag_outliers_iqr    (adds amount_outlier)
   -> (clean DataFrame, report dict)
```

Step-by-step build

Step 1: `pyproject.toml` . This tells tools the package name, version and what it needs.

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "tidyshop"
version = "0.1.0"
description = "Small, tested helpers for cleaning shop-order tables"
requires-python = ">=3.10"
dependencies = ["pandas>=2.0"]

[tool.setuptools.packages.find]
where = ["src"]
```

`pip install -e .` installs it in editable mode: **not run here**, because pip was blocked. We ran everything with `PYTHONPATH=src` , which gives Python the same import path.

Step 2: Small functions that each return a NEW table. Never change the caller's data.

```
# src/tidyshop/clean.py (excerpt)
import re
import pandas as pd

def standardise_columns(df: pd.DataFrame) -> pd.DataFrame:
    """' Order ID ' -> 'order_id'."""
    out = df.copy()
    out.columns = [re.sub(r"\W+", "_", str(c).strip().lower()).strip("_")
                   for c in out.columns]
    return out

def parse_dates(df: pd.DataFrame, column: str) -> pd.DataFrame:
    """Bad values become NaT, never an exception."""
    out = df.copy()
    out[column] = pd.to_datetime(out[column], errors="coerce", format="%Y-%m-
%d")
    return out

def fill_missing(df: pd.DataFrame, column: str, how: str = "median") ->
pd.DataFrame:
    if how not in {"median", "mean"}:
        raise ValueError(f"how must be 'median' or 'mean', got {how!r}")
    out = df.copy()
    out[column] = out[column].fillna(getattr(out[column], how)())
    return out

def flag_outliers_iqr(df: pd.DataFrame, column: str, k: float = 1.5) ->
pd.DataFrame:
    out = df.copy()
    q1, q3 = out[column].quantile([0.25, 0.75])
    iqr = q3 - q1
    out[f"{column}_outlier"] = ((out[column] < q1 - k * iqr)
                                | (out[column] > q3 + k * iqr))
    return out
```

The IQR rule: a value is flagged if it is below `Q1 − 1.5 × IQR` or above `Q3 + 1.5 × IQR`, where IQR = Q3 − Q1 (the spread of the middle half of the data). `strip_text` (trim and lower-case the named text columns with `.astype("string").str.strip().str.lower()`) and `drop_duplicate_rows` (`df.drop_duplicates(subset=..., keep="first").reset_index(drop=True)`) follow the same pattern.

Step 3: The pipeline with a report. Counting inside the pipeline is what makes it trustworthy.

```
def clean_orders(df):
    report = {"rows_in": len(df)}
    out = standardise_columns(df)
    out = strip_text(out, ["city", "status"])
    out = parse_dates(out, "order_date")
    report["bad_dates"] = int(out["order_date"].isna().sum())
    before = len(out)
    out = drop_duplicate_rows(out, ["order_id"])
    report["duplicates_dropped"] = before - len(out)
    report["missing_amount"] = int(out["amount"].isna().sum())
    out = fill_missing(out, "amount", "median")
    out = flag_outliers_iqr(out, "amount")
    report["outliers_flagged"] = int(out["amount_outlier"].sum())
    report["rows_out"] = len(out)
    return out, report
```

Step 4: Run it on the sample. Predict first: after de-duplication there are 5 rows with amounts 100, 120, missing, 110, 9000. What median will fill the gap?

```
import pandas as pd
from tidyshop import clean_orders
raw = pd.DataFrame({
    "Order ID": [1, 2, 2, 3, 4, 5],
    "City": [" Pune", "pune ", "pune ", "DELHI", "Delhi", "Pune"],
    "Status": ["Shipped"] * 6,
    "Order Date": ["2025-01-01", "2025-01-02", "2025-01-02", "bad", "2025-01-
04", "2025-01-05"],
    "Amount": [100.0, 120.0, 120.0, None, 110.0, 9000.0],
})
out, rep = clean_orders(raw)
print(rep)
print(out.to_string())
```

Real output:

```
{'rows_in': 6, 'bad_dates': 1, 'duplicates_dropped': 1, 'missing_amount': 1,
'outliers_flagged': 1, 'rows_out': 5}
   order_id   city   status order_date  amount  amount_outlier
0         1   pune  shipped 2025-01-01   100.0           False
1         2   pune  shipped 2025-01-02   120.0           False
2         3  delhi  shipped        NaT   115.0           False
3         4  delhi  shipped 2025-01-04   110.0           False
4         5   pune  shipped 2025-01-05  9000.0            True
```

Check by hand: the four known amounts are 100, 110, 120, 9000. The median of four values is the average of the middle two: (110 + 120) / 2 = 115. Order 3 received 115. The 9000 order lies far above

`Q3 + 1.5 × IQR`, so it is flagged. The city values collapsed from 5 spellings to 2 (`pune`, `delhi`). The bad date `"bad"` became `NaT` ("not a time"), a safe empty date.

Step 5: Tests. Each small function has a test, the input is checked for mutation, and the pipeline has one test on exact numbers.

```
def test_input_is_not_changed(self):
    df = pd.DataFrame({"A B": [1]})
    standardise_columns(df)
    self.assertEqual(list(df.columns), ["A B"])

def test_outlier_flag(self):
    df = pd.DataFrame({"x": [10, 11, 12, 13, 14, 500]})
    flags = flag_outliers_iqr(df, "x")["x_outlier"].tolist()
    self.assertEqual(flags, [False] * 5 + [True])
```

```
$ PYTHONPATH=src python3 -m unittest discover -s tests -v
...
Ran 8 tests in 0.006s

OK
```

Step 6: Docs. Put the Step 4 code and output in `docs/usage.md`, and write a docstring on every function. A reader should learn the package from the README and one example.

Evaluation / acceptance criteria

Check	Expected
Report on sample	`duplicates_dropped=1, bad_dates=1, missing_amount=1, outliers_flagged=1, rows_out=5`
Filled value (hand calculation)	115.0
Input table unchanged after any function	test passes
Tests	8 pass
Import works after `pip install -e .`	`python -c "import tidyshop"` (not run here)
Every public function has	`help(tidyshop.fill_missing)` shows text

a docstring

Common pitfalls

1. **Changing the caller's data.** Without `.copy()`, the original table changes silently. Detect with the "input is not changed" test.
2. **Filling missing values before removing duplicates (or outliers).** The median changes depending on order. Decide the order, write it in the README, and test it.
3. `errors="coerce"` **hides bad data.** Bad dates become `NaT` quietly. That is why the report counts them. If the count is large, stop and look at the raw file.
4. **Flat layout imports.** Tests passing only because they run next to the code. The `src/` layout and a clean-environment test catch it.
5. **Silent assumptions.** The pipeline assumes columns named `city`, `status`, `order_date`, `amount`. Detect with a test for a missing column and raise a clear `KeyError` message (extension 1).

Three extensions

1. Check that required columns exist and raise a friendly error listing the missing ones.
2. Add a `--report json` command-line entry point (`[project.scripts]` in `pyproject.toml`).
3. Make the outlier method pluggable (IQR, z-score, fixed limits) and compare them on the same data.

Interview talking points

Q1. Why return a new table instead of changing the old one? Changing inputs creates bugs that appear far from their cause. If every function returns a new table, I can compare before and after, and tests are simple.

Q2. How do you know your cleaning did not damage the data? The pipeline returns a report: rows in and out, duplicates dropped, missing values filled, outliers flagged. I check these counts against a hand calculation on a small sample, and I flag outliers rather than delete them, because the owner of the data should decide.

Q3. Why the `src/` layout? It forces tests to use the installed package, as a user would. With a flat layout, Python can import the local folder by accident and hide packaging errors.

README outline

Purpose; install (`pip install -e .`, not run here); five-line example with real report output; list of functions in a table; the order of the pipeline and why; tests; limits (assumes four column names);

changelog entry for `0.1.0`.

15.4 Project: A Small REST API With a Database

Goal and why employers care

Build a small web service that stores shop orders in a database and offers four actions: list, read one, create, delete. This is the backbone of most back-end work: an HTTP interface, a database, input checks and correct status codes. We use only `http.server` and `sqlite3` so that it runs anywhere. In industry you would use a framework such as FastAPI, and the second half of this project shows that version (not run here), so you see what stays the same and what the framework does for you.

Success criteria: the five tests pass; `POST` returns 201, bad input returns 422, a missing order returns 404, a delete returns 204 and the next delete returns 404; text such as `x'); DROP TABLE orders;--` is stored as plain text.

Skills used

Chapter 12 (HTTP, REST design, status codes, building a small service), Chapter 5 (JSON), Chapter 6 (exceptions), Chapter 7 (context managers, used by `with conn:`), Chapter 10 (testing), Chapter 13 (security: parameterised SQL). The learner's SQL document covers `CREATE TABLE`, `INSERT` and constraints.

Data

The data is the orders your tests send. A table with three columns:

Column	Type	Rule
`id`	integer	assigned by the database
`customer`	text	not empty
`amount`	real	greater than 0 (also a database `CHECK`)

Architecture

```
orders-api/
  orders_api.py         connect(), make_handler(db_path), serve()
  test_orders_api.py    starts a real server on a free port, calls it
  README.md
  orders.db             (created at run time; in .gitignore)
```

```
 client --HTTP--> Handler.do_GET / do_POST / do_DELETE
                       |  parse path, parse JSON, validate
                       v
                  sqlite3 (one short connection per request, "?" placeholders)
                       |
                       v
          JSON body + status (200, 201, 204, 404, 422)
```

Two routes: `/orders` (the collection) and `/orders/<id>` (one order). Nouns in the URL, the method carries the action (Chapter 12).

Step-by-step build

Step 1: The database layer. A `CHECK` constraint makes the database refuse bad amounts even if your Python code forgets.

```
import json, sqlite3
from http.server import BaseHTTPRequestHandler, HTTPServer

SCHEMA = """CREATE TABLE IF NOT EXISTS orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer TEXT NOT NULL,
    amount REAL NOT NULL CHECK (amount > 0)
)"""

def connect(db_path):
    conn = sqlite3.connect(db_path)
    conn.row_factory = sqlite3.Row      # rows behave like dicts
    conn.execute(SCHEMA)
    return conn
```

Step 2: A handler that sends JSON and status codes. One helper does the repeated work.

```
def make_handler(db_path):
    class Handler(BaseHTTPRequestHandler):
        def _send(self, status, payload=None):
            body = b"" if payload is None else json.dumps(payload).encode()
            self.send_response(status)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)

        def _order_id(self):
            parts = self.path.strip("/").split("/")
            if len(parts) == 2 and parts[0] == "orders" and parts[1].isdigit():
                return int(parts[1])
            return None
        # do_GET, do_POST, do_DELETE ...
```

The handler is built inside a function so it knows `db_path`. That is a closure (Chapter 2), and it keeps tests from sharing a global database.

Step 3: Create with validation, using `?` placeholders.

```
        def do_POST(self):
            if self.path.rstrip("/") != "/orders":
                return self._send(404, {"error": "no such route"})
            size = int(self.headers.get("Content-Length", 0))
            try:
                data = json.loads(self.rfile.read(size))
                customer, amount = str(data["customer"]).strip(),
float(data["amount"])
                if not customer or amount <= 0:
                    raise ValueError
            except (ValueError, KeyError, TypeError):
                return self._send(422, {"error": "need customer (text) and
amount > 0"})
            conn = connect(db_path)
            try:
                with conn:      # commits on success, rolls back on error
                    cur = conn.execute(
                        "INSERT INTO orders (customer, amount) VALUES (?, ?)",
                        (customer, amount))
                return self._send(201, {"id": cur.lastrowid,
                                        "customer": customer, "amount":
amount})
            finally:
                conn.close()
```

`do_GET` selects with `WHERE id = ?` and returns 404 when `fetchone()` is `None`. `do_DELETE`

checks `cur.rowcount` (rows deleted): 204 if one, 404 if zero. `log_message` is overridden to stay quiet.

Step 4: Why `?` and never an f-string. Predict: what happens if a customer is called `O'Brien` and you build the SQL with an f-string?

```
name = "O'Brien"
c.execute(f"insert into o values ('{name}')")      # wrong
c.execute("insert into o values (?)", (name,))     # right
```

Real output:

```
OperationalError near "Brien": syntax error
[("O'Brien",)]
```

The apostrophe ended the SQL text early. A malicious name does worse than crash: it can run commands (SQL injection). With `?` the value always travels separately from the command, so it can never become part of it. Our test stores `x'); DROP TABLE orders;--` and reads it back unchanged.

Step 5: Try it end to end. `test_orders_api.py` has a small `call(method, url, body)` helper built on `urllib.request`, and the demo starts the server on port 0 (any free port) in a thread.

```
(201, {'id': 1, 'customer': 'Asha', 'amount': 250.0})
(201, {'id': 2, 'customer': 'Ravi', 'amount': 99.5})
(200, [{'id': 1, 'customer': 'Asha', 'amount': 250.0}, {'id': 2, 'customer':
'Ravi', 'amount': 99.5}])
(404, {'error': 'order not found'})
(422, {'error': 'need customer (text) and amount > 0'})
(204, None)
(200, [{'id': 2, 'customer': 'Ravi', 'amount': 99.5}])
```

Read it line by line: two creates (201 Created, ids 1 and 2), a list of both, a read of id 7 (404), a create with amount −1 (422), deleting id 1 (204 No Content, no body), and a list that now holds only Ravi.

To run it for real: `python3 orders_api.py` serves on port 8000 and you can call it with `curl` (not run here, because the tests use a thread instead).

Step 6: Tests that start a real server. `setUp` makes a temporary database and a server; `tearDown` shuts both down.

```
def setUp(self):
    self.tmp = tempfile.TemporaryDirectory()
    db = str(Path(self.tmp.name) / "t.db")
    self.srv = HTTPServer(("127.0.0.1", 0), make_handler(db))
    threading.Thread(target=self.srv.serve_forever, daemon=True).start()
    self.base = f"http://127.0.0.1:{self.srv.server_address[1]}"

def tearDown(self):
    self.srv.shutdown(); self.srv.server_close(); self.tmp.cleanup()
```

```
$ python3 -m unittest
.....
Ran 5 tests in 2.585s

OK
```

Our first run printed `ResourceWarning: Implicitly cleaning up <HTTPError 422>` twice. The test helper read an error reply but never closed it. We added `e.close()` after reading the body, and the warnings disappeared. A warning is a bug report: read it.

The same API in FastAPI (not run here)

FastAPI needs `pip install fastapi uvicorn`, which was not possible here, so this code is **not run here**. It is the standard shape from the documentation.

```
# main.py  (not run here)
import sqlite3
from fastapi import FastAPI, HTTPException, Response
from pydantic import BaseModel, Field

app = FastAPI()

class OrderIn(BaseModel):
    customer: str = Field(min_length=1)
    amount: float = Field(gt=0)

def db():
    conn = sqlite3.connect("orders.db")
    conn.row_factory = sqlite3.Row
    return conn

@app.post("/orders", status_code=201)
def create(order: OrderIn):
    with db() as conn:
        cur = conn.execute("INSERT INTO orders (customer, amount) VALUES (?,
?)",
                           (order.customer, order.amount))
    return {"id": cur.lastrowid, **order.model_dump()}

@app.get("/orders/{order_id}")
def read(order_id: int):
    row = db().execute("SELECT * FROM orders WHERE id = ?",
(order_id,)).fetchone()
    if row is None:
        raise HTTPException(404, "order not found")
    return dict(row)
```

Run with `uvicorn main:app --reload` and open `/docs` for a generated test page. What the framework did for you: parsing the JSON, checking types and rules (`OrderIn` replaces our `try/except` , and bad input gets a 422 automatically), routing, and documentation. What it did not change: SQL with `?` , status codes, and tests.

Evaluation / acceptance criteria

Check	Expected
Create valid order	201 and an `id`
Create with amount −5 or missing `amount`	422
Read id 99 (does not exist)	404

Delete twice	204, then 404
Injection-style name	stored and returned as the same text
Tests	5 pass

Common pitfalls

1. **SQL built with f-strings.** Gives errors on quotes and opens injection. Detect: search the code for `execute(f"`.
2. **Forgetting `Content-Length`.** The client waits for more data and seems to hang. Detect with a test that has a timeout.
3. **Shared database between tests.** Tests pass alone and fail together. Use a temporary database per test.
4. **Wrong status codes.** Returning 200 for everything makes errors invisible to clients. Test the status, not only the body.
5. **Leaving a server thread running.** A test run that never exits. Always `shutdown()` and `server_close()` in `tearDown`.

Three extensions

1. `PUT /orders/<id>` to change an amount, plus `GET /orders?customer=Asha`.
2. A simple API token in the `Authorization` header, read from an environment variable (Chapter 12).
3. Move to the FastAPI version above and keep the same five tests running against it (use a test client).

Interview talking points

Q1. What is the difference between 404 and 422? 404 means the thing you asked for does not exist. 422 means I understood your request, but the content is not acceptable, for example a negative amount.

Q2. Why parameterised queries? The database receives the command and the values separately, so a value can never change the command. It also handles quotes correctly. It is the first defence against SQL injection.

Q3. Why use a framework if the standard library works? The standard library works for learning and small tools, but I would have to write routing, validation, documentation and concurrency handling myself. A framework gives them tested and consistent. I still need to understand the basics

underneath, or I cannot debug it.

README outline

Purpose; routes table (method, path, success code, error codes); how to run the server and sample `curl` calls (not run here); schema; how to run the tests; security notes (parameterised SQL, no authentication yet); FastAPI version and why.

15.5 Project: A Responsible Web Scraper and Dataset Builder

Goal and why employers care

Build a tool that turns product pages into a clean CSV dataset, while respecting the website. Many analytics jobs start with "we need data that is only available on web pages". Employers want people who can do it **and** know where the line is: read `robots.txt`, honour the terms of use, slow down, identify yourself, and never collect personal data. This project teaches the parsing on a local HTML string (so it runs without any network), and the etiquette that makes the difference.

Success criteria: the parser extracts 3 products from the sample page; prices become numbers, and "sold out" becomes an empty value; `robots.txt` rules are checked before every fetch; the pause between requests is at least the site's requested delay.

Skills used

Chapter 3 (strings and regular expressions), Chapter 5 (CSV), Chapter 7 (classes with state), Chapter 12 (HTTP and scraping), Chapter 10 (tests), Chapter 13 (responsible use).

Data

No real site is touched. We use a small HTML string with three products (one sold out) and a `robots.txt` string. Real scraping would feed the same parser with downloaded pages.

Responsible-scraping rules (read before writing code)

1. **Look for an official API or a downloadable dataset first.** It is easier and welcome.
2. **Read the terms of use.** If they forbid automated access, stop. A technical possibility is not permission.
3. **Check `/robots.txt`.** It lists paths the owner asks bots to avoid and sometimes a `Crawl-delay`. It is a request, not a lock, and you follow it anyway.
4. **Identify yourself** with a clear `User-Agent` that includes a contact address.

5. **Go slowly:** a pause between requests (2 seconds or the site's `Crawl-delay`), few requests, and cache pages you already have.
6. **Collect only what you need.** No personal data (names, phone numbers, photos of people).
7. **Do not republish copyrighted content.** Publish your code and a small derived sample, with the source credited.

Architecture

```
web-scraper/
  scraper/
    fetch.py        polite_get(): robots check, User-Agent, delay, cache (not
run here)
    parse.py        ProductParser, parse_price()
    export.py       to_csv()
  tests/
    test_parse.py   runs on the local HTML string
  data/
    sample/products.csv
  README.md
```

```
 robots.txt --allowed?--> polite_get(url) --HTML--> ProductParser --> items -->
CSV
                |              |  sleep(delay)                |
             skip if not    User-Agent + contact        parse_price(): "Rs.
1,299" -> 1299.0
```

Step-by-step build

Step 1: Check robots.txt rules. The standard library includes `urllib.robotparser`. Here we feed it a string.

```
import urllib.robotparser

ROBOTS = """
User-agent: *
Disallow: /private/
Crawl-delay: 2
"""

def allowed(robots_text, url, agent="study-bot"):
    rp = urllib.robotparser.RobotFileParser()
    rp.parse(robots_text.splitlines())
    return rp.can_fetch(agent, url), rp.crawl_delay(agent)
```

```
(True, 2)      # allowed("https://shop.example/products")
(False, 2)     # allowed("https://shop.example/private/admin")
```

So `/products` may be fetched and `/private/admin` may not, and we must wait at least 2 seconds between requests.

Step 2: Parse HTML with `html.parser`. `HTMLParser` calls your methods when it meets a start tag, text, or an end tag. You keep a small state: "am I inside a product?" and "which field am I reading?"

```
from html.parser import HTMLParser

class ProductParser(HTMLParser):
    def __init__(self):
        super().__init__()
        self.items, self._cur, self._field = [], None, None

    def handle_starttag(self, tag, attrs):
        a = dict(attrs)
        if tag == "div" and a.get("class") == "product":
            self._cur = {"name": "", "price_text": "", "url": ""}
        elif self._cur is not None:
            if tag in ("h2", "span") and a.get("class") in ("name", "price"):
                self._field = "name" if a["class"] == "name" else "price_text"
            elif tag == "a":
                self._cur["url"] = a.get("href", "")

    def handle_data(self, data):
        if self._cur is not None and self._field:
            self._cur[self._field] += data.strip()

    def handle_endtag(self, tag):
        if tag in ("h2", "span"):
            self._field = None
        elif tag == "div" and self._cur is not None:
            self.items.append(self._cur)
            self._cur = None
```

Step 3: Turn price text into numbers.

```
def parse_price(text):
    """'Rs. 1,299' -> 1299.0 ; anything without digits -> None."""
    digits = "".join(ch for ch in text if ch.isdigit() or ch == ".").strip(".")
    return float(digits) if digits else None
```

Step 4: Run on the sample page and export. Predict: what will the price of "Desk Lamp" be in the

CSV?

```
p = ProductParser()
p.feed(PAGE)          # PAGE is the HTML string with three product blocks
for it in p.items:
    print(it)
```

Real output:

```
{'name': 'Steel Bottle', 'price_text': 'Rs. 450', 'url': '/p/101'}
{'name': 'Notebook (A5)', 'price_text': 'Rs. 1,299', 'url': '/p/102'}
{'name': 'Desk Lamp', 'price_text': 'sold out', 'url': '/p/103'}
```

And `to_csv(p.items)` (a `csv.DictWriter` writing `name`, `price`, `url`) produces:

```
name,price,url
Steel Bottle,450.0,/p/101
Notebook (A5),1299.0,/p/102
Desk Lamp,,/p/103
```

"sold out" has no digits, so `parse_price` returns `None`, and the CSV writer leaves the cell empty. An empty cell is honest: we did not guess a price. Note also that we kept the raw text (`price_text`) next to the parsed value while debugging; keep it until you trust the parser.

Step 5: The polite fetcher (not run here). It needs the network, so it was **not run here**. This is the standard-library version.

```
import time, urllib.request

UA = "study-bot/0.1 (contact: you@example.com)"
DELAY = 2.0           # at least the site's Crawl-delay

def polite_get(url, cache={}):
    if url in cache:
        return cache[url]
    time.sleep(DELAY)
    req = urllib.request.Request(url, headers={"User-Agent": UA})
    with urllib.request.urlopen(req, timeout=10) as r:
        cache[url] = r.read().decode("utf-8", errors="replace")
    return cache[url]
```

The `requests` and BeautifulSoup version (not run here, needs `pip install requests beautifulsoup4`):

```
import requests
from bs4 import BeautifulSoup

html = requests.get(url, headers={"User-Agent": UA}, timeout=10).text
soup = BeautifulSoup(html, "html.parser")
for box in soup.select("div.product"):
    name = box.select_one(".name").get_text(strip=True)
    price = box.select_one(".price").get_text(strip=True)
```

`select` takes CSS selectors, so the parser class collapses to two lines. The `User-Agent`, delay and timeout are exactly the same; libraries do not remove your responsibility.

Step 6: Tests. Because parsing runs on a string, tests are fast and need no network.

```
class ParseTests(unittest.TestCase):
    def test_three_products(self):
        p = ProductParser(); p.feed(PAGE)
        self.assertEqual(len(p.items), 3)
    def test_prices(self):
        self.assertEqual(parse_price("Rs. 1,299"), 1299.0)
        self.assertIsNone(parse_price("sold out"))
    def test_robots(self):
        self.assertFalse(allowed(ROBOTS, "https://shop.example/private/x")[0])
```

These three tests were run and pass (`Ran 3 tests ... OK`). Put them in `tests/test_parse.py` and run them as you did in Project 1.

Evaluation / acceptance criteria

Check	Expected
Products parsed from sample page	3
Price of Bottle, Notebook, Lamp	450.0, 1299.0, empty
`/private/admin`	blocked by robots check
Delay between real requests	at least `Crawl-delay` (2 s here)
Request headers	custom `User-Agent` with contact
Dataset	no personal data; source and date written in README

Common pitfalls

1. **Page layout changes.** A site changes `class="price"` and your parser returns blanks. Detect: a test and a run-time check that "items found > 0" and "share of empty prices is below a limit".
2. **Fragile price cleaning.** Our `parse_price` keeps digits and dots, so `Rs.1.5` style text or a currency like `1.299,00` (European format) would be read wrongly. Detect by testing real examples from the page, and handle each format on purpose.
3. **Ignoring etiquette.** No delay, no `User-Agent`, ignoring robots.txt. Detect: you get HTTP 429 (too many requests) or get blocked. Respond by slowing down and rereading the terms; do not hide from the block.
4. **Scraping JavaScript pages with a plain downloader.** The HTML you receive has no products because they load later. Detect: `len(items) == 0` on a page that shows products in a browser. Look for an official API first; browser tools are a last resort.
5. **Saving raw pages and personal data.** Keep only the fields you need.

Three extensions

1. Follow pagination ("next page") with a hard limit of pages and the delay.
2. Save to SQLite and keep a history of prices by date.
3. A small chart of price changes over time with `matplotlib` (see the Data Science track).

Interview talking points

Q1. How do you decide if you may scrape a site? First I look for an API or a download. Then I read the terms of use and `robots.txt`. If automated access is forbidden, I do not scrape. If it is allowed, I go slowly, identify myself, and collect only what I need.

Q2. What happens when the site changes its layout? My parser returns fewer items or empty fields. So I add checks: items found must be above zero and the share of missing prices must stay below a limit. A failed check stops the job and tells me, instead of silently saving bad data.

Q3. Why parse with a parser and not regular expressions? HTML is nested and messy. A regular expression breaks when attributes change order or tags nest. A parser follows the structure of the page.

README outline

Purpose; legal and ethical section (what the site's terms and robots.txt say, date checked, your delay and User-Agent); install; example run with real output; data dictionary (columns); how to run the tests; known limits; how the dataset can be used.

Chapter recap

- A project is a question plus measurable success criteria plus a small plan; write them before coding.
- The same layout works for every project: README, `pyproject.toml`, source, tests, sample data.
- Keep logic (`core`) separate from input and output (`cli`, web handler), so tests are simple.
- Validate at the edge, raise one clear error type, return exit codes (0 good, non-zero bad).
- Use temporary folders, temporary databases and local strings in tests; never touch real data or the network.
- Parameterised SQL (`?`) prevents errors with quotes and prevents injection.
- A cleaning pipeline should return a report of what it changed, and flag rather than delete suspicious data.
- Scraping is allowed only after checking terms and robots.txt, and it must be slow, identified and minimal.
- Read warnings and the count of tests run; `NO TESTS RAN` is not a pass.

Pattern	Command or code
Run all tests	`python3 -m unittest discover -v` (needs `tests/__init__.py`)
Run a module	`python3 -m package_name args`
Exit code of last command	`echo $?`
Safe SQL	`conn.execute("... WHERE id = ?", (value,))`
Free port in tests	`HTTPServer(("127.0.0.1", 0), handler)`
Commit message	`Verb + what + why`, under about 60 characters
robots check	`RobotFileParser().parse(lines)`, then `can_fetch(agent, url)`

Practice set

Problem 15.1 (easy) Write three success criteria for the expense tracker. **Answer:** Any measurable three, for example: `summary --month 2025-03` returns 245.50 for the sample rows; a bad date returns exit code 1 with a message; 6 tests pass.

Problem 15.2 (easy) What does this print: `python3 -m expense_tracker add 2025-02-30 food 10` ? **Answer:** `error: bad date '2025-02-30', use YYYY-MM-DD` (February 30 does not exist, so `fromisoformat` raises `ValueError`), and the exit code is 1.

Problem 15.3 (easy) Which of these commit messages is better, and why: `update` or `Fix: reject negative amounts (test added)` ? **Answer:** The second. It says what changed and that a test protects it.

Problem 15.4 (medium) Your tests print `NO TESTS RAN` . List two causes. **Answer:** The `tests` folder has no `__init__.py` (so discovery does not treat it as a package), or you ran the command from the wrong folder, or test files and methods do not start with `test` .

Problem 15.5 (medium) Find the bug: `conn.execute(f"SELECT * FROM orders WHERE customer = '{name}'")` . **Answer:** SQL built by string formatting breaks on a name like `O'Brien` and allows injection. Use `conn.execute("SELECT * FROM orders WHERE customer = ?", (name,))` .

Problem 15.6 (medium) For the amounts 100, 110, 120 and one missing value, the owner asks you to fill the gap with the median of known values. What value? **Answer:** 110, the middle of 100, 110, 120. Adding an outlier such as 9000 before taking the median would change the answer, which is why the order of cleaning steps matters.

Problem 15.7 (interview) A teammate says "robots.txt does not block me technically, so I can scrape the whole site fast." Reply. **Answer:** `robots.txt` is a request from the owner, and the terms of use may add rules. Being able to fetch a page is not permission. I check the terms, follow robots.txt and its crawl delay, identify my bot, go slowly, and look for an API first.

Problem 15.8 (interview) Explain the difference between 201, 204, 404 and 422 for the orders API. **Answer:** 201: created, returns the new order. 204: success with no body (delete). 404: no such order or route. 422: the request was understood but its content is invalid (negative amount, missing field).

Prompt pack

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles. The topic is planning a small Python portfolio project (a
command-line expense tracker). Help me write: the user, the question, three
measurable success criteria, an out-of-scope list, a repository tree, and a 10-
day plan of one hour a day. Ask me up to five clarifying questions first, one
at a time, then produce the plan. After that, ask me to explain in my own words
why logic should be separate from input and output, and correct me kindly.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles. The topic is reviewing a Python project repository before I
share it with employers. Here is my repository tree and README: [PASTE HERE].
Review it using a definition-of-done checklist: README clarity, tests from a
clean clone, separation of logic from input and output, error handling and exit
codes, secrets, and commit messages. Give a short list ordered by importance,
with one concrete fix for each item, and explain the reason in simple English.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles. The topic is debugging failing tests in a Python project that
uses unittest. Here is my test output and the code under test: [PASTE HERE]. Do
not give the fix immediately. First teach me how to read the traceback from the
last line upward, then ask me what I think the cause is, then give hints one at
a time, and only after my attempt show the fix and a test that would have
caught it.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles. The topic is a small REST API built with http.server and
sqlite3, and its FastAPI equivalent. Quiz me one question at a time on status
codes (200, 201, 204, 404, 422), parameterised SQL, test setup and teardown,
and why a framework helps. Wait for my answer each time, mark it, explain
mistakes with a tiny example, and after eight questions tell me which topic I
should revise.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving
into data roles. The topic is writing a responsible web scraper and a data-
cleaning package as portfolio projects. Give me a mock interview with six
questions: three about scraping ethics and robustness (robots.txt, terms, rate
limits, layout changes) and three about data cleaning (reports, outliers, order
of steps, packaging). Ask one at a time, wait for my answer, grade each from 1
to 5 in simple English, and show a model answer I can learn from. Finish with a
one-paragraph summary of my strengths and gaps.
```

APPENDIX

CHEAT SHEET

One page per idea: the patterns from every chapter, in the order you learned them.

Run each pattern in the REPL once. A cheat sheet only helps if you have already built the thing at least one time.

Chapter 1: Setup and thinking

Need	Pattern
Run a script / one line	`python3 file.py` / `python3 -c "print(1)"`
Types	`int float complex bool str bytes None`
Float compare	`math.isclose(a, b)` (never `==`)
Text to number	`int("480")`, `float("2.5")`
Text to bytes	`s.encode("utf-8")`, `b.decode("utf-8")`
Same value / same object	`a == b` / `a is b` (use `is` only for `None`)
Copy	`x.copy()` (shallow), `copy.deepcopy(x)` (deep)
Unpack	`a, b = b, a`, `first, *rest = items`

Chapter 2: Control flow, functions, scope, recursion

Need	Pattern
Branch	`if / elif / else`, ternary `A if cond else B`
Loops	`for i, x in enumerate(xs, start=1)`, `zip(a, b)`, `while`, `break`, `continue`, loop `else`
Shape match	`match v: case [a, b]: ... case _: ...`
Safe default	`def f(x, bucket=None): if bucket is None: bucket = []`

Flexible args	`*args` (tuple), `**kwargs` (dict), `/` positional-only, `*` keyword-only
Scope order	Local, Enclosing, Global, Built-in; `nonlocal`, `global`
Fix an argument	`functools.partial(f, exp=2)`
Recursion	base case + smaller problem; `@lru_cache` for repeats

Chapter 3: Text and regex

Need	Pattern
Slice	`s[2:5]`, `s[-3:]`, `s[::-1]`
Clean	`s.strip().lower()`, `" ".join(s.split())`
Format	`f"{x:,.2f}"`, `f"{p:.1%}"`, `f"{n:05d}"`, `f"{x=}"`
Regex find	`re.search`, `re.findall`, `re.fullmatch`, `re.sub`, `re.split`
Regex pieces	`\d \w \s . [a-z] + * ? {n} ( ) (?P<name>...) ^ $ \b`
Lazy / greedy	`.+?` / `.+`
Always	raw strings `r"..."`, `re.compile(..., re.X)`

Chapter 4: Collections and toolbox

Need	Pattern
Sort	`sorted(x, key=..., reverse=True)` (new list), `x.sort()` (in place, returns `None`)
Unique + fast `in`	`set(x)`, `x in s`
Dict read / merge	`d.get(k, 0)`, `a \
Group / count	`defaultdict(list)`, `Counter(x).most_common(n)`

Window / queue	`deque(maxlen=n)`
Top-N	`heapq.nlargest(n, x)`
Comprehension	`[f(x) for x in data if test(x)]`, `{k: v ...}`, generator `(...)`
Statistics	`statistics.mean`, `median`, `quantiles(x, n=20)[18]`

Chapter 5: Files, data formats, time

Need	Pattern
Path	`Path("a") / "b.csv"`, `p.read_text(encoding="utf-8")`, `root.rglob("*.log")`
Big file	`with open(p, encoding="utf-8") as f: for line in f:`
JSON	`json.dumps(obj)`, `json.loads(text)`
CSV	`csv.DictReader(f)`, then convert with `float(...)`
Time zones	`datetime(..., tzinfo=ZoneInfo("Asia/Kolkata"))`, `.astimezone(ZoneInfo(...))`
Config	`os.environ.get("NAME", "default")`

Chapter 6: Errors, logging, debugging

Need	Pattern
Handle	`try: ... except ValueError as e: ... else: ... finally: ...`
Raise / chain	`raise AppError("...") from e`
Log	`logging.basicConfig(level=logging.INFO)`, `log.exception("what failed")`
Debug	`breakpoint()`, then `n s c p w q`

Chapter 7: Iterators, generators, decorators, context managers

Need	Pattern
Generator	`def g(): yield x` ; `yield from other`
First N	`list(itertools.islice(gen, N))`
Decorator	`def d(f): @wraps(f) def w(*a, **k): return f(*a, **k); return w`
Context manager	`@contextmanager def cm(): setup; try: yield; finally: cleanup`

Chapter 8: OOP

Need	Pattern
Record	`@dataclass(frozen=True, order=True)` , `field(default_factory=list)`
Computed attribute	`@property` ; alternative constructor `@classmethod`
Interface	`ABC` + `@abstractmethod` , or `Protocol`
Inheritance order	`Class.__mro__`
SOLID	Single responsibility, Open/closed, Liskov, Interface segregation, Dependency inversion

Chapter 9: Modules, environments, types

Need	Pattern
Entry guard	`if __name__ == "__main__":`
Virtual env	`python3 -m venv .venv` , `source .venv/bin/activate`
Save / restore	`pip freeze > requirements.txt` , `pip install -r requirements.txt`

Type hints	`list[int]`, `int \`

Chapter 10: Testing and quality

Need	Pattern
Run tests	`python3 -m unittest discover -v`, `pytest -x -q`
Many cases	`@pytest.mark.parametrize`
Replace a name	`mock.patch("module_where_used.name")`
Quality tools	`ruff format .`, `ruff check .`, `mypy src`, `pre-commit install`

Chapter 11: Concurrency and performance

Need	Pattern
Measure	`timeit`, `python3 -m cProfile -s cumulative script.py`, `tracemalloc`
Waiting (I/O)	threads, `asyncio`
Calculating (CPU)	processes (`ProcessPoolExecutor`)
Little's law	`L = λ × W` (concurrency = throughput x time in system)

Chapter 12: APIs, web, CLI

Need	Pattern
Call an API	`client.get(url, timeout=5)`, `raise_for_status()`
Retry	wait `base * 2**attempt`, only for temporary errors
CLI	`argparse`, `main(argv=None)`, exit code 0 or non-zero
Run a program	`subprocess.run([...], capture_output=True, text=True, check=True)`

Chapter 13: Professional practice

Need	Pattern
Safe SQL	`execute("... WHERE x = ?", (value,))`, never an f-string
Passwords	`hashlib.scrypt` with a random salt; compare with `hmac.compare_digest`
Secrets	in env vars, `.env` in `.gitignore`, commit `.env.example`, rotate leaked keys
CI order	lint, types, unit tests, integration tests, build, deploy
Package	`pyproject.toml`, `python -m build`

Chapter 14: Capstones and interviews

Need	Pattern
Percentile (nearest rank)	`xs[max(0, ceil(n*p/100) - 1)]` after sorting
Two-sum	dict `seen[value] = index`
Shortest path (unweighted)	BFS with `deque` and a `seen` set
Explain-X answer	define, consequence, fix, judgement

An Honest Closing Note

If you worked through these chapters with your hands on the keyboard, you can now read and write real Python: parse a log, build a clean data pipeline in plain Python, design classes, write tests, protect a database query from injection, call and build a small API, and explain generators, decorators, the GIL and MRO in your own words. That is the core a junior data or ML engineer needs from Python.

This document does not make you an expert on its own. It does not cover the data libraries (NumPy, pandas, scikit-learn), Docker, cloud services, or most of the deep topics in the missing-topics list. Several tools (pytest, FastAPI, pydantic, mypy, ruff, httpx) were shown as standard usage that was not run in the environment where this text was prepared, so run them yourself and compare.

Practise next: finish every chapter's practice set again without notes, build the four capstones in Chapter 14 and put them on GitHub with a clear README, and solve one coding problem a day. Then move to the pandas, SQL and statistics documents, where Python becomes a daily tool. Be patient

with yourself. Clear understanding, built slowly, is what employers pay for.